

Analise_De_Dados

September 12, 2026

1 5. Análise de Dados

1.1 Importando Bibliotecas e Carregando Dataset:

Nesta etapa da pesquisa, utilizaram-se as seguintes bibliotecas Python para a Análise Exploratória de Dados (EDA):

- **Pandas (pandas)**: Biblioteca central para manipulação de dados tabulares. Utilizada desde o início para carregar o dataset, fazer transformações em colunas, agrupar por datas e usuários, calcular estatísticas e estruturar dados para visualizações e análises.
- **Plotly Express (plotly.express)**: Usada para criar gráficos interativos, como a visualização do número de tweets por dia e da hashtag mais usada diariamente, facilitando a navegação temporal dos dados.
- **Matplotlib (matplotlib.pyplot, matplotlib.dates, matplotlib.lines.Line2D)**: Utilizada extensivamente para a criação de gráficos estáticos customizados, como séries temporais de tweets, engajamento por tipo ou fase, idade das contas, e distribuição de comprimento de tweets. Permitiu personalizações visuais detalhadas, como marcação de datas e picos.
- **NumPy (numpy)**: Aplicada em operações numéricas e categorização de dados, como a criação de colunas com faixas de comprimento ou idade da conta, além de ajustes em coordenadas de gráficos e verificação de tipos.
- **Seaborn (seaborn)**: Empregada para a geração de gráficos de distribuição mais sofisticados, como histogramas da idade das contas e do comprimento dos tweets, com suporte a suavizações (KDE).
- **Ast (ast)**: Usada para converter strings representando listas (em colunas como emojis, hashtags, mentions) em estruturas de dados reais (listas Python), viabilizando contagens e manipulações subsequentes.
- **Collections.Counter (Counter)**: Aplicada para contar a frequência de elementos em listas, especialmente para descobrir os emojis e hashtags mais usados nos tweets.
- **Re (re)**: Utilizada para aplicar expressões regulares em textos, principalmente para categorizar termos dentro de grupos temáticos, como “Infraestrutura”, “Ameaças”, e “Processo Eleitoral”.
- **WordCloud (wordcloud.WordCloud, STOPWORDS)**: Empregada na geração de nuvens de palavras a partir dos tweets processados, destacando os termos mais frequentes após remoção de stopwords e símbolos.

- **NLTK** (`nltk`, `nltk.corpus.stopwords`): Ferramenta de PLN usada para carregar e aplicar listas de stopwords em português, ajudando a limpar os textos para a nuvem de palavras.
- **PIL** (`PIL.Image`): Usada para carregar imagens/máscaras personalizadas utilizadas na renderização da nuvem de palavras.
- **Functools** (`functools.reduce`): Aplicada para combinar diversos DataFrames com métricas distintas em um único perfil consolidado de usuários, viabilizando análises completas por usuário.
- **Urllib.parse** (`urllib.parse.urlparse`): Embora usada de forma discreta, serve para analisar e decompor URLs presentes nos tweets.

```
[1]: import pandas                as pd
import plotly.express          as px
import matplotlib.pyplot       as plt
import matplotlib.dates        as mdates
from matplotlib.lines import Line2D
import numpy                   as np
import seaborn                 as sns
import ast
from collections import Counter
import re
from wordcloud import WordCloud, STOPWORDS
import nltk
from nltk.corpus import stopwords          as nltk_stopwords
from PIL import Image
from functools import reduce
from urllib.parse import urlparse
```

```
[2]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[3]: # Carreagando dataset
df_raw = pd.read_csv('/content/drive/MyDrive/TCC/Data/TUEBR222.csv',
                    sep=";", low_memory=False,
                    parse_dates=['tweet_created_at', 'user_created_at',
                                ↪ 'collection_date'])
```

```
[4]: df_raw.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 111217 entries, 0 to 111216
Data columns (total 62 columns):
#   Column
---


|   |           |
|---|-----------|
| 0 | tweet_id  |
| 1 | raw_tweet |


```

Non-Null Count	Dtype
111217 non-null	object
111217 non-null	object

2	preprocessed_tweet	111193 non-null	object
3	urls	17025 non-null	object
4	hashtags	111217 non-null	object
5	mentions	111217 non-null	object
6	emojis	111217 non-null	object
7	is_retweet	111217 non-null	int64
8	original_author	69620 non-null	object
9	is_truncated	111217 non-null	int64
10	is_upper	111217 non-null	int64
11	tweet_length	111217 non-null	int64
12	has_keywords	111217 non-null	int64
13	tweet_created_at	111217 non-null	datetime64[ns]
14	reply_to_tweet_id	28331 non-null	object
15	reply_to_user_id	28535 non-null	object
16	retweets	111217 non-null	int64
17	likes	111217 non-null	int64
18	engagement_raw	111217 non-null	int64
19	engagement_rate	111217 non-null	float64
20	engagement_index	111217 non-null	float64
21	url_count	111217 non-null	int64
22	hashtags_count	111217 non-null	int64
23	mentions_count	111217 non-null	int64
24	emoji_count	111217 non-null	int64
25	user_id	111217 non-null	object
26	followers_count	111217 non-null	int64
27	user_created_at	111217 non-null	datetime64[ns]
28	is_verified	111217 non-null	int64
29	account_age_days	111217 non-null	int64
30	collection_date	111217 non-null	datetime64[ns]
31	is_withheld	111217 non-null	int64
32	electoral_phase	111217 non-null	object
33	bot_score_heuristica	111217 non-null	float64
34	bot_score_botometer	111002 non-null	float64
35	date_calculated	111002 non-null	object
36	score_final	111002 non-null	float64
37	reliability	111002 non-null	float64
38	average_distance	111002 non-null	float64
39	automation_level	111002 non-null	object
40	sentiment	111217 non-null	object
41	entities	111217 non-null	object
42	clean_tweet	111152 non-null	object
43	topic_num	111217 non-null	float64
44	topic	111217 non-null	object
45	is_disinformation	111217 non-null	int64
46	confidence	111193 non-null	float64
47	indegree	63580 non-null	float64
48	outdegree	63580 non-null	float64
49	degree	63580 non-null	float64

```

50 cluster                63580 non-null    float64
51 cluster_tamanho        63580 non-null    float64
52 cluster_sentimento_dominante 63580 non-null    object
53 cluster_polaridade_media_por_usuario 63580 non-null    float64
54 cluster_polaridade_media_por_tweet 63580 non-null    float64
55 cluster_prop_bots       63580 non-null    float64
56 cluster_topico_dominante 63580 non-null    object
57 cluster_prop_disseminadores 63580 non-null    float64
58 cluster_prop_tweets_desinformacao 63580 non-null    float64
59 cluster_prop_verificados 63580 non-null    float64
60 cluster_total_tweets    63580 non-null    float64
61 cluster_hashtags_principais 63580 non-null    object
dtypes: datetime64[ns](3), float64(21), int64(17), object(21)
memory usage: 52.6+ MB

```

```

[5]: # Remove valores nulos e strings vazias
df_raw = df_raw[df_raw['preprocessed_tweet'].notna()]
df_raw = df_raw[df_raw['score_final'].notna()]

```

```

[6]: df_raw.info()

```

```

<class 'pandas.core.frame.DataFrame'>
Index: 110978 entries, 0 to 111216
Data columns (total 62 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   tweet_id                             110978 non-null  object
1   raw_tweet                           110978 non-null  object
2   preprocessed_tweet                   110978 non-null  object
3   urls                                16986 non-null   object
4   hashtags                            110978 non-null  object
5   mentions                            110978 non-null  object
6   emojis                              110978 non-null  object
7   is_retweet                          110978 non-null  int64
8   original_author                     69564 non-null   object
9   is_truncated                        110978 non-null  int64
10  is_upper                            110978 non-null  int64
11  tweet_length                        110978 non-null  int64
12  has_keywords                        110978 non-null  int64
13  tweet_created_at                    110978 non-null  datetime64[ns]
14  reply_to_tweet_id                   28189 non-null   object
15  reply_to_user_id                    28391 non-null   object
16  retweets                            110978 non-null  int64
17  likes                               110978 non-null  int64
18  engagement_raw                      110978 non-null  int64
19  engagement_rate                     110978 non-null  float64
20  engagement_index                    110978 non-null  float64
21  url_count                           110978 non-null  int64

```

22	hashtags_count	110978	non-null	int64
23	mentions_count	110978	non-null	int64
24	emoji_count	110978	non-null	int64
25	user_id	110978	non-null	object
26	followers_count	110978	non-null	int64
27	user_created_at	110978	non-null	datetime64[ns]
28	is_verified	110978	non-null	int64
29	account_age_days	110978	non-null	int64
30	collection_date	110978	non-null	datetime64[ns]
31	is_withheld	110978	non-null	int64
32	electoral_phase	110978	non-null	object
33	bot_score_heuristica	110978	non-null	float64
34	bot_score_botometer	110978	non-null	float64
35	date_calculated	110978	non-null	object
36	score_final	110978	non-null	float64
37	reliability	110978	non-null	float64
38	average_distance	110978	non-null	float64
39	automation_level	110978	non-null	object
40	sentiment	110978	non-null	object
41	entities	110978	non-null	object
42	clean_tweet	110937	non-null	object
43	topic_num	110978	non-null	float64
44	topic	110978	non-null	object
45	is_disinformation	110978	non-null	int64
46	confidence	110978	non-null	float64
47	indegree	63572	non-null	float64
48	outdegree	63572	non-null	float64
49	degree	63572	non-null	float64
50	cluster	63572	non-null	float64
51	cluster_tamanho	63572	non-null	float64
52	cluster_sentimento_dominante	63572	non-null	object
53	cluster_polaridade_media_por_usuario	63572	non-null	float64
54	cluster_polaridade_media_por_tweet	63572	non-null	float64
55	cluster_prop_bots	63572	non-null	float64
56	cluster_topico_dominante	63572	non-null	object
57	cluster_prop_disseminadores	63572	non-null	float64
58	cluster_prop_tweets_desinformacao	63572	non-null	float64
59	cluster_prop_verificados	63572	non-null	float64
60	cluster_total_tweets	63572	non-null	float64
61	cluster_hashtags_principais	63572	non-null	object

dtypes: datetime64[ns](3), float64(21), int64(17), object(21)

memory usage: 53.3+ MB

1.1.1 Definição do corpus: por que separar retweets?

O dataset bruto contém **retweets**, **replies** e **tweets originais** misturados. Cada tipo tem uma natureza diferente e **não podem ser tratados como equivalentes** em todas as análises.

1.1.2 df_com_retweets — corpus completo (circulação)

Contém **todas as linhas** do dataset original. É o recorte adequado para análises **de rede e de circulação**, porque o retweet é a própria aresta de difusão:

- Métricas de grafo: `indegree`, `outdegree`, `degree`, `cluster*`
- Amplificação / viralidade de um conteúdo
- Fluxos entre comunidades, homofilia, assortatividade
- Perfil de atividade do usuário (`Retweetou`, `Retweetado`)
- Análise de URLs como “o que circulou”

1.1.3 df_sem_retweets — apenas originais e replies (produção)

Mantém somente `is_retweet == 0`. É o recorte adequado para **tudo que mede discurso, produção ou engajamento real**:

- Engajamento (`likes`, `retweets`, `engagement_raw/rate/index`) → retweets **não têm métricas próprias**: são cópias dos valores do tweet original. Incluí-los infla artificialmente o engajamento (ver p. 17 do relatório).
- Emojis, hashtags, termos, entidades, tópicos, sentimento → incluir retweets gera **dupla contagem**: um original retweetado 1000× contaria 1001× nas contagens de conteúdo.
- Distribuição de comprimento de tweets
- Prevalência de desinformação como **produção** (não como circulação)
- Cascatas de conversação (raízes já devem ser originais)

1.1.4 df_users — um registro por usuário

Colapsa o dataset para **uma linha por user_id** (mantendo a aparição mais recente). Evita que usuários muito ativos distorçam médias e distribuições em análises **por usuário** (idade da conta, verificação, nível de automação, `score_final`).

```
[7]: # =====
# 1.2 SEPARAÇÃO DO CORPUS EM TRÊS RECORTES
# -----
# df_com_retweets : corpus completo (circulação / rede)
# df_sem_retweets : apenas is_retweet == 0 (produção / discurso / engajamento)
# df_retweets : apenas is_retweet == 1 (amplificação)
# df_users      : uma linha por usuário (última aparição)
# =====

# -----
# 0) Partimos do dataframe já limpo (df_raw), após:
#   - remoção de nulos em 'preprocessed_tweet' e 'score_final'
#   - conversão de datas (parse_dates no read_csv)
#   Se 'df_raw' ainda não estiver limpo, refaça os filtros aqui.
# -----
assert 'df_raw' in globals(), "O dataframe 'df_raw' precisa existir antes desta_
→ célula."

# -----
```

```

# 1) df_com_retweets - corpus COMPLETO (com retweets)
# -----
# Cópia independente para evitar SettingWithCopyWarning e vazamento
# de alterações entre os recortes.
df_com_retweets = df_raw.copy()

# -----
# 2) df_sem_retweets - apenas ORIGINAIS e REPLIES
# -----
df_sem_retweets = df_raw[df_raw['is_retweet'] == 0].copy()

# -----
# 3) df_retweets - apenas RETWEETS
# -----
df_retweets = df_raw[df_raw['is_retweet'] == 1].copy()

# -----
# 4) df_users - um registro por usuário (última aparição)
# -----
# 'keep="last"' assume que o df está ordenado cronologicamente.
# Caso contrário, ordenar antes:
# df = df.sort_values('tweet_created_at')

df_users = (
    df_com_retweets
    .sort_values('tweet_created_at')
    .drop_duplicates(subset='user_id', keep='last')
    .copy()
)

# -----
# 5) Verificação de integridade
# -----
resumo = pd.DataFrame({
    'recorte': ['df_com_retweets', 'df_sem_retweets', 'df_retweets', 'df_users'],
    'linhas': [len(df_com_retweets), len(df_sem_retweets), len(df_retweets),
↵, len(df_users)],
    'usuarios_unicos': [
        df_com_retweets['user_id'].nunique(),
        df_sem_retweets['user_id'].nunique(),
        df_retweets['user_id'].nunique(),
        df_users['user_id'].nunique(),
    ],
    'uso_principal': [
        'rede, amplificação, circulação',
        'discurso, engajamento, produção',
    ],
})

```

```

        'amplificação, circulação',
        'perfil e características do usuário'
    ]
})
display(resumo)

# Sanity checks
assert len(df_users) == df_users['user_id'].nunique(), \
    "df_users deve ter 1 linha por usuário."
assert set(df_sem_retweets['tweet_id']).
    ↳issubset(set(df_com_retweets['tweet_id'])), \
    "df_sem_retweets deve ser subconjunto de df_com_retweets."
assert (df_sem_retweets['is_retweet'] == 0).all(), \
    "df_sem_retweets não pode conter retweets."
print(" Separação consistente.")

```

	recorte	linhas	usuarios_unicos \
0	df_com_retweets	110978	84473
1	df_sem_retweets	41414	34234
2	df_retweets	69564	52810
3	df_users	84473	84473

	uso_principal
0	rede, amplificação, circulação
1	discurso, engajamento, produção
2	amplificação, circulação
3	perfil e características do usuário

Separação consistente.

1.2 5.1 Caracterização Geral do Corpus

1.2.1 5.1.1 Análise Temporal e de Tipos

```

[8]: #-----
# Tweets ao longo do tempo (interativo)
#-----

tweets_por_dia = df_com_retweets['tweet_created_at'].dt.date.value_counts().
    ↳sort_index()
tweets_por_dia = tweets_por_dia.reset_index()
tweets_por_dia.columns = ['Data', 'Quantidade']

# Cria o gráfico com eixos corretos
fig = px.line(
    tweets_por_dia,
    x='Data',          # Coluna para eixo X
    y='Quantidade',    # Coluna para eixo Y

```

```

        title='Número de Tweets por Dia'
    )

    # Ajusta os rótulos dos eixos
    fig.update_layout(
        xaxis_title='Dia da Publicação',
        yaxis_title='Quantidade de Tweets'
    )

    fig.show()

```

```

[9]: #-----
# Análise temporal de tweets:
# - Gera gráfico de tweets por dia ao longo do tempo
# - Destaca os dias com volume de tweets acima do percentil 75 (top 25%)
# - Evidencia os dias de votação (02/10 e 30/10) com destaque visual
# - Marca o primeiro e o último dia da série temporal no eixo X
#-----

# Preparação dos dados
df_com_retweets['tweet_created_at_day'] = df_com_retweets['tweet_created_at'].dt.\
    ↪date
tweets_por_dia = df_com_retweets.groupby('tweet_created_at_day').size().\
    ↪reset_index(name='count')
tweets_por_dia['tweet_created_at_day'] = pd.\
    ↪to_datetime(tweets_por_dia['tweet_created_at_day'])

# Identificação dos picos (top 25%)
limiar = tweets_por_dia['count'].quantile(0.75)
picos = tweets_por_dia[tweets_por_dia['count'] > limiar].copy()

# Configuração do gráfico
plt.figure(figsize=(14, 6))
ax = plt.gca()

# Plotagem da linha temporal principal
ax.plot(tweets_por_dia['tweet_created_at_day'],
        tweets_por_dia['count'],
        color='#1f77b4',
        linestyle='-',
        linewidth=2,
        marker='o',
        markersize=4,
        markeredgcolor='#1f77b4',
        label='Tweets por dia')

```

```

# Destaque dos picos
colors = ['green' if date == pd.to_datetime('2022-10-02') or
          date == pd.to_datetime('2022-10-30') else 'red'
          for date in picos['tweet_created_at_day']]
ax.scatter(
    picos['tweet_created_at_day'],
    picos['count'],
    color=colors,
    s=25,
    zorder=2,
    edgecolor='black'
)

# Criar elementos customizados para a legenda
legend_elements = [
    Line2D([0], [0], marker='o',
           color='w', label=f'Picos normais (Top 25% > {int(limiar)} tweets)',
           markerfacecolor='red', markersize=5, markeredgcolor='black'),
    Line2D([0], [0], marker='o', color='w', label='Dias de votação',
           markerfacecolor='green', markersize=5, markeredgcolor='black')
]

# Adicionar a legenda customizada
ax.legend(handles=legend_elements, loc='upper left')

# Configuração do eixo x (apenas algumas datas estratégicas + picos)
# Seleciona algumas datas equidistantes
datas_principais = pd.date_range(
    start=tweets_por_dia['tweet_created_at_day'].min(),
    end=tweets_por_dia['tweet_created_at_day'].max(),
    periods=2 # Primeiro e último dia de coleta
)

# Combina datas principais com datas dos picos
todas_datas_mostrar = sorted(set(datas_principais.
    → union(picos['tweet_created_at_day'])))

ax.set_xticks(todas_datas_mostrar)
ax.xaxis.set_major_formatter(mdates.DateFormatter('%d/%m'))

# Destacar labels dos picos no eixo x
for label in ax.get_xticklabels():
    date_str = label.get_text()
    date = pd.to_datetime(date_str, format='%d/%m', errors='coerce')
    label.set_color('black')
    label.set_fontweight('bold')
    label.set_fontsize(7)

```

```

plt.xticks(rotation=90, ha='center')

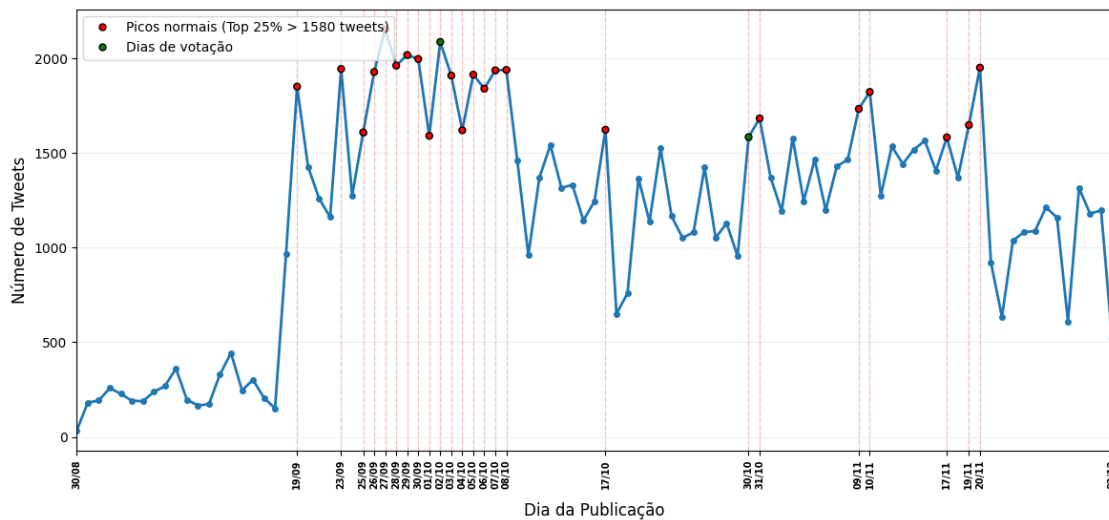
# Linhas de referência para os picos
for date in picos['tweet_created_at_day']:
    ax.axvline(x=date, color='red', alpha=0.2, linestyle='--', linewidth=1)

# Configuração dos limites do eixo x
ax.set_xlim([
    tweets_por_dia['tweet_created_at_day'].min() - pd.Timedelta(days=0),
    tweets_por_dia['tweet_created_at_day'].max() + pd.Timedelta(days=0)
])

# Elementos adicionais do gráfico
ax.set_xlabel('Dia da Publicação', fontsize=12, labelpad=10)
ax.set_ylabel('Número de Tweets', fontsize=12)
ax.grid(True, alpha=0.2)
# ax.legend(loc='upper left')

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/tweets_diaros_com_picos.png',
            dpi=300, bbox_inches='tight')
plt.show()

```



```

[10]: # Datas dos picos (25%) + primeiro e último dia
df_datas = pd.DataFrame({'data': todas_datas_mostrar})

```

```

[11]: # Qnt de dias
df_datas['data'].count()

```

```
[11]: np.int64(26)
```

```
[12]: #-----  
# Tabela: Picos de 25% (Estatísticas Diárias)  
#-----  
"""  
Exibe os dias com maior atividade (top 25%) em:  
- Volume de tweets  
- Interações (likes, retweets)  
- Menções/hashtags  
"""  
  
# Extrai apenas a data (sem hora) da coluna 'tweet_created_at'  
df_com_retweets['date'] = df_com_retweets['tweet_created_at'].dt.date  
  
# Agrupa os dados por data e calcula várias métricas agregadas  
daily_stats = df_com_retweets.groupby('date').agg(  
    # Conta usuários únicos (baseado na coluna 'user_id')  
    Usuários=('user_id', 'nunique'),  
  
    # Conta o total de tweets (baseado na contagem de 'tweet_id')  
    Tweets=('tweet_id', 'count'),  
  
    # Conta tweets originais (onde 'is_retweet' é igual a 0)  
    Original=('is_retweet', lambda x: (x == 0).sum()),  
  
    # Soma os retweets (onde 'is_retweet' é igual a 1)  
    Retweets=('is_retweet', 'sum'),  
  
    # Conta respostas (baseado na coluna 'reply_to_tweet_id')  
    Respostas=('reply_to_tweet_id', 'count'),  
  
    # Soma o total de menções em todos os tweets  
    Menções=('mentions_count', 'sum'),  
  
    # Soma o total de hashtags em todos os tweets  
    Hashtags=('hashtags_count', 'sum'),  
  
    # Soma o total de emojis em todos os tweets  
    Emojis=('emoji_count', 'sum'),  
  
    # Soma o total de URLs em todos os tweets  
    URLs=('url_count', 'sum'),  
  
    # Calcula a média de likes por tweet  
    Média_Likes=('likes', 'mean'),
```

```

# Calcula a média de retweets por tweet
Média_Retweets=('retweets', 'mean'),

# Calcula o comprimento médio dos tweets
Média_Comprimento=('tweet_length', 'mean')
).reset_index() # Remove o agrupamento e transforma em DataFrame normal

# Formata a data como 'dd/mm/yyyy' e renomeia para 'Data'
daily_stats['Data'] = daily_stats['date'].apply(lambda x: x.strftime('%d/%m/%Y'))

# Reorganiza as colunas (Data primeiro) e remove a coluna original 'date'
daily_stats = daily_stats[['Data'] + [col for col in daily_stats.columns if col
↳not in ['date', 'Data']]]

# Arredonda as colunas de média
daily_stats[['Média_Likes', 'Média_Retweets', 'Média_Comprimento']] =
↳daily_stats[
    ['Média_Likes', 'Média_Retweets', 'Média_Comprimento']].round(2)

# Filtra apenas as datas presentes em df_dadas
daily_stats = daily_stats[daily_stats['Data'].isin(df_dadas['data'].dt.
↳strftime('%d/%m/%Y'))]

# --- AJUSTE DE NOMES PARA EXIBIÇÃO ---
# Renomeia as colunas para melhor legibilidade (apenas na visualização)
daily_stats_display = daily_stats.rename(columns={
    'Média_Likes': 'Média de Likes',
    'Média_Retweets': 'Média de Retweets',
    'Média_Comprimento': 'Média de Comprimento'
})

# Exibe o DataFrame formatado
display(daily_stats_display)

```

	Data	Usuários	Tweets	Original	Retweets	Respostas	Menções	\
0	30/08/2022	32	34	34	0	16	29	
20	19/09/2022	1707	1850	712	1138	387	828	
24	23/09/2022	1844	1943	846	1097	488	1053	
26	25/09/2022	1524	1608	535	1073	291	620	
27	26/09/2022	1857	1927	631	1296	291	670	
28	27/09/2022	2068	2152	779	1373	435	830	
29	28/09/2022	1831	1961	589	1372	345	703	
30	29/09/2022	1936	2017	523	1494	299	777	
31	30/09/2022	1959	1996	627	1369	275	754	
32	01/10/2022	1543	1590	461	1129	217	441	
33	02/10/2022	2062	2086	405	1681	149	325	
34	03/10/2022	1882	1908	757	1151	485	890	

35	04/10/2022	1592	1619	682	937	450	946
36	05/10/2022	1833	1913	809	1104	538	1139
37	06/10/2022	1753	1839	872	967	580	1195
38	07/10/2022	1908	1936	280	1656	197	367
39	08/10/2022	1915	1938	378	1560	262	505
48	17/10/2022	1575	1622	592	1030	374	712
61	30/10/2022	1567	1583	344	1239	126	310
62	31/10/2022	1659	1682	466	1216	300	720
71	09/11/2022	1670	1732	567	1165	460	1037
72	10/11/2022	1783	1822	541	1281	452	1017
79	17/11/2022	1550	1582	374	1208	270	665
81	19/11/2022	1536	1647	424	1223	359	800
82	20/11/2022	1874	1950	249	1701	213	550
94	02/12/2022	521	524	71	453	59	118

	Hashtags	Emojis	URLs	Média de Likes	Média de Retweets	\
0	9	4	10	34.00	10.44	
20	375	782	378	2.09	250.92	
24	170	2757	330	0.95	397.99	
26	254	1045	254	0.95	585.80	
27	263	719	637	0.36	1050.19	
28	316	571	624	0.25	838.14	
29	163	566	277	0.07	560.29	
30	142	528	303	0.04	1602.66	
31	224	491	329	0.20	3779.14	
32	188	649	314	0.04	1387.19	
33	115	1147	608	0.00	2166.08	
34	96	325	396	0.00	1733.30	
35	59	274	221	0.00	734.84	
36	68	434	274	0.00	1090.00	
37	163	326	344	0.00	315.41	
38	35	118	192	0.00	6902.57	
39	31	140	199	0.00	7880.05	
48	181	594	176	0.02	596.86	
61	71	781	256	0.00	2082.41	
62	25	229	312	0.00	4053.91	
71	20	376	148	0.00	1277.91	
72	16	306	153	0.00	2854.11	
79	16	611	93	0.00	3329.49	
81	16	309	109	0.00	1084.99	
82	16	193	67	0.00	3162.28	
94	5	24	393	0.00	1069.78	

	Média de Comprimento
0	152.74
20	97.22
24	104.92
26	102.30

27	101.19
28	109.62
29	117.09
30	105.56
31	96.87
32	96.57
33	87.71
34	110.80
35	118.25
36	122.36
37	121.04
38	113.74
39	112.98
48	112.01
61	99.61
62	107.66
71	126.75
72	129.10
79	124.34
81	131.06
82	123.32
94	100.00

```
[13]: # Categorizar os tweets
conditions= [
    (df_com_retweets['is_retweet'] == 1), (df_com_retweets['reply_to_tweet_id'].
    ↪notna() & (df_com_retweets['is_retweet'] == 0)),
    (df_com_retweets['is_retweet'] == 0) & (df_com_retweets['reply_to_tweet_id'].
    ↪isna())
]
choices = ['Retweet', 'Reply', 'Original']
df_com_retweets['tweet_type'] = np.select(conditions, choices,
    ↪default='Original')
# Agrupar por data e tipo
daily_counts = df_com_retweets.groupby([
    pd.Grouper(key='tweet_created_at', freq='D'),
    'tweet_type'
]).size().unstack(fill_value=0)
```

```
[14]: #-----
# Distribuição Entre os Tipos de Tweets:
# Originais, Retweets e Replies (Comentários)
#-----

# Calcular a distribuição incluindo replies
tweet_distribution = df_com_retweets['tweet_type'].value_counts(normalize=True)
```

```

# Ordem desejada e cores
order = ['Original', 'Retweet', 'Reply']
colors = ['#1f77b4', '#ff7f0e', '#2ca02c'] # Cores tradicionais

# Criar figura
plt.figure(figsize=(8, 8))

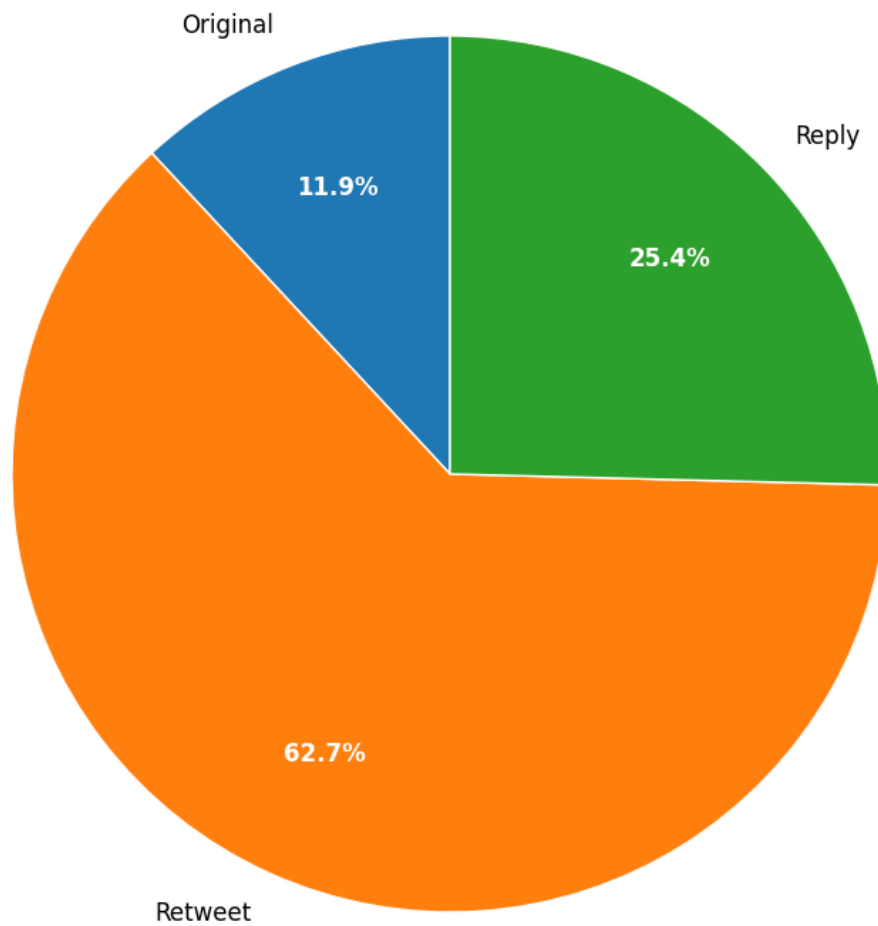
# Plotar gráfico de pizza
plt.pie(
    tweet_distribution[order],
    labels=order,
    colors=colors,
    autopct='%1.1f%%',
    startangle=90,
    wedgeprops={'linewidth': 1, 'edgecolor': 'white'},
    textprops={'fontsize': 12},
    pctdistance=0.7 # Posição das porcentagens
)

# Destacar as porcentagens
for text in plt.gca().texts:
    if '%' in text.get_text():
        text.set_color('white')
        text.set_fontweight('bold')

# Ajustar layout
plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/tipos_de_tweets.png', dpi=300,
            bbox_inches='tight')
plt.show()

```



```
[15]: #-----  
# Distribuição de tweets ao longo do tempo por tipo  
#-----  
  
# Configurações de estilo  
plt.style.use('seaborn-v0_8-darkgrid')  
palette = ['#1f77b4', '#ff7f0e', '#2ca02c'] # Azul, Laranja, Verde  
  
# Criar figura e eixo  
plt.figure(figsize=(16, 8))  
ax = plt.gca()  
  
# Plotar as linhas com estilos diferentes para melhor distinção
```

```

line_styles = ['-', '--', '-.']
for i, col in enumerate(['Original', 'Retweet', 'Reply']):
    daily_counts[col].plot(
        kind='line',
        linewidth=2.5,
        color=palette[i],
        linestyle=line_styles[i],
        marker='o',
        markersize=5,
        markevery=7, # Mostrar marcador a cada 7 pontos
        ax=ax
    )

# Rótulos
plt.xlabel('Dia da Publicação', fontsize=12, labelpad=10)
plt.ylabel('Quantidade de Tweets', fontsize=12, labelpad=10)

# Formatar eixo de datas
ax.xaxis.set_major_formatter(mdates.DateFormatter('%d/%m/%Y'))
plt.xticks(rotation=45)
ax.xaxis.set_major_locator(mdates.WeekdayLocator(interval=1))

# Adicionar grid e ajustes visuais
ax.yaxis.grid(True, linestyle='--', alpha=0.7)
ax.xaxis.grid(False)

# Melhorar legenda
legend = ax.legend(
    title='Tipo de Tweet',
    title_fontsize=12,
    fontsize=11,
    frameon=True,
    framealpha=1,
    edgecolor='#444444'
)

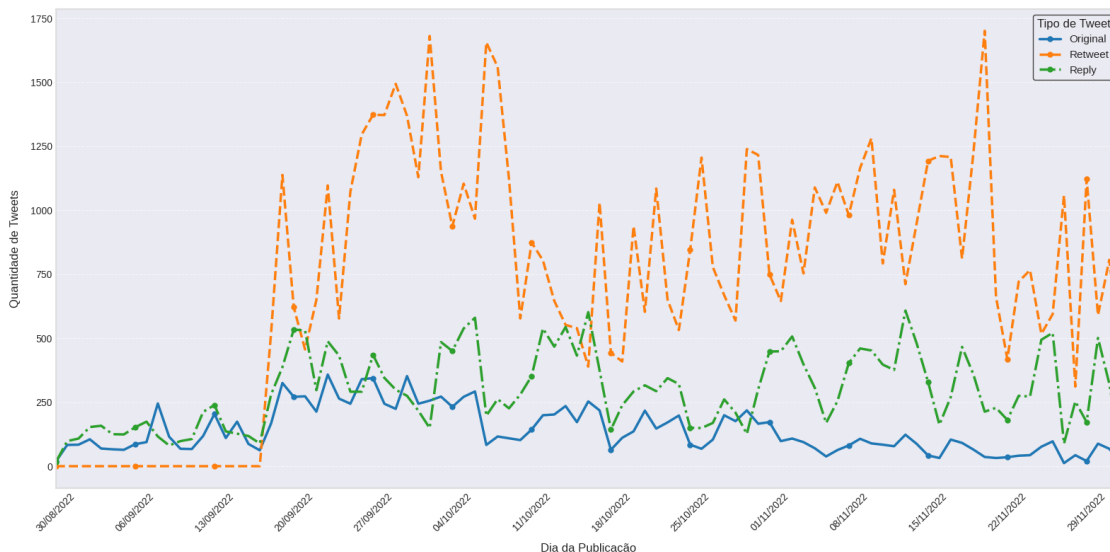
# Ajustar layout
plt.tight_layout()

# Destacar eixos
for spine in ax.spines.values():
    spine.set_edgecolor('#dddddd')
    spine.set_linewidth(1.5)

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/distribuicao_tweets_por_tipo.
→png', dpi=300, bbox_inches='tight')

```

```
plt.show()
```



Durante a fase inicial da coleta de dados, optou-se por ignorar retweets, a fim de garantir que apenas conteúdo original fosse incluído no corpus referente ao período analisado.

```
[16]: #-----
# Contagem de Tweets por Fase Eleitoral (com retweets)
#-----

plt.figure(figsize=(12,5))
phase_counts = df_com_retweets['electoral_phase'].value_counts()
phase_counts.sort_values().plot(kind='barh',

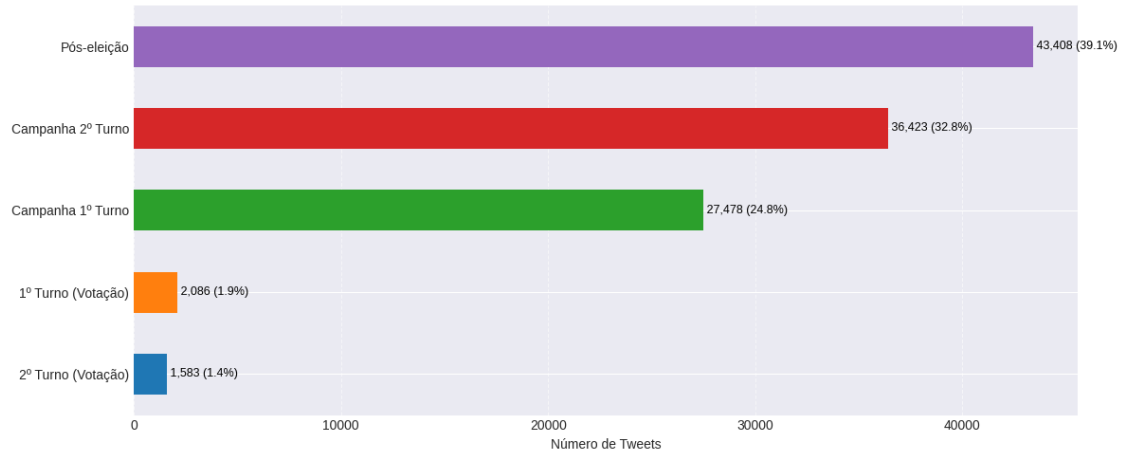
    ↳color=['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd'])

# Adicionar valores nas barras
for i, v in enumerate(phase_counts.sort_values()):
    plt.text(v, i, f" {v:,} ({v/sum(phase_counts):.1%})",
            color='black', va='center', fontsize=9)

plt.xlabel('Número de Tweets')
plt.ylabel('')
plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.tight_layout()

# Salvar e mostrar
```

```
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳ contagem_tweets_com_retweets_por_fase_eleitoral.png', dpi=300,
↳ bbox_inches='tight')
plt.show()
```



1.2.2 5.1.2 Análise dos Tweets

5.1.2.1 Produção (Tweets e Respostas)

```
[17]: # Categorizar os tweets (sem retweets)
conditions= [
    (df_sem_retweets['is_retweet'] == 1), (df_sem_retweets['reply_to_tweet_id'] .
↳ notna() & (df_sem_retweets['is_retweet'] == 0)),
    (df_sem_retweets['is_retweet'] == 0) & (df_sem_retweets['reply_to_tweet_id'] .
↳ isna())
]
choices = ['Retweet', 'Reply', 'Original']
df_sem_retweets['tweet_type'] = np.select(conditions, choices,
↳ default='Original')
# Agrupar por data e tipo
# daily_counts = df_sem_retweets.groupby([
# pd.Grouper(key='tweet_created_at', freq='D'),
# 'tweet_type'
# ]).size().unstack(fill_value=0)
```

```
[18]: #-----
# Média de Engajamento por Fase Eleitoral (sem retweets)
#-----

df_fase = df_sem_retweets.groupby('tweet_type')[[
    'engagement_raw', 'engagement_rate', 'engagement_index']].mean().loc[
    ['Original', 'Reply']]
```

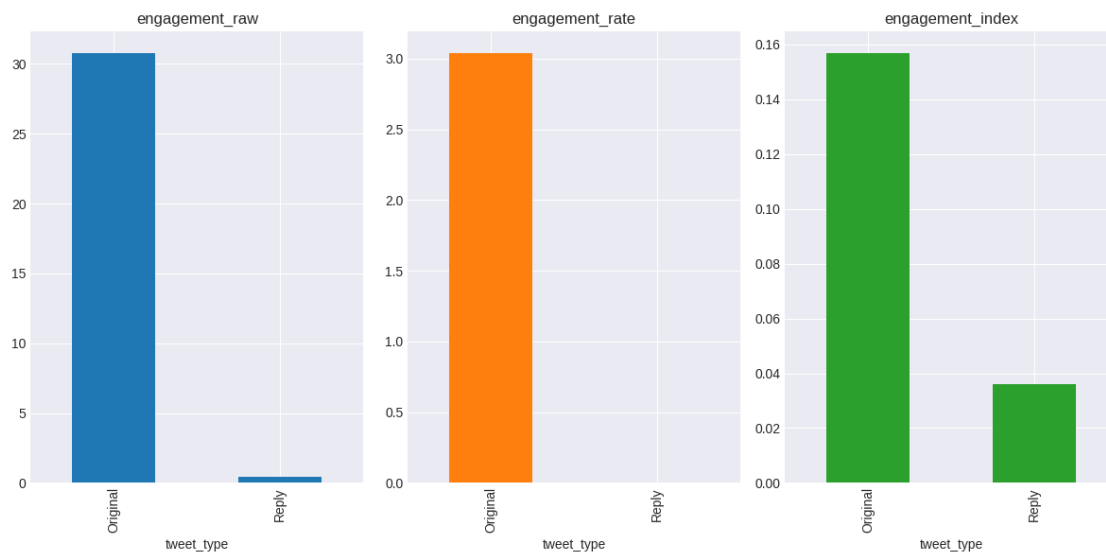
```

]

# Plot
df_fase.plot(kind='bar', figsize=(12, 6), subplots=True,
              layout=(1, 3), sharey=False, legend=False)
plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/media_engajamento_por_fase.
            ↪png', dpi=300, bbox_inches='tight')
plt.show()

```



```

[19]: #-----
# Média de Engajamento ao longo do tempo (sem retweets)
#-----

# Converter coluna de data
df_sem_retweets.loc[:, 'tweet_created_at'] = pd.
    ↪to_datetime(df_sem_retweets['tweet_created_at'])

# Engajamento médio por dia
eng_por_dia = df_sem_retweets.groupby(df_sem_retweets['tweet_created_at'].dt.
    ↪date)['engagement_rate'].mean()

plt.figure(figsize=(12, 5))
eng_por_dia.plot()

# Linha horizontal com a média geral

```

```

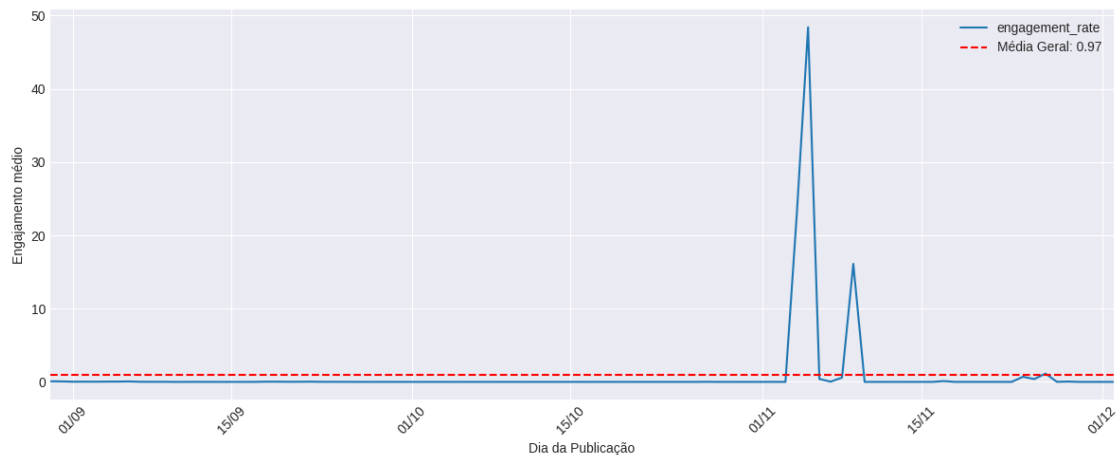
media_geral = df_sem_retweets['engagement_rate'].mean()
plt.axhline(y=media_geral, color='r', linestyle='--', label=f'Média Geral:␣
↳{media_geral:.2f}')

# Formatar eixo X (datas como dd/mm)
ax = plt.gca()
ax.xaxis.set_major_formatter(mdates.DateFormatter('%d/%m'))
plt.xlim([eng_por_dia.index[0], eng_por_dia.index[-1]]) # Ajustar limites

# Labels e legenda
plt.xlabel("Dia da Publicação")
plt.ylabel("Engajamento médio")
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/media_engajamento_temporal.
↳png', dpi=300, bbox_inches='tight')
plt.show()

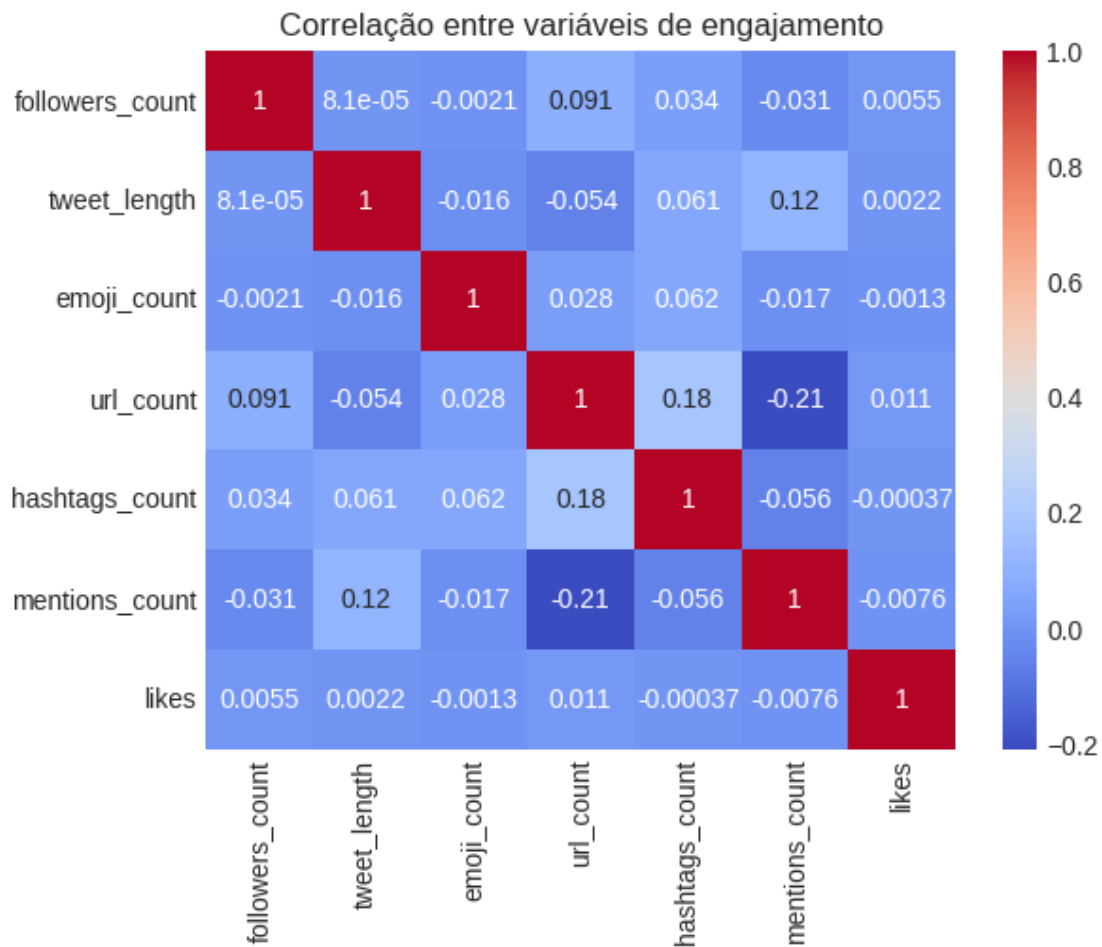
```



```

[20]: # Correlações com engajamento (sem retweets)
cols_eng = ['followers_count', 'tweet_length', 'emoji_count', 'url_count',␣
↳'hashtags_count', 'mentions_count', 'likes']
sns.heatmap(df_sem_retweets[cols_eng].corr(), annot=True, cmap="coolwarm")
plt.title("Correlação entre variáveis de engajamento")
plt.show()

```



```
[21]: #-----
# Proporção de tweets possivelmente alarmistas (sem retweets)
#-----

# Dados
alarmistas = df_sem_retweets['is_upper'].value_counts(normalize=True)
alarmistas.index = ['Normal', 'Alarmista']

colors = ['#1f77b4', '#ff7f0e'] # Azul para Normal, Laranja para Alarmista

# Criar figura
plt.figure(figsize=(8, 8))

plt.pie(
    alarmistas,
```

```

    labels=alarmistas.index,
    colors=colors,
    autopct='%1.1f%%',
    startangle=90,
    wedgeprops={'linewidth': 1, 'edgecolor': 'white'},
    textprops={'fontsize': 12},
    pctdistance=0.7
)

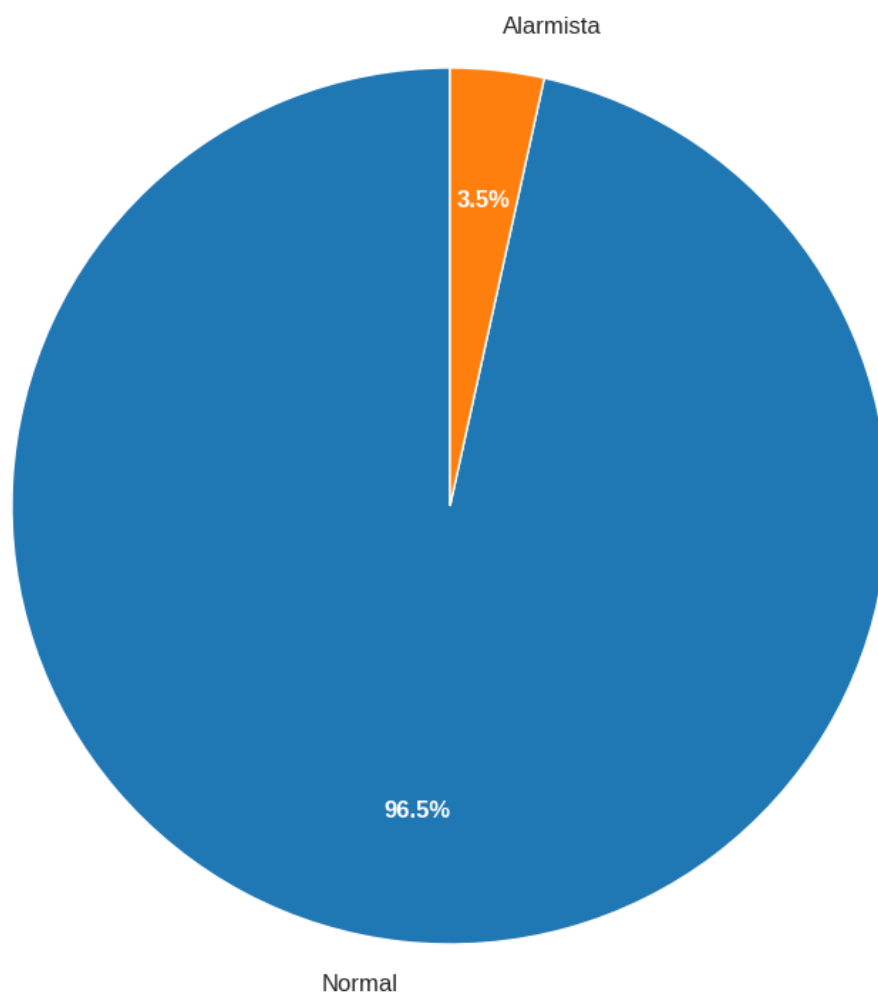
# Destacar porcentagens
for text in plt.gca().texts:
    if '%' in text.get_text():
        text.set_color('white')
        text.set_fontweight('bold')

# Remover rótulo do eixo Y
plt.ylabel('')

# Ajustar layout
plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/tweets_alarmistas.png',
            dpi=300, bbox_inches='tight')
plt.show()

```



```
[22]: #-----
# Contagem de Tweets por Fase Eleitoral (sem retweets)
#-----

plt.figure(figsize=(12,5))
phase_counts = df_sem_retweets['electoral_phase'].value_counts()
phase_counts.sort_values().plot(kind='barh',
    ↳color=['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd'])

# Adicionar valores nas barras
for i, v in enumerate(phase_counts.sort_values()):
    plt.text(v, i, f" {v:,} ({v/sum(phase_counts):.1%})",
```

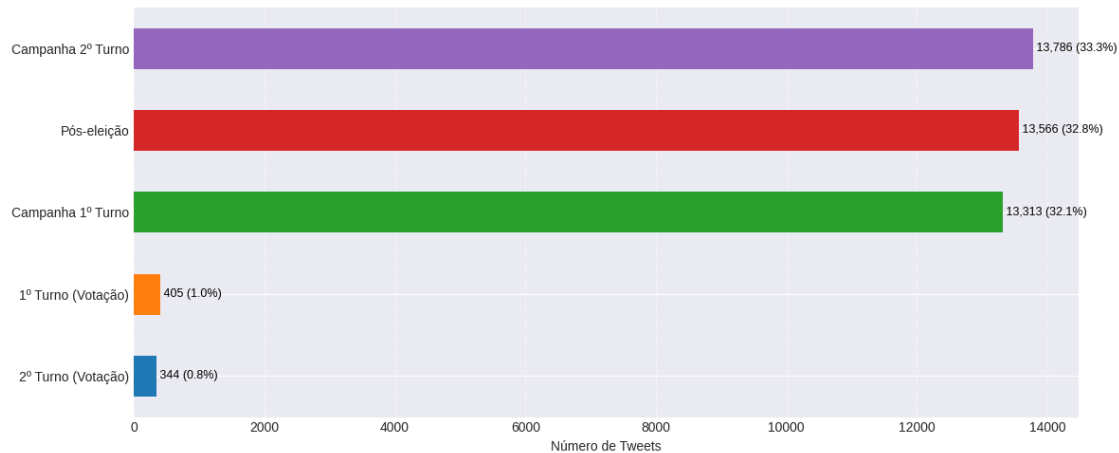
```

        color='black', va='center', fontsize=9)

plt.xlabel('Número de Tweets')
plt.ylabel('')
plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳contagem_tweets_por_fase_eleitoral.png', dpi=300, bbox_inches='tight')
plt.show()

```



```

[23]: # Média de Engajamento por Fase Eleitoral (sem retweets)
df_sem_retweets.groupby('electoral_phase')[['engagement_raw', 'engagement_rate',
                                             'engagement_index']].mean()

```

```

[23]:
          engagement_raw  engagement_rate  engagement_index
electoral_phase
1º Turno (Votação)      0.002469         0.000024         0.000222
2º Turno (Votação)      0.000000         0.000000         0.000000
Campanha 1º Turno       8.258018         0.013497         0.093123
Campanha 2º Turno       0.033512         0.000317         0.001939
Pós-eleição            22.735884         2.960066         0.134488

```

```

[24]: #-----
# Média de Engajamento por Fase Eleitoral (sem retweets)
#-----

# Ordena fases
df_fase = df_sem_retweets.groupby('electoral_phase')[
    ['engagement_raw', 'engagement_rate', 'engagement_index']].mean().loc[

```

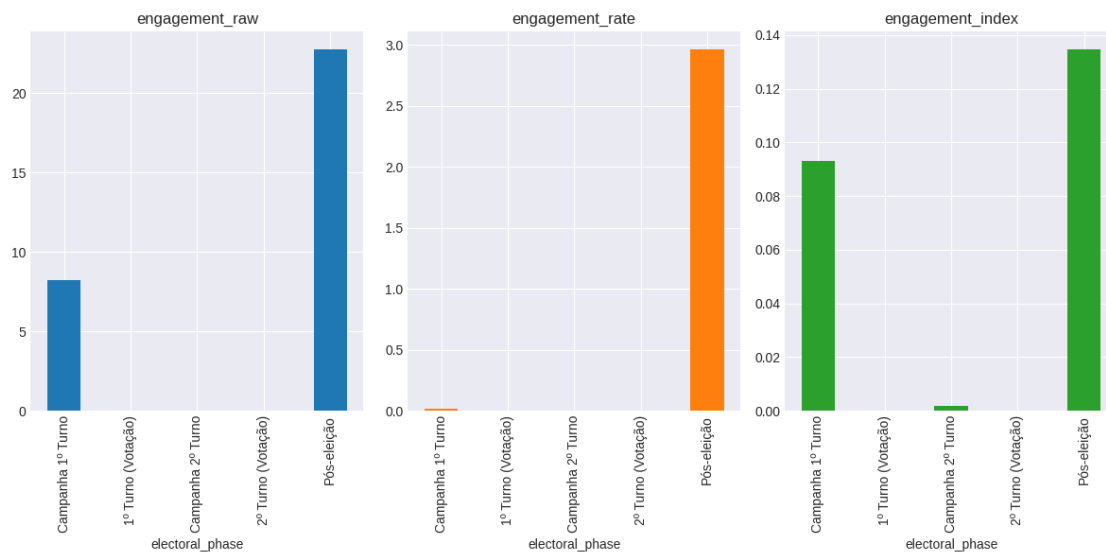
```

['Campanha 1º Turno', '1º Turno (Votação)',
 'Campanha 2º Turno', '2º Turno (Votação)', 'Pós-eleição']
]

# Plot
df_fase.plot(kind='bar', figsize=(12, 6), subplots=True,
              layout=(1, 3), sharey=False, legend=False)
plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳media_engajamento_por_fase_eleitoral.png', dpi=300, bbox_inches='tight')
plt.show()

```



```

[25]: # Comprimento máx e min dos tweets (sem retweets)
min_len = df_sem_retweets['tweet_length'].min()
max_len = df_sem_retweets['tweet_length'].max()
print(f"Tamanho mínimo: {min_len}")
print(f"Tamanho máximo: {max_len}")

```

Tamanho mínimo: 6
Tamanho máximo: 285

```

[26]: #-----
# Distribuição do comprimento dos tweets (sem retweets)
#-----

plt.figure(figsize=(12, 6))

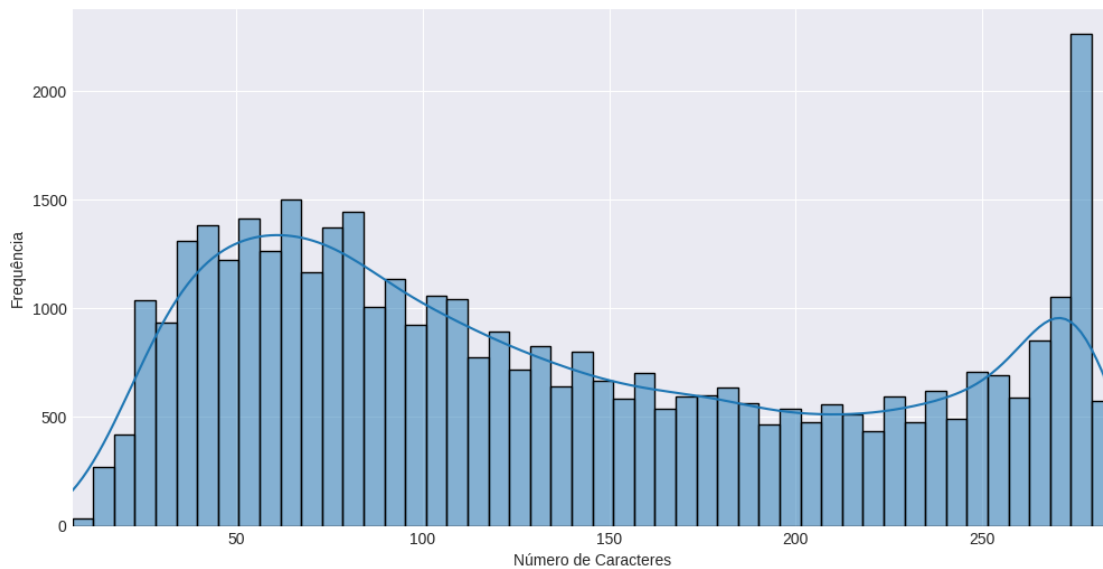
```

```

sns.histplot(df_sem_retweets['tweet_length'], bins=50, kde=True,
             ↳kde_kws={'bw_adjust': 1.2})
plt.xlabel('Número de Caracteres')
plt.ylabel('Frequência')
plt.xlim(min_len, max_len)
plt.grid(True)

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/distribuicao_comprimento_tweet.
           ↳png', dpi=300, bbox_inches='tight')
plt.show()

```



```

[27]: #-----
# Média de Engajamento por Tamanho dos Tweets (sem retweets)
#-----

# Garantir que df_original seja uma cópia independente
df_original = df_sem_retweets.copy()

# Criação da coluna de categoria de comprimento
df_original['length_category'] = pd.cut(
    df_original['tweet_length'],
    bins=[0, 50, 140, 280],
    right=True,
    labels=['Pequeno (<50)', 'Médio (51-140)', 'Longo (141-280)']
)

```

```

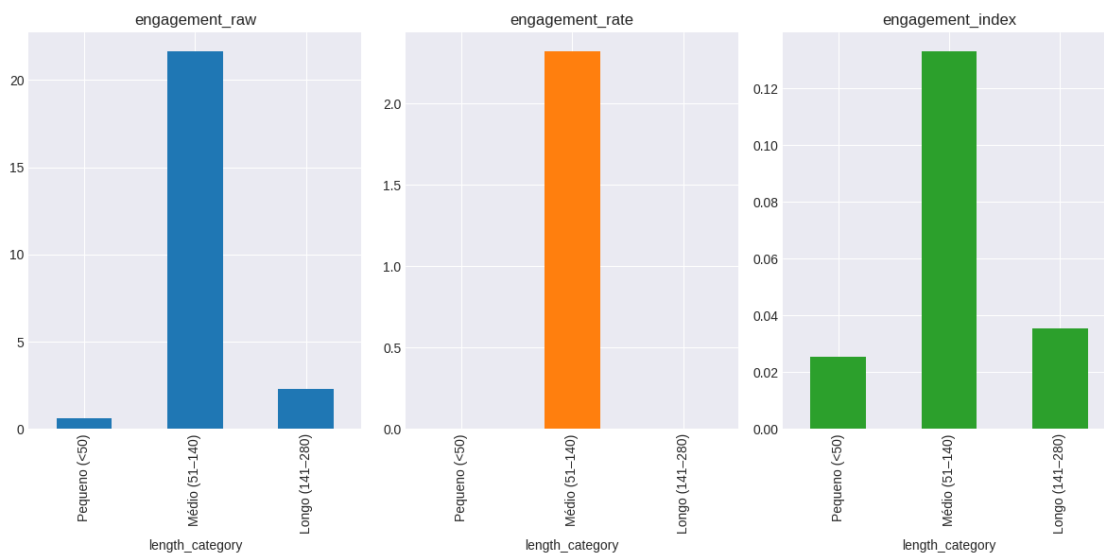
# Agrupamento por categoria de comprimento com média dos indicadores de
↳ engajamento
df_fase = df_original.groupby(
    'length_category',
    observed=False
)[['engagement_raw', 'engagement_rate', 'engagement_index']].mean().loc[
    ['Pequeno (<50)', 'Médio (51-140)', 'Longo (141-280)']
]

# Plotagem dos dados
df_fase.plot(
    kind='bar',
    figsize=(12, 6),
    subplots=True,
    layout=(1, 3),
    sharey=False,
    legend=False
)

plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/media_engajamento_por_tamanho.
↳ png', dpi=300, bbox_inches='tight')
plt.show()

```



5.1.2.2 Amplificação (Retweets)

```
[28]: #-----
# Proporção de retweets possivelmente alarmistas
#-----

# Dados
alarmistas = df_retweets['is_upper'].value_counts(normalize=True)
alarmistas.index = ['Normal', 'Alarmista']

colors = ['#1f77b4', '#ff7f0e'] # Azul para Normal, Laranja para Alarmista

# Criar figura
plt.figure(figsize=(8, 8))

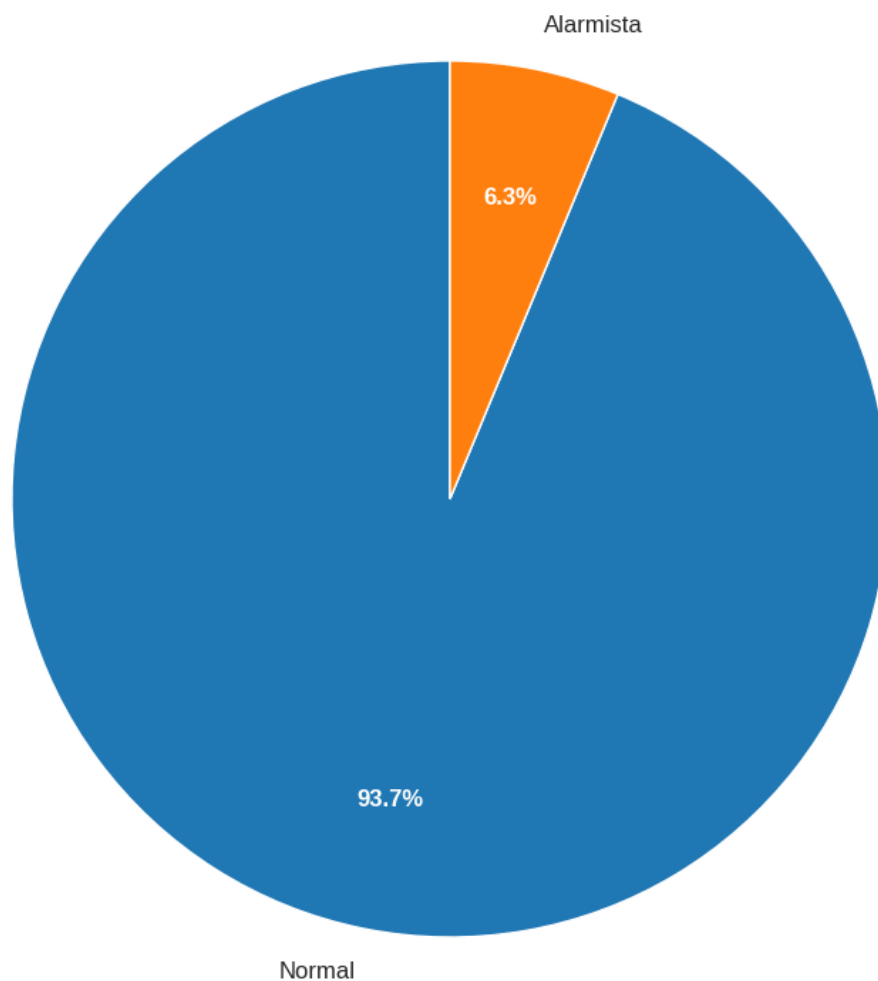
plt.pie(
    alarmistas,
    labels=alarmistas.index,
    colors=colors,
    autopct='%1.1f%%',
    startangle=90,
    wedgeprops={'linewidth': 1, 'edgecolor': 'white'},
    textprops={'fontsize': 12},
    pctdistance=0.7
)

# Destacar porcentagens
for text in plt.gca().texts:
    if '%' in text.get_text():
        text.set_color('white')
        text.set_fontweight('bold')

# Remover rótulo do eixo Y
plt.ylabel('')

# Ajustar layout
plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/retweets_alarmistas.png',
            dpi=300, bbox_inches='tight')
plt.show()
```



```
[29]: #-----
# Contagem de Retweets por Fase Eleitoral
#-----

plt.figure(figsize=(12,5))
phase_counts = df_retweets['electoral_phase'].value_counts()
phase_counts.sort_values().plot(kind='barh',
    ↳color=['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd'])

# Adicionar valores nas barras
for i, v in enumerate(phase_counts.sort_values()):
    plt.text(v, i, f" {v:,} ({v/sum(phase_counts):.1%})",
```

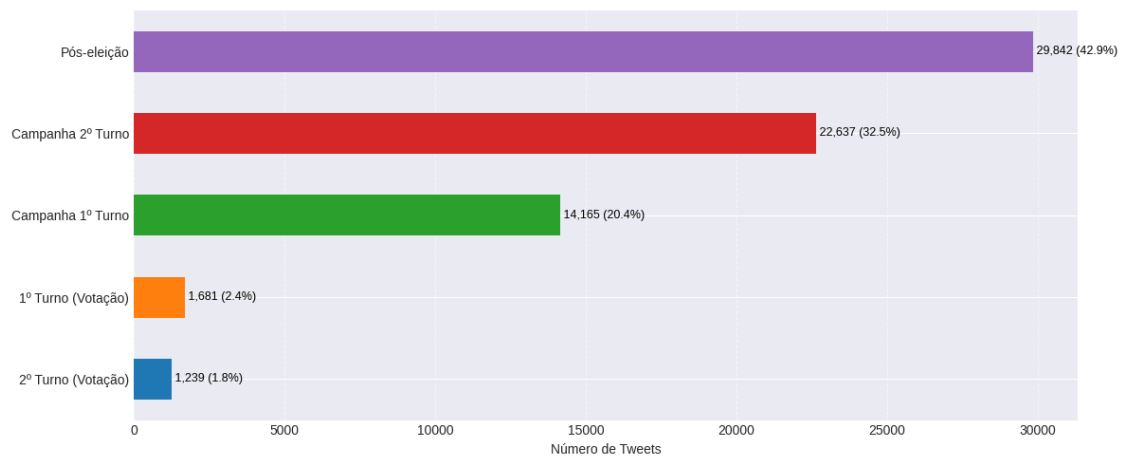
```

        color='black', va='center', fontsize=9)

plt.xlabel('Número de Tweets')
plt.ylabel('')
plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳contagem_retweets_por_fase_eleitoral.png', dpi=300, bbox_inches='tight')
plt.show()

```



```

[30]: # Comprimento máx e min dos retweets
min_len = df_retweets['tweet_length'].min()
max_len = df_retweets['tweet_length'].max()
print(f"Tamanho mínimo: {min_len}")
print(f"Tamanho máximo: {max_len}")

```

Tamanho mínimo: 6
Tamanho máximo: 184

```

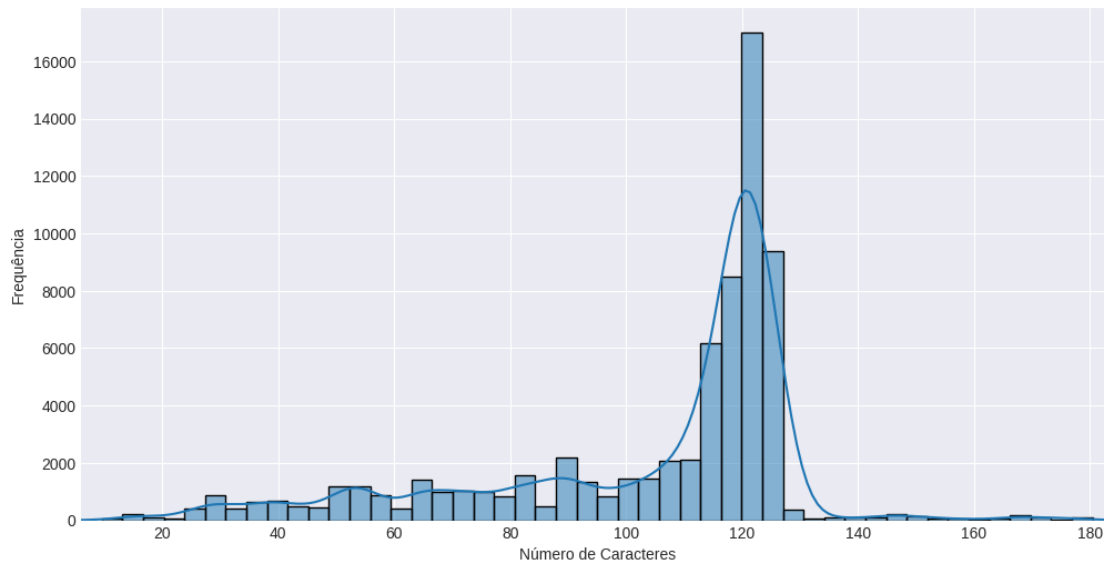
[31]: #-----
# Distribuição do comprimento dos retweets
#-----

plt.figure(figsize=(12, 6))
sns.histplot(df_retweets['tweet_length'], bins=50, kde=True,
↳kde_kws={'bw_adjust': 1.2})
plt.xlabel('Número de Caracteres')
plt.ylabel('Frequência')
plt.xlim(min_len, max_len)

```

```
plt.grid(True)

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳distribuicao_comprimento_retweet.png', dpi=300, bbox_inches='tight')
plt.show()
```



1.2.3 5.1.3 Análise dos Usuários

Análise do Status de Verificação das Contas:

Para analisar os usuários, o dataset foi filtrado para manter apenas a última aparição de cada um, garantindo que suas características mais recentes sejam consideradas dentro da janela de análise. Esse procedimento também evita inflacionar as métricas, já que usuários muito ativos poderiam distorcer os resultados.

```
[32]: df_users['user_id'].nunique()
```

```
[32]: 84473
```

```
[33]: df_users['user_id'].value_counts().head(10)
```

```
[33]: user_id
@USER_1E85F6EB7FFB    1
@USER_EDABE8C22D26    1
@USER_D7E64A0A4978    1
@USER_F31E8D4461E0    1
@USER_8BB74A523D34    1
@USER_F83A7D98B205    1
```

```
@USER_E64C1091441C    1
@USER_108865261CB0    1
@USER_8B30B977D1ED    1
@USER_9DD550E7B748    1
Name: count, dtype: int64
```

```
[34]: # Verificação: conferir se realmente ficou só o mais recente
USER = '@USER_E72B504BC64C'
```

```
[35]: df_users[df_users['user_id'] == USER][['preprocessed_tweet', 'tweet_created_at',
      ↪ 'collection_date']]
```

```
[35]:                                     preprocessed_tweet    tweet_created_at \
7481  Vamos desmoralizar os instituto de pesquisa, q... 2022-09-20 06:44:28

                                     collection_date
7481 2022-09-20 12:22:49
```

```
[36]: #-----
# Proporção de Usuários Verificados
#-----

# Dados
verificados = df_users['is_verified'].value_counts(normalize=True)
verificados.index = ['Não Verificado', 'Verificado']

plt.figure(figsize=(10, 8))
wedges, texts, autotexts = plt.pie(
    verificados,
    labels=verificados.index,
    autopct='%1.1f%%',
    startangle=90,
    wedgeprops={'linewidth': 1, 'edgecolor': 'white'},
    textprops={'fontsize': 12},
    counterclock=False
)

# Apagar texto interno da fatia pequena e usar anotação externa
for i, v in enumerate(verificados):
    if v < 0.01:
        autotexts[i].set_text("") # remove o texto duplicado
        angle = (wedges[i].theta2 - wedges[i].theta1) / 2. + wedges[i].theta1
        x = 0.8 * np.cos(np.deg2rad(angle))
        y = 0.8 * np.sin(np.deg2rad(angle))
        plt.annotate(
            f'{v*100:.1f}%',
            xy=(x, y),
```

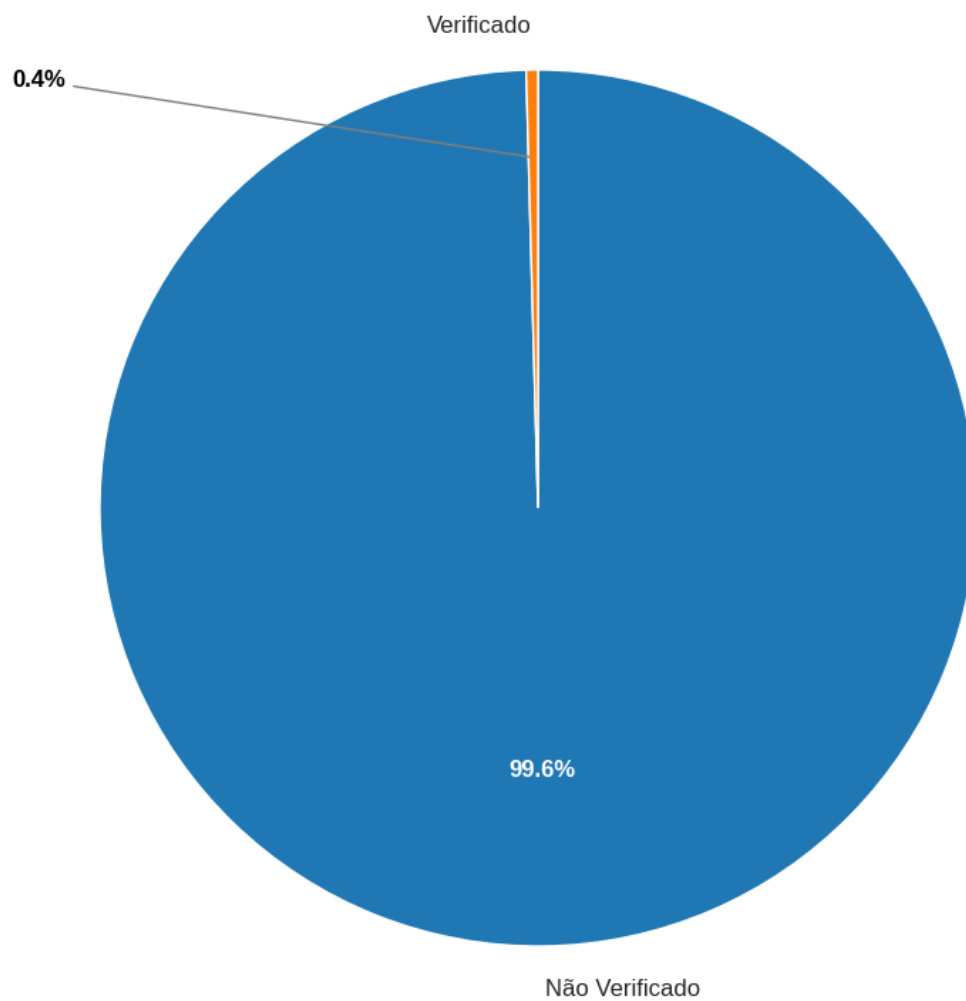
```

        xytext=(1.2*np.sign(x), 1.2*y),
        fontsize=12,
        fontweight='bold',
        color='black',
        arrowprops=dict(arrowstyle="-", color='gray', lw=1)
    )
else:
    autotexts[i].set_color('white')
    autotexts[i].set_fontweight('bold')

plt.ylabel('')
plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳proporcao_de_usuarios_verificados.png', dpi=300, bbox_inches='tight')
plt.show()

```



Análise da Idade das Contas:

```
[37]: # Idade máx e min das contas
df_users['account_age_years'] = df_users['account_age_days'] / 365
min_age = df_users['account_age_years'].min()
max_age = df_users['account_age_years'].max()
print(f"Idade mínimo: {min_age}")
print(f"Idade máximo: {max_age}")
```

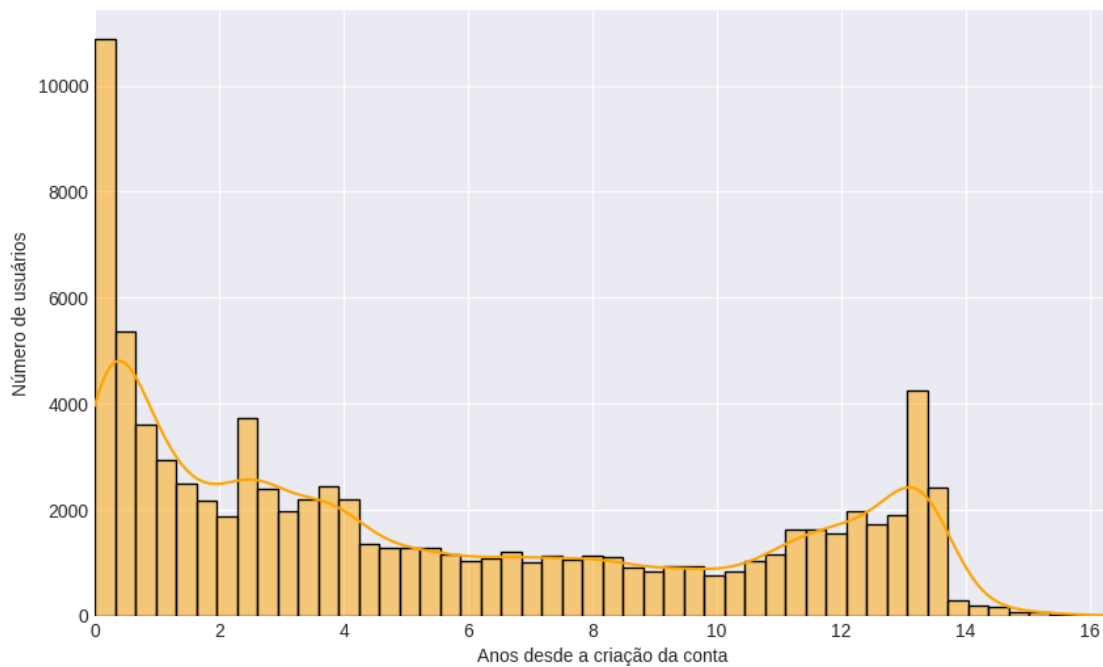
Idade mínimo: 0.0

Idade máximo: 16.317808219178083

```
[38]: #-----
# Distribuição da Idade das Contas (em anos)
#-----

plt.figure(figsize=(10, 6))
sns.histplot(df_users['account_age_years'], bins=50, kde=True, color='orange')
plt.xlabel("Anos desde a criação da conta")
plt.ylabel("Número de usuários")
plt.xlim(min_age, max_age)
plt.grid(True)

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/distribuicao_idade_contas.
↪png', dpi=300, bbox_inches='tight')
plt.show()
```



```
[39]: # =====
# Criar faixas etárias para as contas
# =====
# Definir os limites (em dias)
bins = [0, 30, 90, 180, 365, 730, 1825, float('inf')]
labels = ['< 1 mês', '1-3 meses', '3-6 meses', '6-12 meses',
          '1-2 anos', '2-5 anos', '> 5 anos']

# Aplicar a categorização
```

```

df_users['age_group'] = pd.cut(
    df_users['account_age_days'],
    bins=bins,
    labels=labels,
    right=False # intervalo [início, fim)
)

# Verificar a distribuição geral
print(" Distribuição geral das contas por faixa etária:")
display(df_users['age_group'].value_counts().sort_index())

```

Distribuição geral das contas por faixa etária:

```

age_group
< 1 mês      5193
1-3 meses    4551
3-6 meses    3545
6-12 meses   6739
1-2 anos     7669
2-5 anos    19514
> 5 anos    37262
Name: count, dtype: int64

```

```

[40]: #
      ↪ #-----
# # Média de Engajamento faixa etária das contas
#
      ↪ #-----

# # Agrupamento por faixa etária
# df_fase = df_users.groupby(
#     'idade_faixa',
#     observed=False
# )[['engagement_raw', 'engagement_rate', 'engagement_index']].mean().loc[
#     ['0-1', '1-2', '2-3', '3-5', '5-7', '7-10', '10-16']
# ]

# # Plot
# df_fase.plot(
#     kind='bar',
#     figsize=(12, 6),
#     subplots=True,
#     layout=(1, 3),
#     sharey=False,
#     legend=False
# )

# plt.tight_layout()

```

```
# # Salvar e mostrar
# plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
→media_engajamento_faixa_etaria.png', dpi=300, bbox_inches='tight')
# plt.show()
```

```
[41]: #
→#-----
# # Média de Engajamento por faixa etária das contas (account age)
#
→#-----

# # Definição dos intervalos e rótulos
# bins = [0, 30, 90, 180, 365, 730, 1825, float('inf')]
# labels = ['< 1 mês', '1-3 meses', '3-6 meses', '6-12 meses',
#           '1-2 anos', '2-5 anos', '> 5 anos']

# # Aplicar a categorização
# df_users['age_group'] = pd.cut(
#     df_users['account_age_days'],
#     bins=bins,
#     labels=labels,
#     right=False # intervalo [início, fim)
# )

# # Agrupamento por faixa etária da conta
# df_fase = df_users.groupby(
#     'age_group',
#     observed=False
# )[['engagement_raw', 'engagement_rate', 'engagement_index']].mean().loc[labels]

# # Plot
# df_fase.plot(
#     kind='bar',
#     figsize=(12, 6),
#     subplots=True,
#     layout=(1, 3),
#     sharey=False,
#     legend=False
# )

# plt.tight_layout()

# # Salvar e mostrar
# plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
→media_engajamento_faixa_etaria_conta.png', dpi=300, bbox_inches='tight')
# plt.show()
```

```
[42]: #-----
# Média de Engajamento por faixa etária das contas (account age)
#-----

# Definição dos intervalos e rótulos
bins = [0, 30, 90, 180, 365, 730, 1825, float('inf')]
labels = ['< 1 mês', '1-3 meses', '3-6 meses', '6-12 meses',
          '1-2 anos', '2-5 anos', '> 5 anos']

# Aplicar a categorização
df_users['age_group'] = pd.cut(
    df_users['account_age_days'],
    bins=bins,
    labels=labels,
    right=False # intervalo [início, fim)
)

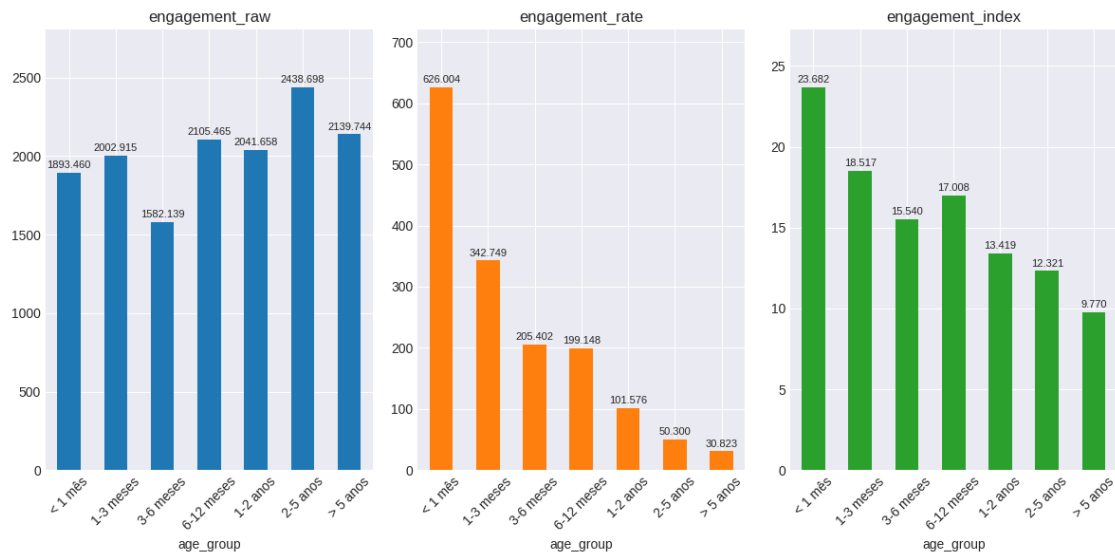
# Agrupamento por faixa etária da conta
df_fase = df_users.groupby(
    'age_group',
    observed=False
)[['engagement_raw', 'engagement_rate', 'engagement_index']].mean().loc[labels]

# Plot
axes = df_fase.plot(
    kind='bar',
    figsize=(12, 6),
    subplots=True,
    layout=(1, 3),
    sharey=False,
    legend=False
)

# Adicionar valores nas barras
for ax in axes.flatten():
    for container in ax.containers:
        ax.bar_label(
            container,
            fmt='%.3f',
            fontsize=8,
            padding=2
        )
    ax.margins(y=0.15) # espaço extra no topo para não cortar os rótulos
    ax.tick_params(axis='x', rotation=45)

plt.tight_layout()
```

```
# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳media_engajamento_faixa_etaria_conta.png', dpi=300, bbox_inches='tight')
plt.show()
```



```
[43]: # Data de criação min
df_sem_retweets['tweet_created_at'].min().strftime('%d/%m/%Y')
```

```
[43]: '30/08/2022'
```

```
[44]: # Data de criação máx
df_com_retweets['tweet_created_at'].max().strftime('%d/%m/%Y')
```

```
[44]: '02/12/2022'
```

```
[45]: #-----
# Média de Idade das Contas ao Longo do Tempo (com Fases Eleitorais)
#-----

# Agrupar por data e calcular média da idade das contas
df_users['tweet_created_at'] = pd.to_datetime(df_users['tweet_created_at'])
df_users['account_age_years'] = df_users['account_age_days'] / 365

media_idade_por_dia = df_users.groupby(df_users['tweet_created_at'].dt.
↳date)['account_age_years'].mean()

# Define os intervalos de campanha (como áreas)
areas = {
```

```

    "Campanha 1º Turno": (df_users['tweet_created_at'].min().
    ↳strftime('%Y-%m-%d'), "2022-10-02"),
    "Campanha 2º Turno": ("2022-10-02", "2022-10-30"),
    "Pós-eleição": ("2022-10-30", df_users['tweet_created_at'].max().
    ↳strftime('%Y-%m-%d'))
}

# Define os dias de votação (como linhas)
eventos = {
    "1º Turno (Votação)": "2022-10-02",
    "2º Turno (Votação)": "2022-10-30"
}

# Converte datas
areas = {fase: (pd.to_datetime(inicio), pd.to_datetime(fim)) for fase, (inicio,
    ↳fim) in areas.items()}
eventos = {nome: pd.to_datetime(data) for nome, data in eventos.items()}

# Cores para áreas
colors = {
    "Campanha 1º Turno": "#f2a640",
    "Campanha 2º Turno": "#cce5ff",
    "Pós-eleição": "#88896e"
}

# Cores para eventos (linhas)
colors_day = {
    "1º Turno (Votação)": "#800080",
    "2º Turno (Votação)": "#ec9d01"
}

# Início do plot
plt.figure(figsize=(14, 6))
media_idade_por_dia.plot(label='Média de idade das contas', color='blue')

# Áreas das campanhas
for fase, (inicio, fim) in areas.items():
    plt.axvspan(inicio, fim, alpha=0.3, label=fase, color=colors[fase])

# Linhas dos dias de votação
for nome, data in eventos.items():
    plt.axvline(data, color=colors_day[nome], linestyle='--', linewidth=1.9,
    ↳label=nome)

# Garante que o eixo x inclua todas as áreas e eventos
datas_inicio_fim = [inicio for inicio, _ in areas.values()] + [fim for _, fim in
    ↳areas.values()]

```

```

datas_eventos = list(eventos.values())
data_min = min(datas_inicio_fim + datas_eventos)
data_max = max(datas_inicio_fim + datas_eventos)

# Garante que o eixo x inclua todas as áreas e eventos
plt.xlim(data_min, data_max)

# Configurar ticks específicos no eixo X
datas_especificas = [
    '30/08/2022', # Data do primeiro dado coletado
    '15/09/2022',
    '02/10/2022', # 1º Turno (Votação)
    '15/10/2022',
    '30/10/2022', # 2º Turno (Votação)
    '15/11/2022',
    '02/12/2022' # Último dia de coleta
]
datas_ticks = [pd.to_datetime(data, format='%d/%m/%Y') for data in
    ↪datas_especificas]
rotulos_ticks = [date.strftime('%d/%m') for date in datas_ticks]

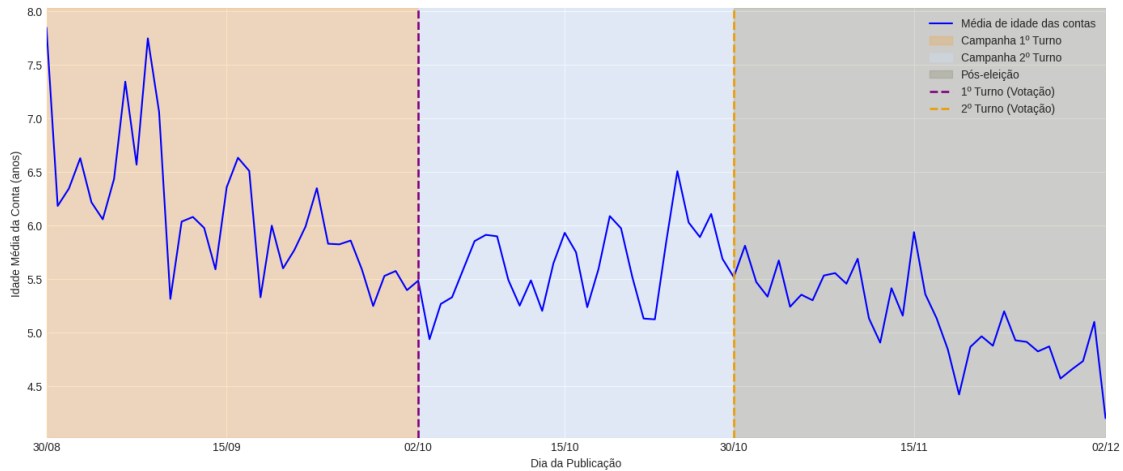
plt.xticks(datas_ticks, rotulos_ticks)

# Formata o eixo X com datas no formato dia/mês
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%d/%m'))

# Personalização
plt.xlabel("Dia da Publicação")
plt.ylabel("Idade Média da Conta (anos)")
plt.legend(loc='upper right')
plt.grid(True)
plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/media_idade_ao_longo_tempo.
    ↪png', dpi=300, bbox_inches='tight')
plt.show()

```



Métricas de Atuação na Rede Por Usuário:

Como as análises a seguir se concentram na atividade dos usuários ao longo do período, o dataset `df_users` não foi usado, garantindo que todas as interações sejam consideradas.

```
[46]: #-----
# Gera perfil de engajamento de usuários com métricas:
# 1. Atividade (tweets, retweets enviados, respostas enviadas, menções feitas)
# 2. Alcance (retweets recebidos, respostas recebidas, menções recebidas)
# 3. Influência (seguidores atualizados)
#-----

# Tratar a coluna mentions
df_com_retweets['mentions'] = df_com_retweets['mentions'].apply(
    lambda x: ast.literal_eval(x) if isinstance(x, str) and x.startswith('[')
    → else x
)

# Calcular métricas individuais:

# Total de tweets por usuário
tweets = df_com_retweets.groupby('user_id')['tweet_id'].count().
    → reset_index(name='Tweets')

# Retweets feitos
retweetou = df_com_retweets[df_com_retweets['is_retweet'] == 1].
    → groupby('user_id').size().reset_index(name='Retweetou')

# Retweets recebidos
```

```

retweetado = df_com_retweets[df_com_retweets['is_retweet'] == 1].
↳groupby('original_author').size().reset_index(name='Retweetado')

# Respostas enviadas
respondeu = df_com_retweets[df_com_retweets['reply_to_user_id'].notnull()].
↳groupby('user_id').size().reset_index(name='Respondeu')

# Respostas recebidas
respondido = df_com_retweets[df_com_retweets['reply_to_user_id'].notnull()].
↳groupby('reply_to_user_id').size().reset_index(name='Respondido')

# Menções feitas
mencionou = df_com_retweets.groupby('user_id')['mentions_count'].sum().
↳reset_index(name='Mencionou')

# Menções recebidas
mentions_exploded = df_com_retweets.explode('mentions')
mentions_exploded = mentions_exploded[mentions_exploded['mentions'].notnull()]
mentions_exploded.loc[:, 'mentions'] = mentions_exploded['mentions'].astype(str).
↳str.strip()
mencionado = mentions_exploded.groupby('mentions').size().
↳reset_index(name='Mencionado')
mencionado = mencionado.rename(columns={'mentions': 'user_id'})

# Unir todas as métricas
dfs = [
    tweets,
    retweetou,
    retweetado.rename(columns={'original_author': 'user_id'}),
    respondeu,
    respondido.rename(columns={'reply_to_user_id': 'user_id'}),
    mencionou,
    mencionado
]

perfil_usuarios = reduce(
    lambda left, right: pd.merge(left, right, on='user_id', how='outer'),
    dfs
).fillna(0)

# Adicionar número de seguidores mais recente
seguidores = df_com_retweets.sort_values('collection_date', ascending=False)
seguidores = seguidores[['user_id', 'followers_count']].
↳drop_duplicates('user_id', keep='last')
perfil_usuarios = pd.merge(perfil_usuarios, seguidores, on='user_id', how='left')

```

```

perfil_usuarios['followers_count'] = perfil_usuarios['followers_count'].
    ↳fillna(0).astype(int)
perfil_usuarios = perfil_usuarios.rename(columns={'followers_count':
    ↳'Seguidores'})

# Adicionar nome de usuário
perfil_usuarios['username'] = perfil_usuarios['user_id']

# Organizar e formatar colunas
perfil_usuarios = perfil_usuarios.rename(columns={'user_id': 'Perfil'})[
    ['Perfil', 'Seguidores', 'Tweets', 'Retweetou', 'Retweetado',
     'Respondeu', 'Respondido', 'Mencionou', 'Mencionado']
]

# Converter colunas numéricas para int
cols_numericas = ['Seguidores', 'Tweets', 'Retweetou', 'Retweetado',
                  'Respondeu', 'Respondido', 'Mencionou', 'Mencionado']
perfil_usuarios[cols_numericas] = perfil_usuarios[cols_numericas].astype(int)

```

```

[47]: #-----
# Ordenar por influência
#-----
perfil_usuarios = perfil_usuarios.sort_values(
    ['Mencionado', 'Retweetado', 'Respondido'],
    ascending=False
).reset_index(drop=True)

perfil_usuarios.head(10)

```

```

[47]:
      Perfil  Seguidores  Tweets  Retweetou  Retweetado  \
0  @EXT_USER_488AA27CB677      0        0         0         0
1      @USER_C34A65EAE2B1  4905621         1         0        40
2      @USER_A87538EEA374  475116        33         3       153
3  @EXT_USER_B615D52D75C2      0        0         0         0
4      @USER_D5970876CB03  4568027         5         1        74
5      @USER_7D99B738F202  1402375         1         0     1689
6  @EXT_USER_562943E1BAEE      0        0         0     1105
7  @EXT_USER_C4768F069EC3      0        0         0         0
8      @USER_772875336776  1453232         1         0        37
9  @EXT_USER_133850CDA778      0        0         0     598

      Respondeu  Respondido  Mencionou  Mencionado
0             0           0           0         1550
1             0         259           0         1479
2             3         436          15         1261
3             0           0           0         1033

```

4	0	260	0	695
5	0	246	0	524
6	0	0	0	511
7	0	0	0	453
8	0	163	0	397
9	0	0	0	349

```
[48]: #-----
# Ordenar por mais retweetados
#-----
perfil_usuarios = perfil_usuarios.sort_values(
    ['Retweetou'],
    ascending=False
).reset_index(drop=True)
perfil_usuarios.head(10)
```

```
[48]:
```

	Perfil	Seguidores	Tweets	Retweetou	Retweetado	Respondeu	\
0	@USER_71040B4ECA6E	1368	28	28	0	0	
1	@USER_670D60090B51	4621	18	18	0	0	
2	@USER_B20D016DE4DC	2008	16	16	0	0	
3	@USER_F307D44A876F	1100	16	15	0	1	
4	@USER_6A212125D8B4	6368	16	15	0	1	
5	@USER_00B4498513EA	1462	15	15	0	0	
6	@USER_FF2CBC90E936	6393	17	15	16	1	
7	@USER_AC8030FB8054	363	15	14	0	0	
8	@USER_EA061A5EFDA2	12307	14	14	0	0	
9	@USER_OE56C7EAD918	545	14	14	0	0	

	Respondido	Mencionou	Mencionado
0	0	7	0
1	0	2	0
2	0	2	0
3	0	7	0
4	0	3	0
5	0	1	0
6	0	6	0
7	0	2	0
8	0	4	0
9	0	1	0

```
[49]: #-----
# Ordenar por mais retweetados
#-----
perfil_usuarios = perfil_usuarios.sort_values(
    ['Retweetado'],
    ascending=False
).reset_index(drop=True)
```

```
perfil_usuarios.head(10)
```

```
[49]:
```

	Perfil	Seguidores	Tweets	Retweetou	Retweetado	\
0	@USER_31902E506AEB	1383249	1	0	3490	
1	@USER_2395F79F21B5	131564	1	0	2162	
2	@EXT_USER_018530429080	0	0	0	1998	
3	@EXT_USER_08A9382CE8CF	0	0	0	1772	
4	@USER_7D99B738F202	1402375	1	0	1689	
5	@USER_F13AB8C04E70	158042	1	0	1502	
6	@USER_052F204E55BE	61875	1	0	1334	
7	@EXT_USER_562943E1BAEE	0	0	0	1105	
8	@USER_B9B4F01D03A1	439757	2	0	1018	
9	@EXT_USER_7498B96FCB21	0	0	0	974	

	Respondeu	Respondido	Mencionou	Mencionado
0	0	123	0	233
1	0	76	0	297
2	0	0	0	102
3	0	0	0	2
4	0	246	0	524
5	0	12	0	28
6	0	43	0	70
7	0	0	0	511
8	0	75	1	112
9	0	0	0	85

```
[50]: #-----
# Ordenar por mais seguidos
#-----
perfil_usuarios = perfil_usuarios.sort_values(
    ['Seguidores'],
    ascending=False
).reset_index(drop=True)
perfil_usuarios.head(10)
```

```
[50]:
```

	Perfil	Seguidores	Tweets	Retweetou	Retweetado	Respondeu	\
0	@USER_47782AFBC34E	14231759	2	0	18	0	
1	@USER_45D2CEC77707	8483697	3	0	72	1	
2	@USER_40D5AFDEBE16	7383179	9	0	54	1	
3	@USER_DD8096CAADF9	7139655	1	0	12	0	
4	@USER_F9CDD6E38E4E	5235486	5	0	20	0	
5	@USER_7AB5375687D1	5054821	1	0	6	0	
6	@USER_C34A65EAE2B1	4905621	1	0	40	0	
7	@USER_D5970876CB03	4568027	5	1	74	0	
8	@USER_2C61C8F4CBD9	3627624	2	1	0	1	
9	@USER_B271BF5CC404	3282058	3	0	39	0	

	Respondido	Mencionou	Mencionado
0	26	0	105
1	177	0	340
2	91	8	174
3	40	0	79
4	26	3	53
5	8	0	20
6	259	0	1479
7	260	0	695
8	11	2	17
9	11	0	17

1.2.4 5.1.4 Análise dos Conteúdos

5.1.4.1 Produção (Tweets e Respostas) Análise dos Emojis Usados:

```
[51]: #-----
# Emojis mais usados
#-----

# Contar todos os emojis em uma única lista
df_sem_retweets['emojis'] = df_sem_retweets['emojis'].apply(lambda x: ast.
↳literal_eval(x) if isinstance(x, str) else x)
all_emojis = [emoji for sublist in df_sem_retweets['emojis'] for emoji in
↳sublist]

# Contar frequência
emoji_counts = Counter(all_emojis)

# Transformar o Counter em DataFrame
emoji_df = pd.DataFrame(emoji_counts.items(), columns=['emoji', 'frequencia'])

# Ordenar por frequência decrescente
emoji_df = emoji_df.sort_values(by='frequencia', ascending=False).
↳reset_index(drop=True)
emoji_df.head(30)
```

```
[51]:      emoji  frequencia
0          1652
1          1459
2          1135
3           589
4           393
5           376
6      22      224
7      13      213
8          176
```

9	175
10	169
11	168
12	162
13	153
14	153
15	134
16	115
17	114
18	112
19	108
20	104
21	104
22	96
23	91
24	85
25	81
26	76
27	75
28	71
29	70

Análise das Hashtags Mais Compartilhadas:

```
[52]: #-----
# Hashtags mais usadas
#-----

# Normalizar: transformar todas as hashtags para minúsculas
df_sem_retweets['hashtags'] = df_sem_retweets['hashtags'].apply(lambda x: ast.
    ↳ literal_eval(x) if isinstance(x, str) else x)

df_sem_retweets['hashtags_norm'] = df_sem_retweets['hashtags'].apply(lambda tags:
    ↳ [tag.lower() for tag in tags])

# Contar hashtags normalizadas
todas_hashtags = [tag for lista in df_sem_retweets['hashtags_norm'] for tag in
    ↳ lista]
contagem = Counter(todas_hashtags)

# Converter para DataFrame e ordenar
resultado = pd.DataFrame(contagem.items(), columns=['hashtag', 'frequencia'])
resultado = resultado.sort_values(by='frequencia', ascending=False).
    ↳ reset_index(drop=True)

resultado.head(30)
```

```
[52]:
```

	hashtag	frequencia
0	#eleições2022	113
1	#eleicoes2022	107
2	#lulapresidente13	96
3	#lulano1ºturno	89
4	#lula	76
5	#brasil	61
6	#lulapresidente13	61
7	#forabolsonaro	61
8	#lulano1oturno	58
9	#bolsonaronoprimeiroturno22	51
10	#bolsonaroreeleitoem2022	50
11	#lulanoprimeiroturno	46
12	#urna	43
13	#lula13	39
14	#vivaosus	37
15	#lulapresidente	32
16	#presidente	31
17	#lulapresidente13	30
18	#lovetwitter	29
19	#brazilwasstolen	26
20	#bolsonaroreeleito22	26
21	#sdv	25
22	#bolsonaronoprimeiroturno	25
23	#bolsonaropresidente22	22
24	#bolsonaro22	22
25	#fraude	22
26	#eleições	22
27	#lulapresidente2022	19
28	#superlivebolsonaro22	19
29	#ptnuncamais	18

```
[53]: #-----
# Hashtag Mais Usada por Dia (iterativo)
#-----

# Explodir a coluna de hashtags
df_exploded = df_sem_retweets.explode('hashtags_norm')
df_exploded['hashtags_norm'] = df_exploded['hashtags_norm'].str.lower().str.
    ↳strip()
df_exploded = df_exploded[df_exploded['hashtags_norm'] != '']

# Adicionar coluna de data
df_exploded['date'] = df_exploded['tweet_created_at'].dt.date

# Contar hashtags por dia
```

```

hashtags_daily = df_exploded.groupby(['date', 'hashtags_norm']).size().
↳reset_index(name='count')

# Selecionar hashtag mais usada por dia
top_daily = hashtags_daily.sort_values(['date', 'count'], ascending=[True,
↳False])
top_daily = top_daily.groupby('date').first().reset_index()

# Criar intervalo completo de datas
full_range = pd.date_range(df_exploded['date'].min(), df_exploded['date'].max())
full_df = pd.DataFrame({'date': full_range})

# Garantir que ambas as colunas estejam no formato datetime64[ns]
full_df['date'] = pd.to_datetime(full_df['date'])
top_daily['date'] = pd.to_datetime(top_daily['date'])

# Merge com todas as datas
top_daily_full = full_df.merge(top_daily, on='date', how='left')

# Preencher ausentes
top_daily_full['count'] = top_daily_full['count'].fillna(0)
top_daily_full['hashtags_norm'] = top_daily_full['hashtags_norm'].fillna('sem
↳dados')

# Criar gráfico
fig = px.line(
    top_daily_full,
    x='date',
    y='count',
    hover_name='hashtags_norm',
    title='Hashtag Mais Usada por Dia',
    labels={'date': 'Data', 'count': 'Quantidade de Tweets'},
    markers=True
)

# Adicionar anotações para picos significativos
for index, row in top_daily_full.iterrows():
    if row['count'] > 29 and row['hashtags_norm'] != 'sem dados':
        fig.add_annotation(
            x=row['date'],
            y=row['count'],
            text=row['hashtags_norm'],
            showarrow=False,
            yanchor='bottom',
            xanchor='left',
            textangle=-45,
            font=dict(size=12)

```

```

    )

# Atualizar layout com ajuste de eixo X
fig.update_layout(
    xaxis_title='Dia da Publicação',
    yaxis_title='Contagem da Hashtag Mais Popular',
    hovermode='x unified',
    template='simple_white',
    margin=dict(r=120),
    xaxis=dict(
        range=[
            top_daily_full['date'].min(),
            top_daily_full['date'].max()
        ]
    )
)

fig.show()

```

```

[54]: #-----
# Volume diário das hashtags mais usadas:
# - Destaca picos (top 15%) e dias de votação (02/10 e 30/10)
#-----

# Preparar dados das hashtags
df_exploded = df_sem_retweets.explode('hashtags_norm')
df_exploded['hashtags_norm'] = df_exploded['hashtags_norm'].str.lower().str.
    ↳strip()
df_exploded = df_exploded[df_exploded['hashtags_norm'] != '']

df_exploded['date'] = df_exploded['tweet_created_at'].dt.date
hashtags_daily = df_exploded.groupby(['date', 'hashtags_norm']).size().
    ↳reset_index(name='count')

top_daily = hashtags_daily.sort_values(['date', 'count'], ascending=[True,
    ↳False])
top_daily = top_daily.groupby('date').first().reset_index()
top_daily['date'] = pd.to_datetime(top_daily['date'])

# Identificação dos picos
limiar = top_daily['count'].quantile(0.85)
picos = top_daily[top_daily['count'] > limiar].copy()

# Garantir destaque dos dias de votação
datas_votacao = [pd.to_datetime('2022-10-02'), pd.to_datetime('2022-10-30')]

```

```

votacao_rows = top_daily[top_daily['date'].isin(datas_votacao)].copy()
votacao_rows['tipo'] = 'votacao'

# Marca os picos
picos['tipo'] = 'pico'

# Combinar picos e dias de votação, sem duplicar
destaques = pd.concat([picos, votacao_rows]).drop_duplicates(subset='date')

# Adicionar cor com base no tipo
destaques['cor'] = destaques['tipo'].map({
    'pico': 'red',
    'votacao': 'green'
})

# Plotagem
plt.figure(figsize=(14, 6))
ax = plt.gca()

# Linha principal
ax.plot(
    top_daily['date'],
    top_daily['count'],
    color='#1f77b4',
    linestyle='-',
    linewidth=2,
    marker='o',
    markersize=4,
    markeredgcolor='#1f77b4',
    label='Hashtag mais usada por dia'
)

# Destaques (picos e dias de votação)
for idx, row in destaques.iterrows():
    ax.scatter(
        row['date'],
        row['count'],
        color=row['cor'],
        s=25,
        zorder=3,
        edgecolor='black'
    )

# Legenda
legend_elements = [
    Line2D([0], [0], marker='o', color='w',
           label=f'Picos (Top 15% > {round(limiar)} hashtags)',

```

```

        markerfacecolor='red', markersize=8, markeredgecolor='black'),
        Line2D([0], [0], marker='o', color='w',
        label='Dias de votação',
        markerfacecolor='green', markersize=8, markeredgecolor='black')
]
ax.legend(handles=legend_elements, loc='upper left')

# Fixo X estratégico
datas_principais = pd.date_range(
    start=top_daily['date'].min(),
    end=top_daily['date'].max(),
    periods=2
)

datas_xticks = sorted(set(datas_principais.union(destaques['date'])))
ax.set_xticks(datas_xticks)
ax.xaxis.set_major_formatter(mdates.DateFormatter('%d/%m'))

for label in ax.get_xticklabels():
    label.set_color('black')
    label.set_fontweight('bold')
    label.set_fontsize(7)

plt.xticks(rotation=90, ha='center')

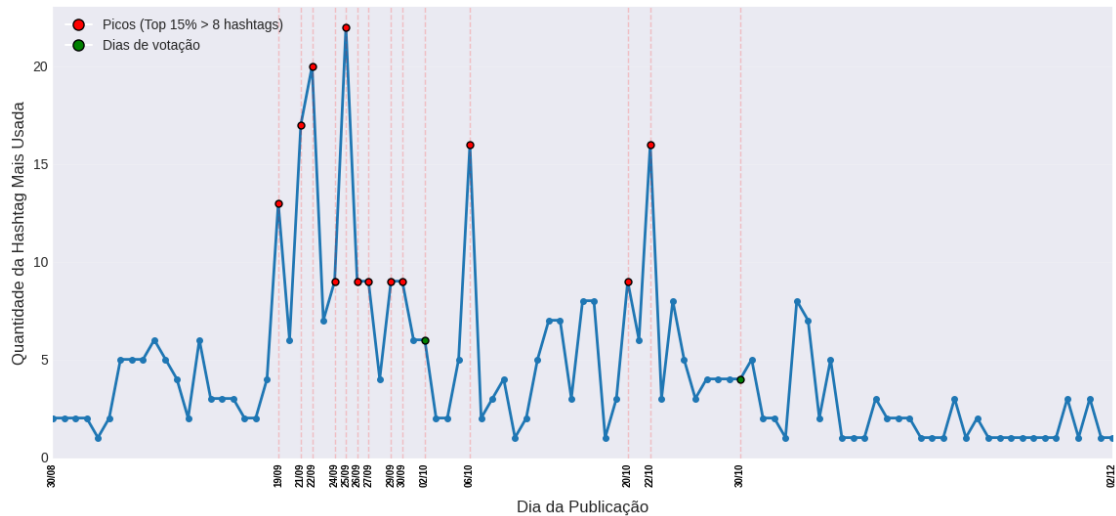
# Linhas verticais nos destaques
for date in destaques['date']:
    ax.axvline(x=date, color='red', alpha=0.2, linestyle='--', linewidth=1)

# Ajustes finais
ax.set_xlim([
    top_daily['date'].min(),
    top_daily['date'].max()
])

ax.set_xlabel('Dia da Publicação', fontsize=12, labelpad=10)
ax.set_ylabel('Quantidade da Hashtag Mais Usada', fontsize=12)
ax.grid(True, alpha=0.2)

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳hashtag_mais_usada_com_votacao_e_picos.png', dpi=300, bbox_inches='tight')
plt.show()

```



```
[55]: #-----
# Tabela: Hashtag Mais Usada por Dia
#-----

# Processamento inicial
df_exploded = df_sem_retweets.explode('hashtags')
df_exploded['hashtags'] = df_exploded['hashtags'].str.lower().str.strip()
df_exploded = df_exploded[df_exploded['hashtags'] != '']

# Converter data e criar coluna de data simplificada
df_exploded['date'] = pd.to_datetime(df_exploded['tweet_created_at']).dt.date

# Agrupar por hashtag e data para encontrar o dia de pico
daily_metrics = df_exploded.groupby(['hashtags', 'date']).agg(
    tweet_count=('tweet_id', 'count')
).reset_index()

# Encontrar o dia com maior uso para cada hashtag
peak_days = daily_metrics.loc[daily_metrics.groupby('hashtags')['tweet_count'].
    ↪idxmax()].reset_index(drop=True)

# Filtrar hashtags com pelo menos 30 tweets no total
hashtag_total_tweets = df_exploded.groupby('hashtags')['tweet_id'].count()
hashtags_validas = hashtag_total_tweets[hashtag_total_tweets >= 30].index
peak_days = peak_days[peak_days['hashtags'].isin(hashtags_validas)]

# Ordenar e formatar a tabela final
resultado = peak_days.sort_values(by='tweet_count', ascending=False)
resultado = resultado.rename(columns={
```

```

        'date': 'dia_do_pico',
        'tweet_count': 'tweets_no_pico'
    })

# Formatar data como dd/mm/yyyy
resultado['dia_do_pico'] = pd.to_datetime(resultado['dia_do_pico']).dt.
    ↳strftime('%d/%m/%Y')

# Mostrar top 30
resultado.head(30)

```

```

[55]:
           hashtags dia_do_pico  tweets_no_pico
838      #lulapresidente13  25/09/2022           22
819      #lulano1oturno    22/09/2022           20
170  #bolsonaronoprimeiroturno22  21/09/2022           17
770      #lula    06/10/2022           16
842      #lulapresidente13  06/10/2022           15
821      #lulano1ºturno    25/09/2022           14
480      #eleições2022    24/09/2022            9
187  #bolsonaroreeleitoem2022  21/09/2022            7
463      #eleicoes2022    02/10/2022            6
1302      #urna    12/09/2022            6
1347      #vivaosus    19/09/2022            6
827      #lulanoprimeiroturno  19/09/2022            6
212      #brasil    19/09/2022            5
840      #lulapresidente13  17/10/2022            5
1069      #presidente    06/10/2022            5
583      #forabolsonaro    20/09/2022            4
772      #lula13    19/09/2022            3
837      #lulapresidente    19/09/2022            3

```

```

[56]: #-----
# Tabela: Hashtag Mais Usada por Período
#-----

# Processamento inicial
df_exploded = df_sem_retweets.explode('hashtags')
df_exploded['hashtags'] = df_exploded['hashtags'].str.lower().str.strip()
df_exploded = df_exploded[df_exploded['hashtags'] != '']

# Converter data e criar coluna de data simplificada
df_exploded['date'] = pd.to_datetime(df_exploded['tweet_created_at']).dt.date

# Agrupar por hashtag para calcular intervalo e métricas consolidadas
hashtag_metrics = df_exploded.groupby('hashtags').agg(
    data_inicio=('date', 'min'),
    data_fim=('date', 'max'),

```

```

total_tweets=('tweet_id', 'count')

).reset_index()

# Filtrar hashtags com pelo menos 30 tweets
hashtag_metrics = hashtag_metrics[hashtag_metrics['total_tweets'] >= 30]

# Formatar datas e ordenar
hashtag_metrics['periodo_circulacao'] = hashtag_metrics.apply(
    lambda x: f"{x['data_inicio'].strftime('%d/%m/%Y')} a {x['data_fim']}.
    ↳strftime('%d/%m/%Y')}",
    axis=1
)

resultado = hashtag_metrics.sort_values('total_tweets', ascending=False).round(2)
resultado = resultado[['hashtags', 'periodo_circulacao', 'total_tweets']]

resultado.head(30)

```

```

[56]:
      hashtags      periodo_circulacao  total_tweets
480  #eleições2022  31/08/2022 a 30/10/2022         113
463  #eleicoes2022  31/08/2022 a 28/11/2022         107
838  #lulapresidente13  30/08/2022 a 26/10/2022          96
821  #lulano1ºturno    02/09/2022 a 02/10/2022          89
770  #lula            01/09/2022 a 02/12/2022          76
583  #forabolsonaro  02/09/2022 a 23/11/2022          61
842  #lulapresidente13  06/10/2022 a 09/11/2022          61
212  #brasil         08/09/2022 a 28/11/2022          61
819  #lulano1oturno    08/09/2022 a 01/10/2022          58
170  #bolsonaronoprimeiroturno22  16/09/2022 a 01/10/2022          51
187  #bolsonaroreeleitoem2022    03/09/2022 a 26/10/2022          50
827  #lulanoprimeiroturno    03/09/2022 a 02/10/2022          46
1302  #urna          08/09/2022 a 02/12/2022          43
772  #lula13         05/09/2022 a 30/10/2022          39
1347  #vivaosus       19/09/2022 a 28/10/2022          37
837  #lulapresidente  15/09/2022 a 09/11/2022          32
1069  #presidente    08/09/2022 a 29/10/2022          31
840  #lulapresidente13  12/10/2022 a 01/11/2022          30

```

Análise de Termos Mais Frequentes:

```

[57]: # Baixa o pacote de stopwords
      nltk.download('stopwords')

```

```

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.

```

```

[57]: True

```

```
[58]: #-----
# Nuvem de palavras
#-----

# Referência: https://medium.com/@m3redithw/wordclouds-with-python-c287887acc8b

# Carregar a máscara
mask = np.array(Image.open("/content/drive/MyDrive/TCC/Estaticos/util/nuvem_mask.
→png"))

stopwords = set(STOPWORDS)
stopwords_pt = set(nltk_stopwords.words('portuguese'))
custom_stopwords = {"pq", "sim", "então", "aí",
                    "tá", "tipo", "rt", "ser", "haver", "pra"}

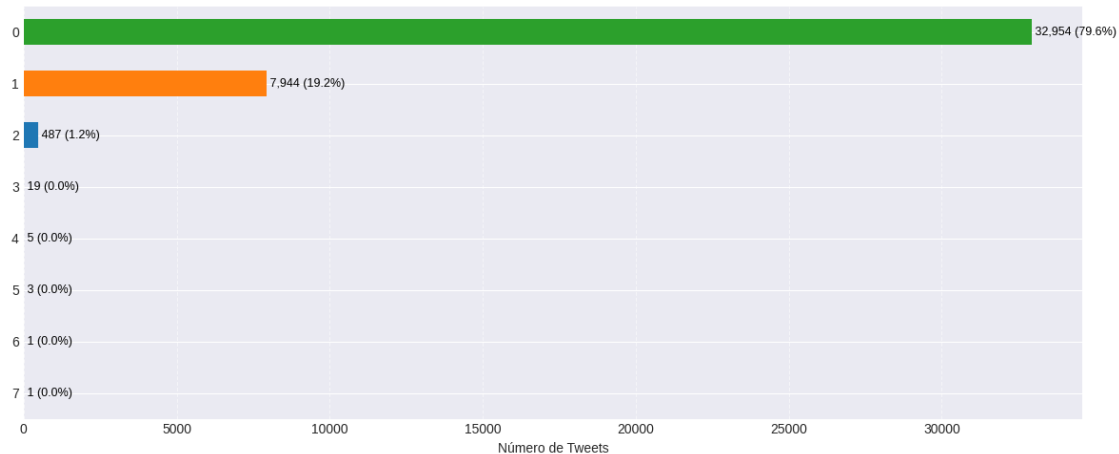
stopwords = stopwords.union(stopwords_pt).union(custom_stopwords)

text = " ".join(df_sem_retweets["preprocessed_tweet"].fillna('')).lower()
tokens = text.split()
filtered_tokens = [w for w in tokens if len(w) > 2 and w not in stopwords]
text_filtered = " ".join(filtered_tokens)

wordcloud = WordCloud(
    width=800,
    height=400,
    mask=mask,
    font_path="/content/drive/MyDrive/TCC/Estaticos/util/Roboto-Black.ttf",
    background_color="white",
    colormap="viridis",
    max_words=100
).generate(text_filtered)

plt.figure(figsize=(12,6))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/nuvem_de_palavras.png',
→dpi=300, bbox_inches='tight')
plt.show()
```

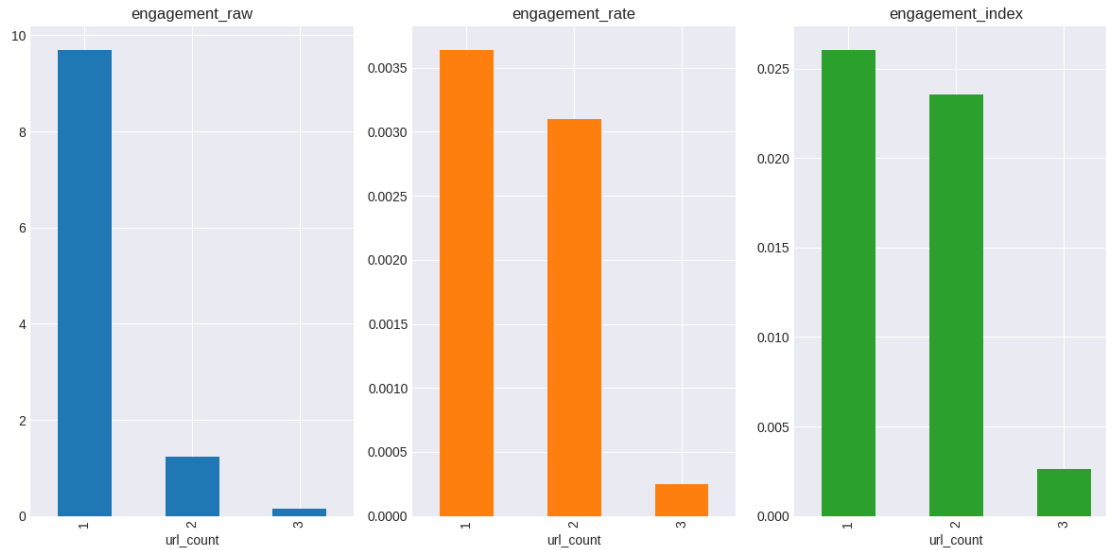



```
[60]: #-----
# Média de engajamento por quantidade de URLs compartilhadas (sem retweets)
# Tweets com mais de três URLs são pouco representativos e foram desconsiderados
#-----

df_fase = df_sem_retweets.groupby('url_count')[
    ['engagement_raw', 'engagement_rate', 'engagement_index']].mean().loc[
    [1, 2, 3] # 3 URLs
]

# Plot
df_fase.plot(kind='bar', figsize=(12, 6), subplots=True,
              layout=(1, 3), sharey=False, legend=False)
plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/media_engajamento_quant_urls.
            ↪png', dpi=300, bbox_inches='tight')
plt.show()
```



```
[61]: #-----
# Contagem das URLs mais frequentes
#-----

# Garantir que os valores nulos sejam tratados como lista vazia para literal_eval
df_sem_retweets['urls'] = df_sem_retweets['urls'].fillna('[]').apply(ast.
    ↳literal_eval)

# Explodir as URLs
df_urls = df_sem_retweets.explode('urls')

# Contar URLs mais comuns
df_urls['urls'].value_counts().head(30)
```

```
[61]: urls
https://www.tse.jus.br/comunicacao/noticias/2022/Agosto/forcas-armadas-
inspecionam-codigos-fonte-da-urna-161670
51
https://voltacomseuex.com/
42
https://twitter.com/lavern_spicer/status/1586851789751652353
37
https://agenciabrasil.ebc.com.br/politica/noticia/2022-08/forcas-armadas-
inspecionam-codigo-fonte-de-urna-eletronica-no-tse
35
https://www.tse.jus.br/eleicoes/urna-eletronica/seguranca-da-urna/apresentacao-
dos-codigos-fonte
30
```

<https://twitter.com/Lunasia64DS/status/1569512538433536001>
 21
<https://noticias.uol.com.br/eleicoes/2022/09/24/desespero-medo-de-sair-na-rua-diz-jovem-agredida-por-bolsonarista-no-rj.htm>
 20
<https://www.youtube.com/watch?v=txRYQPP4byc&feature=youtu.be>
 19
<https://twitter.com/rauquetxt/status/1585380666229002240>
 15
<https://www.youtube.com/watch?v=4wrMLzqgKEI&feature=youtu.be>
 14
<https://twitter.com/UOLNoticias/status/1583715716099817473?t=kt6jQcnehBKt7PbQj-UEw&s=19>
 14
<https://twitter.com/Lunasia64DS/status/1569513720258215937>
 13
<https://www.brasil247.com/regionais/sudeste/lula-forcas-armadas-tem-missao-muito-mais-nobre-que-e-cuidar-da-soberania-brasileira-e-nao-ser-fiscal-de-urna-eletronica>
 13
<https://noticias.uol.com.br/ultimas-noticias/agencia-estado/2022/10/07/carmen-lucia-manda-bolsonaro-remover-video-sobre-prova-de-vida-do-inss-na-urna.htm>
 12
<https://agenciabrasil.ebc.com.br/politica/noticia/2022-08/forças-armadas-inspecionam-código-fonte-de-urna-eletronica-no-tse?amp>
 12
<https://rumble.com/v1tvw3y-relatorio-das-foras-armadas-urna-eletrnica.html>
 12
<https://www.youtube.com/watch?v=8fss2sbeiSg&feature=youtu.be>
 12
<https://www.youtube.com/watch?v=l4BmQ-hylTk&feature=youtu.be>
 11
<https://www.correiobraziliense.com.br/politica/2022/09/5033948-e-impossivel-fraude-com-urna-eletronica-diz-carlos-ayres-britto.html>
 11
https://www.em.com.br/app/noticia/politica/2022/09/12/interna_politica,1393150/padre-ironiza-seguranca-de-urna-eletronica-em-missa.shtml
 11
<https://noticias.uol.com.br/eleicoes/2022/10/11/bolsonaro-pelotas-urnas-ataque-suspeita-lula-votacao.htm>
 10
https://twitter.com/DriSouzaaasz/status/1588732188509360128?t=_syB5vQsX8INU1r7bXuj_Q&s=19
 10
<https://baraodeitarare.org.br/2022/08/31/contra-fantasma-da-fraude-e-do-golpismo-barao-promove-lancamento-de-livro-sobre-a-urna-eletronica-brasileira/>
 10
<https://jovempan.com.br/noticias/politica/eleicoes-2022/eleitores-terao-tempo-a->

```

mais-para-conferencia-do-voto-na-urna-eletronica.html
9
https://br.search.yahoo.com/?fr2=p:newsrd,mkt:br
8
https://www.youtube.com/watch?v=yGdf8SLikHE&feature=youtu.be
8
https://www1.folha.uol.com.br/poder/2021/06/so-brasil-bangladesh-e-butao-usam-
urna-eletronica-sem-comprovante-do-voto-impresso.shtml
8
https://www.youtube.com/watch?v=pJUc7lC_tb0&feature=youtu.be
7
https://www1.folha.uol.com.br/mundo/2022/09/direita-vence-na-italia-e-abre-
caminho-para-giorgia-meloni-indica-boca-de-
urna.shtml?utm_source=twitter&utm_medium=social&utm_campaign=comptw      7
https://www.tecmundo.com.br/mercado/254919-nao-houve-fraude-urna-engenheiros-
ita-repudiam-relatorio-do-pl.htm
7
Name: count, dtype: int64

```

```

[62]: #-----
# URLs com MAIORES PICOS DIÁRIOS de compartilhamento
#-----

# Filtrar apenas URLs não vazias
df_urls = df_urls[df_urls['urls'].apply(lambda x: isinstance(x, str) and x.
    ↪strip() != '')].copy()

# Garantir que a data esteja no formato datetime
df_urls['tweet_created_at'] = pd.to_datetime(df_urls['tweet_created_at'],
    ↪errors='coerce').dt.floor('d')

# Contar o número de vezes que cada URL apareceu por dia
url_daily_counts = df_urls.groupby(['urls', 'tweet_created_at']).size().
    ↪reset_index(name='count')

# Para cada URL, pegar a data com maior número de ocorrências
idx = url_daily_counts.groupby('urls')['count'].idxmax()
url_peak_dates = url_daily_counts.loc[idx].sort_values(by='count',
    ↪ascending=False).copy()

# Formatar a data para dd/mm/yyyy
url_peak_dates['tweet_created_at'] = url_peak_dates['tweet_created_at'].dt.
    ↪strftime('%d/%m/%Y')

# Renomear a coluna para "dia_do_pico"
url_peak_dates.rename(columns={'tweet_created_at': 'dia_do_pico'}, inplace=True)

```

```
# Resultado final: URLs mais compartilhadas e sua data de pico
url_peak_dates.head(30)
```

```
[62]:
```

	urls	dia_do_pico	count
7277	https://voltacomseuex.com/	21/10/2022	35
5536	https://twitter.com/lavern_spicer/status/15868...	31/10/2022	31
522	https://noticias.uol.com.br/eleicoes/2022/09/2...	25/09/2022	20
2498	https://twitter.com/Lunasia64DS/status/1569512...	13/09/2022	15
7356	https://www.brasil247.com/regionais/sudeste/lu...	15/09/2022	12
7407	https://www.correiobraziliense.com.br/politica...	02/09/2022	11
1565	https://twitter.com/DriSouzaaasz/status/158873...	05/11/2022	10
745	https://rumble.com/v1tvw3y-relatrio-das-foras-...	12/11/2022	10
527	https://noticias.uol.com.br/eleicoes/2022/10/1...	11/10/2022	9
83	https://agenciabrasil.ebc.com.br/politica/noti...	21/11/2022	9
7840	https://www.tse.jus.br/comunicacao/noticias/20...	10/11/2022	9
6400	https://twitter.com/rauquetxt/status/158538066...	28/10/2022	9
7443	https://www.em.com.br/app/noticia/politica/202...	13/09/2022	9
2500	https://twitter.com/Lunasia64DS/status/1569513...	13/09/2022	7
8194	https://www.youtube.com/watch?v=yGdf8SLikHE&fe...	04/10/2022	7
3499	https://twitter.com/UOLNoticias/status/1583715...	23/10/2022	7
8154	https://www.youtube.com/watch?v=pJUc7lC_tb0&fe...	01/12/2022	7
8169	https://www.youtube.com/watch?v=txRYQPP4byc&fe...	06/10/2022	6
4376	https://twitter.com/cibeIedourado/status/15883...	04/11/2022	6
435	https://jovempan.com.br/noticias/politica/elei...	12/09/2022	6
596	https://oglobo.globo.com/politica/eleicoes-202...	13/09/2022	6
7363	https://www.brasildefato.com.br/2022/09/25/ext...	26/09/2022	5
4862	https://twitter.com/flaviocostaf/status/156725...	06/09/2022	5
266	https://flaviaribeiroy2.jusbrasil.com.br/artigo...	20/09/2022	5
7762	https://www.tecmundo.com.br/mercado/254919-nao...	25/11/2022	5
78	https://agenciabrasil.ebc.com.br/politica/noti...	28/11/2022	5
252	https://extra.globo.com/casos-de-policia/justi...	12/10/2022	5
2714	https://twitter.com/MidiaNINJA/status/15729420...	23/09/2022	5
8193	https://www.youtube.com/watch?v=yBTbvK7Hl0w&fe...	28/09/2022	5
5616	https://twitter.com/lima1956/status/1564980649...	31/08/2022	5

A maior parte das URLs refere-se ao próprio Twitter/X, provavelmente representando retweets, vídeos ou imagens internas da plataforma. A seguir, essas URLs foram filtradas para possibilitar uma análise focada nas fontes externas efetivamente compartilhadas pelos usuários, permitindo uma avaliação mais clara dos domínios externos presentes nos tweets.

```
[63]: #-----
# URLs EXTERNAS mais frequentes (excluindo twitter.com)
#-----

# Filtragem de URLs
# Remover URLs vazias e do Twitter
df_urls = df_urls[
```

```

df_urls['urls'].apply(
    lambda x: isinstance(x, str)
    and x.strip() != ''
    and 'twitter.com' not in x.lower()
)
].copy()

# Contagem simples de URLs
# Contar URLs mais comuns
url_counts = df_urls['urls'].value_counts().head(30)
url_counts

```

```

[63]: urls
https://www.tse.jus.br/comunicacao/noticias/2022/Agosto/forcas-armadas-
inspecionam-codigos-fonte-da-urna-161670
51
https://voltacomseuex.com/
42
https://agenciabrasil.ebc.com.br/politica/noticia/2022-08/forcas-armadas-
inspecionam-codigo-fonte-de-urna-eletronica-no-tse
35
https://www.tse.jus.br/eleicoes/urna-eletronica/seguranca-da-urna/apresentacao-
dos-codigos-fonte
30
https://noticias.uol.com.br/eleicoes/2022/09/24/desespero-medo-de-sair-na-rua-
diz-jovem-agredida-por-bolsonarista-no-rj.htm
20
https://www.youtube.com/watch?v=txRYQPP4byc&feature=youtu.be
19
https://www.youtube.com/watch?v=4wrMLzqgKEI&feature=youtu.be
14
https://www.brasil247.com/regionais/sudeste/lula-forcas-armadas-tem-missao-
muito-mais-nobre-que-e-cuidar-da-soberania-brasileira-e-nao-ser-fiscal-de-urna-
eletronica
13
https://agenciabrasil.ebc.com.br/politica/noticia/2022-08/forcas-armadas-
inspecionam-codigo-fonte-de-urna-eletronica-no-tse?amp
12
https://rumble.com/v1tvw3y-relatrio-das-foras-armadas-urna-eletrnica.html
12
https://noticias.uol.com.br/ultimas-noticias/agencia-estado/2022/10/07/carmen-
lucia-manda-bolsonaro-remover-video-sobre-prova-de-vida-do-inss-na-urna.htm
12
https://www.youtube.com/watch?v=8fss2sbeiSg&feature=youtu.be
12
https://www.em.com.br/app/noticia/politica/2022/09/12/interna_politica,1393150/p
adre-ironiza-seguranca-de-urna-eletronica-em-missa.shtml
11

```

<https://www.youtube.com/watch?v=l4BmQ-hylTk&feature=youtu.be>
 11
<https://www.correiobraziliense.com.br/politica/2022/09/5033948-e-impossivel-fraude-com-urna-eletronica-diz-carlos-ayres-britto.html>
 11
<https://noticias.uol.com.br/eleicoes/2022/10/11/bolsonaro-pelotas-urnas-ataque-suspeita-lula-votacao.htm>
 10
<https://baraodeitarare.org.br/2022/08/31/contra-fantasma-da-fraude-e-do-golpismo-barao-promove-lancamento-de-livro-sobre-a-urna-eletronica-brasileira/>
 10
<https://jovempan.com.br/noticias/politica/eleicoes-2022/eleitores-terao-tempo-a-mais-para-conferencia-do-voto-na-urna-eletronica.html>
 9
<https://www1.folha.uol.com.br/poder/2021/06/so-brasil-bangladesh-e-butao-usam-urna-eletronica-sem-comprovante-do-voto-impresso.shtml>
 8
<https://br.search.yahoo.com/?fr2=p:newsrd,mkt:br>
 8
<https://www.youtube.com/watch?v=yGdf8SLikHE&feature=youtu.be>
 8
<https://www.tecmundo.com.br/mercado/254919-nao-houve-fraude-urna-engenheiros-ita-repudiam-relatorio-do-pl.htm>
 7
https://www1.folha.uol.com.br/mundo/2022/09/direita-vence-na-italia-e-abre-caminho-para-giorgia-meloni-indica-boca-de-urna.shtml?utm_source=twitter&utm_medium=social&utm_campaign=comptw 7
<https://www.brasildefato.com.br/2022/09/25/extrema-direita-vence-eleicoes-na-italia-com-mais-de-41-dos-votos-segundo-boca-de-urna/>
 7
https://www.youtube.com/watch?v=pJUc7lC_tb0&feature=youtu.be
 7
<https://oglobo.globo.com/politica/eleicoes-2022/noticia/2022/09/tse-diz-que-bone-e-elemento-de-identidade-cultural-e-autoriza-uso-em-foto-exibida-na-urna-eletronica.ghtml> 6
<https://www.poder360.com.br/eleicoes/militares-inspecionam-codigo-fonte-da-urna-no-tse/>
 6
<https://www.dw.com/pt-br/cinco-curiosidades-sobre-a-urna-eletr%C3%B4nica/a-63006622>
 6
<https://g1.globo.com/politica/noticia/2021/10/04/urna-eletronica-tse-abertura-codigo-fonte.ghtml>
 6
<https://plenarinho.leg.br/urna-eletronica-do-plenarinho/>
 6
 Name: count, dtype: int64

```
[64]: #-----
# URLs EXTERNAS mais frequentes com MAIORES PICOS DIÁRIOS de compartilhamento
#-----

# Explodir as URLs
df_urls = df_sem_retweets.explode('urls')

# Filtragem de URLs
# Remover URLs vazias e do Twitter
df_urls = df_urls[
    df_urls['urls'].apply(
        lambda x: isinstance(x, str)
        and x.strip() != ''
        and 'twitter.com' not in x.lower()
    )
].copy()

# Contagem de URLs
# Contar URLs mais comuns
url_counts = df_urls['urls'].value_counts().head(30)

# Análise de datas de pico
# Converter para datetime
df_urls['tweet_created_at'] = pd.to_datetime(df_urls['tweet_created_at'],
    →errors='coerce')

# Agrupar por URL e data
url_daily_counts = df_urls.groupby(['urls', pd.Grouper(key='tweet_created_at',
    →freq='D')]).size().reset_index(name='count')

# Encontrar datas de pico
idx = url_daily_counts.groupby('urls')['count'].idxmax()
url_peak_dates = url_daily_counts.loc[idx].sort_values(by='count',
    →ascending=False)

# Formatar datas
url_peak_dates['peak_date'] = url_peak_dates['tweet_created_at'].dt.strftime('%d/
    →%m/%Y')
url_peak_dates = url_peak_dates[['urls', 'peak_date', 'count']].
    →rename(columns={'peak_date': 'dia_do_pico'})

url_peak_dates.head(30)
```

```
[64]:
```

	urls	dia_do_pico	count
850	https://voltacomseuex.com/	21/10/2022	35
522	https://noticias.uol.com.br/eleicoes/2022/09/2...	25/09/2022	20

929	https://www.brasil247.com/regionais/sudeste/lu...	15/09/2022	12
980	https://www.correiobraziliense.com.br/politica...	02/09/2022	11
745	https://rumble.com/v1tvw3y-relatrio-das-foras-...	12/11/2022	10
527	https://noticias.uol.com.br/eleicoes/2022/10/1...	11/10/2022	9
1413	https://www.tse.jus.br/comunicacao/noticias/20...	10/11/2022	9
83	https://agenciabrasil.ebc.com.br/politica/noti...	21/11/2022	9
1016	https://www.em.com.br/app/noticia/politica/202...	13/09/2022	9
1767	https://www.youtube.com/watch?v=yGdf8SLikHE&fe...	04/10/2022	7
1727	https://www.youtube.com/watch?v=pJUc7lC_tb0&fe...	01/12/2022	7
1742	https://www.youtube.com/watch?v=txRYQPP4byc&fe...	06/10/2022	6
435	https://jovempan.com.br/noticias/politica/elei...	12/09/2022	6
596	https://oglobo.globo.com/politica/eleicoes-202...	13/09/2022	6
252	https://extra.globo.com/casos-de-policia/justi...	12/10/2022	5
1766	https://www.youtube.com/watch?v=yBTbvK7Hl0w&fe...	28/09/2022	5
1335	https://www.tecmundo.com.br/mercado/254919-nao...	25/11/2022	5
266	https://flaviaribeiro2.jusbrasil.com.br/artigo...	20/09/2022	5
78	https://agenciabrasil.ebc.com.br/politica/noti...	28/11/2022	5
1837	https://www12.senado.leg.br/noticias/materias/...	20/09/2022	5
1328	https://www.tecmundo.com.br/mercado/246042-con...	04/09/2022	5
936	https://www.brasildefato.com.br/2022/09/25/ext...	26/09/2022	5
1836	https://www12.senado.leg.br/noticias/materias/...	20/09/2022	5
1153	https://www.jornaldacidadeonline.com.br/notici...	30/09/2022	4
493	https://noticias.cancaonova.com/brasil/resumo-...	01/09/2022	4
545	https://noticias.uol.com.br/politica/ultimas-n...	10/11/2022	4
1005	https://www.dw.com/pt-br/cinco-curiosidades-so...	04/09/2022	4
1711	https://www.youtube.com/watch?v=l4BmQ-hylTk&fe...	30/09/2022	4
327	https://g1.globo.com/politica/noticia/2021/10/...	14/11/2022	4
695	https://radiojornal.ne10.uol.com.br/eleicoes/2...	22/09/2022	4

Verificando os domínios mais frequentes:

```
[65]: #-----
# Domínios mais frequentes
#-----

# Explodir as listas de URLs em múltiplas linhas
df_urls = df_sem_retweets.explode('urls')

# Filtrar somente URLs que são strings e não estão vazias ou em branco
df_urls = df_urls[df_urls['urls'].apply(lambda x: isinstance(x, str) and x.
    ↳strip() != '')].copy()

# Extrair o domínio das URLs
df_urls.loc[:, 'domain'] = df_urls['urls'].apply(lambda x: urlparse(x).netloc.
    ↳lower())

# Remover o prefixo 'www.'
```

```
df_urls.loc[:, 'domain'] = df_urls['domain'].str.replace(r'^www\.|', '',
↳ regex=True)

# Filtrar domínios não vazios
df_urls = df_urls[df_urls['domain'].str.strip() != '']

# Contar os domínios mais comuns
df_urls['domain'].value_counts().head(30)
```

```
[65]: domain
twitter.com          6756
youtube.com          291
tse.jus.br           176
noticias.uol.com.br  110
gl.globo.com          85
www1.folha.uol.com.br 69
agenciabrasil.ebc.com.br 57
voltacomseuex.com     42
tecmundo.com.br       40
justicaeleitoral.jus.br 39
brasil247.com         33
resultados.tse.jus.br 32
instagram.com         31
estadao.com.br        28
google.com            24
correiobraziliense.com.br 24
oglobo.globo.com      24
em.com.br             21
www12.senado.leg.br   19
poder360.com.br       19
cnnbrasil.com.br      17
terrabrasilnoticias.com 16
jovempan.com.br       16
rumble.com            16
metropoles.com        13
uol.com.br            13
msn.com               12
brasildefato.com.br   11
terra.com.br          11
mla.bs                10
Name: count, dtype: int64
```

```
[66]: #-----
# Domínios mais frequentes por dia de pico
#-----

# Explodir listas de URLs e criar cópia independente
```

```

df_urls = df_sem_retweets.explode('urls').copy()

# Filtrar URLs válidas (strings não vazias)
df_urls = df_urls[df_urls['urls'].str.strip().ne('') & df_urls['urls'].notna()]

# Extrair domínios de forma vetorizada
from urllib.parse import urlparse

# Função vetorizada para extração de domínio
def extract_domain(url):
    parsed = urlparse(url)
    domain = parsed.netloc.lower()
    return domain.removeprefix('www.') if domain.startswith('www.') else domain

# Aplicar extração de domínio
df_urls.loc[:, 'domain'] = df_urls['urls'].apply(extract_domain)

# Remover domínios vazios
df_urls = df_urls[df_urls['domain'].str.strip().ne('')]

# Converter data para datetime (apenas data, sem hora)
df_urls.loc[:, 'tweet_created_at'] = pd.to_datetime(
    df_urls['tweet_created_at'],
    errors='coerce'
).dt.floor('d') # Mantém apenas parte da data

# Contar ocorrências por domínio e data
domain_daily_counts = (
    df_urls.groupby(['domain', 'tweet_created_at'])
    .size()
    .reset_index(name='count')
)

# Encontrar datas de pico usando transform
peak_dates = (
    domain_daily_counts
    .loc[domain_daily_counts.groupby('domain')['count'].transform('max') == ]
    ↪ domain_daily_counts['count']]
    .copy()
)

# Formatar datas e ordenar resultados
peak_dates['dia_do_pico'] = peak_dates['tweet_created_at'].dt.strftime('%d/%m/
    ↪ %Y')
peak_dates = (
    peak_dates
    .drop(columns='tweet_created_at')

```

```

.sort_values('count', ascending=False)
.drop_duplicates('domain') # Garantir 1 entrada por domínio
.head(30)
)

# Exibir resultado
peak_dates

```

```

[66]:
      domain  count dia_do_pico
1226  twitter.com    186  23/09/2022
1338  voltacomseuex.com    35  21/10/2022
742   noticias.uol.com.br    20  25/09/2022
1420  youtube.com    17  04/10/2022
1169  tse.jus.br    15  10/11/2022
154   brasil247.com    13  15/09/2022
231   correiobraziliense.com.br    11  02/09/2022
998   rumble.com    10  12/11/2022
1382  www12.senado.leg.br    10  20/09/2022
970   resultados.tse.jus.br    10  05/10/2022
31    agenciabrasil.ebc.com.br    10  21/11/2022
315   em.com.br    9  13/09/2022
1047  tecmundo.com.br    8  04/09/2022
1355  www1.folha.uol.com.br    8  26/09/2022
805   oglobo.globo.com    8  13/09/2022
434   g1.globo.com    6  08/11/2022
609   jovempan.com.br    6  12/09/2022
169   brasildefato.com.br    6  26/09/2022
138   boatos.org    5  13/10/2022
599   jornalimpacto.com.br    5  09/09/2022
392   flaviaribeiro2.jusbrasil.com.br    5  20/09/2022
364   extra.globo.com    5  12/10/2022
627   justicaeleitoral.jus.br    5  28/09/2022
305   dw.com    4  04/09/2022
952   radiojornal.ne10.uol.com.br    4  13/09/2022
339   estado.com.br    4  24/09/2022
495   google.com    4  25/11/2022
939   possible-gambler-e78.notion.site    4  29/09/2022
332   esporte paulista.com.br    4  15/09/2022
582   jornaldacidadeonline.com.br    4  30/09/2022

```

5.1.4.2 Amplificação (Retweets) Análise dos Emojis Usados:

```

[67]: #-----
# Emojis mais usados
#-----

# Contar todos os emojis em uma única lista

```

```

df_retweets['emojis'] = df_retweets['emojis'].apply(lambda x: ast.
↳literal_eval(x) if isinstance(x, str) else x)
all_emojis = [emoji for sublist in df_retweets['emojis'] for emoji in sublist]

# Contar frequência
emoji_counts = Counter(all_emojis)

# Transformar o Counter em DataFrame
emoji_df = pd.DataFrame(emoji_counts.items(), columns=['emoji', 'frequencia'])

# Ordenar por frequência decrescente
emoji_df = emoji_df.sort_values(by='frequencia', ascending=False).
↳reset_index(drop=True)
emoji_df.head(30)

```

```

[67]:      emoji  frequencia
0                2529
1    22          1190
2                1072
3                828
4                763
5                670
6                535
7                451
8                450
9                393
10               376
11               304
12               293
13               249
14               244
15    13          231
16                230
17               223
18               205
19               203
20               199
21               198
22               197
23               189
24     2          167
25               166
26               165
27               161
28               150
29               145

```

Análise das Hashtags Mais Compartilhadas:

```
[68]: #-----
# Hashtags mais usadas
#-----

# Normalizar: transformar todas as hashtags para minúsculas
df_retweets['hashtags'] = df_retweets['hashtags'].apply(lambda x: ast.
↳literal_eval(x) if isinstance(x, str) else x)

df_retweets['hashtags_norm'] = df_retweets['hashtags'].apply(lambda tags: [tag.
↳lower() for tag in tags])

# Contar hashtags normalizadas
todas_hashtags = [tag for lista in df_retweets['hashtags_norm'] for tag in lista]
contagem = Counter(todas_hashtags)

# Converter para DataFrame e ordenar
resultado = pd.DataFrame(contagem.items(), columns=['hashtag', 'frequencia'])
resultado = resultado.sort_values(by='frequencia', ascending=False).
↳reset_index(drop=True)

resultado.head(30)
```

```
[68]:
```

	hashtag	frequencia
0	#paz	159
1	#13	150
2	#brasildaesperanca	140
3	#sãopaulopodemais	130
4	#lulapresidente13	123
5	#lulano1ºturno	122
6	#bomdia	117
7	#eleicoes2022	114
8	#brazilwasstolen	88
9	#bolsonaro22	66
10	#mourãosenador	61
11	#vote100	61
12	#maríliaé77	48
13	#vote13	44
14	#lulanoprimeirotu	44
15	#capitaonopilhadoefigueir	42
16	#eleições2022	34
17	#bolsonaronarecord	34
18	#sabatinabolsonaro	34
19	#debatenaglobo	29
20	#brasildaesperan	26
21	#bolsonney	26
22	#lulapresid	24
23	#brazilwasstollen	22

24	#brazilia	22
25	#debatenaband	21
26	#lula	21
27	#lulanaband	20
28	#fraude	20
29	#lulanoflow	19

```
[69]: #-----
# Hashtag Mais Usada por Dia (interativo)
#-----

# Explodir a coluna de hashtags
df_exploded = df_retweets.explode('hashtags_norm')
df_exploded['hashtags_norm'] = df_exploded['hashtags_norm'].str.lower().str.
    ↳strip()
df_exploded = df_exploded[df_exploded['hashtags_norm'] != '']

# Adicionar coluna de data
df_exploded['date'] = df_exploded['tweet_created_at'].dt.date

# Contar hashtags por dia
hashtags_daily = df_exploded.groupby(['date', 'hashtags_norm']).size().
    ↳reset_index(name='count')

# Selecionar hashtag mais usada por dia
top_daily = hashtags_daily.sort_values(['date', 'count'], ascending=[True,
    ↳False])
top_daily = top_daily.groupby('date').first().reset_index()

# Criar intervalo completo de datas
full_range = pd.date_range(df_exploded['date'].min(), df_exploded['date'].max())
full_df = pd.DataFrame({'date': full_range})

# Garantir que ambas as colunas estejam no formato datetime64[ns]
full_df['date'] = pd.to_datetime(full_df['date'])
top_daily['date'] = pd.to_datetime(top_daily['date'])

# Merge com todas as datas
top_daily_full = full_df.merge(top_daily, on='date', how='left')

# Preencher ausentes
top_daily_full['count'] = top_daily_full['count'].fillna(0)
top_daily_full['hashtags_norm'] = top_daily_full['hashtags_norm'].fillna('sem
    ↳dados')

# Criar gráfico
fig = px.line(
```

```

top_daily_full,
x='date',
y='count',
hover_name='hashtags_norm',
title='Hashtag Mais Usada por Dia',
labels={'date': 'Data', 'count': 'Quantidade de Tweets'},
markers=True
)

# Adicionar anotações para picos significativos
for index, row in top_daily_full.iterrows():
    if row['count'] > 29 and row['hashtags_norm'] != 'sem dados':
        fig.add_annotation(
            x=row['date'],
            y=row['count'],
            text=row['hashtags_norm'],
            showarrow=False,
            yanchor='bottom',
            xanchor='left',
            textangle=-45,
            font=dict(size=12)
        )

# Atualizar layout com ajuste de eixo X
fig.update_layout(
    xaxis_title='Dia da Publicação',
    yaxis_title='Contagem da Hashtag Mais Popular',
    hovermode='x unified',
    template='simple_white',
    margin=dict(r=120),
    xaxis=dict(
        range=[
            top_daily_full['date'].min(),
            top_daily_full['date'].max()
        ]
    )
)

fig.show()

```

```

[70]: #-----
# Volume diário das hashtags mais usadas:
# - Destaca picos (top 15%) e dias de votação (02/10 e 30/10)
#-----

# Preparar dados das hashtags

```

```

df_exploded = df_retweets.explode('hashtags_norm')
df_exploded['hashtags_norm'] = df_exploded['hashtags_norm'].str.lower().str.
    ↳strip()
df_exploded = df_exploded[df_exploded['hashtags_norm'] != '']

df_exploded['date'] = df_exploded['tweet_created_at'].dt.date
hashtags_daily = df_exploded.groupby(['date', 'hashtags_norm']).size().
    ↳reset_index(name='count')

top_daily = hashtags_daily.sort_values(['date', 'count'], ascending=[True,
    ↳False])
top_daily = top_daily.groupby('date').first().reset_index()
top_daily['date'] = pd.to_datetime(top_daily['date'])

# Identificação dos picos
limiar = top_daily['count'].quantile(0.85)
picos = top_daily[top_daily['count'] > limiar].copy()

# Garantir destaque dos dias de votação
datas_votacao = [pd.to_datetime('2022-10-02'), pd.to_datetime('2022-10-30')]
votacao_rows = top_daily[top_daily['date'].isin(datas_votacao)].copy()
votacao_rows['tipo'] = 'votacao'

# Marca os picos
picos['tipo'] = 'pico'

# Combinar picos e dias de votação, sem duplicar
destaques = pd.concat([picos, votacao_rows]).drop_duplicates(subset='date')

# Adicionar cor com base no tipo
destaques['cor'] = destaques['tipo'].map({
    'pico': 'red',
    'votacao': 'green'
})

# Plotagem
plt.figure(figsize=(14, 6))
ax = plt.gca()

# Linha principal
ax.plot(
    top_daily['date'],
    top_daily['count'],
    color='#1f77b4',
    linestyle='--',
    linewidth=2,

```

```

        marker='o',
        markersize=4,
        markeredgcolor='#1f77b4',
        label='Hashtag mais usada por dia'
    )

    # Destaques (picos e dias de votação)
    for idx, row in destaques.iterrows():
        ax.scatter(
            row['date'],
            row['count'],
            color=row['cor'],
            s=25,
            zorder=3,
            edgecolor='black'
        )

    # Legenda
    legend_elements = [
        Line2D([0], [0], marker='o', color='w',
            label=f'Picos (Top 15% > {round(limiar)} hashtags)',
            markerfacecolor='red', markersize=8, markeredgcolor='black'),
        Line2D([0], [0], marker='o', color='w',
            label='Dias de votação',
            markerfacecolor='green', markersize=8, markeredgcolor='black')
    ]
    ax.legend(handles=legend_elements, loc='upper left')

    # Eixo X estratégico
    datas_principais = pd.date_range(
        start=top_daily['date'].min(),
        end=top_daily['date'].max(),
        periods=2
    )

    datas_xticks = sorted(set(datas_principais.union(destaques['date'])))
    ax.set_xticks(datas_xticks)
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%d/%m'))

    for label in ax.get_xticklabels():
        label.set_color('black')
        label.set_fontweight('bold')
        label.set_fontsize(7)

    plt.xticks(rotation=90, ha='center')

```

```

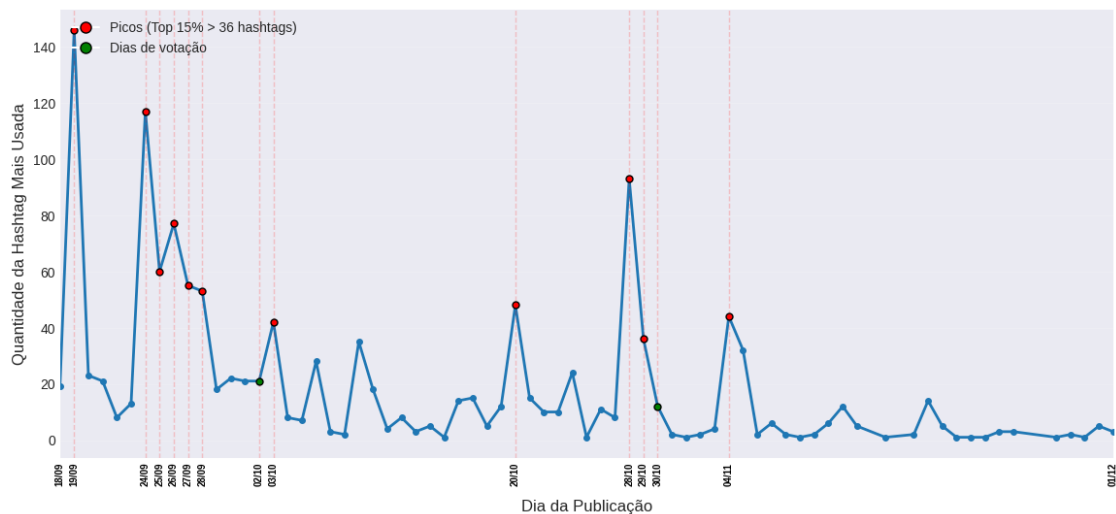
# Linhas verticais nos destaques
for date in destaques['date']:
    ax.axvline(x=date, color='red', alpha=0.2, linestyle='--', linewidth=1)

# Ajustes finais
ax.set_xlim([
    top_daily['date'].min(),
    top_daily['date'].max()
])

ax.set_xlabel('Dia da Publicação', fontsize=12, labelpad=10)
ax.set_ylabel('Quantidade da Hashtag Mais Usada', fontsize=12)
ax.grid(True, alpha=0.2)

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳ hashtag_mais_usada_com_votacao_e_picos_retweets.png', dpi=300,
↳ bbox_inches='tight')
plt.show()

```



```

# Converter data e criar coluna de data simplificada
df_exploded['date'] = pd.to_datetime(df_exploded['tweet_created_at']).dt.date

# Agrupar por hashtag e data para encontrar o dia de pico
daily_metrics = df_exploded.groupby(['hashtags', 'date']).agg(
    tweet_count=('tweet_id', 'count')
).reset_index()

# Encontrar o dia com maior uso para cada hashtag
peak_days = daily_metrics.loc[daily_metrics.groupby('hashtags')['tweet_count'].
    ↪idxmax()].reset_index(drop=True)

# Filtrar hashtags com pelo menos 30 tweets no total
hashtag_total_tweets = df_exploded.groupby('hashtags')['tweet_id'].count()
hashtags_validas = hashtag_total_tweets[hashtag_total_tweets >= 30].index
peak_days = peak_days[peak_days['hashtags'].isin(hashtags_validas)]

# Ordenar e formatar a tabela final
resultado = peak_days.sort_values(by='tweet_count', ascending=False)
resultado = resultado.rename(columns={
    'date': 'dia_do_pico',
    'tweet_count': 'tweets_no_pico'
})

# Formatar data como dd/mm/yyyy
resultado['dia_do_pico'] = pd.to_datetime(resultado['dia_do_pico']).dt.
    ↪strftime('%d/%m/%Y')

# Mostrar top 30
resultado.head(30)

```

```

[71]:
      hashtags dia_do_pico  tweets_no_pico
1          #13  19/09/2022             146
304         #paz  24/09/2022             117
364  #sãopaulopodemais  28/10/2022             93
63   #brasildaesperanca  26/09/2022             77
56         #bomdia  25/09/2022             60
241   #lulano1ºturno  28/09/2022             53
265   #lulapresidente13  20/10/2022             48
71   #brazilwasstolen  04/11/2022             44
35   #bolsonaro22  27/09/2022             43
388   #vote100  27/09/2022             43
290   #mourãosenador  27/09/2022             43
129   #eleicoes2022  03/10/2022             42
79   #capitaonopilhadoefigueir  09/10/2022             35
43   #bolsonaronarecord  24/10/2022             24
342  #sabatinabolsonaro  24/10/2022             24

```

248	#lulanoprimeirotu	20/09/2022	23
134	#eleições2022	02/10/2022	20
279	#maríliaé77	18/09/2022	19
389	#vote13	09/10/2022	12

```
[72]: #-----
# Tabela: Hashtag Mais Usada por Período
#-----

# Processamento inicial
df_exploded = df_retweets.explode('hashtags')
df_exploded['hashtags'] = df_exploded['hashtags'].str.lower().str.strip()
df_exploded = df_exploded[df_exploded['hashtags'] != '']

# Converter data e criar coluna de data simplificada
df_exploded['date'] = pd.to_datetime(df_exploded['tweet_created_at']).dt.date

# Agrupar por hashtag para calcular intervalo e métricas consolidadas
hashtag_metrics = df_exploded.groupby('hashtags').agg(
    data_inicio=('date', 'min'),
    data_fim=('date', 'max'),
    total_tweets=('tweet_id', 'count')
).reset_index()

# Filtrar hashtags com pelo menos 30 tweets
hashtag_metrics = hashtag_metrics[hashtag_metrics['total_tweets'] >= 30]

# Formatar datas e ordenar
hashtag_metrics['periodo_circulacao'] = hashtag_metrics.apply(
    lambda x: f"{x['data_inicio'].strftime('%d/%m/%Y')} a {x['data_fim']}.
    ↳strftime('%d/%m/%Y')}",
    axis=1
)

resultado = hashtag_metrics.sort_values('total_tweets', ascending=False).round(2)
resultado = resultado[['hashtags', 'periodo_circulacao', 'total_tweets']]

resultado.head(30)
```

```
[72]:
```

	hashtags	periodo_circulacao	total_tweets
304	#paz	24/09/2022 a 28/09/2022	159
1	#13	19/09/2022 a 20/09/2022	150
63	#brasildaesperanca	26/09/2022 a 25/10/2022	140
364	#sãopaulopodemais	28/10/2022 a 11/11/2022	130
265	#lulapresidente13	19/09/2022 a 24/10/2022	123
241	#lulano1ºturno	19/09/2022 a 02/10/2022	122

56	#bomdia	25/09/2022 a 26/09/2022	117
129	#eleicoes2022	19/09/2022 a 01/11/2022	114
71	#brazilwasstolen	04/11/2022 a 12/11/2022	88
35	#bolsonaro22	27/09/2022 a 27/11/2022	66
388	#vote100	27/09/2022 a 27/11/2022	61
290	#mourãosenador	27/09/2022 a 27/11/2022	61
279	#maríliaé77	18/09/2022 a 29/10/2022	48
389	#vote13	01/10/2022 a 20/10/2022	44
248	#lulanoprimeirotu	20/09/2022 a 21/09/2022	44
79	#capitaonopilhadoefigueir	09/10/2022 a 12/10/2022	42
43	#bolsonaronarecord	23/10/2022 a 24/10/2022	34
134	#eleições2022	24/09/2022 a 30/10/2022	34
342	#sabatinabolsonaro	23/10/2022 a 24/10/2022	34

Análise de Termos Mais Frequentes:

```
[73]: # Baixa o pacote de stopwords
      nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
[73]: True
```

```
[74]: #-----
      # Nuvem de palavras
      #-----

      # Referência: https://medium.com/@m3redithw/wordclouds-with-python-c287887acc8b

      # Carregar a máscara
      mask = np.array(Image.open("/content/drive/MyDrive/TCC/Estaticos/util/nuvem_mask.
      ↪png"))

      stopwords = set(STOPWORDS)
      stopwords_pt = set(nltk_stopwords.words('portuguese'))
      custom_stopwords = {"pq", "sim", "então", "aí",
                          "tá", "tipo", "rt", "ser", "haver", "pra"}

      stopwords = stopwords.union(stopwords_pt).union(custom_stopwords)

      text = " ".join(df_retweets["preprocessed_tweet"].fillna('')).lower()
      tokens = text.split()
      filtered_tokens = [w for w in tokens if len(w) > 2 and w not in stopwords]
      text_filtered = " ".join(filtered_tokens)

      wordcloud = WordCloud(
```



```

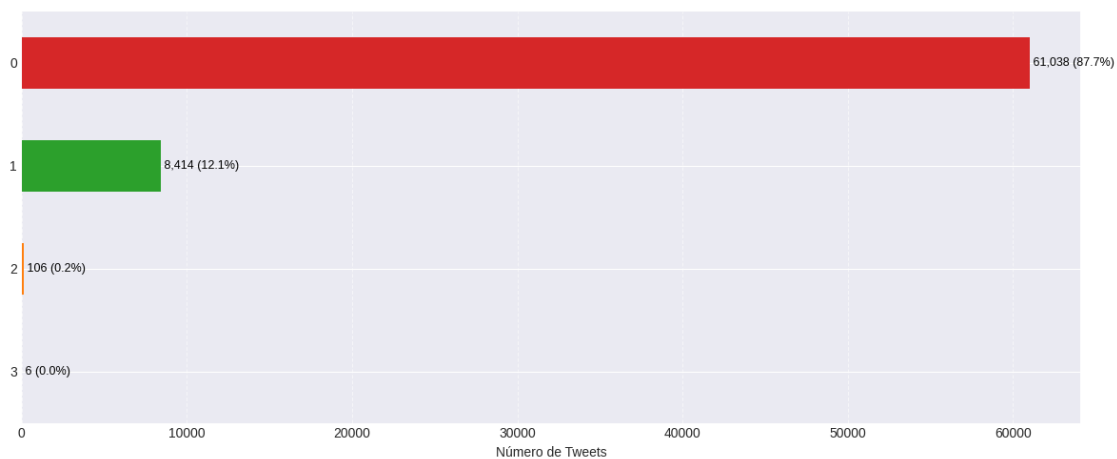
↳color=['#1f77b4','#ff7f0e','#2ca02c','#d62728','#9467bd'])

# Adicionar valores nas barras
for i, v in enumerate(phase_counts.sort_values()):
    plt.text(v, i, f" {v:,} ({v/sum(phase_counts):.1%})",
            color='black', va='center', fontsize=9)

plt.xlabel('Número de Tweets')
plt.ylabel('')
plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/contagem_retweets_por_qnt_urls.
↳png', dpi=300, bbox_inches='tight')
plt.show()

```



```

[76]: #-----
# Contagem das URLs mais frequentes
#-----

# Garantir que os valores nulos sejam tratados como lista vazia para literal_eval
df_retweets['urls'] = df_retweets['urls'].fillna('[]').apply(ast.literal_eval)

# Explodir as URLs
df_urls = df_retweets.explode('urls')

# Contar URLs mais comuns
df_urls['urls'].value_counts().head(30)

```

[76]: urls
https://twitter.com/Miltonneves/status/1574462776780754964/video/1
562
https://twitter.com/obrunos/status/1585271035024343041/video/1
409
https://twitter.com/AugustoNPistola/status/1598514710445318145/video/1
352
https://twitter.com/lamarinearaujo5/status/1571390215314411522/photo/1
208
https://dadosabertos.tse.jus.br/dataset/resultados-2022-boletim-de-urna
160
https://twitter.com/qqrmeme/status/1572006326582083584/video/1
150
https://twitter.com/senadorhumberto/status/1574510242612006913/video/1
136
https://twitter.com/tarcisiogdf/status/1586085299653734400/photo/1
130
https://twitter.com/lazarorosa25/status/1595788270360989696/video/1
130
https://twitter.com/Itspedrito/status/1585363546782982144/video/1
124
https://twitter.com/anacarolinars/status/1586836500800569346/photo/1
119
https://twitter.com/CCiocler/status/1578424526844878848/video/1
112
https://twitter.com/desmentindobofo/status/1593403201209892864/photo/1
110
https://twitter.com/bangtapixels/status/1575603552801550337/video/1
109
https://twitter.com/ErikakHilton/status/1592295546563497986/photo/1
104
https://twitter.com/rolealeatorio/status/1584559263602839553/video/1
102
https://twitter.com/demetrinando/status/817002216682913792/video/1
97
https://twitter.com/choquei/status/1587097485226528771/photo/1
90
https://twitter.com/brussel_ive/status/1583045901035384832/video/1
90
https://twitter.com/Bernilton/status/1572402861417566208/video/1
90
https://twitter.com/ThiAGoMGBR/status/1576565130510274561/video/1
84
https://twitter.com/saintliberte/status/1575988906499375104/photo/1
83
https://twitter.com/Marcoslulaofici/status/1571820419392442370/photo/1
80

```

https://valor.globo.com/politica/noticia/2022/11/08/auditoria-do-tcu-nao-
identifica-falhas-em-urna-eletronica-no-2o-turno.ghtml      77
https://twitter.com/MonarkTalks/status/1598399533402030081/video/1
73
https://twitter.com/dudunaweb/status/1324010036390174721/video/1
72
https://www.bbc.com/portuguese/brasil-62311882
68
https://twitter.com/mahaertt/status/1577424620268838912/photo/1
61
https://twitter.com/PtNuncaMaisEu/status/1586187485092380672/video/1
59
https://terrabrasilnoticias.com/2022/10/tcu-pede-boletins-de-urna-de-4-mil-
secoes-para-auditoria/                                     58
Name: count, dtype: int64

```

```

[77]: #-----
# URLs com MAIORES PICOS DIÁRIOS de compartilhamento
#-----

# Filtrar apenas URLs não vazias
df_urls = df_urls[df_urls['urls'].apply(lambda x: isinstance(x, str) and x.
    ↳strip() != '')].copy()

# Garantir que a data esteja no formato datetime
df_urls['tweet_created_at'] = pd.to_datetime(df_urls['tweet_created_at'],
    ↳errors='coerce').dt.floor('d')

# Contar o número de vezes que cada URL apareceu por dia
url_daily_counts = df_urls.groupby(['urls', 'tweet_created_at']).size().
    ↳reset_index(name='count')

# Para cada URL, pegar a data com maior número de ocorrências
idx = url_daily_counts.groupby('urls')['count'].idxmax()
url_peak_dates = url_daily_counts.loc[idx].sort_values(by='count',
    ↳ascending=False).copy()

# Formatar a data para dd/mm/yyyy
url_peak_dates['tweet_created_at'] = url_peak_dates['tweet_created_at'].dt.
    ↳strftime('%d/%m/%Y')

# Renomear a coluna para "dia_do_pico"
url_peak_dates.rename(columns={'tweet_created_at': 'dia_do_pico'}, inplace=True)

# Resultado final: URLs mais compartilhadas e sua data de pico
url_peak_dates.head(30)

```

[77]:

	urls	dia_do_pico	count
237	https://twitter.com/AugustoNPistola/status/159...	02/12/2022	352
562	https://twitter.com/Miltonneves/status/1574462...	27/09/2022	275
1237	https://twitter.com/obrunos/status/15852710350...	27/10/2022	178
1108	https://twitter.com/lamarinearaujo5/status/157...	18/09/2022	133
1308	https://twitter.com/qqrmeme/status/15720063265...	02/10/2022	132
1410	https://twitter.com/tarcisiogdf/status/1586085...	28/10/2022	93
919	https://twitter.com/desmentindobocho/status/159...	18/11/2022	90
791	https://twitter.com/anacarolinars/status/15868...	31/10/2022	84
431	https://twitter.com/Itspedrito/status/15853635...	27/10/2022	83
261	https://twitter.com/CCiocler/status/1578424526...	07/10/2022	76
877	https://twitter.com/choquei/status/15870974852...	31/10/2022	75
1366	https://twitter.com/senadorhumberto/status/157...	26/09/2022	75
527	https://twitter.com/Marcoslulaofici/status/157...	19/09/2022	74
349	https://twitter.com/ErikakHilton/status/159229...	15/11/2022	74
1123	https://twitter.com/lazarorosa25/status/159578...	24/11/2022	72
37	https://dadosabertos.tse.jus.br/dataset/result...	05/11/2022	72
1334	https://twitter.com/rolealeatorio/status/15845...	24/10/2022	71
811	https://twitter.com/bangtapixels/status/157560...	30/09/2022	69
1349	https://twitter.com/saintliberte/status/157598...	01/10/2022	65
906	https://twitter.com/demetrinando/status/817002...	02/10/2022	65
847	https://twitter.com/brussel_live/status/1583045...	20/10/2022	61
610	https://twitter.com/PtNuncaMaisEu/status/15861...	29/10/2022	58
246	https://twitter.com/Bernilton/status/157240286...	20/09/2022	51
569	https://twitter.com/MonarkTalks/status/1598399...	01/12/2022	49
1501	https://valor.globo.com/politica/noticia/2022/...	08/11/2022	48
701	https://twitter.com/ThiAGoMGBR/status/15765651...	03/10/2022	46
1173	https://twitter.com/mahaertt/status/1577424620...	05/10/2022	42
185	https://terrabrasilnoticias.com/2022/10/tcu-pe...	08/10/2022	38
635	https://twitter.com/RazerBrazil/status/1576587...	03/10/2022	37
329	https://twitter.com/DireitaSDVSEMPR/status/159...	19/11/2022	37

A maior parte das URLs refere-se ao próprio Twitter/X, provavelmente representando retweets, vídeos ou imagens internas da plataforma. A seguir, essas URLs foram filtradas para possibilitar uma análise focada nas fontes externas efetivamente compartilhadas pelos usuários, permitindo uma avaliação mais clara dos domínios externos presentes nos tweets.

[78]:

```
#-----  
# URLs EXTERNAS mais frequentes (excluindo twitter.com)  
#-----  
  
# Filtragem de URLs  
# Remover URLs vazias e do Twitter  
df_urls = df_urls[  
    df_urls['urls'].apply(  
        lambda x: isinstance(x, str)  
        and x.strip() != ''  
    )  
]
```

```

        and 'twitter.com' not in x.lower()
    )
].copy()

# Contagem simples de URLs
# Contar URLs mais comuns
url_counts = df_urls['urls'].value_counts().head(30)
url_counts

```

```

[78]: urls
https://dadosabertos.tse.jus.br/dataset/resultados-2022-boletim-de-urna
160
https://valor.globo.com/politica/noticia/2022/11/08/auditoria-do-tcu-nao-
identifica-falhas-em-urna-eletronica-no-2o-turno.ghtml
77
https://www.bbc.com/portuguese/brasil-62311882
68
https://terrabrasilnoticias.com/2022/10/tcu-pede-boletins-de-urna-de-4-mil-
secoes-para-auditoria/
58
https://revistaoeste.com/politica/eleicoes-2022/moraes-apertar-numero-na-urna-e-
mentir-que-apareceu-outro-sera-considerado-crime/
49
https://www1.folha.uol.com.br/poder/2022/10/moraes-derruba-canal-de-
bolsonarista-no-telegram-que-defendeu-quebrar-
urna.shtml?utm_source=twitter&utm_medium=social&utm_campaign=twfolha
42
https://www.youtube.com/watch?v=u-ZGL90PfeI&feature=youtu.be
42
https://gazetabrasil.com.br/eleicoes-2022-2/2022/10/06/moro-faz-campanha-contr-
abstencao-vem-pra-urna-vamos-votar-e-derrotar-o-pt/
40
https://www.estadao.com.br/internacional/boca-de-urna-mostra-partido-de-extrema-
direita-na-frente-nas-eleicoes-na-
italia/?utm_source=twitter:newsfeed&utm_medium=social-
organic&utm_campaign=redes-sociais:092022:e&utm_content=:::&utm_term=      35
https://brasilsemmedo.com/nao-ha-como-checar-eventuais-erros-da-urna-sem-violar-
o-sigilo-do-voto-diz-especialista/
35
https://www.poder360.com.br/eleicoes/eleicao-2022-nao-tera-pesquisa-boca-de-
urna/
34
https://www.brasil247.com/tanostrends/600-no-bolso-e-13-na-urna-e-assunto-mais-
comentado-do-twitter-apos-bolsonaro-tentar-comprar-eleicao
28
https://noticias.uol.com.br/eleicoes/2022/10/27/sergio-moro-defesa-voto-
impresso.htm?utm_source=twitter&utm_medium=social-

```

media&utm_content=geral&utm_campaign=noticias

27

<https://terrabrasilnoticias.com/2022/09/estranho-datafolha-e-ipec-nao-vao-fazer-pesquisas-boca-de-urna-no-domingo-2/>

26

/oficial

21

https://noticias.uol.com.br/ultimas-noticias/agencia-estado/2022/10/07/carmen-lucia-manda-bolsonaro-remover-video-sobre-prova-de-vida-do-inss-na-urna.htm?utm_source=twitter&utm_medium=social-media&utm_content=geral&utm_campaign=uol

20

<https://revistaforum.com.br/politica/2022/9/19/ipec-lula-bate-em-55-dos-votos-validos-na-pesquisa-espontanea-123521.html>

19

/morto-em-urna-segura/

18

<https://www.youtube.com/watch?v=gi8vhaMLIz4&feature=youtu.be>

14

<https://web.archive.org/web/20221009033826/https://dadosabertos.tse.jus.br/datas-et/resultados-2022-boletim-de-urna>

14

<https://terrabrasilnoticias.com/2022/09/apertar-numero-na-urna-e-mentir-que-apareceu-outro-sera-considerado-crime-alerta-moraes/>

14

<https://www.brasil247.com/brasil/diretor-do-ipespe-faz-alerta-de-golpe-pesquisa-boca-de-urna-fake-a-vista>

13

<https://gazetabrasil.com.br/destaques/2022/09/25/giorgia-meloni-boca-de-urna-aponta-vitoria-da-direita-na-italia/>

13

https://noticias.uol.com.br/ultimas-noticias/agencia-estado/2022/09/22/bolsonaristas-impulsionam-no-telegram-teoria-de-fraude-na-urna.htm?utm_source=twitter&utm_medium=social-media&utm_content=geral&utm_campaign=noticias

12

<https://terrabrasilnoticias.com/2022/11/professor-de-computacao-admite-que-existe-problema-com-urna-mas-justifica-que-boletim-resolve/>

11

<https://www.brasil247.com/brasil/pai-da-urna-eletronica-desmonta-parecer-do-pl>

10

<https://noticias.uol.com.br/ultimas-noticias/agencia-estado/2022/10/07/carmen-lucia-manda-bolsonaro-remover-video-sobre-prova-de-vida-do-inss-na-urna.htm>

10

<https://terrabrasilnoticias.com/2022/10/tse-entrega-ao-tcu-boletins-de-urna-do-1o-turno-para-auditoria/>

10

<https://www.brasil247.com/brasil/justica-afasta-chefe-de-secao-eleitoral-que-fez-boca-de-urna-para-bolsonaro>

9

<https://www.tse.jus.br/comunicacao/noticias/2018/Outubro/tse-determina-exclusao-de-video-em-que-jair-bolsonaro-critica-urnas-eletronicas>

8

Name: count, dtype: int64

```
[79]: #-----
# URLs EXTERNAS mais frequentes com MAIORES PICOS DIÁRIOS de compartilhamento
#-----

# Explodir as URLs
df_urls = df_retweets.explode('urls')

# Filtragem de URLs
# Remover URLs vazias e do Twitter
df_urls = df_urls[
    df_urls['urls'].apply(
        lambda x: isinstance(x, str)
        and x.strip() != ''
        and 'twitter.com' not in x.lower()
    )
].copy()

# Contagem de URLs
# Contar URLs mais comuns
url_counts = df_urls['urls'].value_counts().head(30)

# Análise de datas de pico
# Converter para datetime
df_urls['tweet_created_at'] = pd.to_datetime(df_urls['tweet_created_at'],
    →errors='coerce')

# Agrupar por URL e data
url_daily_counts = df_urls.groupby(['urls', pd.Grouper(key='tweet_created_at',
    →freq='D')]).size().reset_index(name='count')

# Encontrar datas de pico
idx = url_daily_counts.groupby('urls')['count'].idxmax()
url_peak_dates = url_daily_counts.loc[idx].sort_values(by='count',
    →ascending=False)

# Formatar datas
url_peak_dates['peak_date'] = url_peak_dates['tweet_created_at'].dt.strftime('%d/
    →%m/%Y')
url_peak_dates = url_peak_dates[['urls', 'peak_date', 'count']].
    →rename(columns={'peak_date': 'dia_do_pico'})
```

```
url_peak_dates.head(30)
```

```
[79]:
```

	urls	dia_do_pico	count
37	https://dadosabertos.tse.jus.br/dataset/result...	05/11/2022	72
192	https://valor.globo.com/politica/noticia/2022/...	08/11/2022	48
185	https://terrabrasilnoticias.com/2022/10/tcu-pe...	08/10/2022	38
64	https://gazetabrasil.com.br/eleicoes-2022-2/20...	06/10/2022	36
157	https://revistaoeste.com/politica/eleicoes-202...	27/09/2022	35
312	https://www.youtube.com/watch?v=u-ZGL90PfeI&fe...	09/10/2022	35
201	https://www.bbc.com/portuguese/brasil-62311882	21/09/2022	33
239	https://www.estadao.com.br/internacional/boca-...	26/09/2022	28
276	https://www.poder360.com.br/eleicoes/eleicao-2...	29/09/2022	27
178	https://terrabrasilnoticias.com/2022/09/estran...	29/09/2022	24
222	https://www.brasil247.com/tanostrends/600-no-b...	21/10/2022	23
327	https://www1.folha.uol.com.br/poder/2022/10/mo...	28/10/2022	21
31	https://brasilsemmedo.com/nao-ha-como-checar-e...	28/09/2022	19
5	/morto-em-urna-segura/	26/11/2022	18
101	https://noticias.uol.com.br/eleicoes/2022/10/2...	27/10/2022	15
114	https://noticias.uol.com.br/ultimas-noticias/a...	12/10/2022	15
154	https://revistaforum.com.br/politica/2022/9/19...	19/09/2022	15
210	https://www.brasil247.com/brasil/diretor-do-ip...	26/09/2022	13
304	https://www.youtube.com/watch?v=gi8vhaMLIz4&fe...	23/09/2022	13
106	https://noticias.uol.com.br/ultimas-noticias/a...	22/09/2022	12
197	https://web.archive.org/web/20221009033826/htt...	05/11/2022	12
8	/oficial	21/11/2022	11
186	https://terrabrasilnoticias.com/2022/10/tse-en...	25/10/2022	10
175	https://terrabrasilnoticias.com/2022/09/aperta...	27/09/2022	9
181	https://terrabrasilnoticias.com/2022/09/tcu-co...	21/09/2022	8
62	https://gazetabrasil.com.br/destaques/2022/09/...	25/09/2022	8
128	https://oglobo.globo.com/politica/eleicoes-202...	23/09/2022	8
212	https://www.brasil247.com/brasil/justica-afast...	21/10/2022	7
187	https://terrabrasilnoticias.com/2022/11/profes...	20/11/2022	7
180	https://terrabrasilnoticias.com/2022/09/qual-a...	20/09/2022	7

Verificando os domínios mais frequentes:

```
[80]: #-----  
# Domínios mais frequentes  
#-----  
  
# Explodir as listas de URLs em múltiplas linhas  
df_urls = df_retweets.explode('urls')  
  
# Filtrar somente URLs que são strings e não estão vazias ou em branco  
df_urls = df_urls[df_urls['urls'].apply(lambda x: isinstance(x, str) and x.  
→strip() != '')].copy()
```

```

# Extrair o domínio das URLs
df_urls.loc[:, 'domain'] = df_urls['urls'].apply(lambda x: urlparse(x).netloc.
→lower())

# Remover o prefixo 'www.'
df_urls.loc[:, 'domain'] = df_urls['domain'].str.replace(r'^www\.', '',
→regex=True)

# Filtrar domínios não vazios
df_urls = df_urls[df_urls['domain'].str.strip() != '']

# Contar os domínios mais comuns
df_urls['domain'].value_counts().head(30)

```

```

[80]: domain
twitter.com                7292
dadosabertos.tse.jus.br    160
terrabrasilnoticias.com    142
noticias.uol.com.br        118
valor.globo.com            82
brasil247.com              78
youtube.com                77
revistaoeste.com           68
bbc.com                    68
www1.folha.uol.com.br      56
gazetabrasil.com.br        55
estadao.com.br             43
poder360.com.br            42
brasilsemmedo.com          38
g1.globo.com               27
revistaforum.com.br        24
oglobo.globo.com           23
gazetadopovo.com.br        15
metropoles.com             14
web.archive.org            14
resultados.tse.jus.br      13
tse.jus.br                 13
agoranoticiasbrasil.com.br 12
cartacapital.com.br        9
cnnbrasil.com.br           7
oantagonista.com.br        7
fiscaisdobolsonaro.com.br  6
pt.linkedin.com            4
jornalggn.com.br           4
google.com                 3
Name: count, dtype: int64

```

```
[81]: #-----
# Domínios mais frequentes por dia de pico
#-----

# Explodir listas de URLs e criar cópia independente
df_urls = df_retweets.explode('urls').copy()

# Filtrar URLs válidas (strings não vazias)
df_urls = df_urls[df_urls['urls'].str.strip().ne('') & df_urls['urls'].notna()]

# Extrair domínios de forma vetorizada
from urllib.parse import urlparse

# Função vetorizada para extração de domínio
def extract_domain(url):
    parsed = urlparse(url)
    domain = parsed.netloc.lower()
    return domain.removeprefix('www.') if domain.startswith('www.') else domain

# Aplicar extração de domínio
df_urls.loc[:, 'domain'] = df_urls['urls'].apply(extract_domain)

# Remover domínios vazios
df_urls = df_urls[df_urls['domain'].str.strip().ne('')]

# Converter data para datetime (apenas data, sem hora)
df_urls.loc[:, 'tweet_created_at'] = pd.to_datetime(
    df_urls['tweet_created_at'],
    errors='coerce'
).dt.floor('d') # Mantém apenas parte da data

# Contar ocorrências por domínio e data
domain_daily_counts = (
    df_urls.groupby(['domain', 'tweet_created_at'])
    .size()
    .reset_index(name='count')
)

# Encontrar datas de pico usando transform
peak_dates = (
    domain_daily_counts
    .loc[domain_daily_counts.groupby('domain')['count'].transform('max') == ]
    domain_daily_counts['count']
    .copy()
)

# Formatar datas e ordenar resultados
```

```

peak_dates['dia_do_pico'] = peak_dates['tweet_created_at'].dt.strftime('%d/%m/
↳ %Y')
peak_dates = (
    peak_dates
    .drop(columns='tweet_created_at')
    .sort_values('count', ascending=False)
    .drop_duplicates('domain') # Garantir 1 entrada por domínio
    .head(30)
)

# Exibir resultado
peak_dates

```

```

[81]:

```

	domain	count	dia_do_pico
250	twitter.com	542	02/10/2022
54	dadosabertos.tse.jus.br	72	05/11/2022
314	valor.globo.com	48	08/11/2022
224	terrabrasilnoticias.com	38	08/10/2022
90	gazetabrasil.com.br	36	06/10/2022
346	youtube.com	36	09/10/2022
196	revistaoeste.com	35	27/09/2022
17	bbc.com	33	21/09/2022
59	estadao.com.br	31	26/09/2022
32	brasil247.com	31	21/10/2022
172	poder360.com.br	27	29/09/2022
330	www1.folha.uol.com.br	21	28/10/2022
39	brasilsemmedo.com	21	28/09/2022
139	noticias.uol.com.br	20	12/10/2022
190	revistaforum.com.br	15	19/09/2022
320	web.archive.org	12	05/11/2022
156	oglobo.globo.com	8	23/09/2022
64	fiscaisdobolsonaro.com.br	6	23/10/2022
81	g1.globo.com	6	30/10/2022
47	cnnbrasil.com.br	6	09/11/2022
124	metropoles.com	6	07/11/2022
186	resultados.tse.jus.br	6	05/10/2022
230	tse.jus.br	6	12/10/2022
101	gazetadopovo.com.br	4	24/11/2022
154	oantagonista.com.br	4	23/11/2022
181	pt.linkedin.com	4	23/09/2022
16	aovivo.folha.uol.com.br	3	30/09/2022
10	agoranoticiasbrasil.com.br	3	10/11/2022
43	cartacapital.com.br	3	19/10/2022
113	jornalggn.com.br	3	28/09/2022

1.3 5.2 Análise Integrada

1.3.1 5.2.1 Sentimento, Tópicos e Entidades (Produção)

```
[82]: # Considerando apenas tweets e respostas
df = df_sem_retweets.copy()
```

```
[83]: df.groupby("sentiment").size().reset_index(name="qnt").assign(
    porcentagem=lambda x: ((x["qnt"] / x["qnt"].sum()) * 100).round(2)
)
```

```
[83]:  sentiment    qnt  porcentagem
0  Negativo  17125         41.35
1   Neutro  19160         46.26
2  Positivo   5129         12.38
```

```
[84]: # =====
# 2. CONVERTER A COLUNA 'entities' (string) para DICT
# =====
# Como a coluna vem como string, usamos ast.literal_eval
df['entities'] = df['entities'].apply(ast.literal_eval)

# =====
# 3. CONTAR ENTIDADES
# =====

# Contador global (todas as categorias juntas)
contador_global = Counter()

# Contadores separados por categoria
categorias = ['PESSOAS', 'LOCAIS', 'ORGANIZAÇÕES', 'OUTROS']
contadores_categoria = {cat: Counter() for cat in categorias}

# Percorre cada linha do DataFrame
for entidades_dict in df['entities']:
    for categoria, lista_entidades in entidades_dict.items():
        # Atualiza o contador global com todos os itens da lista
        contador_global.update(lista_entidades)

        # Atualiza o contador específico da categoria
        if categoria in contadores_categoria:
            contadores_categoria[categoria].update(lista_entidades)

# =====
# 4. EXIBIR OS RESULTADOS
# =====

print("\n" + "="*50)
```

```

print("TOP 10 ENTIDADES MAIS FREQUENTES (TODAS AS CATEGORIAS)")
print("="*50)
for entidade, freq in contador_global.most_common(10):
    print(f"{freq:>5}x  {entidade}")

print("\n" + "="*50)
print("TOP 5 POR CATEGORIA")
print("="*50)
for categoria, counter in contadores_categoria.items():
    print(f"\n[{categoria}]")
    for entidade, freq in counter.most_common(5):
        print(f"  {freq:>3}x  {entidade}")

```

```

=====
TOP 10 ENTIDADES MAIS FREQUENTES (TODAS AS CATEGORIAS)
=====

```

```

2563x  Bolsonaro
1996x  Lula
1415x  TSE
1382x  Brasil
 587x  PT
 448x  STF
 398x  Urna
 347x  Vc
 332x  Deus
 323x  bolsonaro

```

```

=====
TOP 5 POR CATEGORIA
=====

```

```

[PESSOAS]
 1986x  Lula
 1237x  Bolsonaro
  312x  Vc
  306x  Ciro
  287x  Deus

```

```

[LOCAIS]
 1382x  Brasil
  637x  Bolsonaro
  448x  STF
  365x  Urna
  295x  bolsonaro

```

```

[ORGANIZAÇÕES]
 1384x  TSE

```

```

689x  Bolsonaro
587x  PT
182x  BU
98x   Forças Armadas

```

[OUTROS]

```

197x  L
180x  Bozo
145x  Urna Eletrônica
130x  Presidente
126x  bozo

```

```

[85]: # 1. Calcular contagem e porcentagem (sem total)
result = (
    df.groupby('topic')
    .size()
    .to_frame('count')
    .assign(percentage=lambda x: (100 * x['count'] / x['count'].sum()).round(2))
)

# 2. Ordenar do maior para o menor pela contagem
result = result.sort_values('count', ascending=False)

# 3. Adicionar a linha de Total (soma e 100%)
total_count = result['count'].sum()
total_row = pd.DataFrame({'count': [total_count], 'percentage': [100.0]},
    index=['Total'])
result = pd.concat([result, total_row])

# 4. Transformar porcentagem em string com '%' para exibição
result['percentage'] = result['percentage'].astype(str) + '%'

# 5. Renomear a coluna
result = result.rename(columns={'percentage': 'percentage_value'})

result

```

```

[85]:

```

	count	percentage_value
Urna Eletrônica	18716	45.19%
Uso da Urna	10056	24.28%
Narrativa Política	2678	6.47%
Disputa Eleitoral	1732	4.18%
Pesquisas Eleitorais	1268	3.06%
Mídias Sociais	1211	2.92%
Fraude	1065	2.57%
Código-Fonte	970	2.34%
Auditoria	752	1.82%

Voto Impresso	476	1.15%
Fake News	392	0.95%
Narrativa Religiosa	381	0.92%
Celular na Cabine	368	0.89%
Logs da Urna	299	0.72%
Militares e Fiscalização	298	0.72%
Conexão com a Internet	292	0.71%
Ataque Hacker	277	0.67%
Live do Argentino	161	0.39%
Outlier	22	0.05%
Total	41414	100.0%

1.3.2 5.2.2 Sentimento, Tópicos e Entidades (Amplificação)

```
[86]: # Considerando apenas retweets
df = df_retweets.copy()
```

```
[87]: df.groupby("sentiment").size().reset_index(name="qnt").assign(
    porcentagem=lambda x: ((x["qnt"] / x["qnt"].sum()) * 100).round(2)
)
```

```
[87]:   sentiment    qnt  porcentagem
0  Negativo  22262         32.00
1   Neutro  40236         57.84
2  Positivo   7066         10.16
```

```
[88]: # =====
# 2. CONVERTER A COLUNA 'entities' (string) para DICT
# =====
# Como a coluna vem como string, usamos ast.literal_eval
df['entities'] = df['entities'].apply(ast.literal_eval)

# =====
# 3. CONTAR ENTIDADES
# =====

# Contador global (todas as categorias juntas)
contador_global = Counter()

# Contadores separados por categoria
categorias = ['PESSOAS', 'LOCAIS', 'ORGANIZAÇÕES', 'OUTROS']
contadores_categoria = {cat: Counter() for cat in categorias}

# Percorre cada linha do DataFrame
for entidades_dict in df['entities']:
    for categoria, lista_entidades in entidades_dict.items():
        # Atualiza o contador global com todos os itens da lista
```

```

        contador_global.update(lista_entidades)

        # Atualiza o contador específico da categoria
        if categoria in contadores_categoria:
            contadores_categoria[categoria].update(lista_entidades)

# =====
# 4. EXIBIR OS RESULTADOS
# =====

print("\n" + "="*50)
print("TOP 10 ENTIDADES MAIS FREQUENTES (TODAS AS CATEGORIAS)")
print("="*50)
for entidade, freq in contador_global.most_common(10):
    print(f"{freq:>5}x  {entidade}")

print("\n" + "="*50)
print("TOP 5 POR CATEGORIA")
print("="*50)
for categoria, counter in contadores_categoria.items():
    print(f"\n[{categoria}]")
    for entidade, freq in counter.most_common(5):
        print(f"  {freq:>3}x  {entidade}")

```

```

=====
TOP 10 ENTIDADES MAIS FREQUENTES (TODAS AS CATEGORIAS)
=====

```

```

3767x  Bolsonaro
2826x  TSE
2367x  Lula
2015x  Brasil
1810x  VOTE SEM
1639x  PT
1496x  PL
1131x  Carlos Rocha
1091x  Ciro
1054x  Partido Liberal

```

```

=====
TOP 5 POR CATEGORIA
=====

```

```

[PESSOAS]
2566x  Bolsonaro
2366x  Lula
1810x  VOTE SEM
1496x  PL

```

1131x Carlos Rocha

[LOCAIS]

2015x Brasil
983x STF
886x Bolsonaro
849x Congresso
707x ELECTRÔNICA

[ORGANIZAÇÕES]

2819x TSE
1639x PT
1054x Partido Liberal
530x FA
486x Forças Armadas

[OUTROS]

823x Dilm
621x IMPOSSÍVEL
606x Twitter
590x LOOLA
590x NOMEADO

```
[89]: # 1. Calcular contagem e porcentagem (sem total)
result = (
    df.groupby('topic')
    .size()
    .to_frame('count')
    .assign(percentage=lambda x: (100 * x['count'] / x['count'].sum()).round(2))
)

# 2. Ordenar do maior para o menor pela contagem
result = result.sort_values('count', ascending=False)

# 3. Adicionar a linha de Total (soma e 100%)
total_count = result['count'].sum()
total_row = pd.DataFrame({'count': [total_count], 'percentage': [100.0]},
    index=['Total'])
result = pd.concat([result, total_row])

# 4. Transformar porcentagem em string com '%' para exibição
result['percentage'] = result['percentage'].astype(str) + '%'

# 5. Renomear a coluna
result = result.rename(columns={'percentage': 'percentage_value'})

result
```

```
[89]:
```

	count	percentage_value
Urna Eletrônica	28091	40.38%
Uso da Urna	12973	18.65%
Narrativa Política	8019	11.53%
Narrativa Religiosa	3529	5.07%
Disputa Eleitoral	3017	4.34%
Mídias Sociais	2193	3.15%
Pesquisas Eleitorais	1736	2.5%
Logs da Urna	1303	1.87%
Militares e Fiscalização	1280	1.84%
Auditoria	1226	1.76%
Live do Argentino	1176	1.69%
Fraude	1161	1.67%
Voto Impresso	968	1.39%
Ataque Hacker	893	1.28%
Fake News	610	0.88%
Código-Fonte	581	0.84%
Celular na Cabine	407	0.59%
Conexão com a Internet	380	0.55%
Outlier	21	0.03%
Total	69564	100.0%

1.3.3 5.2.2 Nível de Automação

```
[90]: # Calcula a quantidade e o percentual de usuários por nível de automação,
# incluindo usuários não classificados, e coloca "Não classificado" no final
df_user_counts = (
    df_users
    .assign(automation_level=lambda x: x['automation_level'].fillna('Não_
↳classificado'))
    .groupby('automation_level', as_index=False)
    .agg(qtd_usuarios=('user_id', 'nunique'))
)

# Calcular percentual
df_user_counts['percentual'] = round(
    df_user_counts['qtd_usuarios'] / df_user_counts['qtd_usuarios'].sum() * 100,
↳2
)

# Ordenar colocando 'Não classificado' no final
df_user_counts = df_user_counts.sort_values(
    by='automation_level', key=lambda x: x == 'Não classificado'
)

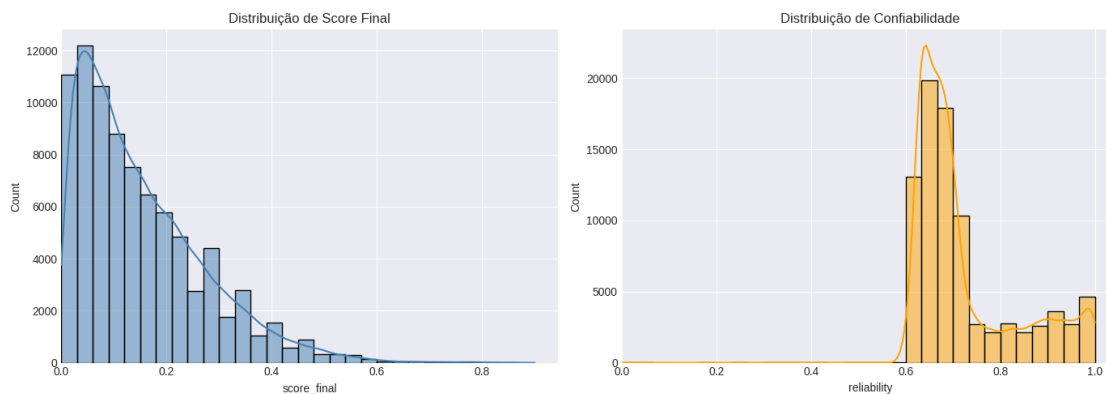
df_user_counts
```

```
[90]: automation_level  qtd_usuarios  percentual
0           Alto           123          0.15
1           Baixo       80538         95.34
2           Médio        3812          4.51
```

```
[91]: fig, ax = plt.subplots(1, 2, figsize=(14, 5))

# Histograma de Score Final
sns.histplot(df_users['score_final'], kde=True, bins=30, ax=ax[0],
             color="steelblue")
ax[0].set_title("Distribuição de Score Final")
ax[0].set_xlim(left=0)

# Histograma de Confiabilidade
sns.histplot(df_users['reliability'], kde=True, bins=30, ax=ax[1],
             color="orange")
ax[1].set_title("Distribuição de Confiabilidade")
ax[1].set_xlim(left=0)
plt.tight_layout()
plt.show()
```



```
[92]: # -----
# Média diária do Score Final (com Fases Eleitorais)
# -----

# Garantir que a data está no formato correto
df_users['tweet_created_at'] = pd.to_datetime(df_users['tweet_created_at'])

# Agrupar por data e calcular média diária do score_final
media_score_por_dia = df_users.groupby(df_users['tweet_created_at'].dt.
                                     date)['score_final'].mean()

# Define os intervalos de campanha (como áreas)
```

```

areas = {
    "Campanha 1º Turno": (df_users['tweet_created_at'].min().
↳strftime('%Y-%m-%d'), "2022-10-02"),
    "Campanha 2º Turno": ("2022-10-02", "2022-10-30"),
    "Pós-eleição": ("2022-10-30", df_users['tweet_created_at'].max().
↳strftime('%Y-%m-%d'))
}

# Define os dias de votação (como linhas)
eventos = {
    "1º Turno (Votação)": "2022-10-02",
    "2º Turno (Votação)": "2022-10-30"
}

# Converte datas
areas = {fase: (pd.to_datetime(inicio), pd.to_datetime(fim)) for fase, (inicio,
↳fim) in areas.items()}
eventos = {nome: pd.to_datetime(data) for nome, data in eventos.items()}

# Cores para áreas
colors = {
    "Campanha 1º Turno": "#f2a640",
    "Campanha 2º Turno": "#cce5ff",
    "Pós-eleição": "#88896e"
}

# Cores para eventos (linhas)
colors_day = {
    "1º Turno (Votação)": "#800080",
    "2º Turno (Votação)": "#ec9d01"
}

# Início do plot
plt.figure(figsize=(14, 6))
media_score_por_dia.plot(label='Média diária do Score Final', color='blue')

# Áreas das campanhas
for fase, (inicio, fim) in areas.items():
    plt.axvspan(inicio, fim, alpha=0.3, label=fase, color=colors[fase])

# Linhas dos dias de votação
for nome, data in eventos.items():
    plt.axvline(data, color=colors_day[nome], linestyle='--', linewidth=1.9,
↳label=nome)

# Garante que o eixo x inclua todas as áreas e eventos

```

```

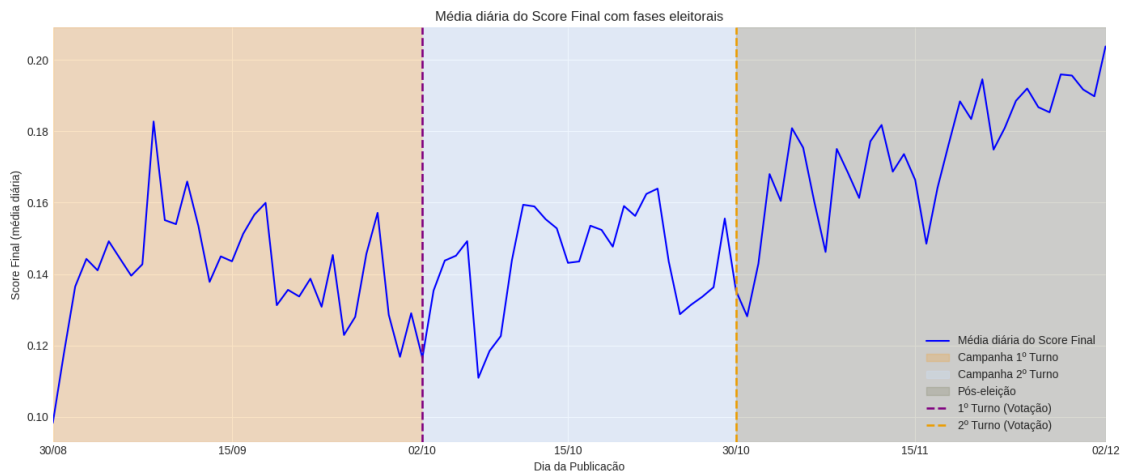
datas_inicio_fim = [inicio for inicio, _ in areas.values()] + [fim for _, fim in
    ↪areas.values()]
datas_eventos = list(eventos.values())
data_min = min(datas_inicio_fim + datas_eventos)
data_max = max(datas_inicio_fim + datas_eventos)
plt.xlim(data_min, data_max)

# Configurar ticks específicos no eixo X
datas_especificas = [
    '30/08/2022',
    '15/09/2022',
    '02/10/2022',
    '15/10/2022',
    '30/10/2022',
    '15/11/2022',
    '02/12/2022'
]
datas_ticks = [pd.to_datetime(data, format='%d/%m/%Y') for data in
    ↪datas_especificas]
rotulos_ticks = [date.strftime('%d/%m') for date in datas_ticks]
plt.xticks(datas_ticks, rotulos_ticks)

# Formata o eixo X
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%d/%m'))

# Personalização
plt.xlabel("Dia da Publicação")
plt.ylabel("Score Final (média diária)")
plt.title("Média diária do Score Final com fases eleitorais")
plt.legend(loc='lower right')
plt.grid(True)
plt.tight_layout()
plt.show()

```



```
[93]: #-----
# Média diária do Score Final + Média Móvel (com Fases Eleitorais)
#-----

# Garantir que a data está no formato correto
df_users['tweet_created_at'] = pd.to_datetime(df_users['tweet_created_at'])

# Agrupar por data e calcular média diária
media_score_por_dia = df_users.groupby(df_users['tweet_created_at'].dt.
    ↳date)['score_final'].mean()

# Calcular a média móvel de 7 dias
media_movel_7d = media_score_por_dia.rolling(window=7, min_periods=1).mean()

# Define os intervalos de campanha (como áreas)
areas = {
    "Campanha 1º Turno": (df_users['tweet_created_at'].min().
    ↳strftime('%Y-%m-%d'), "2022-10-02"),
    "Campanha 2º Turno": ("2022-10-02", "2022-10-30"),
    "Pós-eleição": ("2022-10-30", df['tweet_created_at'].max().
    ↳strftime('%Y-%m-%d'))
}

# Define os dias de votação (como linhas)
eventos = {
    "1º Turno (Votação)": "2022-10-02",
    "2º Turno (Votação)": "2022-10-30"
}

# Converte datas
areas = {fase: (pd.to_datetime(inicio), pd.to_datetime(fim)) for fase, (inicio,
    ↳fim) in areas.items()}
eventos = {nome: pd.to_datetime(data) for nome, data in eventos.items()}

# Cores para áreas
colors = {
    "Campanha 1º Turno": "#f2a640",
    "Campanha 2º Turno": "#cce5ff",
    "Pós-eleição": "#88896e"
}

# Cores para eventos
colors_day = {
    "1º Turno (Votação)": "#800080",

```

```

    "2º Turno (Votação)": "#ec9d01"
}

# Início do plot
plt.figure(figsize=(14, 6))

# Média diária
media_score_por_dia.plot(label='Média diária do Score Final', color='blue')

# Média móvel 7 dias
media_movel_7d.plot(label='Média móvel (7 dias)', color='red', linewidth=2)

# Áreas das campanhas
for fase, (inicio, fim) in areas.items():
    plt.axvspan(inicio, fim, alpha=0.3, label=fase, color=colors[fase])

# Linhas dos dias de votação
for nome, data in eventos.items():
    plt.axvline(data, color=colors_day[nome], linestyle='--', linewidth=1.9,
    →label=nome)

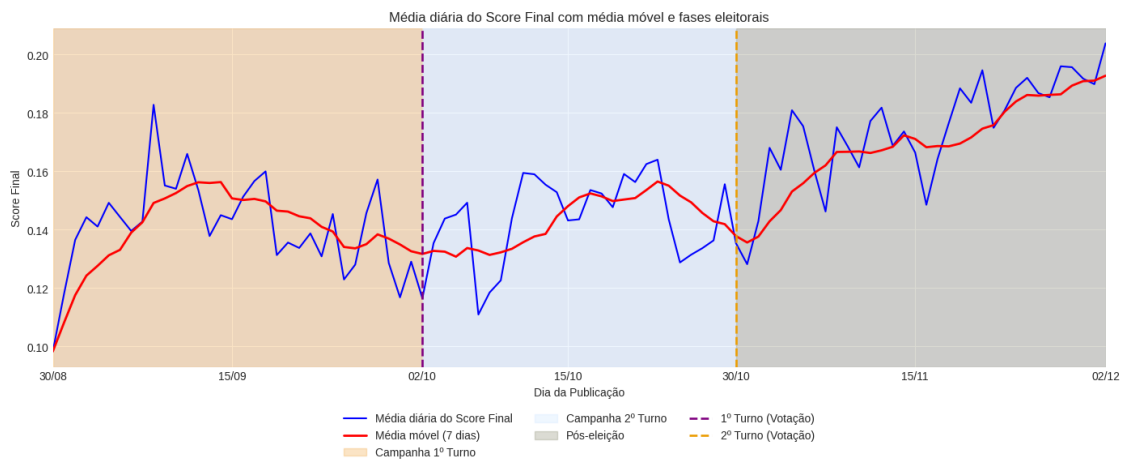
# Ajusta limites do eixo X
datas_inicio_fim = [inicio for inicio, _ in areas.values()] + [fim for _, fim in
    →areas.values()]
datas_eventos = list(eventos.values())
data_min = min(datas_inicio_fim + datas_eventos)
data_max = max(datas_inicio_fim + datas_eventos)
plt.xlim(data_min, data_max)

# Ticks do eixo X
datas_especificas = [
    '30/08/2022',
    '15/09/2022',
    '02/10/2022',
    '15/10/2022',
    '30/10/2022',
    '15/11/2022',
    '02/12/2022'
]
datas_ticks = [pd.to_datetime(data, format='%d/%m/%Y') for data in
    →datas_especificas]
rotulos_ticks = [date.strftime('%d/%m') for date in datas_ticks]
plt.xticks(datas_ticks, rotulos_ticks)
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%d/%m'))

# Personalização
plt.xlabel("Dia da Publicação")

```

```
plt.ylabel("Score Final")
plt.title("Média diária do Score Final com média móvel e fases eleitorais")
plt.legend(loc='lower center', bbox_to_anchor=(0.5, -0.3), ncol=3) # Legenda
    ↳ embaixo
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
[94]: #-----
# Atividade diária de contas suspeitas (automation_level == 'Alto')
#-----

# Filtrar apenas contas suspeitas
suspeitas = df_users[df_users['automation_level'] == "Alto"].copy()

# Agrupar por dia e contar número de tweets
atividade_diaria = (
    suspeitas.groupby(suspeitas['tweet_created_at'].dt.date)['user_id']
    .count()
)

# Define os intervalos de campanha (como áreas)
areas = {
    "Campanha 1º Turno": (df_users['tweet_created_at'].min().
    ↳ strftime('%Y-%m-%d'), "2022-10-02"),
    "Campanha 2º Turno": ("2022-10-02", "2022-10-30"),
    "Pós-eleição": ("2022-10-30", df_users['tweet_created_at'].max().
    ↳ strftime('%Y-%m-%d'))
}
```

```

# Define os dias de votação (como linhas)
eventos = {
    "1º Turno (Votação)": "2022-10-02",
    "2º Turno (Votação)": "2022-10-30"
}

# Converte datas
areas = {fase: (pd.to_datetime(inicio), pd.to_datetime(fim)) for fase, (inicio, fim) in areas.items()}
eventos = {nome: pd.to_datetime(data) for nome, data in eventos.items()}

# Cores para áreas
colors = {
    "Campanha 1º Turno": "#f2a640",
    "Campanha 2º Turno": "#cce5ff",
    "Pós-eleição": "#88896e"
}

# Cores para eventos (linhas)
colors_day = {
    "1º Turno (Votação)": "#800080",
    "2º Turno (Votação)": "#ec9d01"
}

# Início do plot
plt.figure(figsize=(14, 6))
atividade_diaria.plot(label='Atividade diária (contas suspeitas)', color='blue')

# Áreas das campanhas
for fase, (inicio, fim) in areas.items():
    plt.axvspan(inicio, fim, alpha=0.3, label=fase, color=colors[fase])

# Linhas dos dias de votação
for nome, data in eventos.items():
    plt.axvline(data, color=colors_day[nome], linestyle='--', linewidth=1.9, label=nome)

# Garante que o eixo x inclua todas as áreas e eventos
datas_inicio_fim = [inicio for inicio, _ in areas.values()] + [fim for _, fim in areas.values()]
datas_eventos = list(eventos.values())
data_min = min(datas_inicio_fim + datas_eventos)
data_max = max(datas_inicio_fim + datas_eventos)
plt.xlim(data_min, data_max)

# Configurar ticks específicos no eixo X
datas_especificas = [

```

```

'30/08/2022',
'15/09/2022',
'02/10/2022',
'15/10/2022',
'30/10/2022',
'15/11/2022',
'02/12/2022'
]
datas_ticks = [pd.to_datetime(data, format='%d/%m/%Y') for data in datas_especificas]
rotulos_ticks = [date.strftime('%d/%m') for date in datas_ticks]
plt.xticks(datas_ticks, rotulos_ticks)

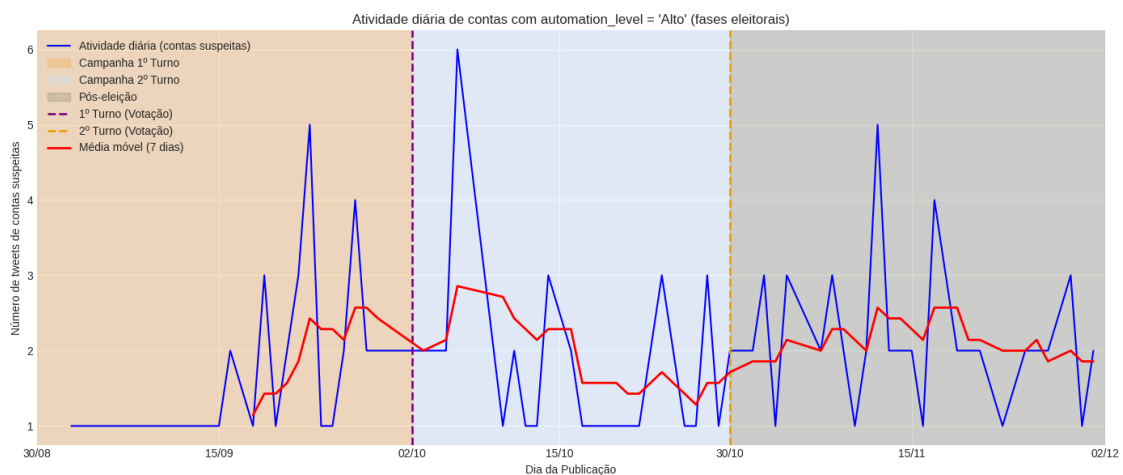
# Formata o eixo X
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%d/%m'))

# Calcular média móvel de 7 dias
media_movel = atividade_diaria.rolling(window=7).mean()

# Adicionar ao gráfico
media_movel.plot(label='Média móvel (7 dias)', color='red', linewidth=2)

# Personalização
plt.xlabel("Dia da Publicação")
plt.ylabel("Número de tweets de contas suspeitas")
plt.title("Atividade diária de contas com automation_level = 'Alto' (fases eleitorais)
→eleitorais)")
plt.legend(loc='upper left')
plt.grid(True)
plt.tight_layout()
plt.show()

```



```
[95]: #-----
# Distribuição de idade das contas por nível de automação (escala log)
#-----

# Definição dos intervalos e rótulos (mesma lógica usada na análise de
→engajamento)
bins = [0, 30, 90, 180, 365, 730, 1825, float('inf')]
labels = ['< 1 mês', '1-3 meses', '3-6 meses', '6-12 meses',
          '1-2 anos', '2-5 anos', '> 5 anos']

# Aplicar a categorização (usa account_age_days)
df_users['age_group'] = pd.cut(
    df_users['account_age_days'],
    bins=bins,
    labels=labels,
    right=False # intervalo [início, fim)
)

# Preencher automation_level vazio
df_users['automation_level'] = df_users['automation_level'].fillna('Não
→classificado')

# Configurações gerais
niveis = df_users['automation_level'].unique()
n_niveis = len(niveis)
cores = plt.cm.tab10.colors
categorias = labels # ordem fixa das faixas

# Criar figura com subplots
fig, axs = plt.subplots(1, n_niveis, figsize=(18, 5),
                        sharey=True,
                        gridspec_kw={'wspace': 0.1})

if n_niveis == 1:
    axs = [axs]

# Plot por nível de automação
for i, nivel in enumerate(niveis):
    df_nivel = df_users[df_users['automation_level'] == nivel]
    contagem = df_nivel['age_group'].value_counts().reindex(categorias,
→fill_value=0)

    container = axs[i].bar(categorias, contagem.values,
                           color=cores[i % len(cores)], width=0.7)
```

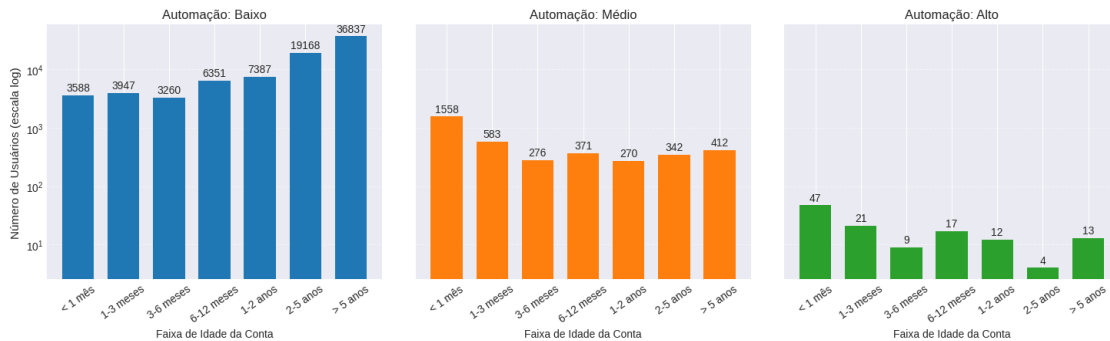
```

# Rótulos
labels_vals = [f'{val}' if val > 0 else '' for val in contagem.values]
axs[i].bar_label(container, labels=labels_vals, label_type='edge',
padding=2, fontsize=10)

axs[i].set_title(f"Automação: {nivel}", fontsize=12)
axs[i].set_xlabel("Faixa de Idade da Conta", fontsize=10)
axs[i].tick_params(axis='x', rotation=35)
axs[i].grid(axis='y', linestyle='--', alpha=0.6)
axs[i].set_yscale("log") # escala log no Y

# Configurações gerais
axs[0].set_ylabel("Número de Usuários (escala log)", fontsize=11)
fig.subplots_adjust(wspace=0.1, top=0.88, bottom=0.22)
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
distribuicao_idade_conta_por_automacao.png',
dpi=300, bbox_inches='tight')
plt.show()

```



```

[96]: # -----
# Distribuição de idade das contas por nível de automação
# -----

bins = [0, 30, 90, 180, 365, 730, 1825, float('inf')]
labels = ['< 1 mês', '1-3 meses', '3-6 meses', '6-12 meses',
'1-2 anos', '2-5 anos', '> 5 anos']

df_users['age_group'] = pd.cut(
    df_users['account_age_days'],
    bins=bins,
    labels=labels,
    right=False
)

```

```

df_users['automation_level'] = df_users['automation_level'].fillna('Não_
↳classificado')

niveis = df_users['automation_level'].unique()
n_niveis = len(niveis)
cores = plt.cm.tab10.colors
categorias = labels

fig, axs = plt.subplots(1, n_niveis, figsize=(18, 5), sharey=True)

if n_niveis == 1:
    axs = [axs]

for i, nivel in enumerate(niveis):
    df_nivel = df_users[df_users['automation_level'] == nivel]
    contagem = df_nivel['age_group'].value_counts().reindex(categorias,
↳fill_value=0)

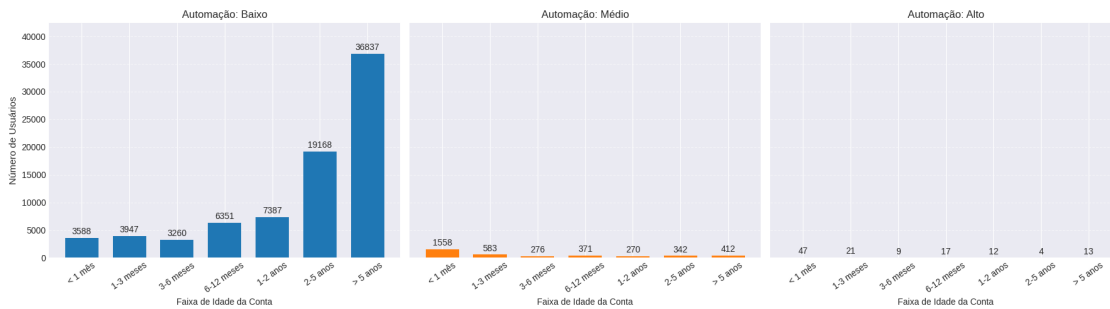
    container = axs[i].bar(categorias, contagem.values,
                           color=cores[i % len(cores)], width=0.7)

    labels_vals = [f'{val}' if val > 0 else '' for val in contagem.values]
    axs[i].bar_label(container, labels=labels_vals, label_type='edge',
↳padding=2, fontsize=10)

    axs[i].set_title(f"Automação: {nivel}", fontsize=12)
    axs[i].set_xlabel("Faixa de Idade da Conta", fontsize=10)
    axs[i].tick_params(axis='x', rotation=35)
    axs[i].grid(axis='y', linestyle='--', alpha=0.6)
    axs[i].margins(y=0.15)

axs[0].set_ylabel("Número de Usuários", fontsize=11)
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳distribuicao_idade_conta_por_automacao.png',
            dpi=300, bbox_inches='tight')
plt.show()

```



```
[97]: #-----
# Distribuição de contagem de URLs por nível de automação
#-----

# Encontrar valores mínimos e máximos para o eixo X
min_url = df_users['url_count'].min()
max_url = df_users['url_count'].max()

niveis = df_users['automation_level'].unique()
n_niveis = len(niveis)
cores = plt.cm.tab10.colors

# Criar figura com subplots compartilhando eixos
fig, axs = plt.subplots(1, n_niveis, figsize=(15, 5),
                        sharey=True, sharex=True,
                        gridspec_kw={'wspace': 0.05})

# Se só houver 1 nível, transformar em array para manter consistência
if n_niveis == 1:
    axs = [axs]

for i, nivel in enumerate(niveis):
    df_nivel = df_users[df_users['automation_level'] == nivel]
    contagem = df_nivel['url_count'].value_counts().sort_index()

    # Plot com barras
    container = axs[i].bar(contagem.index, contagem.values,
                           color=cores[i % len(cores)], width=0.7)
    axs[i].set_title(f"Automação: {nivel}", fontsize=12)
    axs[i].set_xlabel("URLs por usuário", fontsize=10)

    # Escala logarítmica
    axs[i].set_yscale("log")

    # Adicionar rótulos apenas para barras com valor > 5
```

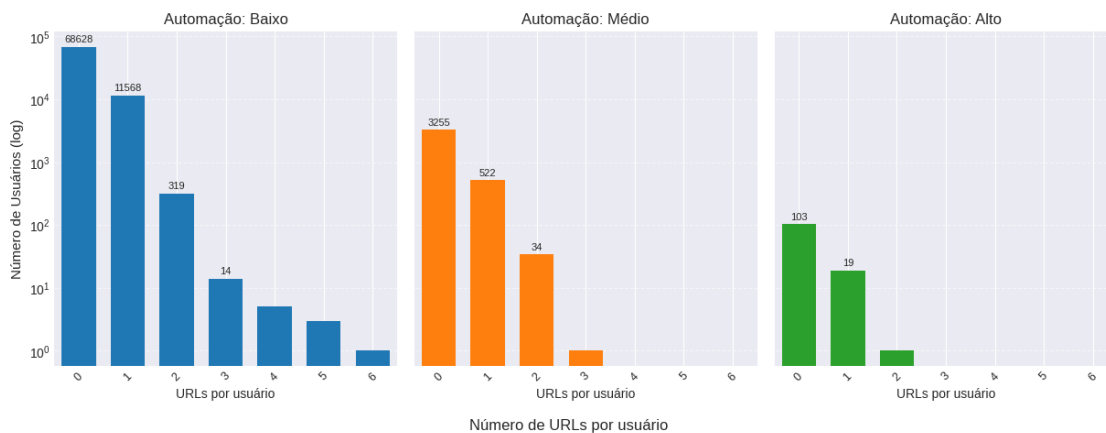
```

labels = [f'{val}' if val > 5 else '' for val in contagem.values]
axs[i].bar_label(container,
                  label_type='edge',
                  padding=2,
                  fontsize=8,
                  labels=labels)

# Grade e limites
axs[i].grid(axis='y', linestyle='--', alpha=0.6)
axs[i].set_xlim(min_url - 0.5, max_url + 0.5)
axs[i].tick_params(axis='x', rotation=45, labels=9)

# Configurações gerais
axs[0].set_ylabel("Número de Usuários (log)", fontsize=11)
# plt.suptitle("Distribuição de URLs por Nível de Automação", fontsize=14)
fig.text(0.5, 0.01, 'Número de URLs por usuário', ha='center', fontsize=12)
plt.subplots_adjust(bottom=0.15)
plt.show()

```



```

[98]: #-----
# Distribuição de contagem de Hashtags por nível de automação (escala log)
#-----

# Encontrar valores mínimos e máximos para o eixo X
min_url = df_users['hashtags_count'].min()
max_url = df_users['hashtags_count'].max()

niveis = df_users['automation_level'].unique()
n_niveis = len(niveis)
cores = plt.cm.tab10.colors

```

```

# Criar figura com subplots compartilhando eixos
fig, axs = plt.subplots(1, n_niveis, figsize=(15, 5),
                        sharey=True, sharex=True,
                        gridspec_kw={'wspace': 0.05})

# Se só houver 1 nível, transformar em array para manter consistência
if n_niveis == 1:
    axs = [axs]

for i, nivel in enumerate(niveis):
    df_nivel = df_users[df_users['automation_level'] == nivel]
    contagem = df_nivel['hashtags_count'].value_counts().sort_index()

    # Plot com barras
    container = axs[i].bar(contagem.index, contagem.values,
                           color=cores[i % len(cores)], width=0.7)
    axs[i].set_title(f"Automação: {nivel}", fontsize=12)
    axs[i].set_xlabel("Hashtags por usuário", fontsize=10)

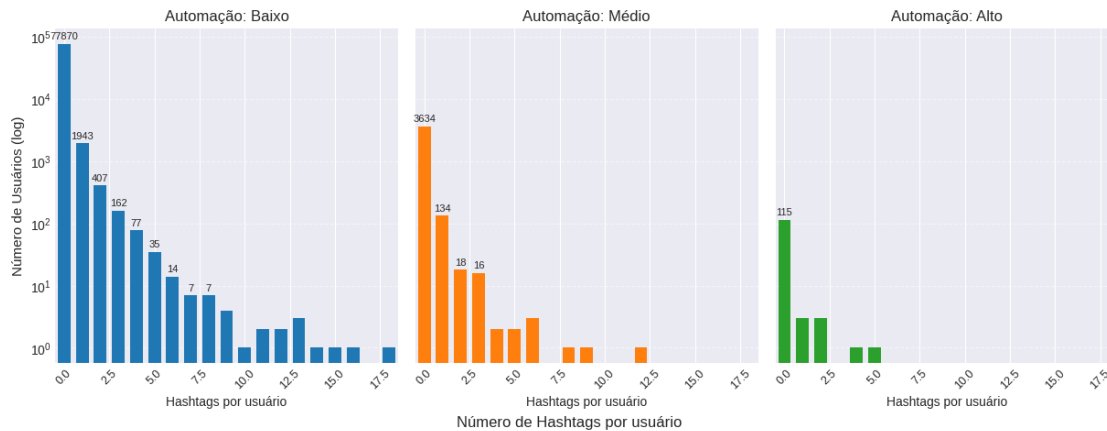
    # Escala logarítmica
    axs[i].set_yscale("log")

    # Adicionar rótulos apenas para barras com valor > 5
    labels = [f'{val}' if val > 5 else '' for val in contagem.values]
    axs[i].bar_label(container,
                     label_type='edge',
                     padding=2,
                     fontsize=8,
                     labels=labels)

    # Grade e limites
    axs[i].grid(axis='y', linestyle='--', alpha=0.6)
    axs[i].set_xlim(min_url - 0.5, max_url + 0.5)
    axs[i].tick_params(axis='x', rotation=45, labelsize=9)

# Configurações gerais
axs[0].set_ylabel("Número de Usuários (log)", fontsize=11)
fig.text(0.5, 0.01, 'Número de Hashtags por usuário', ha='center', fontsize=12)
plt.subplots_adjust(bottom=0.15)
plt.show()

```



```
[99]: #-----
# Distribuição de tamanho dos tweets por nível de automação (escala log)
#-----

# Criação da coluna de categoria de comprimento
df_users['length_category'] = pd.cut(
    df_users['tweet_length'],
    bins=[0, 50, 140, 500],
    right=True,
    labels=['Pequeno (<50)', 'Médio (51-140)', 'Longo (141-280)']
)

# Configurações gerais
niveis = df_users['automation_level'].unique()
n_niveis = len(niveis)
cores = plt.cm.tab10.colors
categorias = ['Pequeno (<50)', 'Médio (51-140)', 'Longo (141-280)']

# Criar figura com subplots
fig, axs = plt.subplots(1, n_niveis, figsize=(15, 5),
                        sharey=True,
                        gridspec_kw={'wspace': 0.1})

# Caso especial para único nível
if n_niveis == 1:
    axs = [axs]

# Plot para cada nível
for i, nivel in enumerate(niveis):
    df_nivel = df_users[df_users['automation_level'] == nivel]
    contagem = df_nivel['length_category'].value_counts().reindex(categorias,
        ↪fill_value=0)
```

```

container = axs[i].bar(categorias, contagem.values,
                        color=cores[i % len(cores)], width=0.7)

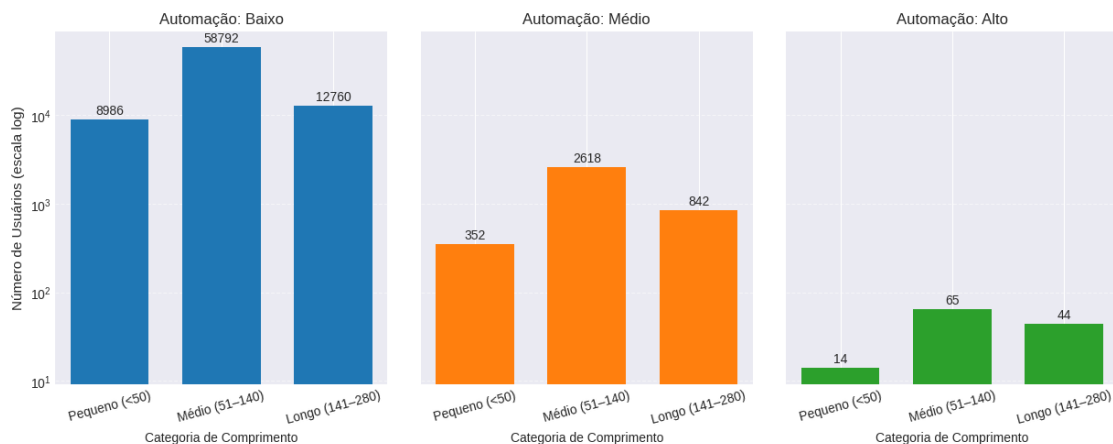
# Rótulos nas barras
labels = [f'{val}' if val > 0 else '' for val in contagem.values]
axs[i].bar_label(container, labels=labels, label_type='edge', padding=2,
→ fontsize=10)

axs[i].set_title(f"Automação: {nivel}", fontsize=12)
axs[i].set_xlabel("Categoria de Comprimento", fontsize=10)
axs[i].tick_params(axis='x', rotation=15)
axs[i].grid(axis='y', linestyle='--', alpha=0.6)

# Escala logarítmica no eixo Y
axs[i].set_yscale("log")

# Configurações gerais
axs[0].set_ylabel("Número de Usuários (escala log)", fontsize=11)
plt.show()

```



1.3.4 5.2.3 Clusters

```

[100]: # Tweets, respostas e retweets
df = df_raw.copy()

```

```

[101]: # =====
# IDADE MÉDIA DA CONTA POR CLUSTER
# =====

import matplotlib.pyplot as plt

```

```

import pandas as pd

# Calcular idade em anos (usando a mesma base de dados)
df['account_age_years'] = df['account_age_days'] / 365

# Filtrar apenas registros com cluster não nulo
df_com_cluster = df[df['cluster'].notna()]

# 1. Calcular a média por cluster (ordenado pelo número do cluster)
medias = df_com_cluster.groupby('cluster')['account_age_years'].mean().
    ↪sort_index()

# 2. Ver resultado em tabela
print("Idade média da conta (anos) por cluster:")
medias.round(2)

```

Idade média da conta (anos) por cluster:

```

[101]: cluster
0.0      6.72
1.0      4.81
2.0      4.91
3.0      5.94
4.0      4.52
5.0      6.55
6.0      6.59
7.0      6.04
8.0      6.01
9.0      3.78
10.0     5.26
11.0     6.44
Name: account_age_years, dtype: float64

```

```

[102]: import pandas as pd
import matplotlib.pyplot as plt

# Filtrar apenas registros que têm cluster (63.580 linhas)
df_cluster = df[df['cluster'].notna()]

# Tabela cruzada: cluster (linhas) x tópico (colunas)
tabela = pd.crosstab(df_cluster['cluster'], df_cluster['topic'])

# Ver as primeiras linhas
print("Distribuição de tópicos por cluster:")
tabela

```

Distribuição de tópicos por cluster:

```
[102]: topic      Ataque Hacker  Auditoria  Celular na Cabine  Conexão com a Internet  \
cluster
0.0          0          0          0          0
1.0          0          0          0          0
2.0         88        1210        195        208
3.0        776          1          4          2
4.0          9          0          0          9
5.0          2          0          0          0
6.0          0          0          0          0
7.0        102        179        131        295
8.0          1          0          0          0
9.0          3          0          0          3
10.0         0          1          2          0
11.0         0          1          0          0
```

```
topic      Código-Fonte  Disputa Eleitoral  Fake News  Fraude  \
cluster
0.0          0          39          0          0
1.0          0          1          0          0
2.0        670        406        266        621
3.0          6          9          0          5
4.0          3          0          3          2
5.0          1         12          0          1
6.0          0          0          1          0
7.0        253        2195        223        381
8.0          0          2          0          1
9.0          4          1          0          1
10.0         1          0          0          2
11.0         4          3          0          0
```

```
topic      Live do Argentino  Logs da Urna  Militares e Fiscalização  \
cluster
0.0          0          0          0
1.0          0          0          0
2.0        1074        1123        181
3.0          0          3          2
4.0          0          1          1
5.0          0          0          2
6.0          0          0          0
7.0        103        146        322
8.0          0          0          1
9.0          0          2          0
10.0         0          0          0
11.0         0          0          0
```

```
topic      Mídias Sociais  Narrativa Política  Narrativa Religiosa  Outlier  \
cluster
```

0.0	0	1	0	0
1.0	0	2	1	0
2.0	795	4156	187	19
3.0	11	49	17	0
4.0	2	6	1	0
5.0	1	6	5	0
6.0	32	1	1	0
7.0	537	2702	1848	3
8.0	1	3	2	0
9.0	2	3	1	0
10.0	2	2	2	0
11.0	1	2	1	0

topic	Pesquisas Eleitorais	Urna Eletrônica	Uso da Urna	Voto Impresso
cluster				
0.0	0	4	2	0
1.0	0	1	148	0
2.0	1141	17892	5246	451
3.0	2	368	41	0
4.0	1	41	12	2
5.0	2	79	540	1
6.0	0	10	5	0
7.0	586	10186	4418	129
8.0	0	17	328	0
9.0	2	12	4	1
10.0	0	31	13	1
11.0	1	144	4	1

```
[103]: # Filtrar apenas registros com cluster
df_cluster = df[df['cluster'].notna()]
# Tabela cruzada: cluster x sentimento
tabela_sentimento = pd.crosstab(df_cluster['cluster'], df_cluster['sentiment'])
```

```
[104]: # Qual sentimento é o mais frequente em cada cluster?
sentimento_dominante = tabela_sentimento.idxmax(axis=1)
contagem_max = tabela_sentimento.max(axis=1)

resultado = pd.DataFrame({
    'Cluster': sentimento_dominante.index,
    'Sentimento_Dominante': sentimento_dominante.values,
    'Qtd_Tweets': contagem_max.values,
    'Total_Cluster': tabela_sentimento.sum(axis=1).values
})
resultado
```

```
[104]: Cluster Sentimento_Dominante Qtd_Tweets Total_Cluster
0      0.0      Negativo      41      46
```

1	1.0	Neutro	139	153
2	2.0	Neutro	22617	35929
3	3.0	Negativo	1031	1296
4	4.0	Neutro	46	93
5	5.0	Neutro	558	652
6	6.0	Neutro	40	50
7	7.0	Neutro	12278	24739
8	8.0	Negativo	335	356
9	9.0	Neutro	26	39
10	10.0	Neutro	29	57
11	11.0	Neutro	112	162

```
[105]: # df = pd.read_csv('/content/drive/MyDrive/TCC/Data/TUEBR222.csv',
#                 sep=",", low_memory=False,
#                 parse_dates=['tweet_created_at', 'user_created_at',
#                 ↪ 'collection_date'])
# df = df[df['preprocessed_tweet'].notna()]
# df = df[df['score_final'].notna()]
```

```
[106]: # import pandas as pd
# import numpy as np

# def parse_hashtags(valor):
#     """Converte qualquer entrada em lista de hashtags (strings)"""

#     # 1. Se for numpy array, converte para lista e chama novamente
#     if isinstance(valor, np.ndarray):
#         return parse_hashtags(valor.tolist())

#     # 2. Se for lista Python, trata cada elemento
#     if isinstance(valor, list):
#         # Converte todos para string e filtra vazios
#         return [str(t).strip() for t in valor if str(t).strip() != '']

#     # 3. Agora que sabemos que não é array nem lista, podemos testar NaN
#     if pd.isna(valor):
#         return []

#     # 4. Se for string, separa por vírgula
#     if isinstance(valor, str):
#         if valor.strip() == '':
#             return []
#         return [t.strip() for t in valor.split(',') if t.strip() != '']

#     # 5. Outros tipos (ex: int, float) - converte para string
#     return [str(valor).strip()] if str(valor).strip() != '' else []
```

```
[107]: def parse_hashtags(valor):
        """Converte qualquer entrada em lista de hashtags (strings)."""
        import numpy as np
        import pandas as pd

        # 1. numpy array -> lista (o tolist já converte NaN para float('nan'))
        if isinstance(valor, np.ndarray):
            return parse_hashtags(valor.tolist())

        # 2. Lista Python: trata cada elemento individualmente
        if isinstance(valor, list):
            resultado = []
            for t in valor:
                # pula nulos ANTES de virar string
                if t is None:
                    continue
                try:
                    if pd.isna(t):
                        continue
                except (TypeError, ValueError):
                    # pd.isna em listas/arrays retorna array; ignora
                    pass

                s = str(t).strip()
                if s and s.lower() not in ('nan', 'none', 'nat', '<na>'):
                    resultado.append(s)
            return resultado

        # 3. Escalar NaN / None / NaT
        try:
            if valor is None or pd.isna(valor):
                return []
        except (TypeError, ValueError):
            pass

        # 4. String: separa por vírgula
        if isinstance(valor, str):
            if valor.strip() == '':
                return []
            return [t.strip() for t in valor.split(',') if t.strip() != '']

        # 5. Outros tipos (int, float, etc.)
        s = str(valor).strip()
        return [s] if s else []
```

```
[108]: import ast
```

```
def to_list(x):
    if isinstance(x, list):
        return x
    if isinstance(x, str):
        s = x.strip()
        if s in ('', '[]', 'nan', 'None'):
            return []
        try:
            return ast.literal_eval(s) # "'['a','b']" -> ['a','b']
        except (ValueError, SyntaxError):
            return [s]
    return []

df['hashtags'] = df['hashtags'].apply(to_list)
df['hashtags_list'] = df['hashtags'].apply(parse_hashtags)
df['num_hashtags'] = df['hashtags_list'].apply(len)
```

```
[109]: df['hashtags'] = df['hashtags'].apply(to_list)
df['hashtags_list'] = df['hashtags'].apply(parse_hashtags)
df['num_hashtags'] = df['hashtags_list'].apply(len)

# Explodir (cria uma linha por hashtag)
df_exploded = df[df['num_hashtags'] > 0].explode('hashtags_list')
df_exploded.rename(columns={'hashtags_list': 'hashtag'}, inplace=True)

print(f"Total de tweets: {len(df)}")
print(f"Tweets com pelo menos 1 hashtag: {len(df_exploded)}")
print(f"Hashtags únicas: {df_exploded['hashtag'].nunique()}")
```

Total de tweets: 110978
Tweets com pelo menos 1 hashtag: 7009
Hashtags únicas: 1793

```
[110]: # Filtrar apenas registros que têm cluster
df_exploded_cluster = df_exploded[df_exploded['cluster'].notna()]

print(f"Total de registros com cluster: {len(df_exploded_cluster)}")
```

Total de registros com cluster: 3558

```
[111]: # Para cada cluster, pegar as top 5 hashtags mais frequentes
top_por_cluster = (df_exploded_cluster.groupby(['cluster', 'hashtag'])
                    .size()
                    .groupby('cluster', group_keys=False)
                    .nlargest(5)
                    .reset_index(name='freq'))

# Ordenar por cluster para visualização
```

```
top_por_cluster = top_por_cluster.sort_values(['cluster', 'freq'],
↪ascending=[True, False])
```

```
# Visualizar como tabela
```

```
top_por_cluster
```

```
[111]:
```

	cluster	hashtag	freq
0	0.0	#Eleicoes2022	1
1	2.0	#BrazilWasStolen	80
2	2.0	#Bolsonaro22	78
3	2.0	#MourãoSenador	62
4	2.0	#Vote100	62
5	2.0	#PAZ	51
6	3.0	#Voto80	1
7	3.0	#VotoLeoNo1turno	1
8	7.0	#LulaPresidente13	155
9	7.0	#LulaNo1ºTurno	151
10	7.0	#BrasilDaEsperanca	145
11	7.0	#Eleições2022	84
12	7.0	#13	69
13	10.0	#13	1
14	10.0	#BolsonaroReeleitoNoPrimeiroTurno	1
15	11.0	#Eleicoes2022	1
16	11.0	#PolianaMoça	1

```
[112]: # Filtrar apenas registros com cluster não nulo
```

```
df_cluster = df[df['cluster'].notna()]
```

```
# Tabela cruzada (clusters como linhas, níveis de automação como colunas)
```

```
tabela_automacao = pd.crosstab(df_cluster['cluster'],
```

```
↪df_cluster['automation_level'])
```

```
# Ordenar clusters numericamente
```

```
tabela_automacao = tabela_automacao.sort_index()
```

```
print("Distribuição de níveis de automação por cluster:")
```

```
tabela_automacao
```

Distribuição de níveis de automação por cluster:

```
[112]:
```

automation_level	Alto	Baixo	Médio
cluster			
0.0	0	46	0
1.0	0	153	0
2.0	25	34112	1792
3.0	0	1280	16
4.0	0	85	8
5.0	0	651	1

6.0	1	48	1
7.0	24	23891	824
8.0	0	349	7
9.0	1	36	2
10.0	0	51	6
11.0	0	159	3

1.4 5.3 Desinformação

1.4.1 5.3.1 Geral

```
[113]: # =====
# ANÁLISE DA DESINFORMAÇÃO (OBJETIVO ii)
# =====

# 1. Visão geral da classificação
total_desinfo = df_com_retweets['is_disinformation'].sum()
total_nao_desinfo = len(df_com_retweets) - total_desinfo
print(f"Total de tweets: {len(df_com_retweets)}")
print(f"Desinformação: {total_desinfo} ({total_desinfo/len(df_com_retweets):.2%})")
print(f"Não desinformação: {total_nao_desinfo} ({total_nao_desinfo/len(df_com_retweets):.2%})")
```

Total de tweets: 110978
Desinformação: 9905 (8.93%)
Não desinformação: 101073 (91.07%)

```
[114]: # Paenas desinformação
df_com_retweets_des = df_com_retweets[df_com_retweets['is_disinformation'] == 1].copy()

# Categorizar os tweets
conditions= [
    (df_com_retweets_des['is_retweet'] == 1), (df_com_retweets_des['reply_to_tweet_id'].notna() &
    (df_com_retweets_des['is_retweet'] == 0)),
    (df_com_retweets_des['is_retweet'] == 0) &
    (df_com_retweets_des['reply_to_tweet_id'].isna())
]
choices = ['Retweet', 'Reply', 'Original']
df_com_retweets_des['tweet_type'] = np.select(conditions, choices,
    default='Original')

# Agrupar por data e tipo
daily_counts = df_com_retweets_des.groupby([
    pd.Grouper(key='tweet_created_at', freq='D'),
    'tweet_type'
]).size().unstack(fill_value=0)
```

```
[115]: #-----
# Distribuição Entre os Tipos de Tweets:
# Originais, Retweets e Replies(Comentários) (DESINFORMÇÃO)
#-----

# Calcular a distribuição incluindo replies
tweet_distribution = df_com_retweets_des['tweet_type'].
    ↳value_counts(normalize=True)

# Ordem desejada e cores
order = ['Original', 'Retweet', 'Reply']
colors = ['#1f77b4', '#ff7f0e', '#2ca02c'] # Cores tradicionais

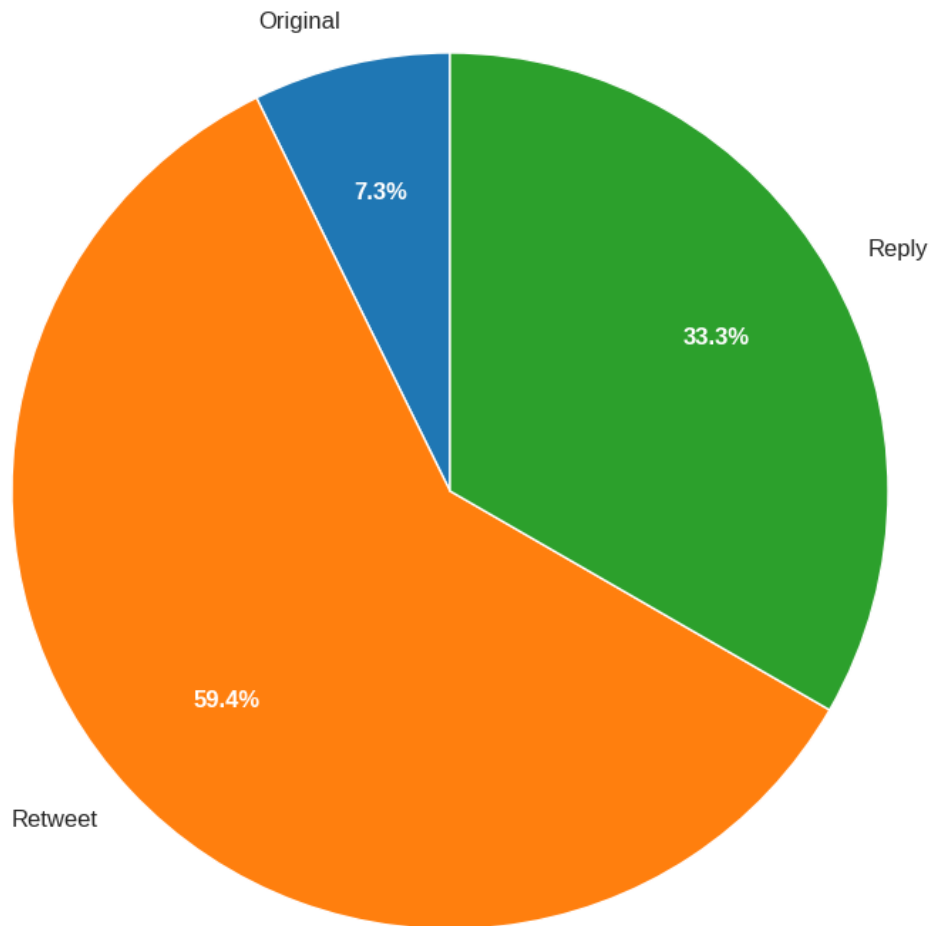
# Criar figura
plt.figure(figsize=(8, 8))

# Plotar gráfico de pizza
plt.pie(
    tweet_distribution[order],
    labels=order,
    colors=colors,
    autopct='%1.1f%%',
    startangle=90,
    wedgeprops={'linewidth': 1, 'edgecolor': 'white'},
    textprops={'fontsize': 12},
    pctdistance=0.7 # Posição das porcentagens
)

# Destacar as porcentagens
for text in plt.gca().texts:
    if '%' in text.get_text():
        text.set_color('white')
        text.set_fontweight('bold')

# Ajustar layout
plt.tight_layout()

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/tipos_de_tweets_desinformacao.
    ↳png', dpi=300, bbox_inches='tight')
plt.show()
```



```
[116]: df_com_retweets_des['tweet_type'].value_counts()
```

```
[116]: tweet_type
Retweet    5883
Reply      3301
Original    721
Name: count, dtype: int64
```

```
[117]: #-----
# Distribuição de tweets ao longo do tempo por tipo (DESINFORMAÇÃO)
# com Fases Eleitorais
#-----
```

```

import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

# Configurações de estilo
plt.style.use('seaborn-v0_8-darkgrid')
palette = ['#1f77b4', '#ff7f0e', '#2ca02c'] # Azul, Laranja, Verde

# --- Define os intervalos de campanha (como áreas) ---
df_com_retweets_des['tweet_created_at'] = pd.
    ↳to_datetime(df_com_retweets_des['tweet_created_at'])

areas = {
    "Campanha 1º Turno": (df_com_retweets_des['tweet_created_at'].min().
    ↳strftime('%Y-%m-%d'), "2022-10-02"),
    "Campanha 2º Turno": ("2022-10-02", "2022-10-30"),
    "Pós-eleição": ("2022-10-30", df_com_retweets_des['tweet_created_at'].
    ↳max().strftime('%Y-%m-%d'))
}

# Define os dias de votação (como linhas)
eventos = {
    "1º Turno (Votação)": "2022-10-02",
    "2º Turno (Votação)": "2022-10-30"
}

# Converte datas
areas = {fase: (pd.to_datetime(inicio), pd.to_datetime(fim)) for fase,
    ↳ (inicio, fim) in areas.items()}
eventos = {nome: pd.to_datetime(data) for nome, data in eventos.items()}

# Cores para áreas
colors = {
    "Campanha 1º Turno": "#f2a640",
    "Campanha 2º Turno": "#cce5ff",
    "Pós-eleição": "#88896e"
}

# Cores para eventos (linhas)
colors_day = {
    "1º Turno (Votação)": "#800080",
    "2º Turno (Votação)": "#ec9d01"
}

# --- Início do plot ---
plt.figure(figsize=(16, 8))
ax = plt.gca()

```

```

# Áreas das campanhas (atrás das linhas)
for fase, (inicio, fim) in areas.items():
    ax.axvspan(inicio, fim, alpha=0.3, label=fase, color=colors[fase], zorder=0)

# Linhas dos dias de votação
for nome, data in eventos.items():
    ax.axvline(data, color=colors_day[nome], linestyle='--', linewidth=1.9,
               label=nome, zorder=1)

# Plotar as linhas com estilos diferentes para melhor distinção
line_styles = ['-', '--', '-.']
for i, col in enumerate(['Original', 'Retweet', 'Reply']):
    daily_counts[col].plot(
        kind='line',
        linewidth=2.5,
        color=palette[i],
        linestyle=line_styles[i],
        marker='o',
        markersize=5,
        markevery=7,
        ax=ax,
        label=col,
        zorder=3
    )

# Rótulos
plt.xlabel('Dia da Publicação', fontsize=12, labelpad=10)
plt.ylabel('Quantidade de Tweets', fontsize=12, labelpad=10)

# Garante que o eixo x inclua todas as áreas e eventos
datas_inicio_fim = [inicio for inicio, _ in areas.values()] + [fim for _, fim in
    ↪ areas.values()]
datas_eventos = list(eventos.values())
data_min = min(datas_inicio_fim + datas_eventos)
data_max = max(datas_inicio_fim + datas_eventos)
plt.xlim(data_min, data_max)

# Configurar ticks específicos no eixo X
datas_especificas = [
    '30/08/2022', # Data do primeiro dado coletado
    '15/09/2022',
    '02/10/2022', # 1º Turno (Votação)
    '15/10/2022',
    '30/10/2022', # 2º Turno (Votação)
    '15/11/2022',
    '02/12/2022' # Último dia de coleta

```

```

]
datas_ticks = [pd.to_datetime(data, format='%d/%m/%Y') for data in
↳datas_especificas]
rotulos_ticks = [date.strftime('%d/%m') for date in datas_ticks]

plt.xticks(datas_ticks, rotulos_ticks)

# Formata o eixo X
ax.xaxis.set_major_formatter(mdates.DateFormatter('%d/%m'))

# Grid
ax.yaxis.grid(True, linestyle='--', alpha=0.7)
ax.xaxis.grid(False)

# Melhorar legenda (título "Tipo de Tweet" + fases + eventos)
legend = ax.legend(
    title='Tipo de Tweet / Fases Eleitorais',
    title_fontsize=12,
    fontsize=10,
    loc='upper right',
    frameon=True,
    framealpha=1,
    edgecolor='#444444',
    ncol=2
)

# Ajustar layout
plt.tight_layout()

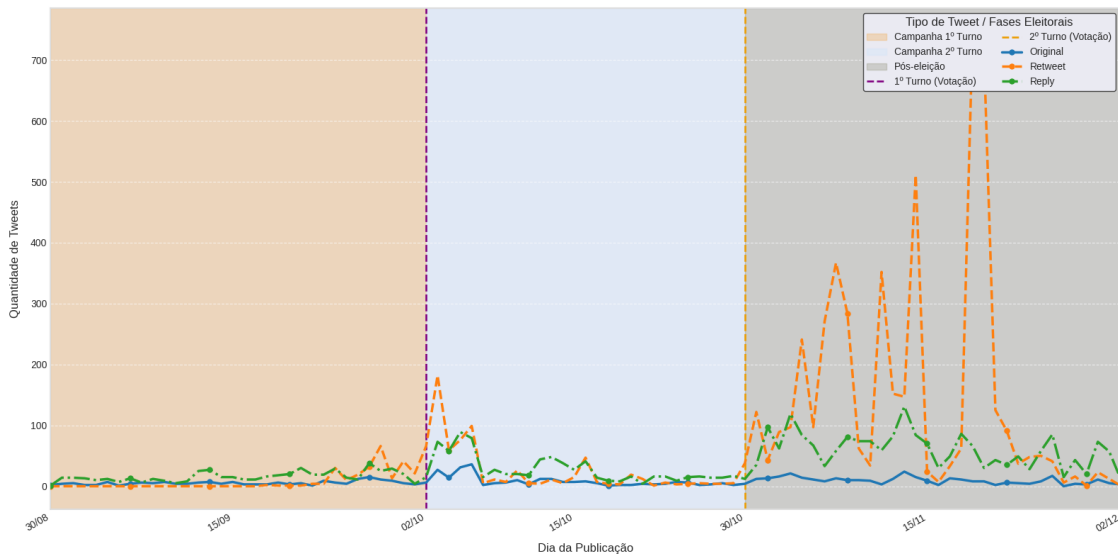
# Destacar eixos
for spine in ax.spines.values():
    spine.set_edgecolor('#dddddd')
    spine.set_linewidth(1.5)

# Salvar e mostrar
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳distribuicao_tweets_desinformacao_por_tipo.png',
            dpi=300, bbox_inches='tight')
plt.show()

```

/usr/local/lib/python3.13/dist-packages/pandas/plotting/_matplotlib/core.py:981:
UserWarning:

This axis already has a converter set and is updating to a potentially
incompatible converter



1.4.2 5.3.2 Produção

```
[118]: # Considerando apenas tweets e respostas
df = df_sem_retweets.copy()

# 2. Visão produção da classificação
total_desinfo = df['is_disinformation'].sum()
total_nao_desinfo = len(df) - total_desinfo
print(f"Total de tweets: {len(df)}")
print(f"Desinformação: {total_desinfo} ({total_desinfo/len(df):.2%})")
print(f"Não desinformação: {total_nao_desinfo} ({total_nao_desinfo/len(df):.2%})")
```

Total de tweets: 41414

Desinformação: 4022 (9.71%)

Não desinformação: 37392 (90.29%)

5.3.2.1 Distribuição temporal

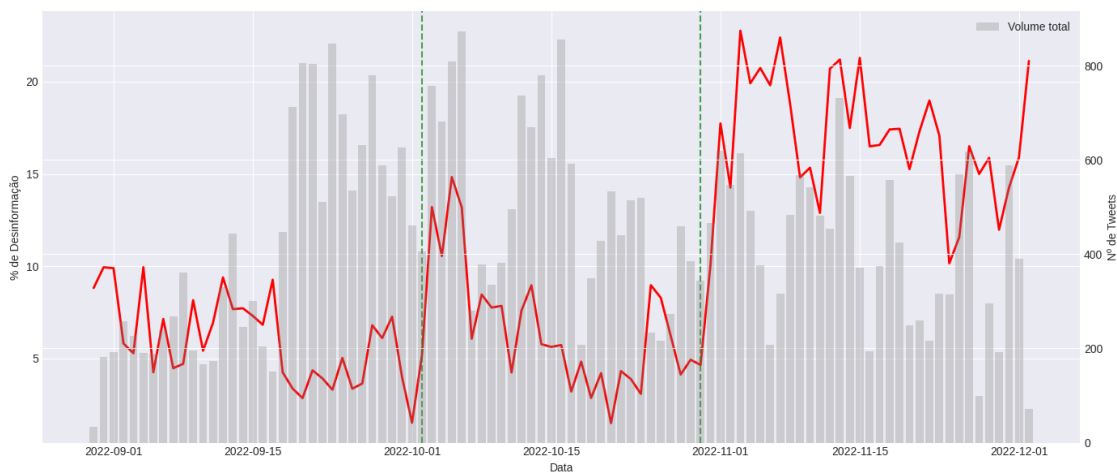
```
[119]: # 2. Distribuição temporal da desinformação (proporção diária)
desinfo_diario = df.groupby(df['tweet_created_at'].dt.date)['is_disinformation'].
    .mean() * 100
volume_diario = df.groupby(df['tweet_created_at'].dt.date).size()

fig, ax = plt.subplots(figsize=(14,6))
ax.plot(desinfo_diario.index, desinfo_diario, color='red', linewidth=2, label='%↳Desinformação')
ax.set_ylabel('% de Desinformação')
ax2 = ax.twinx()
```

```

ax2.bar(volume_diario.index, volume_diario, alpha=0.3, color='gray',
        ↳label='Volume total')
ax2.set_ylabel('Nº de Tweets')
ax.set_xlabel('Data')
# Destacar dias de votação
for dia in ['2022-10-02', '2022-10-30']:
    ax.axvline(pd.to_datetime(dia), color='green', linestyle='--', alpha=0.7)
# plt.title('Proporção diária de desinformação sobre as urnas')
plt.legend()
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/desinformacao_temporal.png',
        ↳dpi=300)
plt.show()

```



```

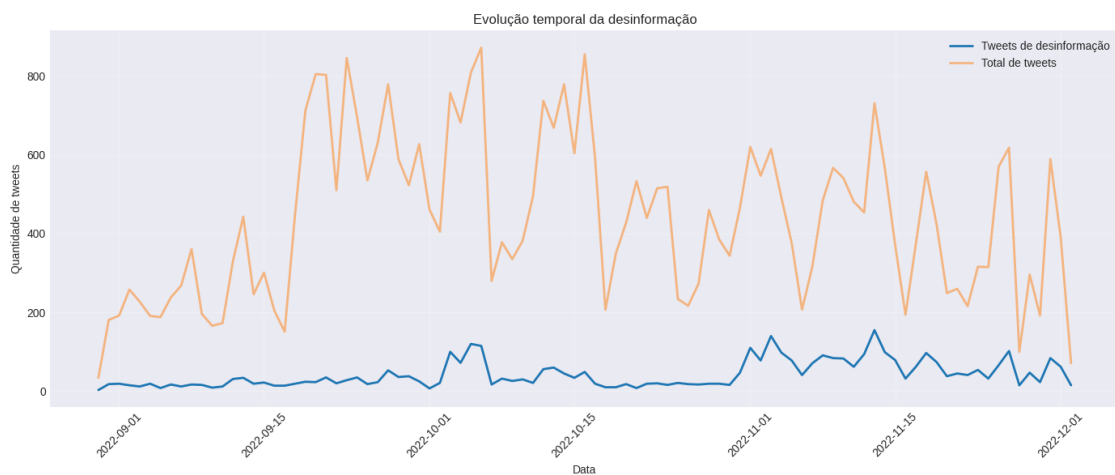
[120]: # =====
# ANÁLISE 1 - DESINFORMAÇÃO AO LONGO DO TEMPO
# =====
df['tweet_created_at'] = pd.to_datetime(df['tweet_created_at'])
# Quantidade total e quantidade de desinformação por dia
desinfo_dia = (
df.groupby(df['tweet_created_at'].dt.date)
.agg(
total_tweets=('tweet_id', 'count'),
tweets_desinformacao=('is_disinformation', 'sum')
)
.reset_index()
)
desinfo_dia['tweet_created_at'] = pd.to_datetime(
desinfo_dia['tweet_created_at']
)

```

```

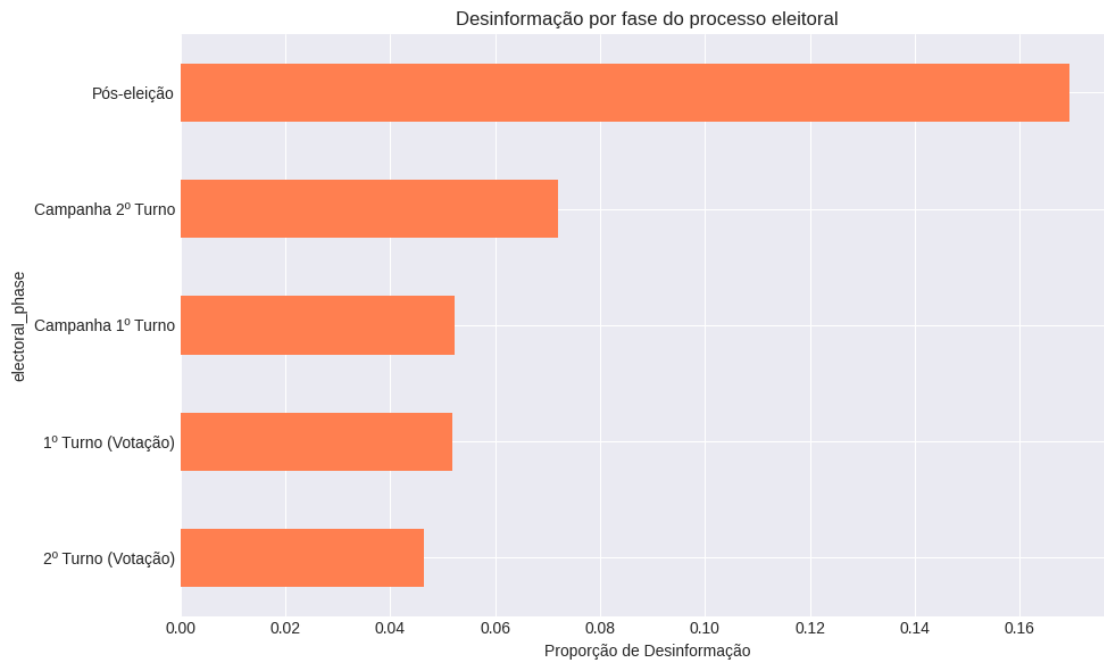
# Proporção de desinformação
desinfo_dia['prop_desinformacao'] = (
desinfo_dia['tweets_desinformacao'] /
desinfo_dia['total_tweets']
)
# Gráfico da quantidade
plt.figure(figsize=(14, 6))
plt.plot(desinfo_dia['tweet_created_at'],
desinfo_dia['tweets_desinformacao'],
linewidth=2,
label='Tweets de desinformação'
)
plt.plot(
desinfo_dia['tweet_created_at'],
desinfo_dia['total_tweets'],
linewidth=2,
alpha=0.5,
label='Total de tweets'
)
plt.xlabel('Data')
plt.ylabel('Quantidade de tweets')
plt.title('Evolução temporal da desinformação')
plt.xticks(rotation=45)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```



[121]: # 3. Desinformação por fase eleitoral

```
fase_desinfo = df.groupby('electoral_phase')['is_disinformation'].mean().
    ↪sort_values()
fase_desinfo.plot(kind='barh', figsize=(10,6), color='coral')
plt.xlabel('Proporção de Desinformação')
plt.title('Desinformação por fase do processo eleitoral')
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/desinfo_por_fase.png', dpi=300)
plt.show()
```



5.3.2.2 Tamanho tweet

```
[122]: import pandas as pd
import matplotlib.pyplot as plt

# 1. Criar categoria de comprimento
df['length_category'] = pd.cut(
    df['tweet_length'],
    bins=[0, 50, 140, 500],
    right=True,
    labels=['Pequeno (<50)', 'Médio (51-140)', 'Longo (141-280)'],
    include_lowest=True
)

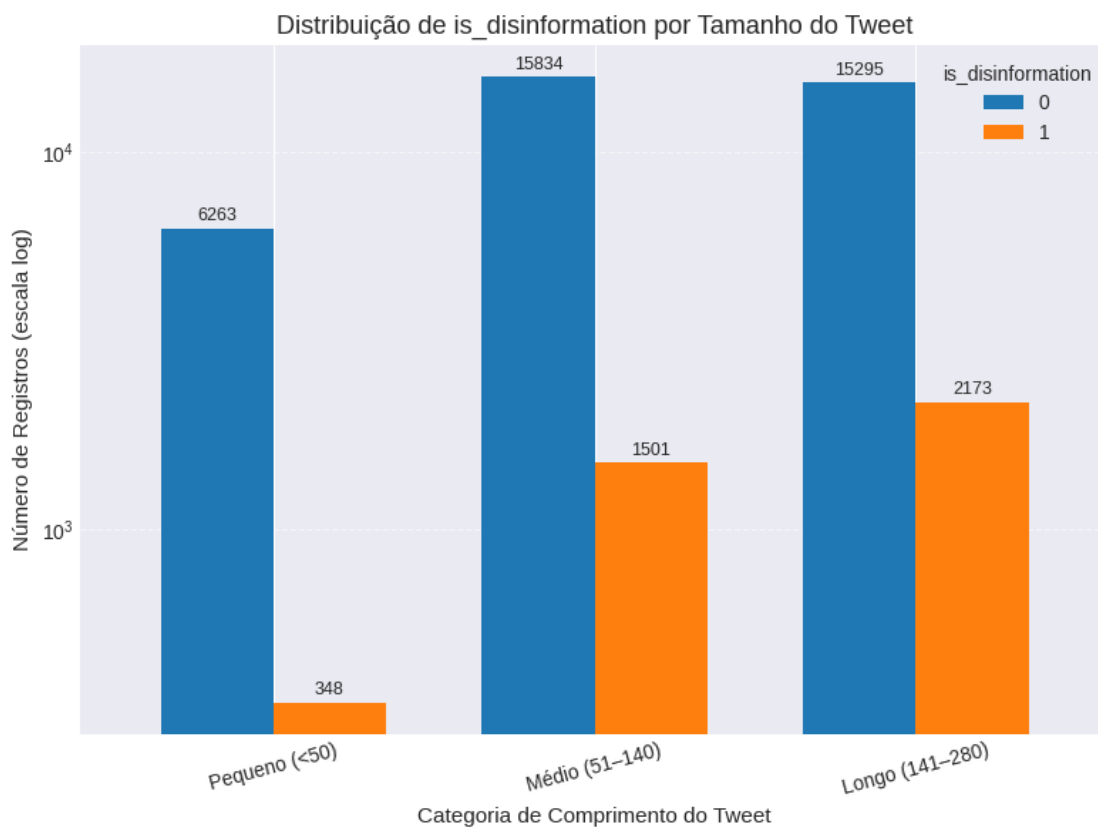
# 2. Contagem cruzada
tabela = pd.crosstab(df['length_category'], df['is_disinformation'])
```

```
# 3. Gráfico único
fig, ax = plt.subplots(figsize=(8, 6))
tabela.plot(kind='bar', ax=ax, width=0.7, rot=15)

for container in ax.containers:
    ax.bar_label(container, label_type='edge', padding=2, fontsize=9)

ax.set_ylabel('Número de Registros (escala log)', fontsize=11)
ax.set_xlabel('Categoria de Comprimento do Tweet', fontsize=11)
ax.set_title('Distribuição de is_disinformation por Tamanho do Tweet',
             ↪ fontsize=13)
ax.legend(title='is_disinformation')
ax.grid(axis='y', linestyle='--', alpha=0.6)
ax.set_yscale('log')

plt.tight_layout()
plt.show()
```

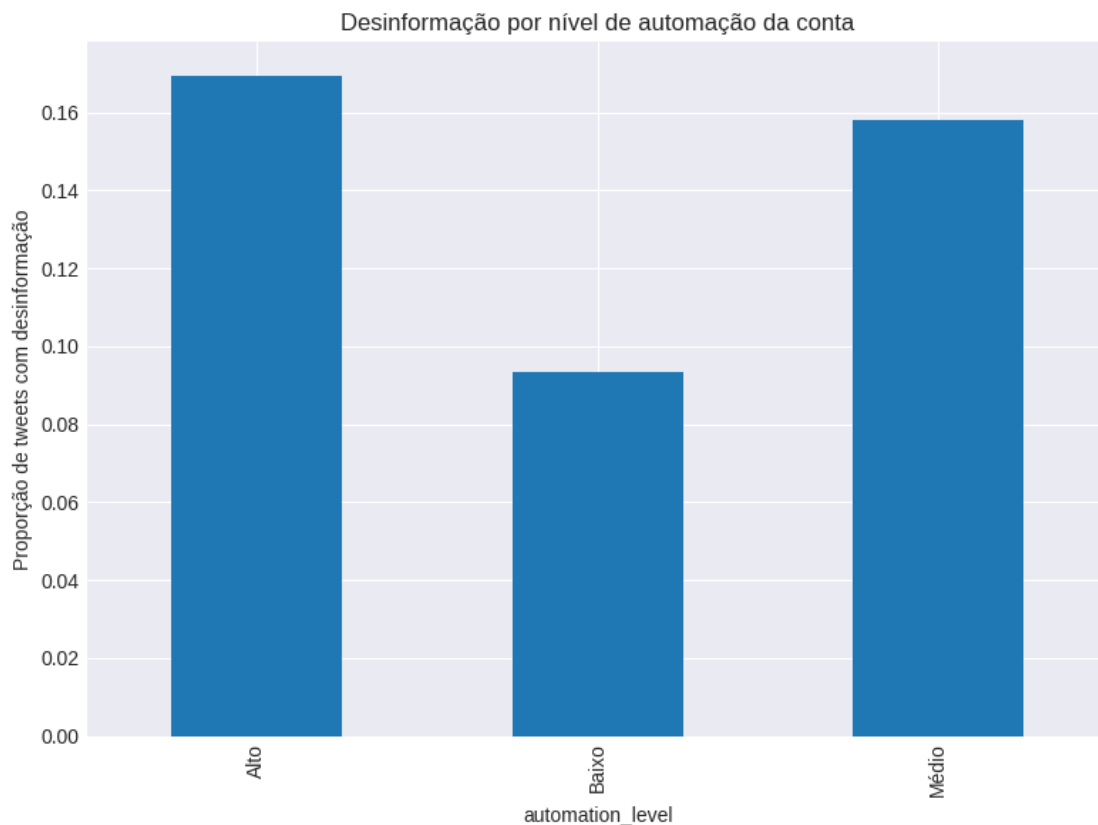


[123]: tabela

```
[123]: is_disinformation      0      1
length_category
Pequeno (<50)           6263   348
Médio (51-140)         15834  1501
Longo (141-280)        15295  2173
```

5.3.2.3 Atores (Usuários)

```
[124]: # Quem amplifica? (por nível de automação)
automacao_desinfo = df.groupby(['automation_level', 'is_disinformation']).size().
    ↳unstack(fill_value=0)
automacao_desinfo['total'] = automacao_desinfo.sum(axis=1)
automacao_desinfo['prop_desinfo'] = automacao_desinfo[1] /↳
    ↳automacao_desinfo['total']
automacao_desinfo[['prop_desinfo']].plot(kind='bar', figsize=(8,6), legend=False)
plt.ylabel('Proporção de tweets com desinformação')
plt.title('Desinformação por nível de automação da conta')
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/desinfo_por_automacao.png',↳
    ↳dpi=300)
plt.show()
```



```
[125]: # =====
# DESINFORMAÇÃO x AUTOMAÇÃO
# =====

# Tabela absoluta
tabela_automacao = pd.crosstab(
    df['automation_level'],
    df['is_disinformation']
)

tabela_automacao.columns = [
    'Não desinformação',
    'Desinformação'
]

display(tabela_automacao)
```

	Não desinformação	Desinformação
automation_level		
Alto	98	20
Baixo	35404	3647
Médio	1890	355

```
[126]: # Proporção de desinformação dentro de cada nível de automação

tabela_automacao_prop = pd.crosstab(
    df['automation_level'],
    df['is_disinformation'],
    normalize='index'
) * 100

tabela_automacao_prop.columns = [
    'Não desinformação (%)',
    'Desinformação (%)'
]

display(tabela_automacao_prop.round(2))
```

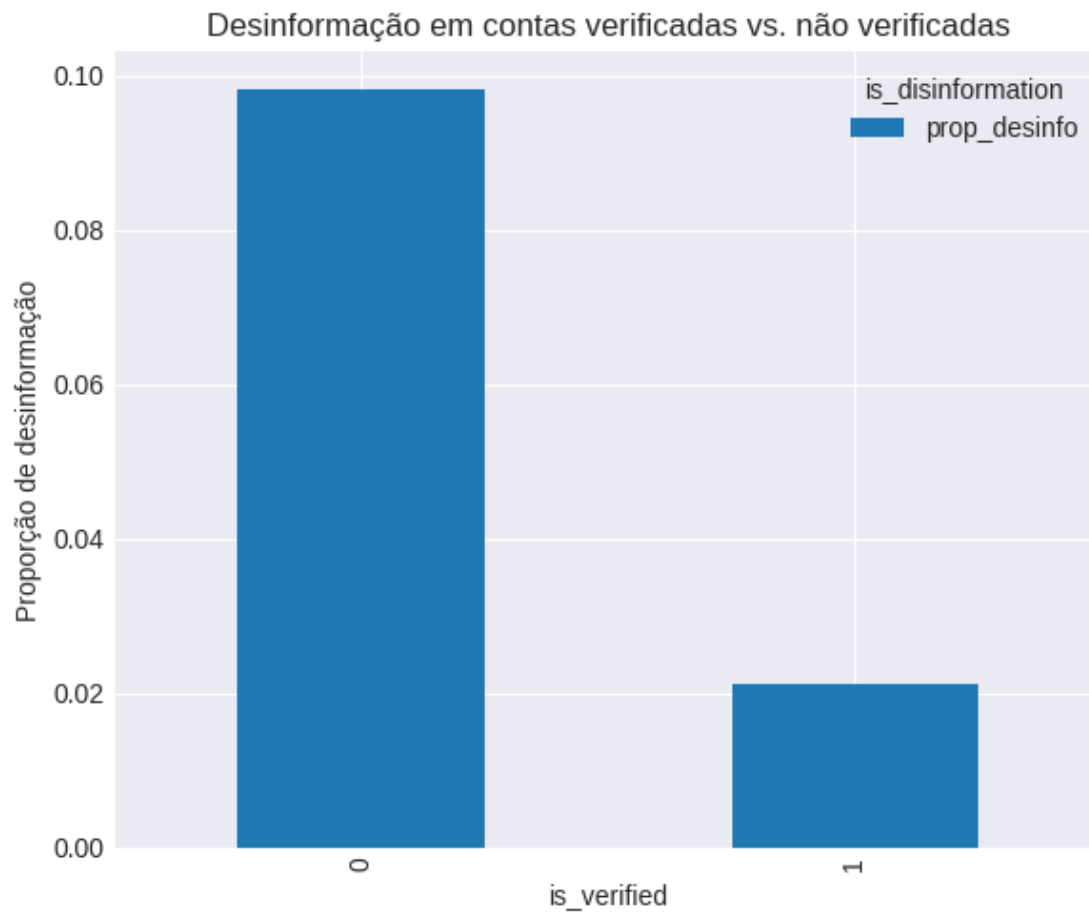
	Não desinformação (%)	Desinformação (%)
automation_level		
Alto	83.05	16.95
Baixo	90.66	9.34
Médio	84.19	15.81

```
[127]: # Amplificação por contas verificadas vs. não verificadas
verificacao_desinfo = df.groupby(['is_verified', 'is_disinformation']).size().
    ↪unstack(fill_value=0)
```

```

verificacao_desinfo['prop_desinfo'] = verificacao_desinfo[1] /
    ↳verificacao_desinfo.sum(axis=1)
verificacao_desinfo[['prop_desinfo']].plot(kind='bar', figsize=(6,5))
plt.ylabel('Proporção de desinformação')
plt.title('Desinformação em contas verificadas vs. não verificadas')
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/desinfo_por_verificacao.png',
    ↳dpi=300)
plt.show()

```



Idade

```

[128]: from scipy.stats import mannwhitneyu

idades_desinfo = df[df['is_disinformation'] == 1]['account_age_days'].dropna()
idades_nao = df[df['is_disinformation'] == 0]['account_age_days'].dropna()

stat, p_valor = mannwhitneyu(idades_desinfo, idades_nao, alternative='less')
print(f" Teste de Mann-Whitney (idades_desinfo < idades_nao): p = {p_valor:.6f}")

```

```

if p_valor < 0.05:
    print(" Diferença estatisticamente significativa: contas que disseminam_
    ↳desinformação são mais novas.")
else:
    print(" Não há diferença estatística significativa entre os grupos.")

```

Teste de Mann-Whitney (idades_desinfo < idades_ao): p = 0.000000
 Diferença estatisticamente significativa: contas que disseminam desinformação
 são mais novas.

```

[129]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# =====
# Criar faixas etárias para as contas
# =====
# Definir os limites (em dias)
bins = [0, 30, 90, 180, 365, 730, 1825, float('inf')]
labels = ['< 1 mês', '1-3 meses', '3-6 meses', '6-12 meses',
          '1-2 anos', '2-5 anos', '> 5 anos']

# Aplicar a categorização
df['age_group'] = pd.cut(
    df['account_age_days'],
    bins=bins,
    labels=labels,
    right=False # intervalo [início, fim)
)

# Verificar a distribuição geral
print(" Distribuição geral das contas por faixa etária:")
display(df['age_group'].value_counts().sort_index())

```

Distribuição geral das contas por faixa etária:

```

age_group
< 1 mês      2522
1-3 meses    2374
3-6 meses    2042
6-12 meses   3228
1-2 anos     3914
2-5 anos     8775
> 5 anos    18559
Name: count, dtype: int64

```

```
[130]: # =====
# Calcular proporção de desinformação em cada faixa
# =====

age_desinfo = df.groupby('age_group', observed=False)['is_disinformation'].agg(
    total_tweets='count',
    tweets_desinfo='sum'
).reset_index()

age_desinfo['proporcao_desinfo'] = (
    age_desinfo['tweets_desinfo'] / age_desinfo['total_tweets'] * 100
)

# Ordenar pela idade (crescente)
age_desinfo = age_desinfo.sort_values(
    'age_group',
    key=lambda x: pd.Categorical(x, categories=labels, ordered=True)
)

print(" Proporção de desinformação por faixa etária da conta:")
display(age_desinfo)
```

Proporção de desinformação por faixa etária da conta:

	age_group	total_tweets	tweets_desinfo	proporcao_desinfo
0	< 1 mês	2522	404	16.019033
1	1-3 meses	2374	320	13.479360
2	3-6 meses	2042	277	13.565132
3	6-12 meses	3228	416	12.887237
4	1-2 anos	3914	337	8.610118
5	2-5 anos	8775	735	8.376068
6	> 5 anos	18559	1533	8.260143

```
[131]: #-----
# Comparação: Desinformação vs Legítimo por faixa etária da conta
#-----

# Agregação por faixa etária
age_desinfo = df.groupby('age_group', observed=False)['is_disinformation'].agg(
    total_tweets='count',
    tweets_desinfo='sum'
).reset_index()

# Contagem de legítimos e proporções
age_desinfo['tweets_legit'] = age_desinfo['total_tweets'] -
    ↳ age_desinfo['tweets_desinfo']
age_desinfo['proporcao_desinfo'] = (
    age_desinfo['tweets_desinfo'] / age_desinfo['total_tweets'] * 100
```

```

)
age_desinfo['proporcao_legit'] = (
    age_desinfo['tweets_legit'] / age_desinfo['total_tweets'] * 100
)

# Ordenar pela idade (crescente)
age_desinfo = age_desinfo.sort_values(
    'age_group',
    key=lambda x: pd.Categorical(x, categories=labels, ordered=True)
).reset_index(drop=True)

print(" Proporção de desinformação por faixa etária da conta:")
display(age_desinfo)

```

Proporção de desinformação por faixa etária da conta:

	age_group	total_tweets	tweets_desinfo	tweets_legit	proporcao_desinfo \
0	< 1 mês	2522	404	2118	16.019033
1	1-3 meses	2374	320	2054	13.479360
2	3-6 meses	2042	277	1765	13.565132
3	6-12 meses	3228	416	2812	12.887237
4	1-2 anos	3914	337	3577	8.610118
5	2-5 anos	8775	735	8040	8.376068
6	> 5 anos	18559	1533	17026	8.260143

	proporcao_legit
0	83.980967
1	86.520640
2	86.434868
3	87.112763
4	91.389882
5	91.623932
6	91.739857

```

[132]: #-----
# Barras agrupadas: contagem absoluta (desinfo vs legítimo)
#-----
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

x = np.arange(len(age_desinfo))
width = 0.4

# Contagem absoluta
b1 = axes[0].bar(x - width/2, age_desinfo['tweets_desinfo'], width,
                 label='Desinformação', color='#d62728')
b2 = axes[0].bar(x + width/2, age_desinfo['tweets_legit'], width,
                 label='Legítimo', color='#2ca02c')

```

```

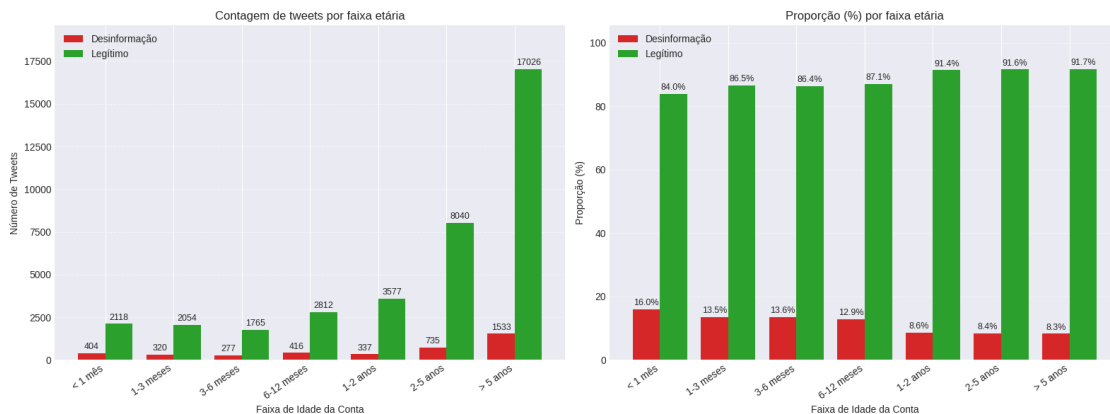
axes[0].bar_label(b1, padding=2, fontsize=9)
axes[0].bar_label(b2, padding=2, fontsize=9)
axes[0].set_xticks(x)
axes[0].set_xticklabels(age_desinfo['age_group'], rotation=35, ha='right')
axes[0].set_title('Contagem de tweets por faixa etária', fontsize=12)
axes[0].set_xlabel('Faixa de Idade da Conta')
axes[0].set_ylabel('Número de Tweets')
axes[0].legend()
axes[0].grid(axis='y', linestyle='--', alpha=0.6)
axes[0].margins(y=0.15)

# Proporção (%)
b3 = axes[1].bar(x - width/2, age_desinfo['proporcao_desinfo'], width,
                 label='Desinformação', color='#d62728')
b4 = axes[1].bar(x + width/2, age_desinfo['proporcao_legit'], width,
                 label='Legítimo', color='#2ca02c')

axes[1].bar_label(b3, fmt='%.1f%%', padding=2, fontsize=9)
axes[1].bar_label(b4, fmt='%.1f%%', padding=2, fontsize=9)
axes[1].set_xticks(x)
axes[1].set_xticklabels(age_desinfo['age_group'], rotation=35, ha='right')
axes[1].set_title('Proporção (%) por faixa etária', fontsize=12)
axes[1].set_xlabel('Faixa de Idade da Conta')
axes[1].set_ylabel('Proporção (%)')
axes[1].legend()
axes[1].grid(axis='y', linestyle='--', alpha=0.6)
axes[1].margins(y=0.15)

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳desinfo_vs_legit_por_idade_conta.png',
            dpi=300, bbox_inches='tight')
plt.show()

```



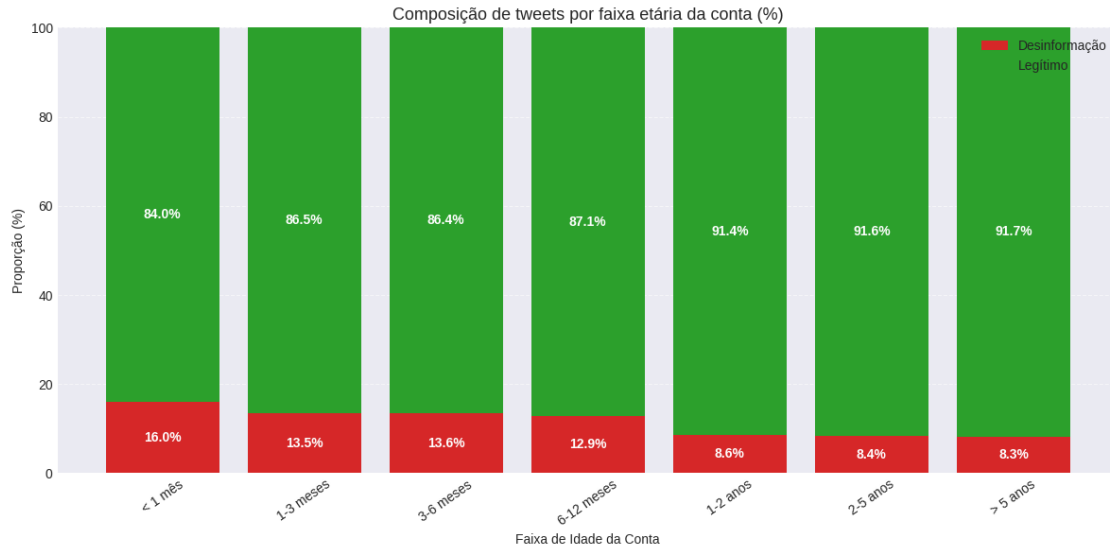
```
[133]: #-----
# Barras empilhadas em % (100% por faixa)
#-----
fig, ax = plt.subplots(figsize=(12, 6))

b_desinfo = ax.bar(age_desinfo['age_group'], age_desinfo['proporcao_desinfo'],
                    label='Desinformação', color='#d62728')
b_legit = ax.bar(age_desinfo['age_group'], age_desinfo['proporcao_legit'],
                  bottom=age_desinfo['proporcao_desinfo'],
                  label='Legítimo', color='#2ca02c')

# Rótulos dentro das barras
ax.bar_label(b_desinfo, fmt='%.1f%%', label_type='center',
              color='white', fontsize=10, fontweight='bold')
ax.bar_label(b_legit, fmt='%.1f%%', label_type='center',
              color='white', fontsize=10, fontweight='bold')

ax.set_title('Composição de tweets por faixa etária da conta (%)', fontsize=13)
ax.set_xlabel('Faixa de Idade da Conta')
ax.set_ylabel('Proporção (%)')
ax.set_ylim(0, 100)
ax.tick_params(axis='x', rotation=35)
ax.legend(loc='upper right')
ax.grid(axis='y', linestyle='--', alpha=0.6)

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳desinfo_vs_legit_empilhado_idade_conta.png',
            dpi=300, bbox_inches='tight')
plt.show()
```



```
[134]: # Contar frequência de cada valor
df['is_disinformation'].value_counts()
```

```
[134]: is_disinformation
0      37392
1       4022
Name: count, dtype: int64
```

5.3.2.4 Sentimento, Tópicos e Entidades

```
[135]: # =====
# SENTIMENTO x DESINFORMAÇÃO
# =====

tabela_sentimento = (
    df.groupby('sentiment')
      .agg(
        total_tweets=('tweet_id', 'count'),
        tweets_desinformacao=('is_disinformation', 'sum')
      )
      .reset_index()
)

tabela_sentimento['percentual_desinformacao'] = (
    tabela_sentimento['tweets_desinformacao'] /
    tabela_sentimento['total_tweets'] * 100
)

display(
```

```
tabela_sentimento.sort_values(
    'percentual_desinformacao',
    ascending=False
)
)
```

	sentiment	total_tweets	tweets_desinformacao	percentual_desinformacao
0	Negativo	17125	1872	10.931387
1	Neutro	19160	1880	9.812109
2	Positivo	5129	270	5.264184

```
[136]: # =====
# TÓPICOS x DESINFORMAÇÃO
# =====

tabela_topicos = (
    df.groupby('topic')
    .agg(
        total_tweets=('tweet_id', 'count'),
        tweets_desinformacao=('is_disinformation', 'sum')
    )
    .reset_index()
)

tabela_topicos['percentual_desinformacao'] = (
    tabela_topicos['tweets_desinformacao'] /
    tabela_topicos['total_tweets'] * 100
)

tabela_topicos = tabela_topicos.sort_values(
    'percentual_desinformacao',
    ascending=False
)

display(tabela_topicos)
```

	topic	total_tweets	tweets_desinformacao \
1	Auditoria	752	211
7	Fraude	1065	268
8	Live do Argentino	161	31
4	Código-Fonte	970	143
16	Urna Eletrônica	18716	2419
2	Celular na Cabine	368	32
18	Voto Impresso	476	40
0	Ataque Hacker	277	22
10	Militares e Fiscalização	298	23
9	Logs da Urna	299	22
15	Pesquisas Eleitorais	1268	89

5	Disputa Eleitoral	1732	89
6	Fake News	392	19
3	Conexão com a Internet	292	14
11	Mídias Sociais	1211	55
17	Uso da Urna	10056	426
12	Narrativa Política	2678	110
13	Narrativa Religiosa	381	9
14	Outlier	22	0

	percentual_desinformacao
1	28.058511
7	25.164319
8	19.254658
4	14.742268
16	12.924770
2	8.695652
18	8.403361
0	7.942238
10	7.718121
9	7.357860
15	7.018927
5	5.138568
6	4.846939
3	4.794521
11	4.541701
17	4.236277
12	4.107543
13	2.362205
14	0.000000

```
[137]: # Top 15 tópicos com maior proporção de desinformação

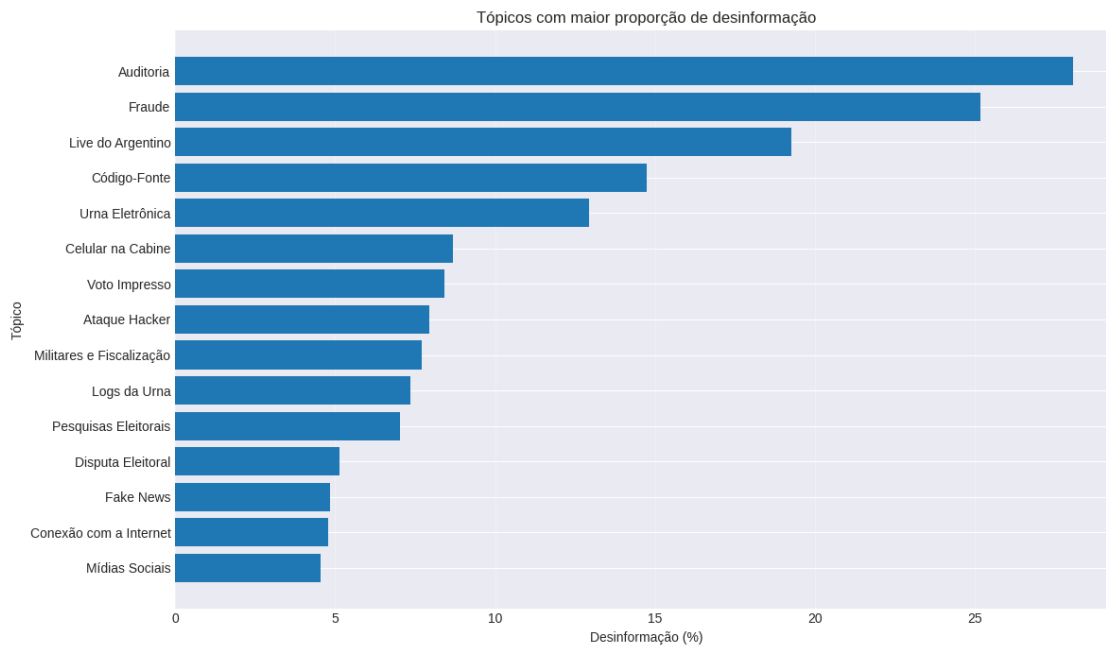
topicos_plot = tabela_topicos.head(15).copy()

plt.figure(figsize=(12, 7))

plt.barh(
    topicos_plot['topic'].astype(str),
    topicos_plot['percentual_desinformacao']
)

plt.xlabel('Desinformação (%)')
plt.ylabel('Tópico')
plt.title('Tópicos com maior proporção de desinformação')
plt.gca().invert_yaxis()
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
```

```
plt.show()
```



```
[138]: # =====
# 1. Garantir que a coluna 'entities' seja um dicionário
# =====
import ast
from collections import Counter
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Se a coluna ainda estiver como string, converter
df['entities'] = df['entities'].apply(
    lambda x: ast.literal_eval(x) if isinstance(x, str) else x
)

# =====
# 2. Filtrar tweets desinformativos
# =====
df_desinfo = df[df['is_disinformation'] == 1].copy()
df_nao = df[df['is_disinformation'] == 0].copy()

# =====
# 3. Função para extrair todas as entidades de uma categoria
# =====
def extrair_entidades(df, categoria):
```

```

"""
Retorna um Counter com todas as entidades da categoria especificada
(ex.: 'PESSOAS', 'LOCAIS', 'ORGANIZAÇÕES', 'OUTROS')
"""

todas = []
for entidades_dict in df['entities']:
    if categoria in entidades_dict:
        lista = entidades_dict[categoria]
        if isinstance(lista, list):
            todas.extend(lista)
return Counter(todas)

# =====
# 4. Top entidades por categoria em desinformação
# =====
categorias = ['PESSOAS', 'LOCAIS', 'ORGANIZAÇÕES', 'OUTROS']
top_n = 10

for cat in categorias:
    counter_desinfo = extrair_entidades(df_desinfo, cat)
    top_desinfo = pd.DataFrame(
        counter_desinfo.most_common(top_n),
        columns=['entidade', 'frequencia']
    )
    print(f"\n TOP {top_n} {cat} em DESINFORMAÇÃO:")
    display(top_desinfo)

```

TOP 10 PESSOAS em DESINFORMAÇÃO:

	entidade	frequencia
0	Lula	220
1	Bolsonaro	142
2	Xandão	26
3	lula	25
4	Democrático	24
5	PL	21
6	Deus	21
7	Vc	19
8	Alexandre de Moraes	13
9	Dilma	11

TOP 10 LOCAIS em DESINFORMAÇÃO:

	entidade	frequencia
0	Brasil	217
1	Bolsonaro	89
2	STF	77

3	Urna	53
4	bolsonaro	42
5	Pq	30
6	Nordeste	24
7	Macapá	24
8	F-D-P	24
9	Urnas	23

TOP 10 ORGANIZAÇÕES em DESINFORMAÇÃO:

	entidade	frequencia
0	TSE	267
1	Bolsonaro	88
2	PT	45
3	TRIBUNAL CORRUPTO	42
4	BU	29
5	Forças Armadas	19
6	CAIXA	19
7	FRAUDE	13
8	ZERO	12
9	FA	9

TOP 10 OUTROS em DESINFORMAÇÃO:

	entidade	frequencia
0	Presidente	28
1	ELEIÇÃO POR URNA ELETRÔNICA	24
2	Ladrão	22
3	SOS FFAA	20
4	petrolao	19
5	BNDES Correios	19
6	FFAA	18
7	Urna Eletrônica	17
8	IMPRESSO	11
9	BUs	11

5.3.2.5 Anexos Hashtags:

```
[139]: # =====
# Filtrar apenas tweets classificados como desinformação
# =====
df_desinfo = df[df['is_disinformation'] == 1].copy()

# Garantir que a coluna 'hashtags' esteja no formato de lista
import ast
df_desinfo['hashtags'] = df_desinfo['hashtags'].apply(
    lambda x: ast.literal_eval(x) if isinstance(x, str) else x
```

```

)

# Explodir a lista de hashtags para uma linha por hashtag
hashtags_desinfo = df_desinfo.explode('hashtags')

# Remover hashtags vazias ou nulas
hashtags_desinfo = hashtags_desinfo[hashtags_desinfo['hashtags'].notna()]
hashtags_desinfo['hashtags'] = hashtags_desinfo['hashtags'].str.lower().str.
    ↳strip()
hashtags_desinfo = hashtags_desinfo[hashtags_desinfo['hashtags'] != '']

# Contar frequência
from collections import Counter
contagem_desinfo = Counter(hashtags_desinfo['hashtags'])

# Criar DataFrame com as top N
top_n = 20
top_hashtags_desinfo = pd.DataFrame(
    contagem_desinfo.most_common(top_n),
    columns=['hashtag', 'frequencia']
)

print(" Hashtags mais usadas em TWEETS DESINFORMATIVOS:")
display(top_hashtags_desinfo)

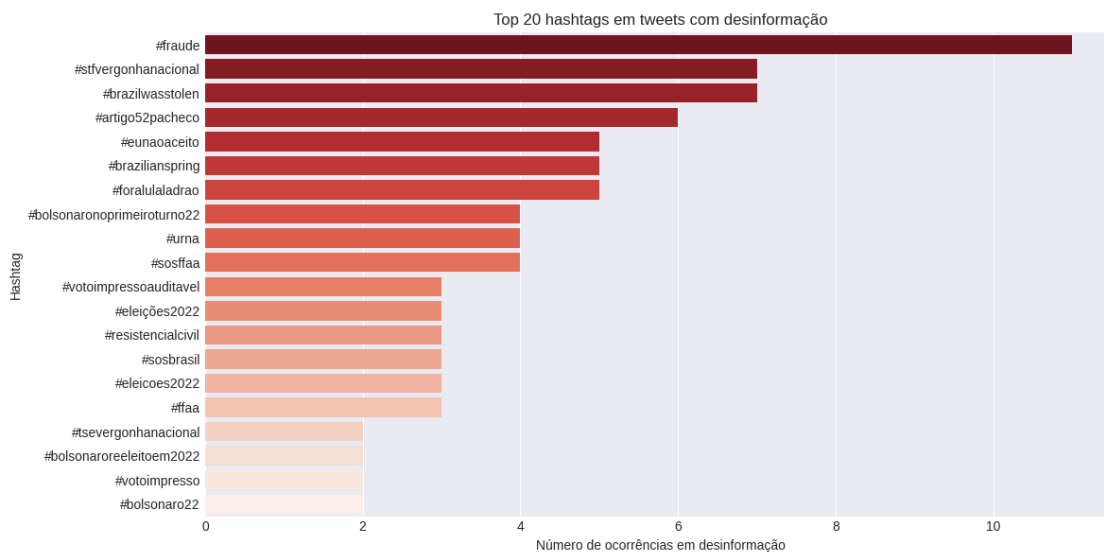
```

Hashtags mais usadas em TWEETS DESINFORMATIVOS:

	hashtag	frequencia
0	#fraude	11
1	#stfvergonhanacional	7
2	#brazilwasstolen	7
3	#artigo52pacheco	6
4	#eunaoaceito	5
5	#brazilianspring	5
6	#foralulaladrao	5
7	#bolsonaronoprimeiroturno22	4
8	#urna	4
9	#sosffaa	4
10	#votoimpressoauditavel	3
11	#eleicoes2022	3
12	#resistencialcivil	3
13	#sosbrasil	3
14	#eleicoes2022	3
15	#ffaa	3
16	#tsevergonhanacional	2
17	#bolsonaroreeleitoem2022	2
18	#votoimpresso	2
19	#bolsonaro22	2

```
[140]: import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 6))
sns.barplot(
    data=top_hashtags_desinfo,
    y='hashtag',
    x='frequencia',
    hue='hashtag',
    palette='Reds_r',
    legend=False
)
plt.xlabel('Número de ocorrências em desinformação')
plt.ylabel('Hashtag')
plt.title(f'Top {top_n} hashtags em tweets com desinformação')
plt.tight_layout()
plt.show()
```



```
[141]: # =====
# Hashtags em tweets NÃO desinformativos (para comparação)
# =====
df_nao_desinfo = df[df['is_disinformation'] == 0].copy()
df_nao_desinfo['hashtags'] = df_nao_desinfo['hashtags'].apply(
    lambda x: ast.literal_eval(x) if isinstance(x, str) else x
)

hashtags_nao = df_nao_desinfo.explode('hashtags')
hashtags_nao = hashtags_nao[hashtags_nao['hashtags'].notna()]
```

```

hashtags_nao['hashtags'] = hashtags_nao['hashtags'].str.lower().str.strip()
hashtags_nao = hashtags_nao[hashtags_nao['hashtags'] != '']

contagem_nao = Counter(hashtags_nao['hashtags'])
top_hashtags_nao = pd.DataFrame(
    contagem_nao.most_common(top_n),
    columns=['hashtag', 'frequencia']
)

# Juntar para comparar lado a lado
comparacao = top_hashtags_desinfo.merge(
    top_hashtags_nao,
    on='hashtag',
    how='outer',
    suffixes=('_desinfo', '_nao')
).fillna(0)

comparacao = comparacao.sort_values('frequencia_desinfo', ascending=False)
display(comparacao)

```

	hashtag	frequencia_desinfo	frequencia_nao
14	#fraude	11.0	0.0
7	#brazilwasstolen	7.0	0.0
29	#stfvergonhanacional	7.0	0.0
0	#artigo52pacheco	6.0	0.0
6	#brazilianspring	5.0	0.0
13	#foralulaladrao	5.0	0.0
10	#eunaoaceito	5.0	0.0
28	#sosffaa	4.0	0.0
2	#bolsonaronoprimeiroturno22	4.0	47.0
31	#urna	4.0	39.0
26	#resistencialcivil	3.0	0.0
9	#eleicoes2022	3.0	110.0
11	#ffaa	3.0	0.0
34	#votoimpressoauditavel	3.0	0.0
27	#sosbrasil	3.0	0.0
8	#eleicoes2022	3.0	104.0
1	#bolsonaro22	2.0	0.0
4	#bolsonaroreeleitoem2022	2.0	48.0
30	#tsevergonhanacional	2.0	0.0
33	#votoimpresso	2.0	0.0
18	#lulanoloturno	0.0	57.0
16	#lula	0.0	75.0
17	#lula13	0.0	38.0
12	#forabolsonaro	0.0	59.0
3	#bolsonaroreeleito22	0.0	25.0
5	#brasil	0.0	59.0
15	#lovetwitter	0.0	29.0

25	#presidente	0.0	31.0
24	#lulapresidente13	0.0	61.0
23	#lulapresidente13	0.0	30.0
22	#lulapresidente13	0.0	96.0
20	#lulanoprimeiroturno	0.0	45.0
21	#lulapresidente	0.0	32.0
19	#lulano1ºturno	0.0	88.0
32	#vivaosus	0.0	37.0

```
[142]: # =====
# PREPARAR DATA (tweet_created_at)
# =====
coluna_data = 'tweet_created_at'

hashtags_desinfo[coluna_data] = pd.to_datetime(
    hashtags_desinfo[coluna_data],
    errors='coerce',
    utc=True          # Twitter geralmente vem em UTC
)

hashtags_desinfo = hashtags_desinfo.dropna(subset=[coluna_data])

hashtags_desinfo['dia'] = hashtags_desinfo[coluna_data].dt.date

# =====
# DIA DE PICO POR HASHTAG (top 20)
# =====
top_hashtags_lista = [h for h, _ in contagem_desinfo.most_common(top_n)]

hashtags_dia = (
    hashtags_desinfo[hashtags_desinfo['hashtags'].isin(top_hashtags_lista)]
    .groupby(['dia', 'hashtags'])
    .size()
    .reset_index(name='frequencia')
)

pico_por_hashtag = (
    hashtags_dia
    .sort_values(['hashtags', 'frequencia'], ascending=[True, False])
    .groupby('hashtags')
    .head(1)
    .sort_values('frequencia', ascending=False)
    .reset_index(drop=True)
)

print(" DIA DE PICO de cada hashtag desinformativa (top 20):")
```

```
display(pico_por_hashtag)
```

DIA DE PICO de cada hashtag desinformativa (top 20):

	dia	hashtags	frequencia
0	2022-10-31	#eunaoaceito	5
1	2022-11-04	#fraude	5
2	2022-11-07	#foralulaladrao	4
3	2022-11-04	#brazilwasstolen	3
4	2022-11-05	#resistencialcivil	3
5	2022-09-12	#stfvergonhanacional	2
6	2022-09-12	#artigo52pacheco	2
7	2022-09-20	#bolsonaro22	1
8	2022-09-03	#bolsonaroreeleitoem2022	1
9	2022-09-23	#bolsonaronoprimeiroturno22	1
10	2022-10-04	#eleicoes2022	1
11	2022-11-05	#eleicoes2022	1
12	2022-11-13	#ffaa	1
13	2022-11-07	#brazilianspring	1
14	2022-11-05	#sosbrasil	1
15	2022-11-17	#sosffaa	1
16	2022-08-31	#tsevergonhanacional	1
17	2022-10-04	#urna	1
18	2022-09-11	#votoimpresso	1
19	2022-09-18	#votoimpressoauditavel	1

```
[143]: # =====
# PERÍODO DE ATIVIDADE POR HASHTAG (primeira → última aparição)
# =====
top_hashtags_lista = [h for h, _ in contagem_desinfo.most_common(top_n)]

df_top = hashtags_desinfo[hashtags_desinfo['hashtags'].isin(top_hashtags_lista)]

periodo_por_hashtag = (
    df_top
    .groupby('hashtags')
    .agg(
        primeira_aparicao=(coluna_data, 'min'),
        ultima_aparicao  =(coluna_data, 'max'),
        total_tweets    =(coluna_data, 'size'),
        dias_ativos      =('dia', 'nunique')
    )
    .reset_index()
)

periodo_por_hashtag['duracao_dias'] = (
    periodo_por_hashtag['ultima_aparicao'] -
    periodo_por_hashtag['primeira_aparicao']
)
```

```

).dt.total_seconds() / 86400

periodo_por_hashtag = (
    periodo_por_hashtag
    .sort_values('total_tweets', ascending=False)
    .reset_index(drop=True)
)

periodo_legivel = periodo_por_hashtag.copy()
periodo_legivel['primeira_aparicao'] = periodo_legivel['primeira_aparicao'].dt.
    ↳date
periodo_legivel['ultima_aparicao'] = periodo_legivel['ultima_aparicao'].dt.date
periodo_legivel['duracao_dias'] = periodo_legivel['duracao_dias'].round(1)

print(" PERÍODO DE ATIVIDADE por hashtag desinformativa (top 20):")
display(periodo_legivel)

```

PERÍODO DE ATIVIDADE por hashtag desinformativa (top 20):

	hashtags	primeira_aparicao	ultima_aparicao	\
0	#fraude	2022-10-06	2022-11-05	
1	#brazilwasstolen	2022-11-04	2022-11-20	
2	#stfvergonhanacional	2022-09-12	2022-11-04	
3	#artigo52pacheco	2022-09-12	2022-11-04	
4	#foralulaladrao	2022-11-07	2022-11-26	
5	#brazilianspring	2022-11-07	2022-11-22	
6	#eunaoaceito	2022-10-31	2022-10-31	
7	#bolsonaronoprimeiroturno22	2022-09-23	2022-10-01	
8	#sosffaa	2022-11-17	2022-11-25	
9	#urna	2022-10-04	2022-11-14	
10	#ffaa	2022-11-13	2022-11-29	
11	#eleicoes2022	2022-10-04	2022-10-20	
12	#resistencialcivil	2022-11-05	2022-11-05	
13	#votoimpressoauditavel	2022-09-18	2022-10-04	
14	#eleicoes2022	2022-11-05	2022-11-28	
15	#sosbrasil	2022-11-05	2022-11-20	
16	#bolsonaroreeleitoem2022	2022-09-03	2022-10-01	
17	#bolsonaro22	2022-09-20	2022-10-30	
18	#tsevergonhanacional	2022-08-31	2022-10-14	
19	#votoimpresso	2022-09-11	2022-10-28	

	total_tweets	dias_ativos	duracao_dias
0	11	6	30.1
1	7	5	16.0
2	7	6	52.8
3	6	5	52.8
4	5	2	19.6
5	5	5	15.0

6	5	1	0.0
7	4	4	7.8
8	4	4	8.5
9	4	4	40.9
10	3	3	16.1
11	3	3	15.9
12	3	1	0.0
13	3	3	15.3
14	3	3	23.3
15	3	3	15.7
16	2	2	27.6
17	2	2	39.6
18	2	2	44.2
19	2	2	47.3

```
[144]: # =====
# Série temporal completa - tweets por dia por hashtag
# =====
serie_por_hashtag = (
    df_top
    .groupby(['hashtags', 'dia'])
    .size()
    .reset_index(name='frequencia')
    .sort_values(['hashtags', 'dia'])
)

serie_por_hashtag['primeiro_dia'] = serie_por_hashtag.groupby('hashtags')['dia'].
    ↪transform('min')
serie_por_hashtag['ultimo_dia'] = serie_por_hashtag.groupby('hashtags')['dia'].
    ↪transform('max')

display(serie_por_hashtag.head(30))
```

	hashtags	dia	frequencia	primeiro_dia \
0	#artigo52pacheco	2022-09-12	2	2022-09-12
1	#artigo52pacheco	2022-09-15	1	2022-09-12
2	#artigo52pacheco	2022-10-12	1	2022-09-12
3	#artigo52pacheco	2022-10-16	1	2022-09-12
4	#artigo52pacheco	2022-11-04	1	2022-09-12
5	#bolsonaro22	2022-09-20	1	2022-09-20
6	#bolsonaro22	2022-10-30	1	2022-09-20
7	#bolsonaronoprimeiroturno22	2022-09-23	1	2022-09-23
8	#bolsonaronoprimeiroturno22	2022-09-27	1	2022-09-23
9	#bolsonaronoprimeiroturno22	2022-09-28	1	2022-09-23
10	#bolsonaronoprimeiroturno22	2022-10-01	1	2022-09-23
11	#bolsonaroreeleitoem2022	2022-09-03	1	2022-09-03
12	#bolsonaroreeleitoem2022	2022-10-01	1	2022-09-03
13	#brazilianspring	2022-11-07	1	2022-11-07

14	#brazilianspring	2022-11-09	1	2022-11-07
15	#brazilianspring	2022-11-15	1	2022-11-07
16	#brazilianspring	2022-11-20	1	2022-11-07
17	#brazilianspring	2022-11-22	1	2022-11-07
18	#brazilwasstolen	2022-11-04	3	2022-11-04
19	#brazilwasstolen	2022-11-06	1	2022-11-04
20	#brazilwasstolen	2022-11-07	1	2022-11-04
21	#brazilwasstolen	2022-11-12	1	2022-11-04
22	#brazilwasstolen	2022-11-20	1	2022-11-04
23	#eleicoes2022	2022-11-05	1	2022-11-05
24	#eleicoes2022	2022-11-14	1	2022-11-05
25	#eleicoes2022	2022-11-28	1	2022-11-05
26	#eleicoes2022	2022-10-04	1	2022-10-04
27	#eleicoes2022	2022-10-14	1	2022-10-04
28	#eleicoes2022	2022-10-20	1	2022-10-04
29	#eunaoaceito	2022-10-31	5	2022-10-31

	ultimo_dia
0	2022-11-04
1	2022-11-04
2	2022-11-04
3	2022-11-04
4	2022-11-04
5	2022-10-30
6	2022-10-30
7	2022-10-01
8	2022-10-01
9	2022-10-01
10	2022-10-01
11	2022-10-01
12	2022-10-01
13	2022-11-22
14	2022-11-22
15	2022-11-22
16	2022-11-22
17	2022-11-22
18	2022-11-20
19	2022-11-20
20	2022-11-20
21	2022-11-20
22	2022-11-20
23	2022-11-28
24	2022-11-28
25	2022-11-28
26	2022-10-20
27	2022-10-20
28	2022-10-20
29	2022-10-31

Links:

```
[145]: # =====
# Filtrar tweets desinformativos e tratar coluna 'urls'
# =====
import ast
from collections import Counter
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from urllib.parse import urlparse

# Garantir que a coluna 'urls' seja uma lista
df['urls'] = df['urls'].apply(
    lambda x: ast.literal_eval(x) if isinstance(x, str) else x
)

# Filtrar apenas desinformação
df_desinfo = df[df['is_disinformation'] == 1].copy()

# Explodir a lista de URLs para uma linha por URL
urls_desinfo = df_desinfo.explode('urls')

# Remover URLs vazias ou nulas
urls_desinfo = urls_desinfo[urls_desinfo['urls'].notna()]
urls_desinfo['urls'] = urls_desinfo['urls'].str.strip()
urls_desinfo = urls_desinfo[urls_desinfo['urls'] != '']

# Contar frequência
contagem_links_desinfo = Counter(urls_desinfo['urls'])

# Criar DataFrame com os Top 20
top_n = 20
top_links_desinfo = pd.DataFrame(
    contagem_links_desinfo.most_common(top_n),
    columns=['url', 'frequencia']
)

print(" Links mais compartilhados em TWEETS DESINFORMATIVOS:")
display(top_links_desinfo)
```

Links mais compartilhados em TWEETS DESINFORMATIVOS:

	url	frequencia
0	https://www.youtube.com/watch?v=txRYQPP4byc&fe...	18
1	https://twitter.com/lavern_spicer/status/15868...	6
2	https://www1.folha.uol.com.br/poder/2021/06/so...	3
3	https://twitter.com/Angelalandim3/status/15774...	3
4	https://twitter.com/DriSouzaaasz/status/158873...	3

5	https://www.tse.jus.br/comunicacao/noticias/20...	3
6	https://a.msn.com/01/pt-br/AA11qCkf?ocid=st	2
7	https://twitter.com/lpbragancabr/status/157059...	2
8	https://twitter.com/reporterenato/status/15727...	2
9	https://www.brasil247.com/brasil/diretor-do-ip...	2
10	https://twitter.com/RIntegrarh/status/15776603...	2
11	https://resultados.tse.jus.br/oficial/app/inde...	2
12	https://twitter.com/angelalandim3/status/15774...	2
13	https://twitter.com/AdilsonTx2022/status/15799...	2
14	https://www.youtube.com/watch?v=rdxtAUPDC_w&fe...	2
15	https://resultados.tse.jus.br/oficial/app/inde...	2
16	https://resultados.tse.jus.br/oficial/app/inde...	2
17	https://twitter.com/fernandoconrado/status/158...	2
18	https://drive.google.com/drive/folders/10XXXRD...	2
19	https://www.youtube.com/watch?v=yVOKJbGTTZQ&ab...	2

```
[146]: # Função para extrair domínio
def extrair_dominio(url):
    try:
        parsed = urlparse(url)
        domain = parsed.netloc.lower()
        if domain.startswith('www.'):
            domain = domain[4:]
        return domain
    except:
        return None

# Aplicar a extração de domínio
urls_desinfo['dominio'] = urls_desinfo['urls'].apply(extrair_dominio)

# Remover domínios nulos e vazios
urls_desinfo = urls_desinfo[urls_desinfo['dominio'].notna()]
urls_desinfo = urls_desinfo[urls_desinfo['dominio'] != '']

# Contar domínios
contagem_dominios_desinfo = Counter(urls_desinfo['dominio'])

# Top 15 domínios
top_dominios_desinfo = pd.DataFrame(
    contagem_dominios_desinfo.most_common(15),
    columns=['dominio', 'frequencia']
)

print(" Domínios mais usados em TWEETS DESINFORMATIVOS:")
display(top_dominios_desinfo)
```

Domínios mais usados em TWEETS DESINFORMATIVOS:

dominio	frequencia
---------	------------

0	twitter.com	553
1	youtube.com	38
2	resultados.tse.jus.br	15
3	www1.folha.uol.com.br	5
4	tse.jus.br	5
5	g1.globo.com	4
6	instagram.com	3
7	noticias.uol.com.br	3
8	brasil247.com	3
9	open.spotify.com	2
10	a.msn.com	2
11	msn.com	2
12	correiobraziliense.com.br	2
13	fiscaisdobolsonaro.com.br	2
14	drive.google.com	2

```
[147]: # =====
# URLs em tweets NÃO desinformativos
# =====
df_nao = df[df['is_disinformation'] == 0].copy()
urls_nao = df_nao.explode('urls')
urls_nao = urls_nao[urls_nao['urls'].notna()]
urls_nao['urls'] = urls_nao['urls'].str.strip()
urls_nao = urls_nao[urls_nao['urls'] != '']

# Extrair domínios
urls_nao['dominio'] = urls_nao['urls'].apply(extrair_dominio)
urls_nao = urls_nao[urls_nao['dominio'].notna()]
urls_nao = urls_nao[urls_nao['dominio'] != '']

contagem_dominios_nao = Counter(urls_nao['dominio'])
top_dominios_nao = pd.DataFrame(
    contagem_dominios_nao.most_common(15),
    columns=['dominio', 'frequencia']
)

# Juntar para comparar
comparacao_dominios = top_dominios_desinfo.merge(
    top_dominios_nao,
    on='dominio',
    how='outer',
    suffixes=('_desinfo', '_nao')
).fillna(0)

comparacao_dominios = comparacao_dominios.sort_values('frequencia_desinfo',
↪ascending=False)
display(comparacao_dominios)
```

	dominio	frequencia_desinfo	frequencia_ao
17	twitter.com	553.0	6203.0
20	youtube.com	38.0	253.0
14	resultados.tse.jus.br	15.0	0.0
16	tse.jus.br	5.0	171.0
19	www1.folha.uol.com.br	5.0	64.0
7	g1.globo.com	4.0	81.0
2	brasil247.com	3.0	30.0
11	noticias.uol.com.br	3.0	107.0
8	instagram.com	3.0	28.0
6	fiscaisdobolsonaro.com.br	2.0	0.0
4	drive.google.com	2.0	0.0
3	correioabraziliense.com.br	2.0	22.0
0	a.msn.com	2.0	0.0
10	msn.com	2.0	0.0
13	open.spotify.com	2.0	0.0
1	agenciabrasil.ebc.com.br	0.0	56.0
5	estadao.com.br	0.0	27.0
12	oglobo.globo.com	0.0	24.0
9	justicaeleitoral.jus.br	0.0	38.0
15	tecmundo.com.br	0.0	40.0
18	voltacomseuex.com	0.0	42.0

```
[148]: #-----
# Domínios mais frequentes por dia de pico
#-----

import pandas as pd
from collections import Counter

# 1. Preparar data
urls_desinfo['tweet_created_at'] = pd.to_datetime(
    urls_desinfo['tweet_created_at'], errors='coerce', utc=True
).dt.tz_convert('America/Sao_Paulo')

urls_desinfo['data'] = urls_desinfo['tweet_created_at'].dt.date

# 2. Achar os 5 dias de pico
urls_por_dia = urls_desinfo.groupby('data').size().sort_values(ascending=False)
dias_pico = urls_por_dia.head(5).index.tolist()

# 3. Filtrar e montar a tabela
df_pico = urls_desinfo[urls_desinfo['data'].isin(dias_pico)]

linhas = []
for dia, grupo in df_pico.groupby('data'):
    for dom, freq in Counter(grupo['dominio']).most_common(10):
```

```
linhas.append({'Data': dia, 'Domínio': dom, 'Frequência': freq})

tabela = pd.DataFrame(linhas)
display(tabela)
```

	Data	Domínio	Frequência
0	2022-10-05	twitter.com	38
1	2022-10-05	resultados.tse.jus.br	5
2	2022-10-05	cnnbrasil.com.br	1
3	2022-10-06	twitter.com	21
4	2022-10-06	youtube.com	5
5	2022-10-06	resultados.tse.jus.br	2
6	2022-10-06	fb.watch	1
7	2022-11-01	twitter.com	19
8	2022-11-01	resultados.tse.jus.br	4
9	2022-11-02	twitter.com	19
10	2022-11-02	br.search.yahoo.com	1
11	2022-11-02	xn--seo-nlat.se	1
12	2022-11-02	www1.folha.uol.com.br	1
13	2022-11-02	youtube.com	1
14	2022-11-13	twitter.com	21
15	2022-11-13	public.tableau.com	2
16	2022-11-13	tse.jus.br	2
17	2022-11-13	youtube.com	1

```
[149]: # =====
# PERÍODO DE ATIVIDADE POR DOMÍNIO (primeira → última aparição)
# =====
import pandas as pd

# Preparar data
coluna_data = 'tweet_created_at'

urls_desinfo[coluna_data] = pd.to_datetime(
    urls_desinfo[coluna_data],
    errors='coerce',
    utc=True
)

urls_desinfo = urls_desinfo.dropna(subset=[coluna_data])

urls_desinfo['dia'] = urls_desinfo[coluna_data].dt.date

# Top N domínios
top_n = 20
contagem_dominios = Counter(urls_desinfo['dominio'])
top_dominios_lista = [d for d, _ in contagem_dominios.most_common(top_n)]
```

```

df_top_dom = urls_desinfo[urls_desinfo['dominio'].isin(top_dominios_lista)].
    ↳copy()

# =====
# Primeira → última aparição por domínio
# =====
periodo_por_dominio = (
    df_top_dom
    .groupby('dominio')
    .agg(
        primeira_aparicao=(coluna_data, 'min'),
        ultima_aparicao  =(coluna_data, 'max'),
        total_urls      =(coluna_data, 'size'),
        dias_ativos      =('dia', 'nunique')
    )
    .reset_index()
)

periodo_por_dominio['duracao_dias'] = (
    periodo_por_dominio['ultima_aparicao'] -
    ↳periodo_por_dominio['primeira_aparicao']
).dt.total_seconds() / 86400

periodo_por_dominio = (
    periodo_por_dominio
    .sort_values('total_urls', ascending=False)
    .reset_index(drop=True)
)

periodo_legivel_dom = periodo_por_dominio.copy()
periodo_legivel_dom['primeira_aparicao'] =
    ↳periodo_legivel_dom['primeira_aparicao'].dt.date
periodo_legivel_dom['ultima_aparicao'] =
    ↳periodo_legivel_dom['ultima_aparicao'].dt.date
periodo_legivel_dom['duracao_dias'] = periodo_legivel_dom['duracao_dias'].
    ↳round(1)

print(" PERÍODO DE ATIVIDADE por domínio (top 20):")
display(periodo_legivel_dom)

```

PERÍODO DE ATIVIDADE por domínio (top 20):

	dominio	primeira_aparicao	ultima_aparicao	total_urls \
0	twitter.com	2022-08-31	2022-12-02	553
1	youtube.com	2022-08-31	2022-12-01	38
2	resultados.tse.jus.br	2022-10-05	2022-11-07	15

3	tse.jus.br	2022-09-08	2022-11-18	5
4	www1.folha.uol.com.br	2022-08-30	2022-11-17	5
5	g1.globo.com	2022-09-24	2022-11-08	4
6	noticias.uol.com.br	2022-09-15	2022-11-10	3
7	brasil247.com	2022-09-26	2022-11-23	3
8	instagram.com	2022-09-07	2022-11-27	3
9	dadosabertos.tse.jus.br	2022-11-04	2022-11-05	2
10	a.msn.com	2022-09-03	2022-09-04	2
11	correiobraziliense.com.br	2022-09-14	2022-09-17	2
12	msn.com	2022-09-04	2022-11-26	2
13	google.com	2022-11-06	2022-11-15	2
14	drive.google.com	2022-11-04	2022-11-04	2
15	fiscaisdobolsonaro.com.br	2022-10-24	2022-10-24	2
16	docs.google.com	2022-11-04	2022-11-05	2
17	open.spotify.com	2022-08-31	2022-08-31	2
18	public.tableau.com	2022-11-13	2022-11-13	2
19	page.brasilparalelo.com.br	2022-09-11	2022-09-11	1

	dias_ativos	duracao_dias
0	89	93.3
1	23	91.8
2	6	32.8
3	4	71.0
4	5	78.1
5	4	44.6
6	3	56.3
7	2	58.1
8	3	80.4
9	2	0.4
10	2	0.7
11	2	3.5
12	2	82.4
13	2	9.0
14	1	0.0
15	1	0.0
16	2	0.0
17	1	0.2
18	1	0.4
19	1	0.0

```
[150]: # =====
# Série temporal completa - URLs por dia por domínio
# =====
serie_por dominio = (
    df_top_dom
    .groupby(['dominio', 'dia'])
    .size())
```

```

        .reset_index(name='frequencia')
        .sort_values(['dominio', 'dia'])
    )

serie_por_dominio['primeiro_dia'] = serie_por_dominio.groupby('dominio')['dia'].
    ↪transform('min')
serie_por_dominio['ultimo_dia']    = serie_por_dominio.groupby('dominio')['dia'].
    ↪transform('max')

display(serie_por_dominio.head(30))

```

	dominio	dia	frequencia	primeiro_dia \
0	a.msn.com	2022-09-03	1	2022-09-03
1	a.msn.com	2022-09-04	1	2022-09-03
2	brasil247.com	2022-09-26	2	2022-09-26
3	brasil247.com	2022-11-23	1	2022-09-26
4	correiobraziliense.com.br	2022-09-14	1	2022-09-14
5	correiobraziliense.com.br	2022-09-17	1	2022-09-14
6	dadosabertos.tse.jus.br	2022-11-04	1	2022-11-04
7	dadosabertos.tse.jus.br	2022-11-05	1	2022-11-04
8	docs.google.com	2022-11-04	1	2022-11-04
9	docs.google.com	2022-11-05	1	2022-11-04
10	drive.google.com	2022-11-04	2	2022-11-04
11	fiscaisdobolsonaro.com.br	2022-10-24	2	2022-10-24
12	g1.globo.com	2022-09-24	1	2022-09-24
13	g1.globo.com	2022-10-01	1	2022-09-24
14	g1.globo.com	2022-10-23	1	2022-09-24
15	g1.globo.com	2022-11-08	1	2022-09-24
16	google.com	2022-11-06	1	2022-11-06
17	google.com	2022-11-15	1	2022-11-06
18	instagram.com	2022-09-07	1	2022-09-07
19	instagram.com	2022-11-20	1	2022-09-07
20	instagram.com	2022-11-27	1	2022-09-07
21	msn.com	2022-09-04	1	2022-09-04
22	msn.com	2022-11-26	1	2022-09-04
23	noticias.uol.com.br	2022-09-15	1	2022-09-15
24	noticias.uol.com.br	2022-10-04	1	2022-09-15
25	noticias.uol.com.br	2022-11-10	1	2022-09-15
26	open.spotify.com	2022-08-31	2	2022-08-31
27	page.brasilparalelo.com.br	2022-09-11	1	2022-09-11
28	public.tableau.com	2022-11-13	2	2022-11-13
29	resultados.tse.jus.br	2022-10-05	5	2022-10-05

	ultimo_dia
0	2022-09-04
1	2022-09-04
2	2022-11-23

```

3  2022-11-23
4  2022-09-17
5  2022-09-17
6  2022-11-05
7  2022-11-05
8  2022-11-05
9  2022-11-05
10 2022-11-04
11 2022-10-24
12 2022-11-08
13 2022-11-08
14 2022-11-08
15 2022-11-08
16 2022-11-15
17 2022-11-15
18 2022-11-27
19 2022-11-27
20 2022-11-27
21 2022-11-26
22 2022-11-26
23 2022-11-10
24 2022-11-10
25 2022-11-10
26 2022-08-31
27 2022-09-11
28 2022-11-13
29 2022-11-07

```

5.3.2.6 Engajamento comparativo

```

[151]: # Teste estatístico (t-test ou Mann-Whitney)
# from scipy.stats import mannwhitneyu
# retweets_desinfo = df[df['is_disinformation']==1]['retweets']
# retweets_verdade = df[df['is_disinformation']==0]['retweets']
# stat, p = mannwhitneyu(retweets_desinfo, retweets_verdade,
# → alternative='greater')
# print(f"Teste de Mann-Whitney: retweets em desinformação > verdade? p = {p:.
→ 4f}")

```

```

[152]: # Correlação entre desinformação e engajamento (correlação de ponto-bisserial)
from scipy.stats import pointbiserialr
corr_retweet, _ = pointbiserialr(df['is_disinformation'], df['retweets'])
corr_likes, _ = pointbiserialr(df['is_disinformation'], df['likes'])
print(f"Correlação entre desinformação e retweets: {corr_retweet:.3f}")
print(f"Correlação entre desinformação e curtidas: {corr_likes:.3f}")

```

Correlação entre desinformação e retweets: -0.009

Correlação entre desinformação e curtidas: -0.003

```
[153]: # =====
# DESINFORMAÇÃO x AMPLIFICAÇÃO
# =====

# Usar somente tweets originais para análise de engajamento
df_original = df[df['is_retweet'] == 0].copy()

# Garantir que a variável seja categórica
df_original['grupo_desinfo'] = df_original['is_disinformation'].map({
    0: 'Não desinformação',
    1: 'Desinformação'
})

# Estatísticas descritivas
tabela_amplificacao = (
    df_original
    .groupby('grupo_desinfo')[
        [
            'retweets',
            'likes',
            'engagement_raw',
            'engagement_rate',
            'engagement_index'
        ]
    ]
    .agg(['count', 'mean', 'median'])
)

display(tabela_amplificacao)
```

	retweets			likes			
	count	mean	median	count	mean	median	\
grupo_desinfo							
Desinformação	4022	0.016658	0.0	4022	0.123570	0.0	
Não desinformação	37392	8.585500	0.0	37392	2.600663	0.0	

	engagement_raw			engagement_rate			
	count	mean	median	count	mean		\
grupo_desinfo							
Desinformação	4022	0.140229	0.0	4022	0.003807		
Não desinformação	37392	11.186163	0.0	37392	1.078439		

	engagement_index				
	median	count	mean	median	
grupo_desinfo					
Desinformação	0.0	4022	0.027260	0.0	
Não desinformação	0.0	37392	0.079733	0.0	

```
[154]: # =====
# AMPLIFICAÇÃO ROBUSTA DA DESINFORMAÇÃO
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import mannwhitneyu

# -----
# 1. Trabalhar somente com tweets originais
# -----

df_original = df[df['is_retweet'] == 0].copy()

print("Total de tweets originais:", len(df_original))

# -----
# 2. Variáveis de engajamento existentes
# -----

metricas_engajamento = [
    'retweets',
    'likes',
    'engagement_raw',
    'engagement_rate',
    'engagement_index'
]

# -----
# 3. Estatísticas descritivas
# -----

resultados_amplificacao = []

for variavel in metricas_engajamento:

    desinfo = df_original.loc[
        df_original['is_disinformation'] == 1,
        variavel
    ].dropna()

    nao_desinfo = df_original.loc[
        df_original['is_disinformation'] == 0,
        variavel
    ].dropna()
```

```

resultados_amplificacao.append({
    'Métrica': variavel,

    'N_Desinfo': len(desinfo),
    'N_Nao_Desinfo': len(nao_desinfo),

    'Media_Desinfo': desinfo.mean(),
    'Media_Nao_Desinfo': nao_desinfo.mean(),

    'Mediana_Desinfo': desinfo.median(),
    'Mediana_Nao_Desinfo': nao_desinfo.median(),

    'P75_Desinfo': desinfo.quantile(0.75),
    'P75_Nao_Desinfo': nao_desinfo.quantile(0.75),

    'P90_Desinfo': desinfo.quantile(0.90),
    'P90_Nao_Desinfo': nao_desinfo.quantile(0.90),

    'P95_Desinfo': desinfo.quantile(0.95),
    'P95_Nao_Desinfo': nao_desinfo.quantile(0.95)
})

tabela_amplificacao = pd.DataFrame(resultados_amplificacao)

print("\n=== AMPLIFICAÇÃO - ESTATÍSTICAS DESCRITIVAS ===")

display(
    tabela_amplificacao.round(4)
)

```

Total de tweets originais: 41414

=== AMPLIFICAÇÃO - ESTATÍSTICAS DESCRITIVAS ===

	Métrica	N_Desinfo	N_Nao_Desinfo	Media_Desinfo	\
0	retweets	4022	37392	0.0167	
1	likes	4022	37392	0.1236	
2	engagement_raw	4022	37392	0.1402	
3	engagement_rate	4022	37392	0.0038	
4	engagement_index	4022	37392	0.0273	

	Media_Nao_Desinfo	Mediana_Desinfo	Mediana_Nao_Desinfo	P75_Desinfo	\
0	8.5855	0.0	0.0	0.0	
1	2.6007	0.0	0.0	0.0	
2	11.1862	0.0	0.0	0.0	
3	1.0784	0.0	0.0	0.0	
4	0.0797	0.0	0.0	0.0	

	P75_Nao_Desinfo	P90_Desinfo	P90_Nao_Desinfo	P95_Desinfo	P95_Nao_Desinfo
0	0.0	0.0	0.0	0.0	0.0000
1	0.0	0.0	0.0	0.0	1.0000
2	0.0	0.0	0.0	0.0	2.0000
3	0.0	0.0	0.0	0.0	0.0033
4	0.0	0.0	0.0	0.0	0.0300

```
[155]: # -----
# Teste de Mann-Whitney
# -----

resultados_testes = []

for variavel in metricas_engajamento:

    desinfo = df_original.loc[
        df_original['is_disinformation'] == 1,
        variavel
    ].dropna()

    nao_desinfo = df_original.loc[
        df_original['is_disinformation'] == 0,
        variavel
    ].dropna()

    stat, p = mannwhitneyu(
        desinfo,
        nao_desinfo,
        alternative='two-sided'
    )

    resultados_testes.append({
        'Métrica': variavel,
        'U': stat,
        'p_valor': p
    })

tabela_testes = pd.DataFrame(resultados_testes)

print("\n=== TESTE DE MANN-WHITNEY ===")

display(
    tabela_testes.round(6)
)
```

=== TESTE DE MANN-WHITNEY ===

Métrica	U	p_valor
---------	---	---------

0	retweets	73807466.0	0.0
1	likes	71042370.0	0.0
2	engagement_raw	70780686.0	0.0
3	engagement_rate	71129839.5	0.0
4	engagement_index	71874514.5	0.0

5.3.2.7 Concentração da produção

```
[156]: # =====
# CONCENTRAÇÃO DA DESINFORMAÇÃO POR USUÁRIO
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# -----
# 1. Contar tweets de desinformação por usuário
# -----

desinfo_por_usuario = (
    df[df['is_disinformation'] == 1]
    .groupby('user_id')
    .size()
    .sort_values(ascending=False)
)

print("Número de usuários que publicaram desinformação:",
      len(desinfo_por_usuario))

print("Total de tweets de desinformação:",
      desinfo_por_usuario.sum())

# -----
# 2. Função para calcular concentração
# -----

def calcular_concentracao(series, percentual):

    n_usuarios = len(series)

    n_top = max(
        1,
        int(np.ceil(n_usuarios * percentual))
    )

    top_usuarios = series.head(n_top)
```

```

tweets_top = top_usuarios.sum()

total_tweets = series.sum()

proporcao = tweets_top / total_tweets

return {
    'Percentual_Usuarios': percentual * 100,
    'Numero_Usuarios': n_top,
    'Tweets_Desinformacao': tweets_top,
    'Proporcao_Desinformacao': proporcao * 100
}

```

Número de usuários que publicaram desinformação: 3630
 Total de tweets de desinformação: 4022

```

[157]: # =====
# CONCENTRAÇÃO DA DESINFORMAÇÃO
# =====

desinfo_usuarios = (
    df[df['is_disinformation'] == 1]
    .groupby('user_id')
    .agg(
        tweets_desinformacao=('tweet_id', 'count'),
        retweets_recebidos=('retweets', 'sum'),
        likes_recebidos=('likes', 'sum'),
        engajamento=('engagement_index', 'sum')
    )
    .reset_index()
    .sort_values(
        'tweets_desinformacao',
        ascending=False
    )
)

display(desinfo_usuarios.head(20))

```

	user_id	tweets_desinformacao	retweets_recebidos	\
1676	@USER_7AC76A59AB76	24	0	
469	@USER_2175B20B9366	19	0	
3355	@USER_EDCACB49CCF5	8	0	
953	@USER_431F25A8F354	8	0	
538	@USER_25AFCE56D558	7	0	
310	@USER_1681FA3A1602	6	0	
1882	@USER_889FDBFA5B29	6	0	
1114	@USER_4EEB5C5D3859	5	0	
374	@USER_1B2B98E0FB8F	5	0	

3537	@USER_F8C93C23C2BB	5	0
287	@USER_14E8A8278FA5	5	0
945	@USER_41FF6DEADC99	5	0
309	@USER_16764FFD280D	5	0
3464	@USER_F4606F0419C4	4	1
2018	@USER_90C41F44E87A	4	0
1190	@USER_54A26A891076	4	0
1109	@USER_4EA5CE5A7881	4	0
1658	@USER_79809B186B1E	4	0
1781	@USER_82245B94A83E	4	0
3055	@USER_D913EEFDOEFE	4	0

	likes_recebidos	engajamento
1676	0	0.00
469	0	0.00
3355	0	0.00
953	0	0.00
538	0	0.00
310	5	0.81
1882	0	0.00
1114	0	0.00
374	0	0.00
3537	0	0.00
287	0	0.00
945	2	0.20
309	0	0.00
3464	2	0.01
2018	19	9.83
1190	0	0.00
1109	1	0.04
1658	0	0.00
1781	0	0.00
3055	0	0.00

```
[158]: # -----
# Top 10 usuários
# -----

top_10 = (
    desinfo_por_usuario
    .head(10)
    .reset_index()
)

top_10.columns = [
    'user_id',
    'tweets_desinformacao'
```

```

]

top_10['proporcao_total_desinfo'] = (
    top_10['tweets_desinformacao']
    / desinfo_por_usuario.sum()
    * 100
)

print("\n=== TOP 10 DISSEMINADORES ===")

display(
    top_10.round(2)
)

```

=== TOP 10 DISSEMINADORES ===

	user_id	tweets_desinformacao	proporcao_total_desinfo
0	@USER_7AC76A59AB76	24	0.60
1	@USER_2175B20B9366	19	0.47
2	@USER_EDCACB49CCF5	8	0.20
3	@USER_431F25A8F354	8	0.20
4	@USER_25AFCE56D558	7	0.17
5	@USER_1681FA3A1602	6	0.15
6	@USER_889FDBFA5B29	6	0.15
7	@USER_4EEB5C5D3859	5	0.12
8	@USER_1B2B98E0FB8F	5	0.12
9	@USER_F8C93C23C2BB	5	0.12

```

[159]: # =====
# CONCENTRAÇÃO - TOP 1%, 5% E 10%
# =====

desinfo_usuarios = desinfo_usuarios.sort_values(
    'tweets_desinformacao',
    ascending=False
).reset_index(drop=True)

total_desinfo = desinfo_usuarios['tweets_desinformacao'].sum()
n_usuarios = len(desinfo_usuarios)

resultados_concentracao = []

for percentual in [0.01, 0.05, 0.10]:

    n_top = max(1, int(n_usuarios * percentual))

    tweets_top = (

```

```

        desinfo_usuarios
        .head(n_top)['tweets_desinformacao']
        .sum()
    )

    proporcao = tweets_top / total_desinfo * 100

    resultados_concentracao.append({
        'Grupo': f'Top {percentual:.0%}',
        'Usuários': n_top,
        'Tweets de desinformação': tweets_top,
        '% da desinformação': proporcao
    })

concentracao = pd.DataFrame(resultados_concentracao)

display(concentracao)

```

	Grupo	Usuários	Tweets de desinformação	% da desinformação
0	Top 1%	36	193	4.798608
1	Top 5%	181	516	12.829438
2	Top 10%	363	755	18.771755

```

[160]: # -----
# Top 1%, 5% e 10%
# -----

percentuais = [0.01, 0.05, 0.10]

resultados_concentracao = []

for percentual in percentuais:

    resultado = calcular_concentracao(
        desinfo_por_usuario,
        percentual
    )

    resultados_concentracao.append(resultado)

tabela_concentracao = pd.DataFrame(
    resultados_concentracao
)

print("\n=== CONCENTRAÇÃO DA DESINFORMAÇÃO ===")

```

```
display(
    tabela_concentracao.round(2)
)
```

=== CONCENTRAÇÃO DA DESINFORMAÇÃO ===

	Percentual_Usuarios	Numero_Usuarios	Tweets_Desinformacao \
0	1.0	37	196
1	5.0	182	518
2	10.0	363	755

	Proporcao_Desinformacao
0	4.87
1	12.88
2	18.77

```
[161]: # =====
# Série de PRODUÇÃO (apenas tweets originais)
# =====
prod = (
    df_sem_retweets[df_sem_retweets['is_disinformation'] == 1]
    .groupby('user_id')
    .size()
    .sort_values(ascending=False)
)

# Sanity check
print(f"Produção:    {len(prod)} usuários, {prod.sum()} tweets")
```

Produção: 3630 usuários, 4022 tweets

```
[162]: import numpy as np
import matplotlib.pyplot as plt

def gini(x):
    x = np.sort(x)
    n = len(x)
    cum = np.cumsum(x)
    return (n + 1 - 2 * sum(cum) / cum[-1]) / n

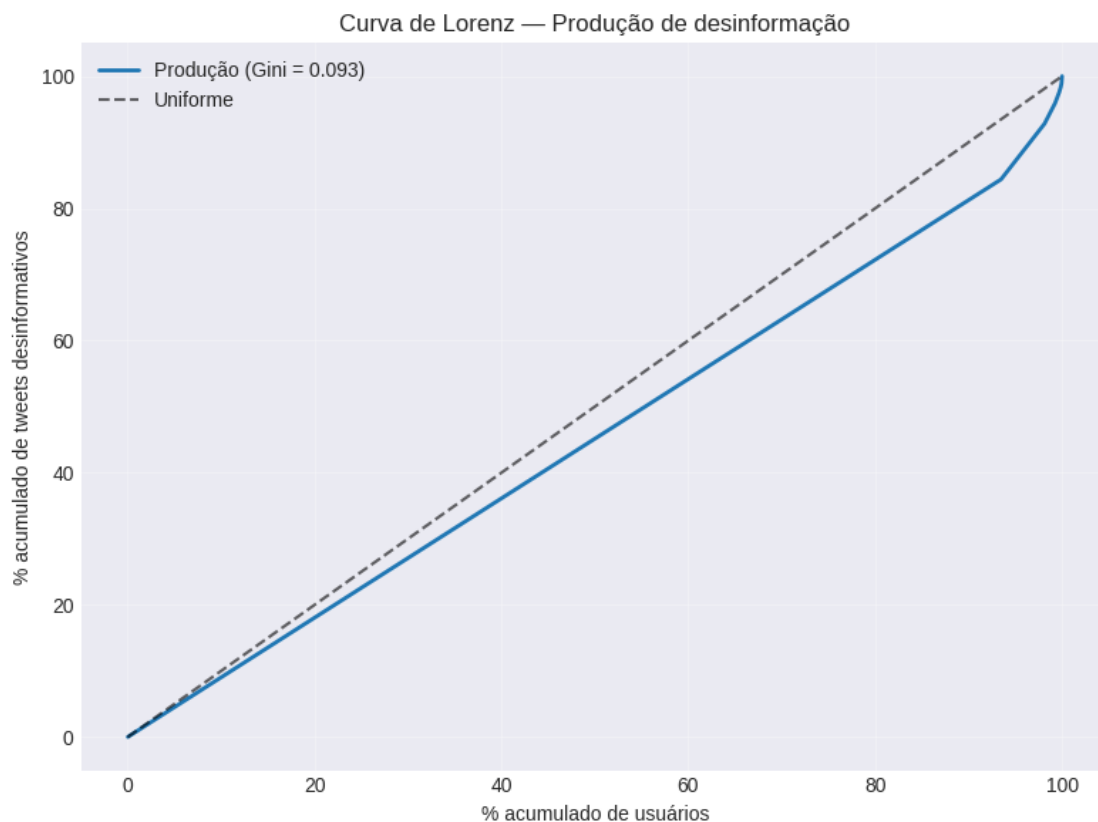
def lorenz(x):
    x = np.sort(x)
    cum = np.cumsum(x)
    return np.arange(1, len(x)+1)/len(x), cum/cum[-1]
```

```
[163]: gini_prod = gini(prod.values)
print(f"Gini produção:    {gini_prod:.3f}")
```

Gini produção: 0.093

```
[164]: gini_prod = gini(prod.values)
p, l = lorenz(prod.values)

plt.figure(figsize=(8, 6))
plt.plot(p*100, l*100, color='#1f77b4', linewidth=2,
         label=f'Produção (Gini = {gini_prod:.3f})')
plt.plot([0, 100], [0, 100], 'k--', alpha=0.6, label='Uniforme')
plt.xlabel('% acumulado de usuários')
plt.ylabel('% acumulado de tweets desinformativos')
plt.title('Curva de Lorenz - Produção de desinformação')
plt.legend(loc='upper left')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/lorenz_producao.png',
            dpi=300, bbox_inches='tight')
plt.show()
```



```
[168]: def resumo_concentracao(serie, nome):
total = serie.sum()
```

```

n = len(serie)
linhas = [{
    'Recorte': nome,
    'N_usuarios': n,
    'N_tweets': total,
    '%': 100.0,
    'Gini': round(gini(serie.values), 3)
}]
for pct in [0.01, 0.05, 0.10]:
    n_top = max(1, int(np.ceil(n * pct)))
    tweets_top = serie.head(n_top).sum()
    linhas.append({
        'Recorte': f'{nome} - Top {pct:.0%}',
        'N_usuarios': n_top,
        'N_tweets': int(tweets_top),
        '%': round(100 * tweets_top / total, 2),
        'Gini': None
    })
return pd.DataFrame(linhas)

tabela_prod = resumo_concentracao(prod, 'Produção')
display(tabela_prod)

```

	Recorte	N_usuarios	N_tweets	%	Gini
0	Produção	3630	4022	100.00	0.093
1	Produção - Top 1%	37	196	4.87	NaN
2	Produção - Top 5%	182	518	12.88	NaN
3	Produção - Top 10%	363	755	18.77	NaN

```

[224]: # Modelo nulo: se cada usuário publicasse Poisson(média)
media = prod.mean() # ~1,11
n = len(prod)

gini_nulo = []
for _ in range(1000):
    simulado = np.random.poisson(media, n)
    gini_nulo.append(gini(simulado))
gini_nulo = np.array(gini_nulo)

p_valor = (gini_nulo <= gini_prod).mean()
print(f"Gini observado: {gini_prod:.3f}")
print(f"Gini nulo (Poisson): {gini_nulo.mean():.3f} ± {gini_nulo.std():.3f}")
print(f"p-valor (observado nulo): {p_valor:.4f}")

```

```

Gini observado: 0.093
Gini nulo (Poisson): 0.502 ± 0.007
p-valor (observado nulo): 0.0000

```

5.3.2.8 Cascatas de conversação

```
[ ]: # =====
# CASCATAS DE CONVERSAÇÃO (RESPOSTAS)
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# -----
# 1. CONFIGURAÇÃO E LIMPEZA
# -----

df['tweet_created_at'] = pd.to_datetime(df['tweet_created_at'], errors='coerce')
df['tweet_id'] = df['tweet_id'].astype(str)
df['reply_to_tweet_id'] = df['reply_to_tweet_id'].astype(str)

# -----
# 2. IDENTIFICAR OS TWEETS RAIZ (INÍCIO DA CASCATA)
# -----
# AGORA: todos os tweets que NÃO são retweets (is_retweet==0)
roots = df[df['is_retweet'] == 0].copy()
print(f"Total de tweets raiz (originais) encontrados: {len(roots)}")

# -----
# 3. CONSTRUIR MAPA DE RELACIONAMENTO (PAI -> FILHOS)
# -----
parent_to_children = {}
for _, row in df[df['reply_to_tweet_id'].notna()].iterrows():
    parent = row['reply_to_tweet_id']
    child = row['tweet_id']
    parent_to_children.setdefault(parent, []).append(child)

# -----
# 4. FUNÇÃO PARA EXTRAIR TODA A CASCATA (BFS)
# -----
def get_cascade_data(root_id):
    visited = set()
    queue = [root_id]
    all_ids = []
    all_times = []

    while queue:
        current_id = queue.pop(0)
        if current_id in visited:
            continue
        visited.add(current_id)
```

```

row = df[df['tweet_id'] == current_id]
if not row.empty:
    all_ids.append(current_id)
    all_times.append(row['tweet_created_at'].iloc[0])

for child in parent_to_children.get(current_id, []):
    if child not in visited:
        queue.append(child)

sorted_data = sorted(zip(all_ids, all_times), key=lambda x: x[1])
sorted_ids = [x[0] for x in sorted_data]
sorted_times = [x[1] for x in sorted_data]

return {
    'ids': sorted_ids,
    'times': sorted_times,
    'total_respostas': len(sorted_ids) - 1
}

# -----
# 5. APLICAR A FUNÇÃO PARA TODAS AS RAIZES
# -----
cascatas = []

for root_id in roots['tweet_id'].values:
    root_row = roots[roots['tweet_id'] == root_id].iloc[0]
    cascata = get_cascade_data(root_id)

    if cascata['total_respostas'] > 0:  # só guarda se houve pelo menos uma
↳resposta
        first_time = cascata['times'][0]
        last_time = cascata['times'][-1]

    cascatas.append({
        'tweet_original_id': root_id,
        'usuario_original': root_row['user_id'],
        'topico': root_row.get('topic', np.nan),
        'sentimento': root_row.get('sentiment', np.nan),
        'cluster_origem': root_row.get('cluster', np.nan),
        'desinformacao': root_row['is_disinformation'],
        'n_respostas': cascata['total_respostas'],
        'n_usuarios': df[df['tweet_id'].isin(cascata['ids'])]['user_id'].
↳nunique(),
        'data_original': first_time,
        'ultima_resposta': last_time,

```

```

        'primeira_resposta': cascata['times'][1] if len(cascata['times']) > 1
→1 else np.nan
    })

cascatas_df = pd.DataFrame(cascatas)

# -----
# 6. CALCULAR DURAÇÃO E VELOCIDADE
# -----
if not cascatas_df.empty:
    cascatas_df['duracao_minutos'] = (
        cascatas_df['ultima_resposta'] - cascatas_df['data_original']
    ).dt.total_seconds() / 60

    cascatas_df['tempo_primeira_resposta_min'] = (
        cascatas_df['primeira_resposta'] - cascatas_df['data_original']
    ).dt.total_seconds() / 60

    cascatas_df['duracao_horas'] = cascatas_df['duracao_minutos'] / 60
    cascatas_df['velocidade_resposta_hora'] = (
        cascatas_df['n_respostas'] / cascatas_df['duracao_horas'].replace(0, np.
→nan)
    )

# -----
# 7. RESUMO POR DESINFORMAÇÃO (SÓ SE HOUVER DADOS)
# -----
if not cascatas_df.empty:
    resumo = (cascatas_df.groupby('desinformacao').agg(
        cascatas=('tweet_original_id', 'count'),
        media_respostas=('n_respostas', 'mean'),
        mediana_respostas=('n_respostas', 'median'),
        max_respostas=('n_respostas', 'max'),
        tempo_medio_primeira_resp=('tempo_primeira_resposta_min', 'mean'),
        duracao_media_min=('duracao_minutos', 'mean'),
        velocidade_media=('velocidade_resposta_hora', 'mean')
    ).reset_index())

    resumo['desinformacao'] = resumo['desinformacao'].map({0: 'Não_
→desinformação', 1: 'Desinformação'})
    print(" Resumo das cascatas de conversação:")
    display(resumo)
else:
    print(" Nenhuma cascata com pelo menos uma resposta foi encontrada.")

```

```

[ ]: # Distribuição do tamanho
plt.figure(figsize=(12, 6))

```

```
plt.hist(cascatas_df[cascatas_df['desinformacao']==0]['n_respostas'], bins=50,
        alpha=0.6, label='Não desinformação')
plt.hist(cascatas_df[cascatas_df['desinformacao']==1]['n_respostas'], bins=50,
        alpha=0.6, label='Desinformação')
plt.yscale('log')
plt.xlabel('Número de respostas na thread')
plt.ylabel('Número de cascatas (escala log)')
plt.title('Distribuição do tamanho das cascatas de conversação')
plt.legend()
plt.show()
```

```
[ ]: # =====
# VELOCIDADE - FAIXAS DE TEMPO ATÉ A PRIMEIRA RESPOSTA
# =====

# Remover linhas com NaN (tweets que tiveram resposta, mas sem timestamp válido)
temp_df = cascatas_df.dropna(subset=['tempo_primeira_resposta_min'])

if not temp_df.empty:

    # Criar as faixas de tempo (exatamente as mesmas do original)
    temp_df['faixa_velocidade'] = pd.cut(
        temp_df['tempo_primeira_resposta_min'],
        bins=[
            -np.inf,
            1,
            5,
            15,
            60,
            360,
            1440,
            np.inf
        ],
        labels=[
            '< 1 minuto',
            '1-5 minutos',
            '5-15 minutos',
            '15-60 minutos',
            '1-6 horas',
            '6-24 horas',
            '> 24 horas'
        ]
    )

    # Tabela cruzada com percentual por coluna (desinformação vs não)
    velocidade_faixas = pd.crosstab(
        temp_df['faixa_velocidade'],
```

```

        temp_df['desinformacao'],
        normalize='columns'
    ) * 100

    # Renomear as colunas para exibição clara
    velocidade_faixas.columns = [
        'Não desinformação',
        'Desinformação'
    ]

    print(" Distribuição percentual do tempo até a primeira resposta:")
    display(velocidade_faixas)

```

```

[ ]: # Plotar o gráfico de barras
velocidade_faixas.plot(
    kind='bar',
    figsize=(12, 6),
    color=['#1f77b4', '#d62728'] # Azul e vermelho para contraste
)

plt.xlabel('Tempo até a primeira resposta')
plt.ylabel('Percentual das cascatas de conversação (%)')
plt.title(
    'Velocidade inicial de engajamento: '
    'Desinformação × Não desinformação (Respostas)'
)

plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

```

```

[ ]: # Mann-Whitney para n_respostas
from scipy.stats import mannwhitneyu
c_des = cascatas_df[cascatas_df['desinformacao']==1]['n_respostas']
c_nao = cascatas_df[cascatas_df['desinformacao']==0]['n_respostas']
stat, p = mannwhitneyu(c_des, c_nao, alternative='two-sided')
print(f"Cascatas n_respostas: U={stat:.0f}, p={p:.4f}")

# Nota explícita sobre N
print(f"N desinfo = {len(c_des)} (pequeno, resultados exploratórios)")
print(f"N não-desinfo = {len(c_nao)}")

```

1.4.3 5.3.3 Aplicação

```
[169]: # Considerando apenas retweets
df = df_retweets.copy()

# 3. Visão aplicação da classificação
total_desinfo = df['is_disinformation'].sum()
total_nao_desinfo = len(df) - total_desinfo
print(f"Total de retweets: {len(df)}")
print(f"Desinformação: {total_desinfo} ({total_desinfo/len(df):.2%})")
print(f"Não desinformação: {total_nao_desinfo} ({total_nao_desinfo/len(df):.2%})")
```

Total de retweets: 69564

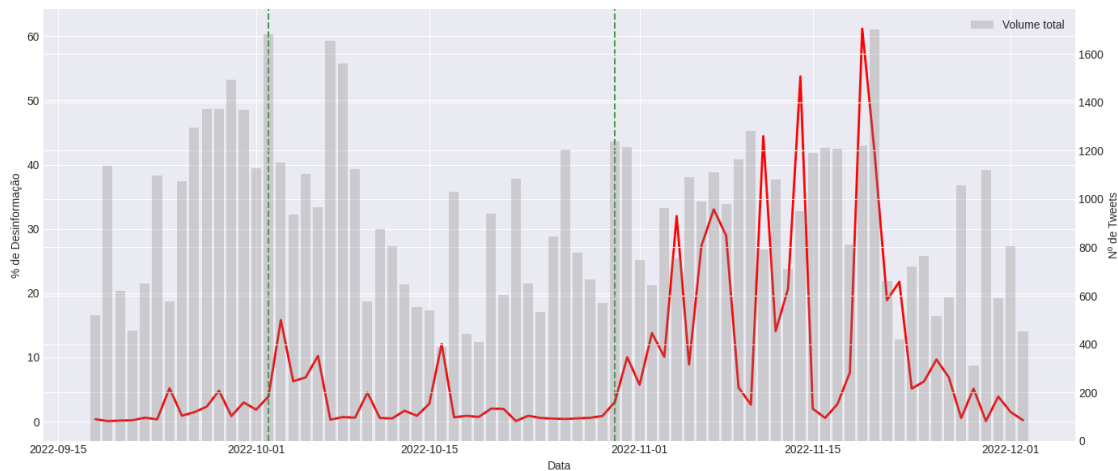
Desinformação: 5883 (8.46%)

Não desinformação: 63681 (91.54%)

5.3.3.1 Distribuição temporal

```
[170]: # 2. Distribuição temporal da desinformação (proporção diária)
desinfo_diario = df.groupby(df['tweet_created_at'].dt.date)['is_disinformation'].
    →mean() * 100
volume_diario = df.groupby(df['tweet_created_at'].dt.date).size()

fig, ax = plt.subplots(figsize=(14,6))
ax.plot(desinfo_diario.index, desinfo_diario, color='red', linewidth=2, label='%
    →Desinformação')
ax.set_ylabel('% de Desinformação')
ax2 = ax.twinx()
ax2.bar(volume_diario.index, volume_diario, alpha=0.3, color='gray',
    →label='Volume total')
ax2.set_ylabel('Nº de Tweets')
ax.set_xlabel('Data')
# Destacar dias de votação
for dia in ['2022-10-02', '2022-10-30']:
    ax.axvline(pd.to_datetime(dia), color='green', linestyle='--', alpha=0.7)
# plt.title('Proporção diária de desinformação sobre as urnas')
plt.legend()
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/desinformacao_temporal.png',
    →dpi=300)
plt.show()
```

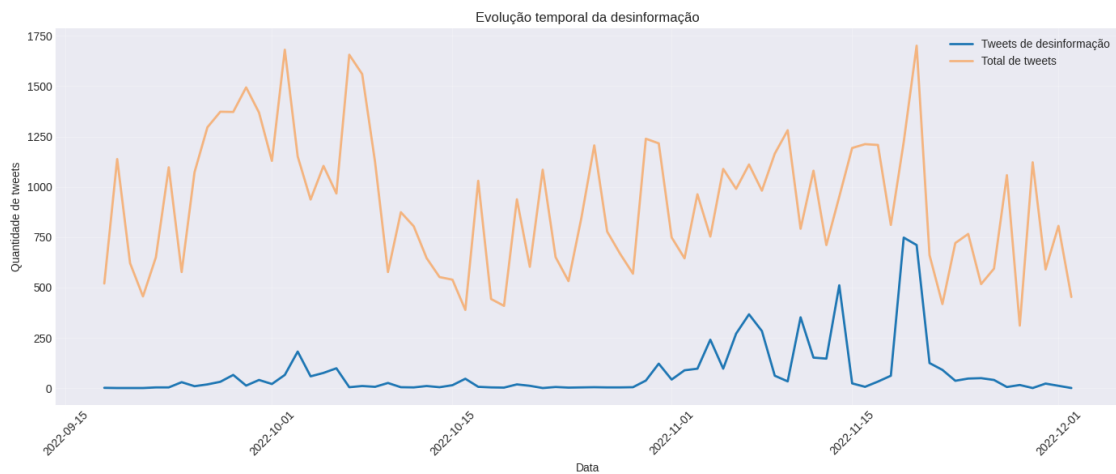


```
[171]: # =====
# ANÁLISE 1 - DESINFORMAÇÃO AO LONGO DO TEMPO
# =====
df['tweet_created_at'] = pd.to_datetime(df['tweet_created_at'])
# Quantidade total e quantidade de desinformação por dia
desinfo_dia = (
df.groupby(df['tweet_created_at'].dt.date)
.agg(
total_tweets=('tweet_id', 'count'),
tweets_desinformacao=('is_disinformation', 'sum')
)
.reset_index()
)
desinfo_dia['tweet_created_at'] = pd.to_datetime(
desinfo_dia['tweet_created_at']
)
# Proporção de desinformação
desinfo_dia['prop_desinformacao'] = (
desinfo_dia['tweets_desinformacao'] /
desinfo_dia['total_tweets']
)
# Gráfico da quantidade
plt.figure(figsize=(14, 6))
plt.plot(desinfo_dia['tweet_created_at'],
desinfo_dia['tweets_desinformacao'],
linewidth=2,
label='Tweets de desinformação'
)
plt.plot(
desinfo_dia['tweet_created_at'],
```

```

desinfo_dia['total_tweets'],
linewidth=2,
alpha=0.5,
label='Total de tweets'
)
plt.xlabel('Data')
plt.ylabel('Quantidade de tweets')
plt.title('Evolução temporal da desinformação')
plt.xticks(rotation=45)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

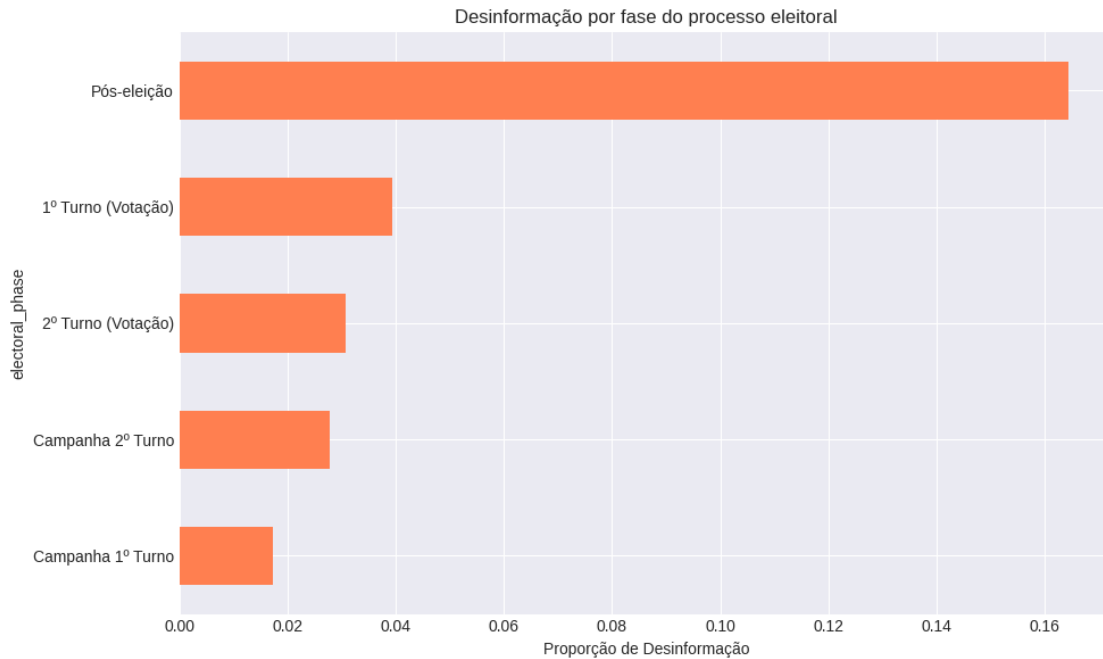
```



```

[172]: # 3. Desinformação por fase eleitoral
fase_desinfo = df.groupby('electoral_phase')['is_disinformation'].mean().
↳ sort_values()
fase_desinfo.plot(kind='barh', figsize=(10,6), color='coral')
plt.xlabel('Proporção de Desinformação')
plt.title('Desinformação por fase do processo eleitoral')
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/desinfo_por_fase.png', dpi=300)
plt.show()

```



5.3.3.2 Tamanho tweet

```
[173]: import pandas as pd
import matplotlib.pyplot as plt

# 1. Criar categoria de comprimento
df['length_category'] = pd.cut(
    df['tweet_length'],
    bins=[0, 50, 140, 500],
    right=True,
    labels=['Pequeno (<50)', 'Médio (51-140)', 'Longo (141-280)'],
    include_lowest=True
)

# 2. Contagem cruzada
tabela = pd.crosstab(df['length_category'], df['is_disinformation'])

# 3. Gráfico único
fig, ax = plt.subplots(figsize=(8, 6))
tabela.plot(kind='bar', ax=ax, width=0.7, rot=15)

for container in ax.containers:
    ax.bar_label(container, label_type='edge', padding=2, fontsize=9)

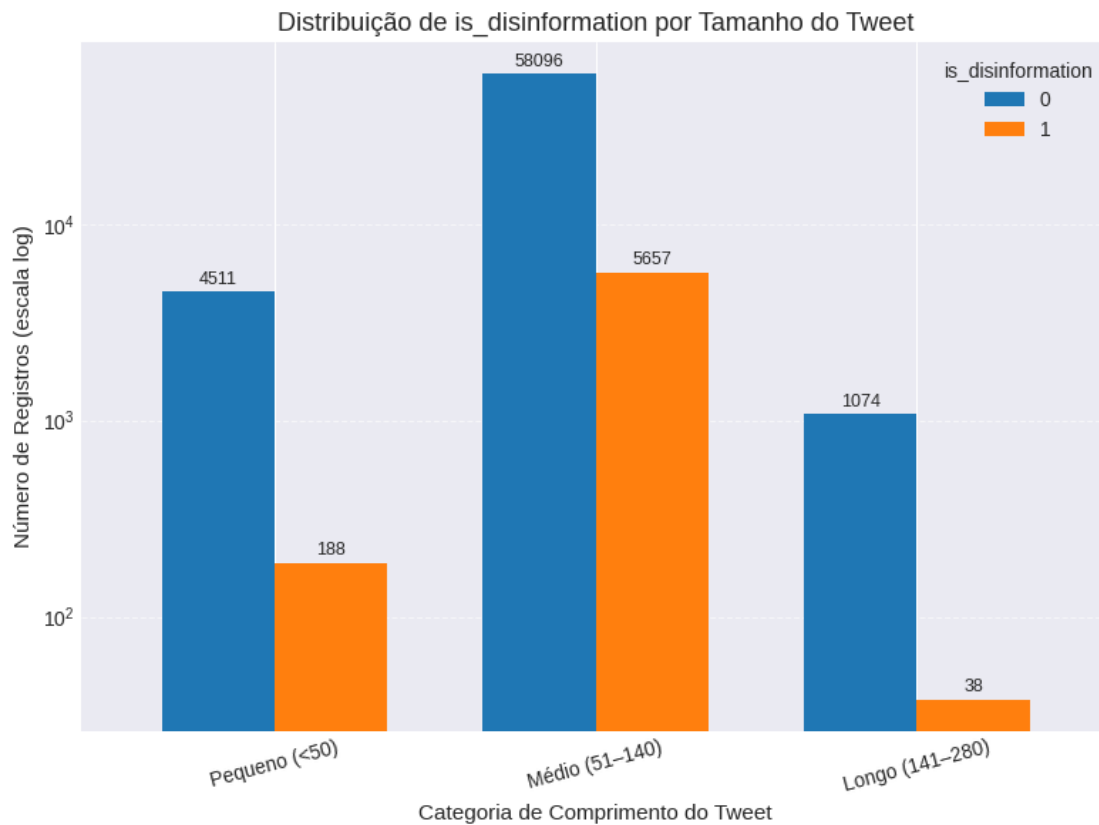
ax.set_ylabel('Número de Registros (escala log)', fontsize=11)
ax.set_xlabel('Categoria de Comprimento do Tweet', fontsize=11)
```

```

ax.set_title('Distribuição de is_disinformation por Tamanho do Tweet',
             ↪fontsize=13)
ax.legend(title='is_disinformation')
ax.grid(axis='y', linestyle='--', alpha=0.6)
ax.set_yscale('log')

plt.tight_layout()
plt.show()

```



```
[174]: tabela
```

```

[174]: is_disinformation    0    1
length_category
Pequeno (<50)           4511   188
Médio (51-140)        58096  5657
Longo (141-280)       1074    38

```

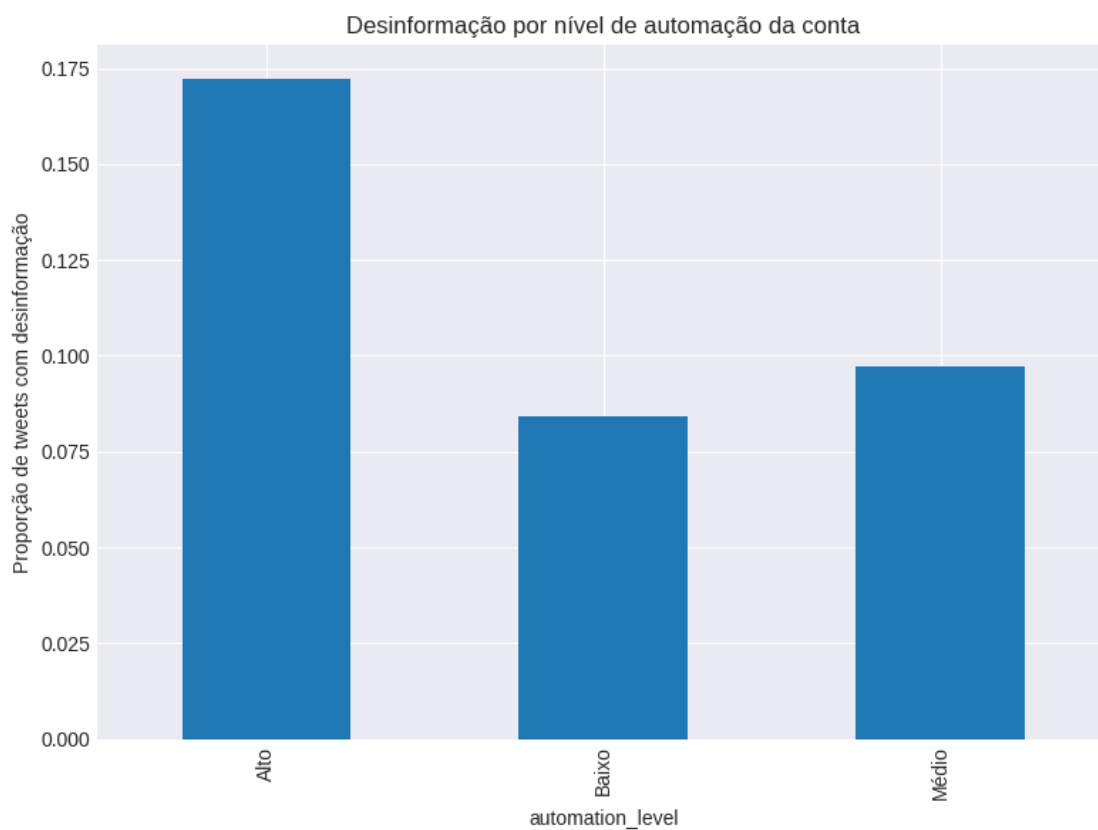
5.3.3.3 Atores (Usuários)

```
[175]: # Quem amplifica? (por nível de automação)
```

```

automacao_desinfo = df.groupby(['automation_level', 'is_disinformation']).size().
    ↳unstack(fill_value=0)
automacao_desinfo['total'] = automacao_desinfo.sum(axis=1)
automacao_desinfo['prop_desinfo'] = automacao_desinfo[1] / ◻
    ↳automacao_desinfo['total']
automacao_desinfo[['prop_desinfo']].plot(kind='bar', figsize=(8,6), legend=False)
plt.ylabel('Proporção de tweets com desinformação')
plt.title('Desinformação por nível de automação da conta')
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/desinfo_por_automacao.png', ◻
    ↳dpi=300)
plt.show()

```



```

[176]: # =====
# DESINFORMAÇÃO × AUTOMAÇÃO
# =====

# Tabela absoluta
tabela_automacao = pd.crosstab(
    df['automation_level'],

```

```

    df['is_disinformation']
)

tabela_automacao.columns = [
    'Não desinformação',
    'Desinformação'
]

display(tabela_automacao)

```

	Não desinformação	Desinformação
automation_level		
Alto	24	5
Baixo	61296	5624
Médio	2361	254

[177]: *# Proporção de desinformação dentro de cada nível de automação*

```

tabela_automacao_prop = pd.crosstab(
    df['automation_level'],
    df['is_disinformation'],
    normalize='index'
) * 100

tabela_automacao_prop.columns = [
    'Não desinformação (%)',
    'Desinformação (%)'
]

display(tabela_automacao_prop.round(2))

```

	Não desinformação (%)	Desinformação (%)
automation_level		
Alto	82.76	17.24
Baixo	91.60	8.40
Médio	90.29	9.71

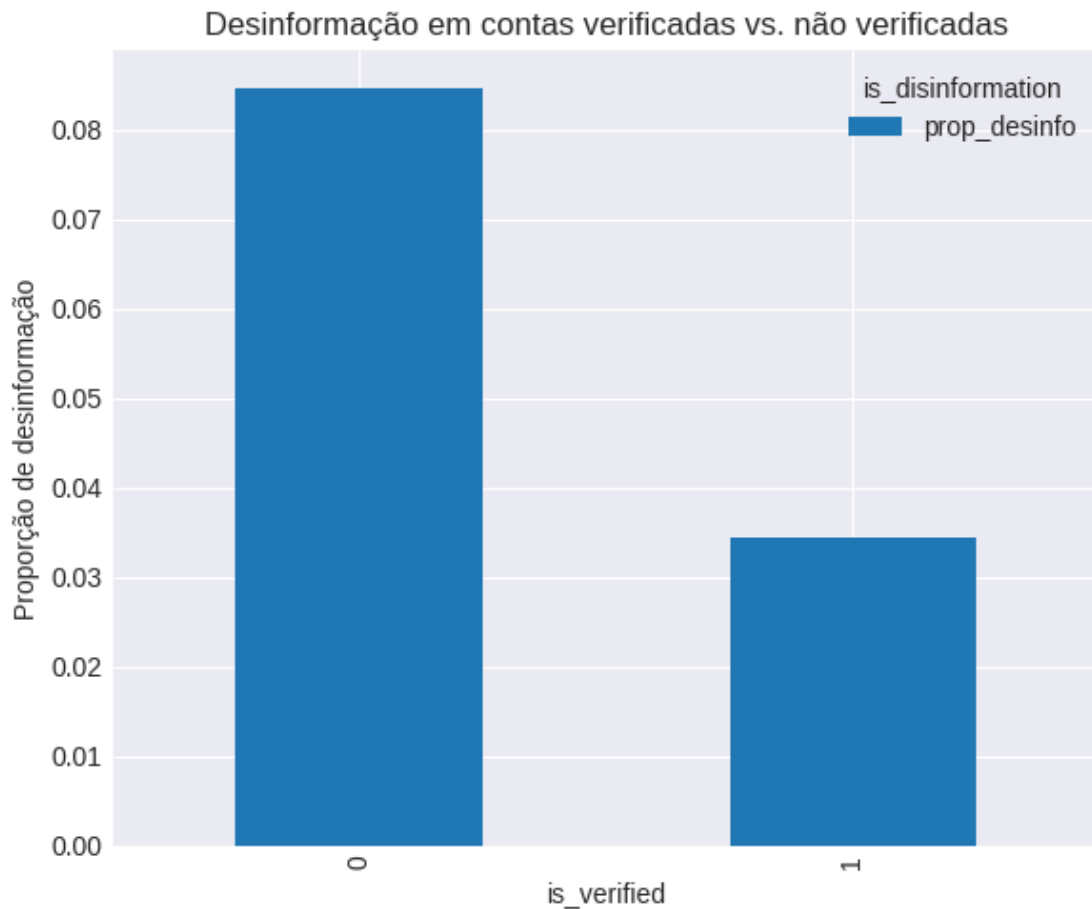
[178]: *# Amplificação por contas verificadas vs. não verificadas*

```

verificacao_desinfo = df.groupby(['is_verified', 'is_disinformation']).size().
    ↪unstack(fill_value=0)
verificacao_desinfo['prop_desinfo'] = verificacao_desinfo[1] /
    ↪verificacao_desinfo.sum(axis=1)
verificacao_desinfo[['prop_desinfo']].plot(kind='bar', figsize=(6,5))
plt.ylabel('Proporção de desinformação')
plt.title('Desinformação em contas verificadas vs. não verificadas')
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/desinfo_por_verificacao.png',
    ↪dpi=300)

```

```
plt.show()
```



Idade

```
[179]: from scipy.stats import mannwhitneyu

idades_desinfo = df[df['is_disinformation'] == 1]['account_age_days'].dropna()
idades_ao = df[df['is_disinformation'] == 0]['account_age_days'].dropna()

stat, p_valor = mannwhitneyu(idades_desinfo, idades_ao, alternative='less')
print(f" Teste de Mann-Whitney (idades_desinfo < idades_ao): p = {p_valor:.6f}")
if p_valor < 0.05:
    print(" Diferença estatisticamente significativa: contas que disseminam  

    ↳desinformação são mais novas.")
else:
    print(" Não há diferença estatística significativa entre os grupos.")
```

```
Teste de Mann-Whitney (idades_desinfo < idades_ao): p = 0.000000
Diferença estatisticamente significativa: contas que disseminam desinformação
```

são mais novas.

```
[180]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# =====
# Criar faixas etárias para as contas
# =====
# Definir os limites (em dias)
bins = [0, 30, 90, 180, 365, 730, 1825, float('inf')]
labels = ['< 1 mês', '1-3 meses', '3-6 meses', '6-12 meses',
          '1-2 anos', '2-5 anos', '> 5 anos']

# Aplicar a categorização
df['age_group'] = pd.cut(
    df['account_age_days'],
    bins=bins,
    labels=labels,
    right=False # intervalo [início, fim)
)

# Verificar a distribuição geral
print(" Distribuição geral das contas por faixa etária:")
display(df['age_group'].value_counts().sort_index())
```

Distribuição geral das contas por faixa etária:

```
age_group
< 1 mês      4469
1-3 meses    3937
3-6 meses    2807
6-12 meses   5419
1-2 anos     6117
2-5 anos    16566
> 5 anos    30249
Name: count, dtype: int64
```

```
[181]: # =====
# Calcular proporção de desinformação em cada faixa
# =====

age_desinfo = df.groupby('age_group', observed=False)['is_disinformation'].agg(
    total_tweets='count',
    tweets_desinfo='sum'
).reset_index()
```

```

age_desinfo['proporcao_desinfo'] = (
    age_desinfo['tweets_desinfo'] / age_desinfo['total_tweets'] * 100
)

# Ordenar pela idade (crescente)
age_desinfo = age_desinfo.sort_values(
    'age_group',
    key=lambda x: pd.Categorical(x, categories=labels, ordered=True)
)

print(" Proporção de desinformação por faixa etária da conta:")
display(age_desinfo)

```

Proporção de desinformação por faixa etária da conta:

	age_group	total_tweets	tweets_desinfo	proporcao_desinfo
0	< 1 mês	4469	489	10.942045
1	1-3 meses	3937	477	12.115824
2	3-6 meses	2807	222	7.908799
3	6-12 meses	5419	616	11.367411
4	1-2 anos	6117	492	8.043158
5	2-5 anos	16566	1337	8.070747
6	> 5 anos	30249	2250	7.438262

```

[182]: #-----
# Comparação: Desinformação vs Legítimo por faixa etária da conta
#-----

# Agregação por faixa etária
age_desinfo = df.groupby('age_group', observed=False)['is_disinformation'].agg(
    total_tweets='count',
    tweets_desinfo='sum'
).reset_index()

# Contagem de legítimos e proporções
age_desinfo['tweets_legit'] = age_desinfo['total_tweets'] -
    ↳ age_desinfo['tweets_desinfo']
age_desinfo['proporcao_desinfo'] = (
    age_desinfo['tweets_desinfo'] / age_desinfo['total_tweets'] * 100
)
age_desinfo['proporcao_legit'] = (
    age_desinfo['tweets_legit'] / age_desinfo['total_tweets'] * 100
)

# Ordenar pela idade (crescente)
age_desinfo = age_desinfo.sort_values(
    'age_group',
    key=lambda x: pd.Categorical(x, categories=labels, ordered=True)
)

```

```

).reset_index(drop=True)

print(" Proporção de desinformação por faixa etária da conta:")
display(age_desinfo)

```

Proporção de desinformação por faixa etária da conta:

	age_group	total_tweets	tweets_desinfo	tweets_legit	proporcao_desinfo \
0	< 1 mês	4469	489	3980	10.942045
1	1-3 meses	3937	477	3460	12.115824
2	3-6 meses	2807	222	2585	7.908799
3	6-12 meses	5419	616	4803	11.367411
4	1-2 anos	6117	492	5625	8.043158
5	2-5 anos	16566	1337	15229	8.070747
6	> 5 anos	30249	2250	27999	7.438262

	proporcao_legit
0	89.057955
1	87.884176
2	92.091201
3	88.632589
4	91.956842
5	91.929253
6	92.561738

```

[183]: #-----
# Barras agrupadas: contagem absoluta (desinfo vs legítimo)
#-----
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

x = np.arange(len(age_desinfo))
width = 0.4

# Contagem absoluta
b1 = axes[0].bar(x - width/2, age_desinfo['tweets_desinfo'], width,
                 label='Desinformação', color='#d62728')
b2 = axes[0].bar(x + width/2, age_desinfo['tweets_legit'], width,
                 label='Legítimo', color='#2ca02c')

axes[0].bar_label(b1, padding=2, fontsize=9)
axes[0].bar_label(b2, padding=2, fontsize=9)
axes[0].set_xticks(x)
axes[0].set_xticklabels(age_desinfo['age_group'], rotation=35, ha='right')
axes[0].set_title('Contagem de tweets por faixa etária', fontsize=12)
axes[0].set_xlabel('Faixa de Idade da Conta')
axes[0].set_ylabel('Número de Tweets')
axes[0].legend()
axes[0].grid(axis='y', linestyle='--', alpha=0.6)

```

```

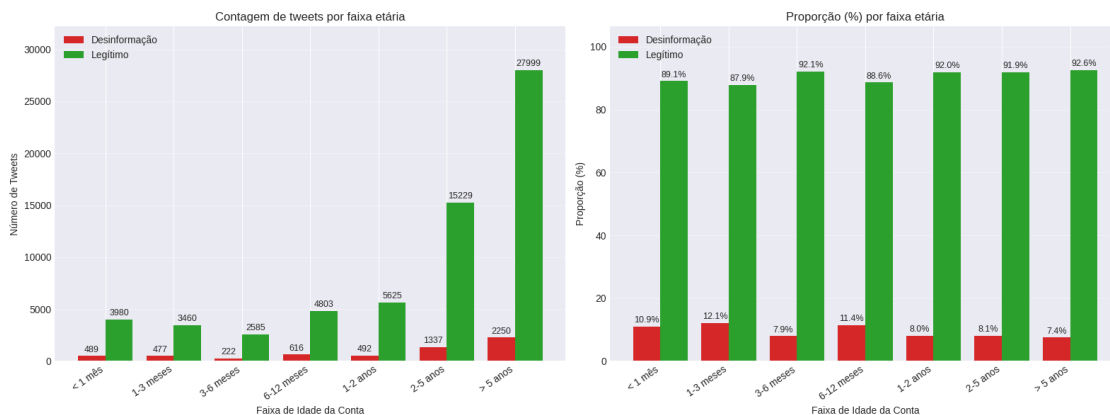
axes[0].margins(y=0.15)

# Proporção (%)
b3 = axes[1].bar(x - width/2, age_desinfo['proporcao_desinfo'], width,
                 label='Desinformação', color='#d62728')
b4 = axes[1].bar(x + width/2, age_desinfo['proporcao_legit'], width,
                 label='Legítimo', color='#2ca02c')

axes[1].bar_label(b3, fmt='%.1f%%', padding=2, fontsize=9)
axes[1].bar_label(b4, fmt='%.1f%%', padding=2, fontsize=9)
axes[1].set_xticks(x)
axes[1].set_xticklabels(age_desinfo['age_group'], rotation=35, ha='right')
axes[1].set_title('Proporção (%) por faixa etária', fontsize=12)
axes[1].set_xlabel('Faixa de Idade da Conta')
axes[1].set_ylabel('Proporção (%)')
axes[1].legend()
axes[1].grid(axis='y', linestyle='--', alpha=0.6)
axes[1].margins(y=0.15)

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳desinfo_vs_legit_por_idade_conta.png',
            dpi=300, bbox_inches='tight')
plt.show()

```



```

[184]: #-----
# Barras empilhadas em % (100% por faixa)
#-----
fig, ax = plt.subplots(figsize=(12, 6))

b_desinfo = ax.bar(age_desinfo['age_group'], age_desinfo['proporcao_desinfo'],
                  label='Desinformação', color='#d62728')

```

```

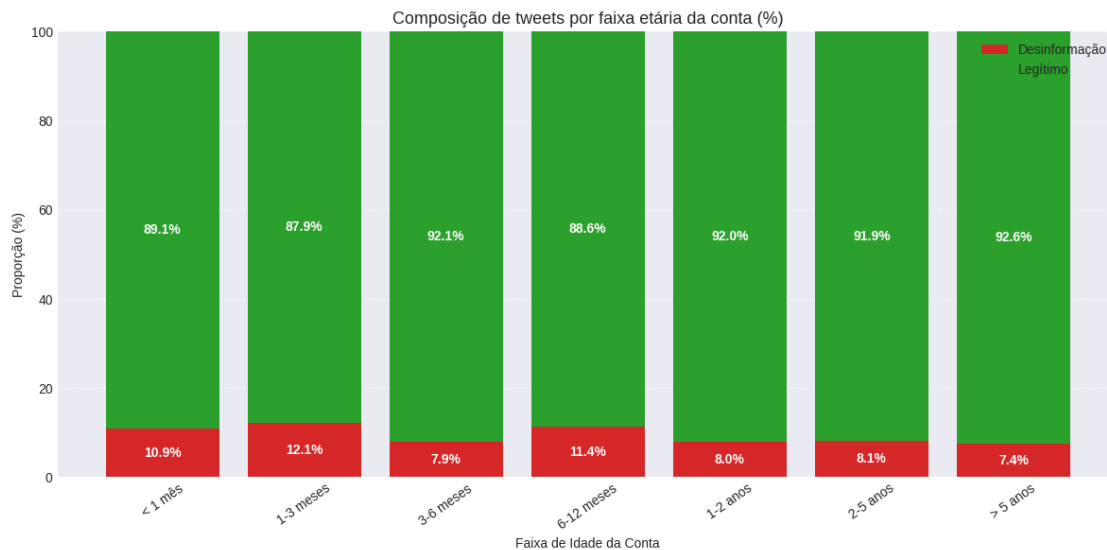
b_legit = ax.bar(age_desinfo['age_group'], age_desinfo['proporcao_legit'],
                 bottom=age_desinfo['proporcao_desinfo'],
                 label='Legítimo', color='#2ca02c')

# Rótulos dentro das barras
ax.bar_label(b_desinfo, fmt='%.1f%%', label_type='center',
             color='white', fontsize=10, fontweight='bold')
ax.bar_label(b_legit, fmt='%.1f%%', label_type='center',
             color='white', fontsize=10, fontweight='bold')

ax.set_title('Composição de tweets por faixa etária da conta (%)', fontsize=13)
ax.set_xlabel('Faixa de Idade da Conta')
ax.set_ylabel('Proporção (%)')
ax.set_ylim(0, 100)
ax.tick_params(axis='x', rotation=35)
ax.legend(loc='upper right')
ax.grid(axis='y', linestyle='--', alpha=0.6)

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/
↳desinfo_vs_legit_empilhado_idade_conta.png',
            dpi=300, bbox_inches='tight')
plt.show()

```



```

[185]: # Contar frequência de cada valor
df['is_disinformation'].value_counts()

```

```
[185]: is_disinformation
0      63681
1       5883
Name: count, dtype: int64
```

5.3.3.4 Sentimento, Tópicos e Entidades

```
[186]: # =====
# SENTIMENTO x DESINFORMAÇÃO
# =====

tabela_sentimento = (
    df.groupby('sentiment')
      .agg(
        total_tweets=('tweet_id', 'count'),
        tweets_desinformacao=('is_disinformation', 'sum')
      )
      .reset_index()
)

tabela_sentimento['percentual_desinformacao'] = (
    tabela_sentimento['tweets_desinformacao'] /
    tabela_sentimento['total_tweets'] * 100
)

display(
    tabela_sentimento.sort_values(
        'percentual_desinformacao',
        ascending=False
    )
)
```

	sentiment	total_tweets	tweets_desinformacao	percentual_desinformacao
1	Neutro	40236	3775	9.382145
0	Negativo	22262	1968	8.840176
2	Positivo	7066	140	1.981319

```
[187]: # =====
# TÓPICOS x DESINFORMAÇÃO
# =====

tabela_topicos = (
    df.groupby('topic')
      .agg(
        total_tweets=('tweet_id', 'count'),
        tweets_desinformacao=('is_disinformation', 'sum')
      )
      .reset_index()
)
```

```

)

tabela_topicos['percentual_desinformacao'] = (
    tabela_topicos['tweets_desinformacao'] /
    tabela_topicos['total_tweets'] * 100
)

tabela_topicos = tabela_topicos.sort_values(
    'percentual_desinformacao',
    ascending=False
)

display(tabela_topicos)

```

	topic	total_tweets	tweets_desinformacao \
1	Auditoria	1226	549
7	Fraude	1161	261
4	Código-Fonte	581	117
16	Urna Eletrônica	28091	3333
14	Outlier	21	2
8	Live do Argentino	1176	109
17	Uso da Urna	12973	945
12	Narrativa Política	8019	399
15	Pesquisas Eleitorais	1736	70
2	Celular na Cabine	407	9
18	Voto Impresso	968	10
10	Militares e Fiscalização	1280	13
5	Disputa Eleitoral	3017	28
9	Logs da Urna	1303	11
6	Fake News	610	5
11	Mídias Sociais	2193	17
0	Ataque Hacker	893	5
3	Conexão com a Internet	380	0
13	Narrativa Religiosa	3529	0

	percentual_desinformacao
1	44.779772
7	22.480620
4	20.137694
16	11.865010
14	9.523810
8	9.268707
17	7.284360
12	4.975683
15	4.032258
2	2.211302
18	1.033058
10	1.015625

5	0.928074
9	0.844206
6	0.819672
11	0.775194
0	0.559910
3	0.000000
13	0.000000

[188]: *# Top 15 tópicos com maior proporção de desinformação*

```

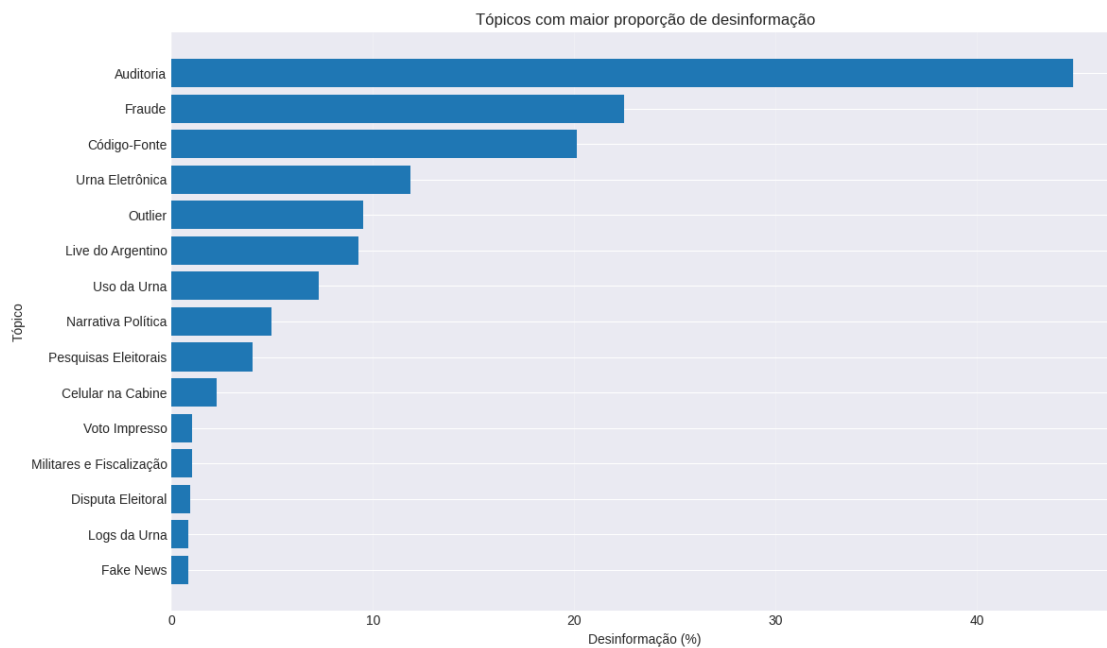
topicos_plot = tabela_topicos.head(15).copy()

plt.figure(figsize=(12, 7))

plt.barh(
    topicos_plot['topic'].astype(str),
    topicos_plot['percentual_desinformacao']
)

plt.xlabel('Desinformação (%)')
plt.ylabel('Tópico')
plt.title('Tópicos com maior proporção de desinformação')
plt.gca().invert_yaxis()
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()

```



```
[189]: # =====
# 1. Garantir que a coluna 'entities' seja um dicionário
# =====
import ast
from collections import Counter
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Se a coluna ainda estiver como string, converter
df['entities'] = df['entities'].apply(
    lambda x: ast.literal_eval(x) if isinstance(x, str) else x
)

# =====
# 2. Filtrar tweets desinformativos
# =====
df_desinfo = df[df['is_disinformation'] == 1].copy()
df_nao = df[df['is_disinformation'] == 0].copy()

# =====
# 3. Função para extrair todas as entidades de uma categoria
# =====
def extrair_entidades(df, categoria):
    """
    Retorna um Counter com todas as entidades da categoria especificada
    (ex.: 'PESSOAS', 'LOCAIS', 'ORGANIZAÇÕES', 'OUTROS')
    """
    todas = []
    for entidades_dict in df['entities']:
        if categoria in entidades_dict:
            lista = entidades_dict[categoria]
            if isinstance(lista, list):
                todas.extend(lista)
    return Counter(todas)

# =====
# 4. Top entidades por categoria em desinformação
# =====
categorias = ['PESSOAS', 'LOCAIS', 'ORGANIZAÇÕES', 'OUTROS']
top_n = 10

for cat in categorias:
    counter_desinfo = extrair_entidades(df_desinfo, cat)
    top_desinfo = pd.DataFrame(
        counter_desinfo.most_common(top_n),
        columns=['entidade', 'frequencia']
    )
```

```
)
print(f"\n TOP {top_n} {cat} em DESINFORMAÇÃO:")
display(top_desinfo)
```

TOP 10 PESSOAS em DESINFORMAÇÃO:

	entidade	frequencia
0	PL	299
1	Achei	291
2	pro Lula	182
3	Lula	73
4	Bolsonaro	60
5	Zero	18
6	Dilmo	17
7	EJACULOU	14
8	Arnaldo	10
9	Molusco	9

TOP 10 LOCAIS em DESINFORMAÇÃO:

	entidade	frequencia
0	BA	330
1	Rio Grande	291
2	Brasil	84
3	Bolsonaro	76
4	São Luís	23
5	Butão	22
6	Site do TSE\n	18
7	Bangladesh	17
8	Esclareçam	16
9	Lisboa	14

TOP 10 ORGANIZAÇÕES em DESINFORMAÇÃO:

	entidade	frequencia
0	TSE	1125
1	FA	478
2	POVO	478
3	PT	158
4	TSE ACABA DE TIRAR	84
5	Forças Armadas	53
6	Bolsonaro	37
7	ZERO	10
8	AC	10
9	BOCA	6

TOP 10 OUTROS em DESINFORMAÇÃO:

	entidade	frequencia
0	LOOLA	590
1	NOMEADO	590
2	Modernidade	478
3	Humanos GELÉIA	478
4	Relatório “	478
5	VIOLÁVEL	478
6	Lola	96
7	QUATORZE!!	44
8	Nine	41
9	Gentili	23

5.3.3.5 Anexos Hashtags:

```
[190]: # =====
# Filtrar apenas tweets classificados como desinformação
# =====
df_desinfo = df[df['is_disinformation'] == 1].copy()

# Garantir que a coluna 'hashtags' esteja no formato de lista
import ast
df_desinfo['hashtags'] = df_desinfo['hashtags'].apply(
    lambda x: ast.literal_eval(x) if isinstance(x, str) else x
)

# Explodir a lista de hashtags para uma linha por hashtag
hashtags_desinfo = df_desinfo.explode('hashtags')

# Remover hashtags vazias ou nulas
hashtags_desinfo = hashtags_desinfo[hashtags_desinfo['hashtags'].notna()]
hashtags_desinfo['hashtags'] = hashtags_desinfo['hashtags'].str.lower().str.
    ↪strip()
hashtags_desinfo = hashtags_desinfo[hashtags_desinfo['hashtags'] != '']

# Contar frequência
from collections import Counter
contagem_desinfo = Counter(hashtags_desinfo['hashtags'])

# Criar DataFrame com as top N
top_n = 20
top_hashtags_desinfo = pd.DataFrame(
    contagem_desinfo.most_common(top_n),
    columns=['hashtag', 'frequencia']
)

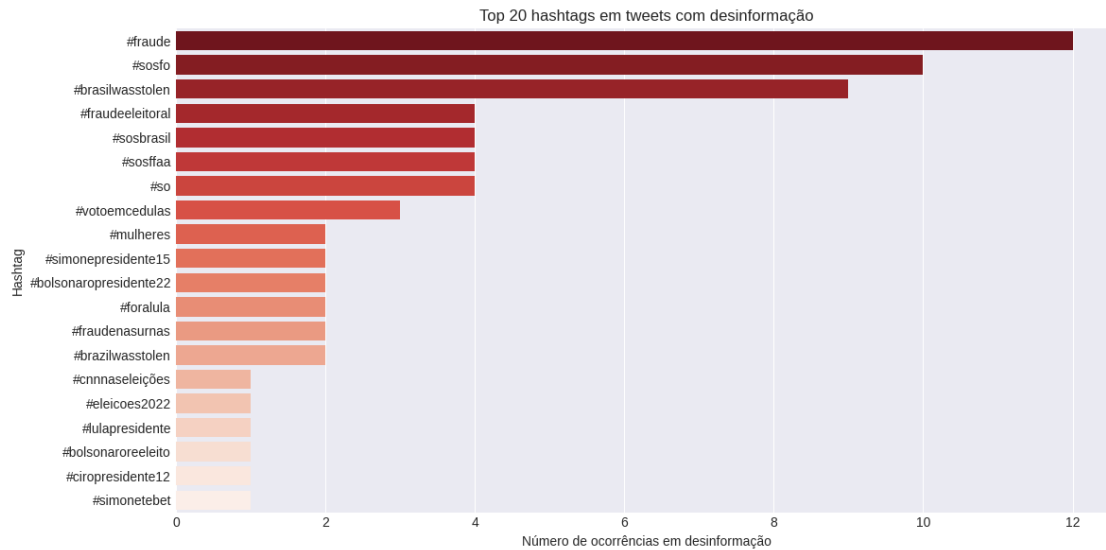
print(" Hashtags mais usadas em TWEETS DESINFORMATIVOS:")
display(top_hashtags_desinfo)
```

Hashtags mais usadas em TWEETS DESINFORMATIVOS:

	hashtag	frequencia
0	#fraude	12
1	#sosfo	10
2	#brasilwasstolen	9
3	#fraudeeleitoral	4
4	#sosbrasil	4
5	#sosffaa	4
6	#so	4
7	#votoemcedulas	3
8	#mulheres	2
9	#simonepresidente15	2
10	#bolsonaropresidente22	2
11	#foralula	2
12	#fraudenasurnas	2
13	#brazilwasstolen	2
14	#cnnnaseleicoes	1
15	#eleicoes2022	1
16	#lulapresidente	1
17	#bolsonaroreeleito	1
18	#ciropresidente12	1
19	#simonetebet	1

```
[191]: import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 6))
sns.barplot(
    data=top_hashtags_desinfo,
    y='hashtag',
    x='frequencia',
    hue='hashtag',
    palette='Reds_r',
    legend=False
)
plt.xlabel('Número de ocorrências em desinformação')
plt.ylabel('Hashtag')
plt.title(f'Top {top_n} hashtags em tweets com desinformação')
plt.tight_layout()
plt.show()
```



```
[192]: # =====
# Hashtags em tweets NÃO desinformativos (para comparação)
# =====
df_nao_desinfo = df[df['is_disinformation'] == 0].copy()
df_nao_desinfo['hashtags'] = df_nao_desinfo['hashtags'].apply(
    lambda x: ast.literal_eval(x) if isinstance(x, str) else x
)

hashtags_nao = df_nao_desinfo.explode('hashtags')
hashtags_nao = hashtags_nao[hashtags_nao['hashtags'].notna()]
hashtags_nao['hashtags'] = hashtags_nao['hashtags'].str.lower().str.strip()
hashtags_nao = hashtags_nao[hashtags_nao['hashtags'] != '']

contagem_nao = Counter(hashtags_nao['hashtags'])
top_hashtags_nao = pd.DataFrame(
    contagem_nao.most_common(top_n),
    columns=['hashtag', 'frequencia']
)

# Juntar para comparar lado a lado
comparacao = top_hashtags_desinfo.merge(
    top_hashtags_nao,
    on='hashtag',
    how='outer',
    suffixes=('_desinfo', '_nao')
).fillna(0)

comparacao = comparacao.sort_values('frequencia_desinfo', ascending=False)
```

```
display(comparacao)
```

	hashtag	frequencia_desinfo	frequencia_nao
16	#fraude	12.0	0.0
33	#sosfo	10.0	0.0
7	#brasilwasstolen	9.0	0.0
17	#fraudeeleitoral	4.0	0.0
30	#so	4.0	0.0
31	#sosbrasil	4.0	0.0
32	#sosffaa	4.0	0.0
37	#votoemcedulas	3.0	0.0
18	#fraudenasurnas	2.0	0.0
28	#simonepresidente15	2.0	0.0
15	#foralula	2.0	0.0
3	#bolsonaropresidente22	2.0	0.0
25	#mulheres	2.0	0.0
8	#brazilwasstolen	2.0	86.0
13	#eleicoes2022	1.0	113.0
29	#simoneteбет	1.0	0.0
11	#cnnnaseleições	1.0	0.0
10	#ciropresidente12	1.0	0.0
4	#bolsonaroreeleito	1.0	0.0
21	#lulapresidente	1.0	0.0
0	#13	0.0	150.0
1	#bolsonaro22	0.0	66.0
5	#bomdia	0.0	117.0
2	#bolsonaronarecord	0.0	34.0
20	#lulanoprimeirotu	0.0	44.0
19	#lulano1ºturno	0.0	122.0
9	#capitaonopilhadoefigueir	0.0	42.0
6	#brasildaesperanca	0.0	140.0
14	#eleições2022	0.0	34.0
12	#debatenaglobo	0.0	29.0
26	#paz	0.0	159.0
27	#sabatinabolsonaro	0.0	34.0
23	#maríliaé77	0.0	48.0
24	#mourãosenador	0.0	61.0
22	#lulapresidente13	0.0	123.0
34	#sãopaulopodemais	0.0	130.0
35	#vote100	0.0	61.0
36	#vote13	0.0	44.0

```
[193]: # =====
# PREPARAR DATA (tweet_created_at)
# =====
coluna_data = 'tweet_created_at'
```

```

hashtags_desinfo[coluna_data] = pd.to_datetime(
    hashtags_desinfo[coluna_data],
    errors='coerce',
    utc=True          # Twitter geralmente vem em UTC
)

hashtags_desinfo = hashtags_desinfo.dropna(subset=[coluna_data])

hashtags_desinfo['dia'] = hashtags_desinfo[coluna_data].dt.date

# =====
# DIA DE PICO POR HASHTAG (top 20)
# =====
top_hashtags_lista = [h for h, _ in contagem_desinfo.most_common(top_n)]

hashtags_dia = (
    hashtags_desinfo[hashtags_desinfo['hashtags'].isin(top_hashtags_lista)]
    .groupby(['dia', 'hashtags'])
    .size()
    .reset_index(name='frequencia')
)

pico_por_hashtag = (
    hashtags_dia
    .sort_values(['hashtags', 'frequencia'], ascending=[True, False])
    .groupby('hashtags')
    .head(1)
    .sort_values('frequencia', ascending=False)
    .reset_index(drop=True)
)

print(" DIA DE PICO de cada hashtag desinformativa (top 20):")
display(pico_por_hashtag)

```

DIA DE PICO de cada hashtag desinformativa (top 20):

	dia	hashtags	frequencia
0	2022-11-04	#brasilwasstolen	9
1	2022-11-07	#sosfo	6
2	2022-11-04	#fraude	5
3	2022-10-04	#votoemcedulas	3
4	2022-11-07	#sosbrasil	2
5	2022-11-04	#brazilwasstolen	2
6	2022-10-05	#bolsonaropresidente22	2
7	2022-10-30	#fraudeeleitoral	2
8	2022-09-23	#simonepresidente15	2
9	2022-09-23	#mulheres	2

10	2022-11-02	#fraudenasurnas	2
11	2022-11-07	#sosffaa	2
12	2022-11-07	#so	2
13	2022-10-03	#bolsonaroreeleito	1
14	2022-10-03	#lulapresidente	1
15	2022-11-02	#foralula	1
16	2022-10-03	#ciropresidente12	1
17	2022-09-30	#cnnnaseleicoes	1
18	2022-10-02	#eleicoes2022	1
19	2022-10-03	#simonetebet	1

```
[194]: # =====
# PERÍODO DE ATIVIDADE POR HASHTAG (primeira → última aparição)
# =====
top_hashtags_lista = [h for h, _ in contagem_desinfo.most_common(top_n)]

df_top = hashtags_desinfo[hashtags_desinfo['hashtags'].isin(top_hashtags_lista)]

periodo_por_hashtag = (
    df_top
    .groupby('hashtags')
    .agg(
        primeira_aparicao=(coluna_data, 'min'),
        ultima_aparicao  =(coluna_data, 'max'),
        total_tweets     =(coluna_data, 'size'),
        dias_ativos      =('dia', 'nunique')
    )
    .reset_index()
)

periodo_por_hashtag['duracao_dias'] = (
    periodo_por_hashtag['ultima_aparicao'] -
    → periodo_por_hashtag['primeira_aparicao']
).dt.total_seconds() / 86400

periodo_por_hashtag = (
    periodo_por_hashtag
    .sort_values('total_tweets', ascending=False)
    .reset_index(drop=True)
)

periodo_legivel = periodo_por_hashtag.copy()
periodo_legivel['primeira_aparicao'] = periodo_legivel['primeira_aparicao'].dt.
→ date
periodo_legivel['ultima_aparicao']   = periodo_legivel['ultima_aparicao'].dt.date
periodo_legivel['duracao_dias']     = periodo_legivel['duracao_dias'].round(1)
```

```
print(" PERÍODO DE ATIVIDADE por hashtag desinformativa (top 20):")
display(periodo_legivel)
```

PERÍODO DE ATIVIDADE por hashtag desinformativa (top 20):

	hashtags	primeira_aparicao	ultima_aparicao	total_tweets	\
0	#fraude	2022-11-02	2022-11-05	12	
1	#sosfo	2022-11-06	2022-11-11	10	
2	#brasilwasstolen	2022-11-04	2022-11-04	9	
3	#so	2022-11-07	2022-11-08	4	
4	#sosffaa	2022-11-07	2022-11-08	4	
5	#sosbrasil	2022-11-07	2022-11-08	4	
6	#fraudeeleitoral	2022-10-30	2022-10-31	4	
7	#votoemcdulas	2022-10-04	2022-10-04	3	
8	#fraudenasurnas	2022-11-02	2022-11-02	2	
9	#brazilwasstolen	2022-11-04	2022-11-04	2	
10	#bolsonaropresidente22	2022-10-05	2022-10-05	2	
11	#foralula	2022-11-02	2022-11-05	2	
12	#simonepresidente15	2022-09-23	2022-09-23	2	
13	#mulheres	2022-09-23	2022-09-23	2	
14	#bolsonaroreeleito	2022-10-03	2022-10-03	1	
15	#eleicoes2022	2022-10-02	2022-10-02	1	
16	#lulapresidente	2022-10-03	2022-10-03	1	
17	#ciropresidente12	2022-10-03	2022-10-03	1	
18	#cnnnaseleições	2022-09-30	2022-09-30	1	
19	#simonetebet	2022-10-03	2022-10-03	1	

	dias_ativos	duracao_dias
0	4	3.3
1	4	4.1
2	1	0.1
3	2	0.9
4	2	0.9
5	2	0.9
6	2	0.7
7	1	0.1
8	1	0.0
9	1	0.0
10	1	0.0
11	2	3.3
12	1	0.2
13	1	0.2
14	1	0.0
15	1	0.0
16	1	0.0
17	1	0.0
18	1	0.0
19	1	0.0

```
[195]: # =====
# Série temporal completa - tweets por dia por hashtag
# =====
serie_por_hashtag = (
    df_top
    .groupby(['hashtags', 'dia'])
    .size()
    .reset_index(name='frequencia')
    .sort_values(['hashtags', 'dia'])
)

serie_por_hashtag['primeiro_dia'] = serie_por_hashtag.groupby('hashtags')['dia'].
    ↪transform('min')
serie_por_hashtag['ultimo_dia'] = serie_por_hashtag.groupby('hashtags')['dia'].
    ↪transform('max')

display(serie_por_hashtag.head(30))
```

	hashtags	dia	frequencia	primeiro_dia	ultimo_dia
0	#bolsonaropresidente22	2022-10-05	2	2022-10-05	2022-10-05
1	#bolsonaroreeleito	2022-10-03	1	2022-10-03	2022-10-03
2	#brasilwasstolen	2022-11-04	9	2022-11-04	2022-11-04
3	#brazilwasstolen	2022-11-04	2	2022-11-04	2022-11-04
4	#ciropresidente12	2022-10-03	1	2022-10-03	2022-10-03
5	#cnnnaseleicoes	2022-09-30	1	2022-09-30	2022-09-30
6	#eleicoes2022	2022-10-02	1	2022-10-02	2022-10-02
7	#foralula	2022-11-02	1	2022-11-02	2022-11-05
8	#foralula	2022-11-05	1	2022-11-02	2022-11-05
9	#fraude	2022-11-02	2	2022-11-02	2022-11-05
10	#fraude	2022-11-03	4	2022-11-02	2022-11-05
11	#fraude	2022-11-04	5	2022-11-02	2022-11-05
12	#fraude	2022-11-05	1	2022-11-02	2022-11-05
13	#fraudeeleitoral	2022-10-30	2	2022-10-30	2022-10-31
14	#fraudeeleitoral	2022-10-31	2	2022-10-30	2022-10-31
15	#fraudenasurnas	2022-11-02	2	2022-11-02	2022-11-02
16	#lulapresidente	2022-10-03	1	2022-10-03	2022-10-03
17	#mulheres	2022-09-23	2	2022-09-23	2022-09-23
18	#simonepresidente15	2022-09-23	2	2022-09-23	2022-09-23
19	#simoneteбет	2022-10-03	1	2022-10-03	2022-10-03
20	#so	2022-11-07	2	2022-11-07	2022-11-08
21	#so	2022-11-08	2	2022-11-07	2022-11-08
22	#sosbrasil	2022-11-07	2	2022-11-07	2022-11-08
23	#sosbrasil	2022-11-08	2	2022-11-07	2022-11-08
24	#sosffaa	2022-11-07	2	2022-11-07	2022-11-08
25	#sosffaa	2022-11-08	2	2022-11-07	2022-11-08
26	#sosfo	2022-11-06	2	2022-11-06	2022-11-11
27	#sosfo	2022-11-07	6	2022-11-06	2022-11-11
28	#sosfo	2022-11-09	1	2022-11-06	2022-11-11

Links:

```
[196]: # =====
# Filtrar tweets desinformativos e tratar coluna 'urls'
# =====

import ast
from collections import Counter
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from urllib.parse import urlparse

# Garantir que a coluna 'urls' seja uma lista
df['urls'] = df['urls'].apply(
    lambda x: ast.literal_eval(x) if isinstance(x, str) else x
)

# Filtrar apenas desinformação
df_desinfo = df[df['is_disinformation'] == 1].copy()

# Explodir a lista de URLs para uma linha por URL
urls_desinfo = df_desinfo.explode('urls')

# Remover URLs vazias ou nulas
urls_desinfo = urls_desinfo[urls_desinfo['urls'].notna()]
urls_desinfo['urls'] = urls_desinfo['urls'].str.strip()
urls_desinfo = urls_desinfo[urls_desinfo['urls'] != '']

# Contar frequência
contagem_links_desinfo = Counter(urls_desinfo['urls'])

# Criar DataFrame com os Top 20
top_n = 20
top_links_desinfo = pd.DataFrame(
    contagem_links_desinfo.most_common(top_n),
    columns=['url', 'frequencia']
)

print(" Links mais compartilhados em TWEETS DESINFORMATIVOS:")
display(top_links_desinfo)
```

Links mais compartilhados em TWEETS DESINFORMATIVOS:

	url	frequencia
0	https://twitter.com/anacarolinar/status/15868...	119
1	https://dadosabertos.tse.jus.br/dataset/result...	86
2	https://twitter.com/VlogdoLisboa/status/158865...	55

3	https://brasilsemmedo.com/nao-ha-como-checar-e...	35
4	https://twitter.com/GuedinhoeFans/status/15736...	25
5	https://twitter.com/fpassos1964/status/1583080...	18
6	https://twitter.com/terapiagora/status/1576678...	14
7	https://twitter.com/adailtonavr/status/1591941...	14
8	https://www.brasil247.com/brasil/diretor-do-ip...	13
9	https://twitter.com/Carolmph/status/1593409742...	10
10	https://twitter.com/SoGasch/status/15919606774...	9
11	https://twitter.com/SgcRoberto/status/15934173...	9
12	https://www.brasil247.com/brasil/pai-da-urna-e...	9
13	https://twitter.com/eliane_varella/status/1587...	8
14	https://twitter.com/marcelolduarte/status/1593...	8
15	https://twitter.com/kellypstorres/status/15908...	6
16	https://twitter.com/Rogerramer/status/15980369...	6
17	https://twitter.com/RobsaoE/status/15738483538...	5
18	https://twitter.com/bruno_carioc2/status/15766...	5
19	https://twitter.com/CentralEleicoes/status/157...	4

```
[197]: # Função para extrair domínio
def extrair_dominio(url):
    try:
        parsed = urlparse(url)
        domain = parsed.netloc.lower()
        if domain.startswith('www.'):
            domain = domain[4:]
        return domain
    except:
        return None

# Aplicar a extração de domínio
urls_desinfo['dominio'] = urls_desinfo['urls'].apply(extrair_dominio)

# Remover domínios nulos e vazios
urls_desinfo = urls_desinfo[urls_desinfo['dominio'].notna()]
urls_desinfo = urls_desinfo[urls_desinfo['dominio'] != '']

# Contar domínios
contagem_dominios_desinfo = Counter(urls_desinfo['dominio'])

# Top 15 domínios
top_dominios_desinfo = pd.DataFrame(
    contagem_dominios_desinfo.most_common(15),
    columns=['dominio', 'frequencia']
)

print(" Domínios mais usados em TWEETS DESINFORMATIVOS:")
display(top_dominios_desinfo)
```

Domínios mais usados em TWEETS DESINFORMATIVOS:

	dominio	frequencia
0	twitter.com	395
1	dadosabertos.tse.jus.br	86
2	brasilsemmedo.com	35
3	brasil247.com	23
4	agoranoticiasbrasil.com.br	6
5	youtube.com	4
6	tse.jus.br	2
7	jornaldacidadeonline.com.br	1
8	g1.globo.com	1
9	ronaldobragaiaguatu.blogspot.com	1
10	kanekoa.news	1
11	resultados.tse.jus.br	1
12	gilbertoleda.com.br	1
13	dados.gov.br	1
14	google.com	1

```
[198]: # =====
# URLs em tweets NÃO desinformativos
# =====
df_nao = df[df['is_disinformation'] == 0].copy()
urls_nao = df_nao.explode('urls')
urls_nao = urls_nao[urls_nao['urls'].notna()]
urls_nao['urls'] = urls_nao['urls'].str.strip()
urls_nao = urls_nao[urls_nao['urls'] != '']

# Extrair domínios
urls_nao['dominio'] = urls_nao['urls'].apply(extrair_dominio)
urls_nao = urls_nao[urls_nao['dominio'].notna()]
urls_nao = urls_nao[urls_nao['dominio'] != '']

contagem_dominios_nao = Counter(urls_nao['dominio'])
top_dominios_nao = pd.DataFrame(
    contagem_dominios_nao.most_common(15),
    columns=['dominio', 'frequencia']
)

# Juntar para comparar
comparacao_dominios = top_dominios_desinfo.merge(
    top_dominios_nao,
    on='dominio',
    how='outer',
    suffixes=('_desinfo', '_nao')
).fillna(0)
```

```
comparacao_dominios = comparacao_dominios.sort_values('frequencia_desinfo',
↪ascending=False)
display(comparacao_dominios)
```

	dominio	frequencia_desinfo	frequencia_ao
21	twitter.com	395.0	6897.0
5	dadosabertos.tse.jus.br	86.0	74.0
3	brasilsemmedo.com	35.0	0.0
2	brasil247.com	23.0	55.0
0	agoranoticiasbrasil.com.br	6.0	0.0
24	youtube.com	4.0	73.0
20	tse.jus.br	2.0	0.0
7	g1.globo.com	1.0	26.0
4	dados.gov.br	1.0	0.0
18	ronaldobraguaguatu.blogspot.com	1.0	0.0
10	google.com	1.0	0.0
11	jornaldacidadeonline.com.br	1.0	0.0
9	gilbertoleda.com.br	1.0	0.0
15	resultados.tse.jus.br	1.0	0.0
12	kanekoa.news	1.0	0.0
1	bbc.com	0.0	68.0
6	estadao.com.br	0.0	43.0
8	gazetabrasil.com.br	0.0	55.0
16	revistaforum.com.br	0.0	24.0
13	noticias.uol.com.br	0.0	118.0
14	poder360.com.br	0.0	42.0
19	terrabrasilnoticias.com	0.0	142.0
17	revistaeste.com	0.0	68.0
22	valor.globo.com	0.0	82.0
23	www1.folha.uol.com.br	0.0	56.0

```
[199]: #-----
# Domínios mais frequentes por dia de pico
#-----

import pandas as pd
from collections import Counter

# 1. Preparar data
urls_desinfo['tweet_created_at'] = pd.to_datetime(
    urls_desinfo['tweet_created_at'], errors='coerce', utc=True
).dt.tz_convert('America/Sao_Paulo')

urls_desinfo['data'] = urls_desinfo['tweet_created_at'].dt.date

# 2. Achar os 5 dias de pico
urls_por_dia = urls_desinfo.groupby('data').size().sort_values(ascending=False)
```

```

dias_pico = urls_por_dia.head(5).index.tolist()

# 3. Filtrar e montar a tabela
df_pico = urls_desinfo[urls_desinfo['data'].isin(dias_pico)]

linhas = []
for dia, grupo in df_pico.groupby('data'):
    for dom, freq in Counter(grupo['dominio']).most_common(10):
        linhas.append({'Data': dia, 'Domínio': dom, 'Frequência': freq})

tabela = pd.DataFrame(linhas)
display(tabela)

```

	Data	Domínio	Frequência
0	2022-09-24	twitter.com	26
1	2022-10-30	twitter.com	38
2	2022-10-31	twitter.com	72
3	2022-11-04	dadosabertos.tse.jus.br	48
4	2022-11-04	twitter.com	31
5	2022-11-05	dadosabertos.tse.jus.br	38
6	2022-11-05	twitter.com	35
7	2022-11-05	dados.gov.br	1

```

[200]: # =====
# PERÍODO DE ATIVIDADE POR DOMÍNIO (primeira → última aparição)
# =====
import pandas as pd

# Preparar data
coluna_data = 'tweet_created_at'

urls_desinfo[coluna_data] = pd.to_datetime(
    urls_desinfo[coluna_data],
    errors='coerce',
    utc=True
)

urls_desinfo = urls_desinfo.dropna(subset=[coluna_data])

urls_desinfo['dia'] = urls_desinfo[coluna_data].dt.date

# Top N domínios
top_n = 20
contagem_dominios = Counter(urls_desinfo['dominio'])
top_dominios_lista = [d for d, _ in contagem_dominios.most_common(top_n)]

```

```

df_top_dom = urls_desinfo[urls_desinfo['dominio'].isin(top_dominios_lista)].
↳copy()

# =====
# Primeira → última aparição por domínio
# =====
periodo_por_dominio = (
    df_top_dom
    .groupby('dominio')
    .agg(
        primeira_aparicao=(coluna_data, 'min'),
        ultima_aparicao  =(coluna_data, 'max'),
        total_urls      =(coluna_data, 'size'),
        dias_ativos     =('dia', 'nunique')
    )
    .reset_index()
)

periodo_por_dominio['duracao_dias'] = (
    periodo_por_dominio['ultima_aparicao'] -
↳periodo_por_dominio['primeira_aparicao']
).dt.total_seconds() / 86400

periodo_por_dominio = (
    periodo_por_dominio
    .sort_values('total_urls', ascending=False)
    .reset_index(drop=True)
)

periodo_legivel_dom = periodo_por_dominio.copy()
periodo_legivel_dom['primeira_aparicao'] =
↳periodo_legivel_dom['primeira_aparicao'].dt.date
periodo_legivel_dom['ultima_aparicao']   =
↳periodo_legivel_dom['ultima_aparicao'].dt.date
periodo_legivel_dom['duracao_dias']      = periodo_legivel_dom['duracao_dias'].
↳round(1)

print(" PERÍODO DE ATIVIDADE por domínio (top 20):")
display(periodo_legivel_dom)

```

PERÍODO DE ATIVIDADE por domínio (top 20):

	dominio	primeira_aparicao	ultima_aparicao	\
0	twitter.com	2022-09-19	2022-12-01	
1	dadosabertos.tse.jus.br	2022-11-04	2022-11-05	
2	brasilsemmedo.com	2022-09-27	2022-09-28	
3	brasil247.com	2022-09-26	2022-11-24	

4	agoranoticiasbrasil.com.br	2022-10-03	2022-11-12
5	youtube.com	2022-10-06	2022-10-11
6	tse.jus.br	2022-11-12	2022-11-12
7	gilbertoleda.com.br	2022-11-02	2022-11-02
8	g1.globo.com	2022-10-02	2022-10-02
9	dados.gov.br	2022-11-05	2022-11-05
10	google.com	2022-11-16	2022-11-16
11	resultados.tse.jus.br	2022-11-02	2022-11-02
12	kanekoa.news	2022-10-10	2022-10-10
13	jornaldacidadeonline.com.br	2022-10-01	2022-10-01
14	ronaldobragaiatu.blogspot.com	2022-10-08	2022-10-08

	total_urls	dias_ativos	duracao_dias
0	395	39	72.5
1	86	2	0.9
2	35	2	0.6
3	23	4	59.2
4	6	4	39.5
5	4	3	5.3
6	2	1	0.0
7	1	1	0.0
8	1	1	0.0
9	1	1	0.0
10	1	1	0.0
11	1	1	0.0
12	1	1	0.0
13	1	1	0.0
14	1	1	0.0

```
[201]: # =====
# Série temporal completa - URLs por dia por domínio
# =====
serie_por dominio = (
    df_top_dom
    .groupby(['dominio', 'dia'])
    .size()
    .reset_index(name='frequencia')
    .sort_values(['dominio', 'dia'])
)

serie_por dominio['primeiro_dia'] = serie_por dominio.groupby('dominio')['dia'].
    ↪transform('min')
serie_por dominio['ultimo_dia'] = serie_por dominio.groupby('dominio')['dia'].
    ↪transform('max')

display(serie_por dominio.head(30))
```

dominio	dia	frequencia	primeiro_dia	\
---------	-----	------------	--------------	---

0	agoranoticiasbrasil.com.br	2022-10-03	1	2022-10-03
1	agoranoticiasbrasil.com.br	2022-11-10	3	2022-10-03
2	agoranoticiasbrasil.com.br	2022-11-11	1	2022-10-03
3	agoranoticiasbrasil.com.br	2022-11-12	1	2022-10-03
4	brasil247.com	2022-09-26	13	2022-09-26
5	brasil247.com	2022-09-27	1	2022-09-26
6	brasil247.com	2022-11-23	4	2022-09-26
7	brasil247.com	2022-11-24	5	2022-09-26
8	brasilsemmedo.com	2022-09-27	16	2022-09-27
9	brasilsemmedo.com	2022-09-28	19	2022-09-27
10	dados.gov.br	2022-11-05	1	2022-11-05
11	dadosabertos.tse.jus.br	2022-11-04	37	2022-11-04
12	dadosabertos.tse.jus.br	2022-11-05	49	2022-11-04
13	g1.globo.com	2022-10-02	1	2022-10-02
14	gilbertoleda.com.br	2022-11-02	1	2022-11-02
15	google.com	2022-11-16	1	2022-11-16
16	jornaldacidadeonline.com.br	2022-10-01	1	2022-10-01
17	kanekoa.news	2022-10-10	1	2022-10-10
18	resultados.tse.jus.br	2022-11-02	1	2022-11-02
19	ronaldobragaiaguatu.blogspot.com	2022-10-08	1	2022-10-08
20	tse.jus.br	2022-11-12	2	2022-11-12
21	twitter.com	2022-09-19	1	2022-09-19
22	twitter.com	2022-09-24	25	2022-09-19
23	twitter.com	2022-09-25	5	2022-09-19
24	twitter.com	2022-09-28	1	2022-09-19
25	twitter.com	2022-10-01	1	2022-09-19
26	twitter.com	2022-10-02	17	2022-09-19
27	twitter.com	2022-10-03	15	2022-09-19
28	twitter.com	2022-10-04	3	2022-09-19
29	twitter.com	2022-10-05	6	2022-09-19

	ultimo_dia
0	2022-11-12
1	2022-11-12
2	2022-11-12
3	2022-11-12
4	2022-11-24
5	2022-11-24
6	2022-11-24
7	2022-11-24
8	2022-09-28
9	2022-09-28
10	2022-11-05
11	2022-11-05
12	2022-11-05
13	2022-10-02
14	2022-11-02
15	2022-11-16

```
16 2022-10-01
17 2022-10-10
18 2022-11-02
19 2022-10-08
20 2022-11-12
21 2022-12-01
22 2022-12-01
23 2022-12-01
24 2022-12-01
25 2022-12-01
26 2022-12-01
27 2022-12-01
28 2022-12-01
29 2022-12-01
```

Circulação

```
[223]: # Circulação: originais + retweets
circ = (
    df_com_retweets[df_com_retweets['is_disinformation'] == 1]
    .groupby('user_id')
    .size()
    .sort_values(ascending=False)
)

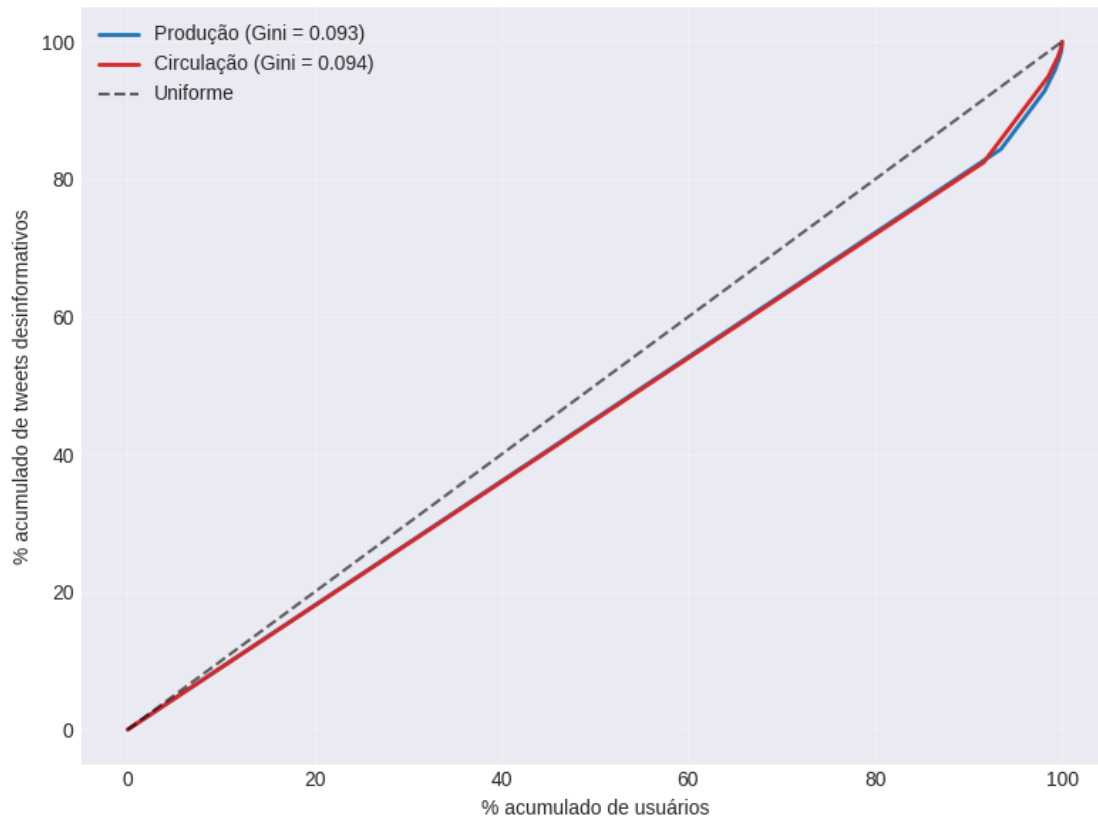
print(f"Circulação: {len(circ)} usuários, {circ.sum()} tweets")

gini_circ = gini(circ.values)
print(f"Gini circulação: {gini_circ:.3f}")

# Lorenz comparação
p_prod, l_prod = lorenz(prod.values)
p_circ, l_circ = lorenz(circ.values)

plt.figure(figsize=(8, 6))
plt.plot(p_prod*100, l_prod*100, color='#1f77b4', linewidth=2,
        label=f'Produção (Gini = {gini_prod:.3f})')
plt.plot(p_circ*100, l_circ*100, color='#d62728', linewidth=2,
        label=f'Circulação (Gini = {gini_circ:.3f})')
plt.plot([0, 100], [0, 100], 'k--', alpha=0.6, label='Uniforme')
plt.xlabel('% acumulado de usuários')
plt.ylabel('% acumulado de tweets desinformativos')
plt.legend(loc='upper left')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/TCC/Estaticos/lorenz_comparacao.png',
            dpi=300, bbox_inches='tight')
plt.show()
```

Circulação: 8912 usuários, 9905 tweets
Gini circulação: 0.094



1.5 5.4 Análise Estrutural da Rede

```
[205]: # Geral
# df = df_com_retweets.copy()

# total_desinfo = df['is_disinformation'].sum()
# total_nao_desinfo = len(df) - total_desinfo
# print(f"Total de tweets: {len(df)}")
# print(f"Desinformação: {total_desinfo} ({total_desinfo/len(df):.2%})")
# print(f"Não desinformação: {total_nao_desinfo} ({total_nao_desinfo/len(df):.
→2%})")
```

```
[206]: # Estrutura/Rede
df = df_com_retweets.copy()
df = df[df['cluster'].notna()]
total_desinfo = df['is_disinformation'].sum()
total_nao_desinfo = len(df) - total_desinfo
print(f"Total de tweets: {len(df)}")
```

```
print(f"Desinformação: {total_desinfo} ({total_desinfo/len(df):.2%}")
print(f"Não desinformação: {total_nao_desinfo} ({total_nao_desinfo/len(df):.
→2%}")
```

Total de tweets: 63572

Desinformação: 6657 (10.47%)

Não desinformação: 56915 (89.53%)

1.5.1 5.4.1 Estrutura da rede

```
[207]: # =====
# ESTRUTURA DA REDE * DESINFORMAÇÃO
# =====

# Comparação das métricas estruturais

metricas_rede = [
    'indegree',
    'outdegree',
    'degree'
]

df_rede = df[
    ['is_disinformation'] + metricas_rede
].copy()

df_rede['grupo_desinfo'] = df_rede['is_disinformation'].map({
    0: 'Não desinformação',
    1: 'Desinformação'
})

tabela_rede = (
    df_rede
    .groupby('grupo_desinfo')[metricas_rede]
    .agg(['mean', 'median', 'max'])
)

display(tabela_rede)
```

	indegree		outdegree			degree \	
	mean	median	max	mean	median	max	mean
grupo_desinfo							
Desinformação	1.847829	0.0	2535.0	2.395073	2.0	34.0	4.242902
Não desinformação	3.134253	0.0	3846.0	2.299816	1.0	55.0	5.434068

	median	max
grupo_desinfo		

Desinformação	2.0	2535.0
Não desinformação	2.0	3846.0

```
[208]: # Percentis P75, P90, P95
tabela_rede = df_rede.groupby('grupo_desinfo')[metricas_rede].agg(
    ['mean', 'median', 'std',
     lambda x: x.quantile(0.75),
     lambda x: x.quantile(0.90),
     lambda x: x.quantile(0.95),
     'max']
)

# Mann-Whitney two-sided
from scipy.stats import mannwhitneyu
for metrica in metricas_rede:
    d = df_rede[df_rede['grupo_desinfo']=='Desinformação'][metrica].dropna()
    nd = df_rede[df_rede['grupo_desinfo']=='Não desinformação'][metrica].dropna()
    stat, p = mannwhitneyu(d, nd, alternative='two-sided')
    print(f"{metrica}: U={stat:.0f}, p={p:.4f}")
```

```
indegree: U=191523974, p=0.0200
outdegree: U=204045644, p=0.0000
degree: U=203679788, p=0.0000
```

1.5.2 5.4.2 Desinformação por comunidade

```
[209]: # =====
# DESINFORMAÇÃO * COMUNIDADES
# =====

# Número total de tweets e desinformação por cluster

tabela_clusters = (
    df.groupby('cluster')
      .agg(
          total_tweets=('tweet_id', 'count'),
          tweets_desinformacao=('is_disinformation', 'sum')
      )
      .reset_index()
)

tabela_clusters['percentual_desinformacao'] = (
    tabela_clusters['tweets_desinformacao'] /
    tabela_clusters['total_tweets'] * 100
)

tabela_clusters = tabela_clusters.sort_values(
    'percentual_desinformacao',
```

```

        ascending=False
    )

display(tabela_clusters)

```

	cluster	total_tweets	tweets_desinformacao	percentual_desinformacao
2	2.0	35929	5624	15.653094
10	10.0	57	7	12.280702
4	4.0	93	7	7.526882
9	9.0	39	2	5.128205
7	7.0	24739	1004	4.058369
0	0.0	46	1	2.173913
3	3.0	1296	11	0.848765
5	5.0	652	1	0.153374
1	1.0	153	0	0.000000
6	6.0	50	0	0.000000
8	8.0	356	0	0.000000
11	11.0	162	0	0.000000

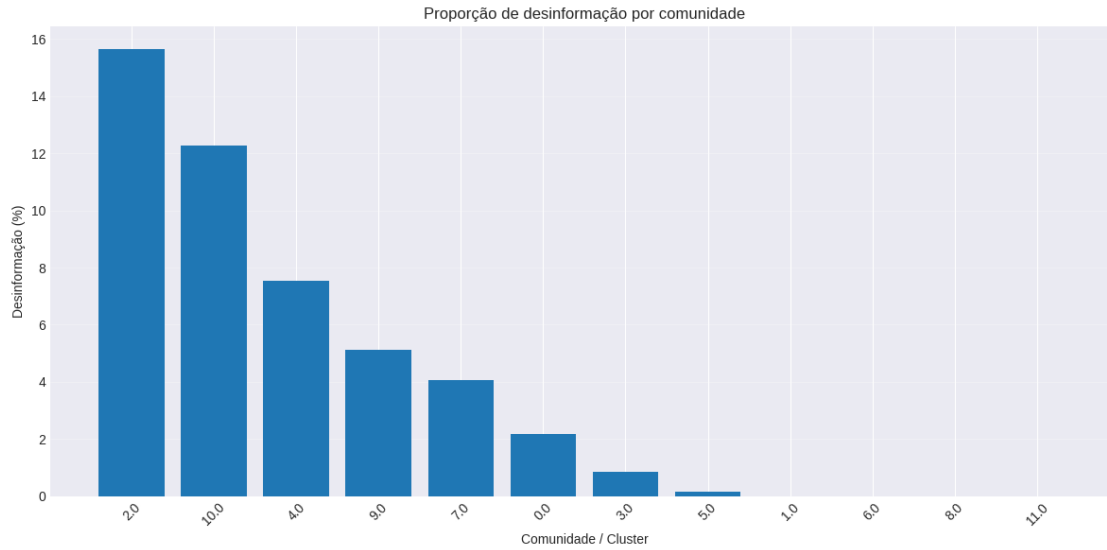
```

[210]: plt.figure(figsize=(12, 6))

plt.bar(
    tabela_clusters['cluster'].astype(str),
    tabela_clusters['percentual_desinformacao']
)

plt.xlabel('Comunidade / Cluster')
plt.ylabel('Desinformação (%)')
plt.title('Proporção de desinformação por comunidade')
plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

```



1.5.3 5.4.3 Fluxo entre comunidades

```
[211]: # =====
# FLUXO DE DESINFORMAÇÃO ENTRE COMUNIDADES
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# -----
# 1. SELECIONAR REGISTROS COM CLUSTER
# -----

df_rede_cluster = df[
    df['cluster'].notna()
].copy()

print('Registros com comunidade:', len(df_rede_cluster))

# -----
# 2. IDENTIFICAR COLUNAS DE ORIGEM/DESTINO
# -----
#
# A ideia é:
#
# usuário que publica/compartilha
#     ↓
```

```

# usuário que recebe/interage
#
# Se sua rede foi construída a partir de retweets,
# precisamos dos IDs dos usuários origem e destino.
#
# AJUSTE OS NOMES ABAIXO conforme seu dataset.

```

```

COLUNA_USUARIO_ORIGEM = 'user_id'
COLUNA_USUARIO_DESTINO = 'original_author'

```

```

# -----
# 3. MAPEAR CLUSTER DO USUÁRIO DE ORIGEM
# -----

```

```

mapa_usuario_cluster = (
    df_rede_cluster[
        ['user_id', 'cluster']
    ]
    .dropna()
    .drop_duplicates('user_id')
)

```

```

mapa_usuario_cluster.columns = [
    'usuario_origem',
    'cluster_origem'
]

```

```

# -----
# 4. MAPEAR CLUSTER DO USUÁRIO DESTINO
# -----

```

```

mapa_destino = (
    df_rede_cluster[
        ['user_id', 'cluster']
    ]
    .dropna()
    .drop_duplicates('user_id')
)

```

```

mapa_destino.columns = [
    'usuario_destino',
    'cluster_destino'
]

```

```

# -----

```

```

# 5. CONSTRUIR EDGES
# -----

fluxos = df_rede_cluster[
    [
        'user_id',
        COLUNA_USUARIO_DESTINO,
        'is_disinformation'
    ]
].copy()

fluxos.rename(
    columns={
        'user_id': 'usuario_origem',
        COLUNA_USUARIO_DESTINO: 'usuario_destino'
    },
    inplace=True
)

fluxos = fluxos.merge(
    mapa_usuario_cluster,
    on='usuario_origem',
    how='left'
)

fluxos = fluxos.merge(
    mapa_destino,
    on='usuario_destino',
    how='left'
)

# Remover conexões sem comunidade identificada
fluxos = fluxos.dropna(
    subset=[
        'cluster_origem',
        'cluster_destino'
    ]
)

# -----
# 6. MATRIZ DE FLUXO ENTRE COMUNIDADES
# -----

matriz_fluxo = pd.crosstab(
    fluxos['cluster_origem'],
    fluxos['cluster_destino']
)

```

```
print('Matriz de fluxo entre comunidades:')
display(matriz_fluxo)
```

Registros com comunidade: 63572

Matriz de fluxo entre comunidades:

cluster_destino \ cluster_origem	1.0	2.0	3.0	4.0	5.0	6.0	7.0	8.0	9.0	10.0	\
0.0	0	0	0	0	0	0	1	0	0	0	
1.0	144	0	0	0	0	0	3	0	0	0	
2.0	0	20141	1	0	0	1	119	0	0	0	
3.0	0	5	1074	0	0	0	22	0	0	0	
4.0	0	3	0	6	0	0	0	0	0	0	
5.0	0	1	2	0	582	0	12	0	0	0	
6.0	0	0	0	0	0	37	1	0	0	0	
7.0	0	238	41	0	7	1	12964	1	0	1	
8.0	0	0	1	0	0	0	2	324	0	0	
9.0	0	1	0	0	0	0	0	0	2	0	
10.0	0	1	0	0	0	0	0	0	0	0	
11.0	0	1	0	0	0	0	0	0	0	0	

cluster_destino \ cluster_origem	11.0
0.0	0
1.0	0
2.0	0
3.0	0
4.0	0
5.0	0
6.0	0
7.0	0
8.0	0
9.0	0
10.0	0
11.0	137

```
[212]: # -----
# FLUXO ESPECÍFICO DE DESINFORMAÇÃO
# -----

fluxo_desinfo = (
    fluxos[
        fluxos['is_disinformation'] == 1
    ]
    .groupby(
        [
            'cluster_origem',
```

```

        'cluster_destino'
    ]
)
.size()
.reset_index(
    name='interacoes_desinformacao'
)
.sort_values(
    'interacoes_desinformacao',
    ascending=False
)
)

display(fluxo_desinfo.head(30))

```

	cluster_origem	cluster_destino	interacoes_desinformacao
1	2.0	2.0	3151
6	7.0	7.0	253
5	7.0	2.0	24
2	2.0	7.0	3
0	0.0	7.0	1
4	3.0	3.0	1
3	3.0	2.0	1

```

[213]: # -----
# PROPORÇÃO DE DESINFORMAÇÃO EM CADA FLUXO
# -----

fluxo_total = (
    fluxos
    .groupby(
        [
            'cluster_origem',
            'cluster_destino'
        ]
    )
    .size()
    .reset_index(
        name='total_interacoes'
    )
)

fluxo_desinfo = fluxo_desinfo.merge(
    fluxo_total,
    on=[
        'cluster_origem',
        'cluster_destino'
    ]
)

```

```

    ],
    how='left'
)

fluxo_desinfo[
    'prop_desinformacao'
] = (
    fluxo_desinfo[
        'interacoes_desinformacao'
    ]
    /
    fluxo_desinfo[
        'total_interacoes'
    ]
    * 100
)

fluxo_desinfo = fluxo_desinfo.sort_values(
    'prop_desinformacao',
    ascending=False
)

print('Fluxos com maior proporção de desinformação:')
display(fluxo_desinfo.head(30))

```

Fluxos com maior proporção de desinformação:

	cluster_origem	cluster_destino	interacoes_desinformacao \
4	0.0	7.0	1
6	3.0	2.0	1
0	2.0	2.0	3151
2	7.0	2.0	24
3	2.0	7.0	3
1	7.0	7.0	253
5	3.0	3.0	1

	total_interacoes	prop_desinformacao
4	1	100.000000
6	5	20.000000
0	20141	15.644705
2	238	10.084034
3	119	2.521008
1	12964	1.951558
5	1074	0.093110

1.5.4 5.4.5 Homofilia

```
[214]: # =====
# HOMOFILIA DA DESINFORMAÇÃO
# =====

if 'fluxos' in locals() and len(fluxos) > 0:

    # -----
    # 1. CLASSIFICAR O TIPO DA ORIGEM/INTERAÇÃO
    # -----

    # Precisamos saber se origem e destino estão
    # associados a usuários que disseminam desinformação.

    desinfo_usuario = (
        df.groupby('user_id')['is_disinformation']
        .mean()
        .reset_index()
    )

    desinfo_usuario['tipo_usuario'] = np.where(
        desinfo_usuario['is_disinformation'] > 0,
        'Desinformação',
        'Não desinformação'
    )

    desinfo_usuario = desinfo_usuario[
        ['user_id', 'tipo_usuario']
    ]

    # -----
    # 2. CLASSIFICAR ORIGEM
    # -----

    fluxos_h = fluxos.merge(
        desinfo_usuario,
        left_on='usuario_origem',
        right_on='user_id',
        how='left'
    )

    fluxos_h.rename(
        columns={
            'tipo_usuario': 'tipo_origem'
        },
        inplace=True
```

```

)

fluxos_h.drop(
    columns=['user_id'],
    inplace=True
)

# -----
# 3. CLASSIFICAR DESTINO
# -----

fluxos_h = fluxos_h.merge(
    desinfo_usuario,
    left_on='usuario_destino',
    right_on='user_id',
    how='left'
)

fluxos_h.rename(
    columns={
        'tipo_usuario': 'tipo_destino'
    },
    inplace=True
)

fluxos_h.drop(
    columns=['user_id'],
    inplace=True
)

# Remover casos sem classificação
fluxos_h = fluxos_h.dropna(
    subset=[
        'tipo_origem',
        'tipo_destino'
    ]
)

```

```

[215]: # -----
# MATRIZ DE HOMOFILIA
# -----

matriz_homofilia = pd.crosstab(
    fluxos_h['tipo_origem'],
    fluxos_h['tipo_destino']
)

```

```
print('Número de interações:')
display(matriz_homofilia)
```

Número de interações:

tipo_destino	Desinformação	Não desinformação
tipo_origem		
Desinformação	2891	3482
Não desinformação	2777	26727

```
[216]: # -----
# PROPORÇÕES
# -----

matriz_homofilia_pct = (
    pd.crosstab(
        fluxos_h['tipo_origem'],
        fluxos_h['tipo_destino'],
        normalize='index'
    ) * 100
)

print(
    'Distribuição percentual das interações:'
)

display(
    matriz_homofilia_pct.round(2)
)
```

Distribuição percentual das interações:

tipo_destino	Desinformação	Não desinformação
tipo_origem		
Desinformação	45.36	54.64
Não desinformação	9.41	90.59

```
[217]: # -----
# CALCULAR HOMOFILIA OBSERVADA
# -----

total_interacoes = len(fluxos_h)

mesma_categoria = (
    (
        fluxos_h['tipo_origem']
        ==
        fluxos_h['tipo_destino']
    )
)
```

```

        .sum()
    )

homofilia_observada = (
    mesma_categoria /
    total_interacoes
)

print(
    f'Homofilia observada: '
    f'{homofilia_observada:.4f}'
)

```

Homofilia observada: 0.8255

```

[218]: # -----
#  ASSORTATIVIDADE
#  -----

# Transformar categorias em valores binários
fluxos_h['origem_bin'] = (
    fluxos_h['tipo_origem']
    == 'Desinformação'
).astype(int)

fluxos_h['destino_bin'] = (
    fluxos_h['tipo_destino']
    == 'Desinformação'
).astype(int)

# Correlação equivalente à assortatividade
assortatividade = (
    fluxos_h[
        ['origem_bin', 'destino_bin']
    ]
    .corr()
    .iloc[0, 1]
)

print(
    f'Coeficiente de homofilia/assortatividade: '
    f'{assortatividade:.4f}'
)

```

Coeficiente de homofilia/assortatividade: 0.3767

```
[219]: import networkx as nx
G = nx.from_pandas_edgelist(fluxos_h, 'usuario_origem', 'usuario_destino',
    ↳ create_using=nx.DiGraph)
# Adicionar atributo binário aos nós
for n in G.nodes():
    G.nodes[n]['disseminador'] = 1 if n in
    ↳ desinfo_usuario[desinfo_usuario['tipo_usuario']=='Desinformação']['user_id'])
    ↳ else 0
r_newman = nx.attribute_assortativity_coefficient(G, 'disseminador')
print(f"Assortatividade de Newman: {r_newman:.4f}")
```

Assortatividade de Newman: 0.3759

```
[220]: # -----
# VISUALIZAÇÃO
# -----

plt.figure(figsize=(8, 6))

matriz_homofilia_pct.plot(
    kind='bar',
    figsize=(8, 6)
)

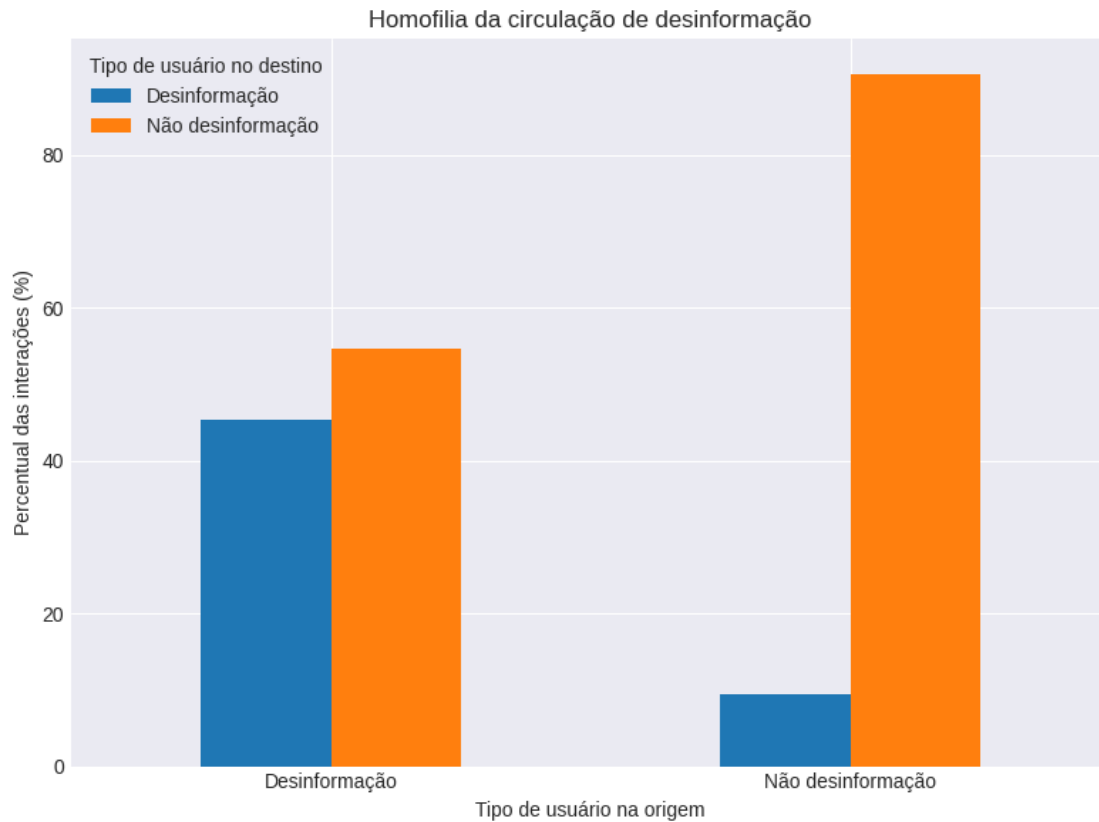
plt.xlabel('Tipo de usuário na origem')
plt.ylabel('Percentual das interações (%)')

plt.title(
    'Homofilia da circulação de desinformação'
)

plt.xticks(rotation=0)
plt.legend(
    title='Tipo de usuário no destino'
)

plt.tight_layout()
plt.show()
```

<Figure size 800x600 with 0 Axes>



```
[221]: # =====
# HOMOFILIA ENTRE COMUNIDADES
# =====

if 'fluxos' in locals() and len(fluxos) > 0:

    # -----
    # 1. TODAS AS INTERAÇÕES
    # -----

    fluxos['mesma_comunidade'] = (
        fluxos['cluster_origem']
        ==
        fluxos['cluster_destino']
    )

    homofilia_comunidades = (
        fluxos['mesma_comunidade']
        .mean()
        * 100
```

```

)

print(
    f'Homofilia geral entre comunidades: '
    f'{homofilia_comunidades:.2f}%'
)

# -----
# 2. APENAS DESINFORMAÇÃO
# -----

fluxos_desinfo = fluxos[
    fluxos['is_disinformation'] == 1
].copy()

if len(fluxos_desinfo) > 0:

    homofilia_desinfo = (
        fluxos_desinfo['mesma_comunidade']
        .mean()
        * 100
    )

    print(
        f'Homofilia da desinformação: '
        f'{homofilia_desinfo:.2f}%'
    )

# -----
# 3. FLUXO INTERNO * FLUXO EXTERNO
# -----

fluxo_interno_externo = (
    fluxos_desinfo[
        'mesma_comunidade'
    ]
    .value_counts(
        normalize=True
    )
    * 100
)

fluxo_interno_externo.index = [
    'Dentro da mesma comunidade'
    if x

```

```

        else 'Entre comunidades diferentes'
        for x in fluxo_interno_externo.index
    ]

    display(
        fluxo_interno_externo
        .rename('percentual')
        .to_frame()
    )

```

Homofilia geral entre comunidades: 98.70%

Homofilia da desinformação: 99.16%

	percentual
Dentro da mesma comunidade	99.155504
Entre comunidades diferentes	0.844496

```

[222]: # -----
# GRÁFICO
# -----

fluxo_interno_externo.plot(
    kind='bar',
    figsize=(9, 6)
)

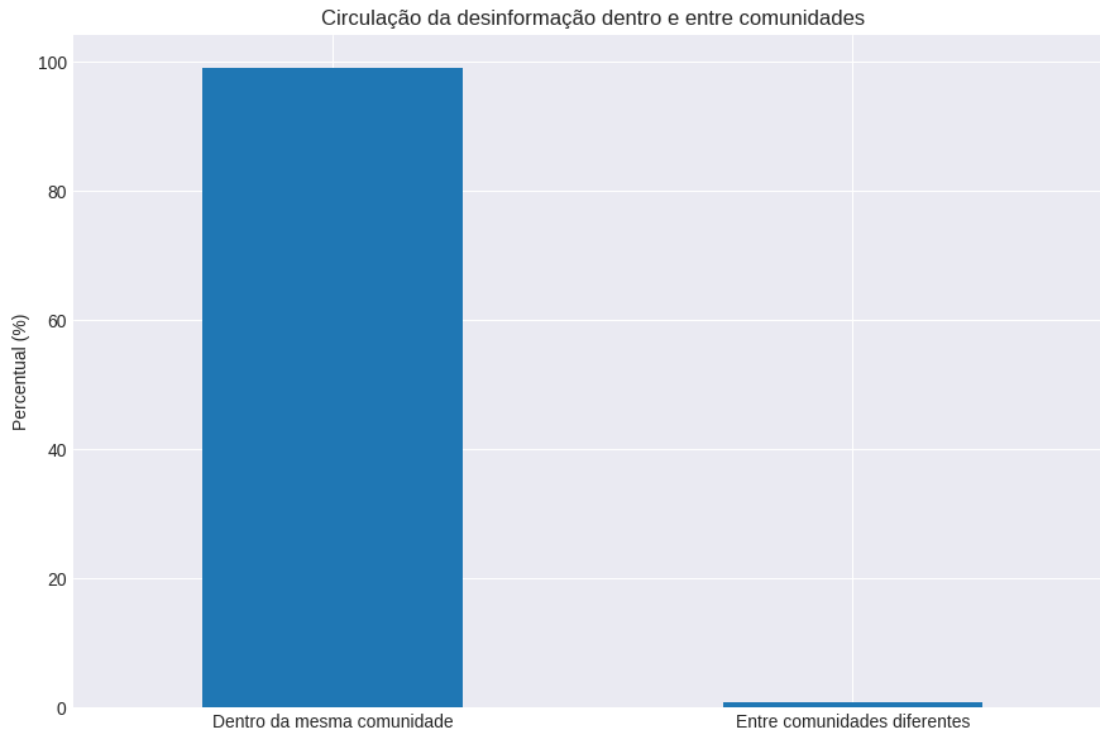
plt.xlabel('')
plt.ylabel('Percentual (%)')

plt.title(
    'Circulação da desinformação '
    'dentro e entre comunidades'
)

plt.xticks(rotation=0)

plt.tight_layout()
plt.show()

```



1.6 Visualização BERTopic

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[ ]: !python --version
```

Python 3.13.15

```
[ ]: !sudo apt-get update -y
!sudo apt-get install python3.12
```

```
0% [Working]          Get:1 https://cli.github.com/packages stable
InRelease [4,685 B]
0% [Connecting to archive.ubuntu.com (91.189.92.23)] [Connecting to security.ub
0% [Connecting to archive.ubuntu.com (91.189.92.23)] [Connecting to security.ub
Get:2 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
[3,632 B]
Get:3 https://r2u.stat.illinois.edu/ubuntu jammy InRelease [6,555 B]
Hit:4 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:5 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:6 https://cli.github.com/packages stable/main amd64 Packages [356 B]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
```

```

Get:8 https://r2u.stat.illinois.edu/ubuntu jammy/main amd64 Packages [3,183 kB]
Get:9 https://r2u.stat.illinois.edu/ubuntu jammy/main all Packages [10.8 MB]
Get:10 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
[18.1 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Hit:12 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Get:13 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages
[4,265 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [4,598
kB]
Get:15 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy/main amd64
Packages [44.3 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages
[7,965 kB]
Get:17 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages
[1,316 kB]
Get:18 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64
Packages [7,674 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages
[1,620 kB]
Fetched 41.9 MB in 13s (3,196 kB/s)
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide
it (sources.list entry misspelt?)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libpython3.12-stdlib
Suggested packages:
  python3.12-venv
The following NEW packages will be installed:
  libpython3.12-stdlib python3.12
0 upgraded, 2 newly installed, 0 to remove and 73 not upgraded.
Need to get 5,383 kB of archives.
After this operation, 22.3 MB of additional disk space will be used.
Get:1 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy/main amd64
libpython3.12-stdlib amd64 3.12.13-1+jammy1 [2,866 kB]
Get:2 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy/main amd64
python3.12 amd64 3.12.13-1+jammy1 [2,517 kB]
Fetched 5,383 kB in 1s (3,751 kB/s)
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based
frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 78,
<> line 2.)
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline

```

```

debconf: (This frontend requires a controlling tty.)
debconf: falling back to frontend: Teletype
dpkg-preconfigure: unable to re-open stdin:
Selecting previously unselected package libpython3.12-stdlib:amd64.
(Reading database ... 118419 files and directories currently installed.)
Preparing to unpack .../libpython3.12-stdlib_3.12.13-1+jammy1_amd64.deb ...
Unpacking libpython3.12-stdlib:amd64 (3.12.13-1+jammy1) ...
Selecting previously unselected package python3.12.
Preparing to unpack .../python3.12_3.12.13-1+jammy1_amd64.deb ...
Unpacking python3.12 (3.12.13-1+jammy1) ...
Setting up libpython3.12-stdlib:amd64 (3.12.13-1+jammy1) ...
Setting up python3.12 (3.12.13-1+jammy1) ...
Processing triggers for mailcap (3.70+nmu1ubuntu1.22.04.1) ...
Processing triggers for man-db (2.10.2-1) ...

```

```
[ ]: !sudo update-alternatives --install /usr/bin/python3 python3 /usr/bin/python3.12
```

```
[ ]: !sudo update-alternatives --config python3
```

There are 3 choices for the alternative python3 (providing /usr/bin/python3).

Selection	Path	Priority	Status
* 0	/usr/bin/python3.13	2	auto mode
1	/usr/bin/python3.10	1	manual mode
2	/usr/bin/python3.12	1	manual mode
3	/usr/bin/python3.13	2	manual mode

```

Press <enter> to keep the current choice[*], or type selection number: 2
update-alternatives: using /usr/bin/python3.12 to provide /usr/bin/python3
(python3) in manual mode

```

```
[ ]: !python --version
```

Python 3.12.13

```
[ ]: !pip list
```

```

Traceback (most recent call last):
  File "/usr/local/bin/pip3", line 4, in <module>
    from pip._internal.cli.main import main
ModuleNotFoundError: No module named 'pip'

```

```
[ ]: !apt-get install python3-pip
```

```

Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:

```

```

javascript-common libjs-sphinxdoc libjs-underscore libpython3.10
libpython3.10-dev libpython3.10-minimal libpython3.10-stdlib python3-dev
python3-pkg-resources python3-setuptools python3-wheel python3.10
python3.10-dev python3.10-minimal
Suggested packages:
  apache2 | lighttpd | httpd python-setuptools-doc python3.10-venv
  python3.10-doc binfmt-support
The following NEW packages will be installed:
  javascript-common libjs-sphinxdoc libjs-underscore python3-dev python3-pip
  python3-setuptools python3-wheel python3.10-dev
The following packages will be upgraded:
  libpython3.10 libpython3.10-dev libpython3.10-minimal libpython3.10-stdlib
  python3-pkg-resources python3.10 python3.10-minimal
7 upgraded, 8 newly installed, 0 to remove and 66 not upgraded.
Need to get 15.0 MB of archives.
After this operation, 10.9 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64
libpython3.10-dev amd64 3.10.12-1~22.04.17 [4,765 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libpython3.10
amd64 3.10.12-1~22.04.17 [1,950 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 python3.10 amd64
3.10.12-1~22.04.17 [508 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64
libpython3.10-stdlib amd64 3.10.12-1~22.04.17 [1,850 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64
python3.10-minimal amd64 3.10.12-1~22.04.17 [2,279 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64
libpython3.10-minimal amd64 3.10.12-1~22.04.17 [817 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy/main amd64 javascript-common all
11+nmu1 [5,936 B]
Get:8 http://archive.ubuntu.com/ubuntu jammy/main amd64 libjs-underscore all
1.13.2~dfsg-2 [118 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy/main amd64 libjs-sphinxdoc all
4.3.2-1 [139 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 python3.10-dev
amd64 3.10.12-1~22.04.17 [508 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 python3-dev
amd64 3.10.6-1~22.04.1 [26.0 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64
python3-wheel all 0.37.1-2ubuntu0.22.04.1 [32.0 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 python3-pip
all 22.0.2+dfsg-1ubuntu0.7 [1,306 kB]
Get:14 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy/main amd64
python3-pkg-resources all 68.1.2-2jammy3 [216 kB]
Get:15 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy/main amd64
python3-setuptools all 68.1.2-2jammy3 [465 kB]
Fetched 15.0 MB in 1s (11.3 MB/s)
(Reading database ... 119107 files and directories currently installed.)

```

```

Preparing to unpack .../00-libpython3.10-dev_3.10.12-1~22.04.17_amd64.deb ...
Unpacking libpython3.10-dev:amd64 (3.10.12-1~22.04.17) over (3.10.12-1~22.04.16)
...
Preparing to unpack .../01-libpython3.10_3.10.12-1~22.04.17_amd64.deb ...
Unpacking libpython3.10:amd64 (3.10.12-1~22.04.17) over (3.10.12-1~22.04.16) ...
Preparing to unpack .../02-python3.10_3.10.12-1~22.04.17_amd64.deb ...
Unpacking python3.10 (3.10.12-1~22.04.17) over (3.10.12-1~22.04.16) ...
Preparing to unpack .../03-libpython3.10-stdlib_3.10.12-1~22.04.17_amd64.deb ...
Unpacking libpython3.10-stdlib:amd64 (3.10.12-1~22.04.17) over
(3.10.12-1~22.04.16) ...
Preparing to unpack .../04-python3.10-minimal_3.10.12-1~22.04.17_amd64.deb ...
Unpacking python3.10-minimal (3.10.12-1~22.04.17) over (3.10.12-1~22.04.16) ...
Preparing to unpack .../05-libpython3.10-minimal_3.10.12-1~22.04.17_amd64.deb
...
Unpacking libpython3.10-minimal:amd64 (3.10.12-1~22.04.17) over
(3.10.12-1~22.04.16) ...
Selecting previously unselected package javascript-common.
Preparing to unpack .../06-javascript-common_11+nmu1_all.deb ...
Unpacking javascript-common (11+nmu1) ...
Selecting previously unselected package libjs-underscore.
Preparing to unpack .../07-libjs-underscore_1.13.2~dfsg-2_all.deb ...
Unpacking libjs-underscore (1.13.2~dfsg-2) ...
Selecting previously unselected package libjs-sphinxdoc.
Preparing to unpack .../08-libjs-sphinxdoc_4.3.2-1_all.deb ...
Unpacking libjs-sphinxdoc (4.3.2-1) ...
Selecting previously unselected package python3.10-dev.
Preparing to unpack .../09-python3.10-dev_3.10.12-1~22.04.17_amd64.deb ...
Unpacking python3.10-dev (3.10.12-1~22.04.17) ...
Selecting previously unselected package python3-dev.
Preparing to unpack .../10-python3-dev_3.10.6-1~22.04.1_amd64.deb ...
Unpacking python3-dev (3.10.6-1~22.04.1) ...
Preparing to unpack .../11-python3-pkg-resources_68.1.2-2~jammy3_all.deb ...
Unpacking python3-pkg-resources (68.1.2-2~jammy3) over
(59.6.0-1.2ubuntu0.22.04.3) ...
Selecting previously unselected package python3-setuptools.
Preparing to unpack .../12-python3-setuptools_68.1.2-2~jammy3_all.deb ...
Unpacking python3-setuptools (68.1.2-2~jammy3) ...
Selecting previously unselected package python3-wheel.
Preparing to unpack .../13-python3-wheel_0.37.1-2ubuntu0.22.04.1_all.deb ...
Unpacking python3-wheel (0.37.1-2ubuntu0.22.04.1) ...
Selecting previously unselected package python3-pip.
Preparing to unpack .../14-python3-pip_22.0.2+dfsg-1ubuntu0.7_all.deb ...
Unpacking python3-pip (22.0.2+dfsg-1ubuntu0.7) ...
Setting up python3-pkg-resources (68.1.2-2~jammy3) ...
Setting up javascript-common (11+nmu1) ...
Setting up python3-setuptools (68.1.2-2~jammy3) ...
Setting up python3-wheel (0.37.1-2ubuntu0.22.04.1) ...
Setting up libpython3.10-minimal:amd64 (3.10.12-1~22.04.17) ...

```

```

Setting up python3-pip (22.0.2+dfsg-1ubuntu0.7) ...
Setting up libjs-underscore (1.13.2~dfsg-2) ...
Setting up python3.10-minimal (3.10.12-1~22.04.17) ...
Setting up libpython3.10-stdlib:amd64 (3.10.12-1~22.04.17) ...
Setting up libjs-sphinxdoc (4.3.2-1) ...
Setting up libpython3.10:amd64 (3.10.12-1~22.04.17) ...
Setting up python3.10 (3.10.12-1~22.04.17) ...
Setting up libpython3.10-dev:amd64 (3.10.12-1~22.04.17) ...
Setting up python3.10-dev (3.10.12-1~22.04.17) ...
Setting up python3-dev (3.10.6-1~22.04.1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.14) ...
/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero_v2.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

Processing triggers for man-db (2.10.2-1) ...
Processing triggers for mailcap (3.70+nmulubuntu1.22.04.1) ...

```

```
[ ]: # pip
!wget https://bootstrap.pypa.io/get-pip.py
!python get-pip.py

--2026-09-02 18:34:25-- https://bootstrap.pypa.io/get-pip.py
Resolving bootstrap.pypa.io (bootstrap.pypa.io)... 151.101.0.175,
151.101.64.175, 151.101.128.175, ...
Connecting to bootstrap.pypa.io (bootstrap.pypa.io)|151.101.0.175|:443...
connected.
HTTP request sent, awaiting response... 200 OK
Length: 2230488 (2.1M) [text/x-python]
Saving to: 'get-pip.py'

get-pip.py          0%[                  ] 0  --.-KB/s
get-pip.py          100%[=====>] 2.13M  --.-KB/s  in 0.01s

2026-09-02 18:34:25 (156 MB/s) - 'get-pip.py' saved [2230488/2230488]

Collecting pip
  Downloading pip-26.2.1-py3-none-any.whl.metadata (4.6 kB)
  Downloading pip-26.2.1-py3-none-any.whl (1.8 MB)
    1.8/1.8 MB
  31.8 MB/s  0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 22.0.2
    Uninstalling pip-22.0.2:
      Successfully uninstalled pip-22.0.2
  Successfully installed pip-26.2.1
```

```
[ ]: !pip list
```

Package	Version
-----	-----
blinker	1.4
cryptography	3.4.8
dbus-python	1.2.18
distro	1.7.0
httplib2	0.20.2
importlib-metadata	4.6.4
jeepney	0.7.1
keyring	23.5.0
launchpadlib	1.10.16
lazr.restfulclient	0.14.4
lazr.uri	1.0.6
Mako	1.1.3
Markdown	3.3.6
MarkupSafe	2.0.1

more-itertools	8.10.0
oauthlib	3.2.0
pip	26.2.1
PyGObject	3.42.1
PyJWT	2.3.0
pyparsing	2.4.7
python-apt	2.4.0+ubuntu4.1
SecretStorage	3.3.1
setuptools	68.1.2
six	1.16.0
wadllib	1.3.6
wheel	0.37.1
zipp	1.0.0

```
[ ]: # Versão de Dez/2025
```

```
!python -m pip install transformers==4.55.4 sentence_transformers==5.1.1
↪bertopic==0.17.4 pynndescent==0.5.13
```

```
Collecting transformers==4.55.4
  Downloading transformers-4.55.4-py3-none-any.whl.metadata (41 kB)
Collecting sentence_transformers==5.1.1
  Downloading sentence_transformers-5.1.1-py3-none-any.whl.metadata (16 kB)
Collecting bertopic==0.17.4
  Downloading bertopic-0.17.4-py3-none-any.whl.metadata (24 kB)
Collecting pynndescent==0.5.13
  Downloading pynndescent-0.5.13-py3-none-any.whl.metadata (6.8 kB)
Collecting filelock (from transformers==4.55.4)
  Downloading filelock-3.32.5-py3-none-any.whl.metadata (2.0 kB)
Collecting huggingface_hub<1.0,>=0.34.0 (from transformers==4.55.4)
  Downloading huggingface_hub-0.36.2-py3-none-any.whl.metadata (15 kB)
Collecting numpy>=1.17 (from transformers==4.55.4)
  Downloading
numpy-2.5.2-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata
(6.6 kB)
Collecting packaging>=20.0 (from transformers==4.55.4)
  Downloading packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pyyaml>=5.1 (from transformers==4.55.4)
  Downloading pyyaml-6.0.3-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(2.4 kB)
Collecting regex!=2019.12.17 (from transformers==4.55.4)
  Downloading regex-2026.9.3-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(40 kB)
Collecting requests (from transformers==4.55.4)
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting tokenizers<0.22,>=0.21 (from transformers==4.55.4)
  Downloading tokenizers-0.21.4-cp39-abi3-
```

```

manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (6.7 kB)
Collecting safetensors>=0.4.3 (from transformers==4.55.4)
  Downloading safetensors-0.8.0-cp310-abi3-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.2 kB)
Collecting tqdm>=4.27 (from transformers==4.55.4)
  Downloading tqdm-4.70.0-py3-none-any.whl.metadata (57 kB)
Collecting torch>=1.11.0 (from sentence_transformers==5.1.1)
  Downloading torch-2.14.0-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (37
kB)
Collecting scikit-learn (from sentence_transformers==5.1.1)
  Downloading scikit_learn-1.9.0-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (11 kB)
Collecting scipy (from sentence_transformers==5.1.1)
  Downloading scipy-1.18.1-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (62 kB)
Collecting Pillow (from sentence_transformers==5.1.1)
  Downloading pillow-12.3.0-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (9.1 kB)
Collecting typing_extensions>=4.5.0 (from sentence_transformers==5.1.1)
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting hdbscan>=0.8.29 (from bertopic==0.17.4)
  Downloading hdbscan-0.8.44-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(15 kB)
Collecting umap-learn>=0.5.0 (from bertopic==0.17.4)
  Downloading umap_learn-0.5.12-py3-none-any.whl.metadata (24 kB)
Collecting pandas>=1.1.5 (from bertopic==0.17.4)
  Downloading pandas-3.0.5-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
Collecting plotly>=4.7.0 (from bertopic==0.17.4)
  Downloading plotly-7.0.0-py3-none-any.whl.metadata (8.7 kB)
Collecting llvmlite>0.36.0 (from bertopic==0.17.4)
  Downloading llvmlite-0.49.0-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (5.0 kB)
Collecting numba>=0.51.2 (from pynndescent==0.5.13)
  Downloading
numba-0.67.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata
(2.7 kB)
Collecting joblib>=0.11 (from pynndescent==0.5.13)
  Downloading joblib-1.6.0-py3-none-any.whl.metadata (6.5 kB)
Collecting fsspec>=2023.5.0 (from huggingface-
hub<1.0,>=0.34.0->transformers==4.55.4)
  Downloading fsspec-2026.7.0-py3-none-any.whl.metadata (10 kB)
Collecting hf-xet<2.0.0,>=1.1.3 (from huggingface-
hub<1.0,>=0.34.0->transformers==4.55.4)
  Downloading
hf_xet-1.6.0-cp38-abi3-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata
(4.9 kB)

```

```

Collecting cloudpickle>=3.0 (from joblib>=0.11->pynndescent==0.5.13)
  Downloading cloudpickle-3.1.2-py3-none-any.whl.metadata (7.1 kB)
Collecting python-dateutil>=2.8.2 (from pandas>=1.1.5->bertopic==0.17.4)
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting narwhals>=1.15.1 (from plotly>=4.7.0->bertopic==0.17.4)
  Downloading narwhals-2.25.0-py3-none-any.whl.metadata (15 kB)
Requirement already satisfied: six>=1.5 in /usr/lib/python3/dist-packages (from
python-dateutil>=2.8.2->pandas>=1.1.5->bertopic==0.17.4) (1.16.0)
Collecting threadpoolctl>=3.5.0 (from scikit-
learn->sentence_transformers==5.1.1)
  Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Collecting setuptools>=77.0.3 (from torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading setuptools-84.0.0-py3-none-any.whl.metadata (6.6 kB)
Collecting sympy>=1.13.3 (from torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading sympy-1.14.0-py3-none-any.whl.metadata (12 kB)
Collecting networkx>=2.5.1 (from torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading networkx-3.6.1-py3-none-any.whl.metadata (6.8 kB)
Collecting Jinja2 (from torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading Jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting cuda-toolkit==13.0.3 (from cuda-toolkit[cublas,cudart,cufft,cufi
le,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system ==
"Linux"->torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading cuda_toolkit-13.0.3.0-py2.py3-none-any.whl.metadata (17 kB)
Collecting cuda-bindings<14,>=13.0.3 (from
torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading cuda_bindings-13.3.1-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (2.5 kB)
Collecting nvidia-cudnn-cu13==9.24.0.43 (from
torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading nvidia_cudnn_cu13-9.24.0.43-py3-none-
manylinux_2_27_x86_64.whl.metadata (1.9 kB)
Collecting nvidia-cusparselt-cu13==0.8.1 (from
torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading nvidia_cusparselt_cu13-0.8.1-py3-none-
manylinux2014_x86_64.whl.metadata (12 kB)
Collecting nvidia-nccl-cu13==2.30.7 (from
torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading nvidia_nccl_cu13-2.30.7-py3-none-
manylinux_2_18_x86_64.whl.metadata (2.1 kB)
Collecting nvidia-nvshmem-cu13==3.4.5 (from
torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading nvidia_nvshmem_cu13-3.4.5-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (2.1 kB)
Collecting triton~=3.8.0 (from torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading triton-3.8.0-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (978 bytes)
Collecting nvidia-cublas==13.1.1.3.* (from cuda-toolkit[cublas,cudart,cufft,cufi
le,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system

```



```

Collecting nvidia-nvtx==13.0.85.* (from cuda-toolkit[cublas,cudart,cufft,cufile,
cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system ==
"Linux"->torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading nvidia_nvtx-13.0.85-py3-none-
manylinux1_x86_64.manylinux_2_5_x86_64.whl.metadata (1.8 kB)
Collecting cuda-pathfinder>=1.4.2 (from cuda-
bindings<14,>=13.0.3->torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading cuda_pathfinder-1.8.1-py3-none-any.whl.metadata (1.9 kB)
Collecting mpmath<1.4,>=1.1.0 (from
sympy>=1.13.3->torch>=1.11.0->sentence_transformers==5.1.1)
  Downloading mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/lib/python3/dist-packages
(from jinja2->torch>=1.11.0->sentence_transformers==5.1.1) (2.0.1)
Collecting charset_normalizer<4,>=2 (from requests->transformers==4.55.4)
  Downloading charset_normalizer-3.5.1-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(45 kB)
Collecting idna<4,>=2.5 (from requests->transformers==4.55.4)
  Downloading idna-3.19-py3-none-any.whl.metadata (9.2 kB)
Collecting urllib3<3,>=1.26 (from requests->transformers==4.55.4)
  Downloading urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2023.5.7 (from requests->transformers==4.55.4)
  Downloading certifi-2026.7.22-py3-none-any.whl.metadata (2.5 kB)
Downloading transformers-4.55.4-py3-none-any.whl (11.3 MB)
  11.3/11.3 MB
59.7 MB/s 0:00:00
Downloading sentence_transformers-5.1.1-py3-none-any.whl (486 kB)
Downloading bertopic-0.17.4-py3-none-any.whl (154 kB)
Downloading pynndescent-0.5.13-py3-none-any.whl (56 kB)
Downloading huggingface_hub-0.36.2-py3-none-any.whl (566 kB)
  566.4/566.4 kB
10.9 MB/s 0:00:00
Downloading
hf_xet-1.6.0-cp38-abi3-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (4.5 MB)
  4.5/4.5 MB
105.6 MB/s 0:00:00
Downloading
tokenizers-0.21.4-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.1
MB)
  3.1/3.1 MB
72.4 MB/s 0:00:00
Downloading fsspec-2026.7.0-py3-none-any.whl (206 kB)
Downloading hdbscan-0.8.44-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (5.9 MB)
  5.9/5.9 MB
108.1 MB/s 0:00:00
Downloading
numpy-2.5.2-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.7

```

```

MB)
    16.7/16.7 MB
    112.1 MB/s  0:00:00
Downloading joblib-1.6.0-py3-none-any.whl (306 kB)
Downloading cloudpickle-3.1.2-py3-none-any.whl (22 kB)
Downloading
llvmlite-0.49.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (59.9
MB)
    59.9/59.9 MB
    55.9 MB/s  0:00:01
Downloading
numba-0.67.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (3.9 MB)
    3.9/3.9 MB
    80.4 MB/s  0:00:00
Downloading packaging-26.3-py3-none-any.whl (129 kB)
Downloading
pandas-3.0.5-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (11.0
MB)
    11.0/11.0 MB
    110.4 MB/s  0:00:00
Downloading plotly-7.0.0-py3-none-any.whl (9.1 MB)
    9.1/9.1 MB
    98.4 MB/s  0:00:00
Downloading narwhals-2.25.0-py3-none-any.whl (467 kB)
Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading pyyaml-6.0.3-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (807 kB)
    807.9/807.9 kB
    16.6 MB/s  0:00:00
Downloading regex-2026.9.3-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (801 kB)
    802.0/802.0 kB
    14.2 MB/s  0:00:00
Downloading
safetensors-0.8.0-cp310-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (516
kB)
Downloading
scikit_learn-1.9.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
(9.1 MB)
    9.1/9.1 MB
    96.4 MB/s  0:00:00
Downloading
scipy-1.18.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (35.3
MB)
    35.3/35.3 MB
    47.1 MB/s  0:00:00
Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Downloading torch-2.14.0-cp312-cp312-manylinux_2_28_x86_64.whl (554.6 MB)

```

554.6/554.6 MB
26.9 MB/s 0:00:16
Downloading cuda_toolkit-13.0.3.0-py2.py3-none-any.whl (2.5 kB)
Downloading nvidia_cudnn_cu13-9.24.0.43-py3-none-manylinux_2_27_x86_64.whl
(553.1 MB)
553.1/553.1 MB
48.7 MB/s 0:00:07
Downloading nvidia_cusparselt_cu13-0.8.1-py3-none-manylinux2014_x86_64.whl
(170.1 MB)
170.1/170.1 MB
61.4 MB/s 0:00:02
Downloading nvidia_nccl_cu13-2.30.7-py3-none-manylinux_2_18_x86_64.whl
(216.0 MB)
216.0/216.0 MB
64.9 MB/s 0:00:03
Downloading nvidia_nvshmem_cu13-3.4.5-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (60.4 MB)
60.4/60.4 MB
62.8 MB/s 0:00:00
Downloading
cuda_bindings-13.3.1-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl
(6.7 MB)
6.7/6.7 MB
78.8 MB/s 0:00:00
Downloading nvidia_cublas-13.1.1.3-py3-none-manylinux_2_27_x86_64.whl
(423.1 MB)
423.1/423.1 MB
43.6 MB/s 0:00:06
Downloading nvidia_cuda_cupti-13.0.85-py3-none-manylinux_2_25_x86_64.whl
(10.7 MB)
10.7/10.7 MB
99.9 MB/s 0:00:00
Downloading nvidia_cuda_nvrtc-13.0.88-py3-none-
manylinux2010_x86_64.manylinux_2_12_x86_64.whl (90.2 MB)
90.2/90.2 MB
51.5 MB/s 0:00:01
Downloading nvidia_cuda_runtime-13.0.96-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (2.2 MB)
2.2/2.2 MB
29.8 MB/s 0:00:00
Downloading nvidia_cufft-12.0.0.61-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (214.1 MB)
214.1/214.1 MB
58.5 MB/s 0:00:03
Downloading nvidia_cufile-1.15.1.6-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (1.2 MB)
1.2/1.2 MB
40.4 MB/s 0:00:00

Downloading nvidia_curand-10.4.0.35-py3-none-manylinux_2_27_x86_64.whl
(59.5 MB)
59.5/59.5 MB
80.7 MB/s 0:00:00

Downloading nvidia_cusolver-12.0.4.66-py3-none-manylinux_2_27_x86_64.whl
(200.9 MB)
200.9/200.9 MB
67.4 MB/s 0:00:02

Downloading nvidia_cuspars-12.6.3.3-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (145.9 MB)
145.9/145.9 MB
64.7 MB/s 0:00:02

Downloading nvidia_nvjitlink-13.3.33-py3-none-
manylinux2010_x86_64.manylinux_2_12_x86_64.whl (40.7 MB)
40.7/40.7 MB
69.1 MB/s 0:00:00

Downloading nvidia_nvtx-13.0.85-py3-none-
manylinux1_x86_64.manylinux_2_5_x86_64.whl (148 kB)

Downloading
triton-3.8.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (248.0
MB)
248.0/248.0 MB
66.5 MB/s 0:00:03

Downloading cuda_pathfinder-1.8.1-py3-none-any.whl (62 kB)

Downloading networkx-3.6.1-py3-none-any.whl (2.1 MB)
2.1/2.1 MB
46.2 MB/s 0:00:00

Downloading setuptools-84.0.0-py3-none-any.whl (818 kB)
818.2/818.2 kB
22.0 MB/s 0:00:00

Downloading sympy-1.14.0-py3-none-any.whl (6.3 MB)
6.3/6.3 MB
87.1 MB/s 0:00:00

Downloading mpmath-1.3.0-py3-none-any.whl (536 kB)
536.2/536.2 kB
14.7 MB/s 0:00:00

Downloading tqdm-4.70.0-py3-none-any.whl (80 kB)

Downloading typing_extensions-4.16.0-py3-none-any.whl (45 kB)

Downloading umap_learn-0.5.12-py3-none-any.whl (91 kB)

Downloading filelock-3.32.5-py3-none-any.whl (100 kB)

Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)

Downloading
pillow-12.3.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (6.9
MB)
6.9/6.9 MB
101.7 MB/s 0:00:00

Downloading requests-2.34.2-py3-none-any.whl (73 kB)

Downloading charset_normalizer-3.5.1-cp312-cp312-

```

manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (248 kB)
Downloading idna-3.19-py3-none-any.whl (68 kB)
Downloading urllib3-2.7.0-py3-none-any.whl (131 kB)
Downloading certifi-2026.7.22-py3-none-any.whl (136 kB)
Installing collected packages: nvidia-cusparselt-cu13, mpmath, cuda-toolkit,
urllib3, typing_extensions, triton, tqdm, threadpoolctl, sympy, setuptools,
safetensors, regex, pyyaml, python-dateutil, Pillow, packaging, nvidia-nvtx,
nvidia-nvshmem-cu13, nvidia-nvjitlink, nvidia-nccl-cu13, nvidia-curand, nvidia-
cufile, nvidia-cuda-runtime, nvidia-cuda-nvrtc, nvidia-cuda-cupti, numpy,
networkx, narwhals, llvmlite, jinja2, idna, hf-xet, fsspec, filelock, cuda-
pathfinder, cloudpickle, charset_normalizer, certifi, scipy, requests, plotly,
pandas, nvidia-cusparse, nvidia-cufft, nvidia-cublas, numba, joblib, cuda-
bindings, scikit-learn, nvidia-cusolver, nvidia-cudnn-cu13, huggingface-hub,
tokenizers, pynndescent, hdbscan, umap-learn, transformers, torch,
sentence_transformers, bertopic
  Attempting uninstall: setuptools
    Found existing installation: setuptools 68.1.2
    Uninstalling setuptools-68.1.2:
      Successfully uninstalled setuptools-68.1.2

```

60/60

[bertopic]

```

Successfully installed Pillow-12.3.0 bertopic-0.17.4 certifi-2026.7.22
charset_normalizer-3.5.1 cloudpickle-3.1.2 cuda-bindings-13.3.1 cuda-
pathfinder-1.8.1 cuda-toolkit-13.0.3.0 filelock-3.32.5 fsspec-2026.7.0
hdbscan-0.8.44 hf-xet-1.6.0 huggingface-hub-0.36.2 idna-3.19 jinja2-3.1.6
joblib-1.6.0 llvmlite-0.49.0 mpmath-1.3.0 narwhals-2.25.0 networkx-3.6.1
numba-0.67.0 numpy-2.5.2 nvidia-cublas-13.1.1.3 nvidia-cuda-cupti-13.0.85
nvidia-cuda-nvrtc-13.0.88 nvidia-cuda-runtime-13.0.96 nvidia-cudnn-
cu13-9.24.0.43 nvidia-cufft-12.0.0.61 nvidia-cufile-1.15.1.6 nvidia-
curand-10.4.0.35 nvidia-cusolver-12.0.4.66 nvidia-cusparse-12.6.3.3 nvidia-
cusparselt-cu13-0.8.1 nvidia-nccl-cu13-2.30.7 nvidia-nvjitlink-13.3.33 nvidia-
nvshmem-cu13-3.4.5 nvidia-nvtx-13.0.85 packaging-26.3 pandas-3.0.5 plotly-7.0.0
pynndescent-0.5.13 python-dateutil-2.9.0.post0 pyyaml-6.0.3 regex-2026.9.3
requests-2.34.2 safetensors-0.8.0 scikit-learn-1.9.0 scipy-1.18.1
sentence_transformers-5.1.1 setuptools-84.0.0 sympy-1.14.0 threadpoolctl-3.6.0
tokenizers-0.21.4 torch-2.14.0 tqdm-4.70.0 transformers-4.55.4 triton-3.8.0
typing_extensions-4.16.0 umap-learn-0.5.12 urllib3-2.7.0

```

```

[ ]: %%script python3
import sys
print("Python no subprocesso:", sys.version)

```

Python no subprocesso: 3.12.13 (main, Mar 4 2026, 09:23:07) [GCC 11.4.0]

```

[ ]: %%script python3
import sys
import bertopic
import numba

```

```

import numpy
import joblib
import transformers
import sentence_transformers as st
import umap

print("Python :", sys.version)
print("BERTopic:", bertopic.__version__)
print('Transformer:', transformers.__version__)
print('ST:', st.__version__)
print("Numba  :", numba.__version__)
print("NumPy  :", numpy.__version__)
print("Joblib  :", joblib.__version__)
print("UMAP   :", umap.__version__)

```

```

Python : 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
BERTopic: 0.17.4
Transformer: 4.55.4
ST: 5.1.1
Numba  : 0.67.0
NumPy  : 2.5.2
Joblib  : 1.6.0
UMAP   : 0.5.12

```

```

[ ]: %%script python3
import pandas as pd
from sentence_transformers import SentenceTransformer
from bertopic import BERTopic
import os

# Carregar dados e modelo
df = pd.read_csv('/content/drive/MyDrive/TCC/Data/TUEBR222.csv',
                 sep=";", low_memory=False,
                 parse_dates=['tweet_created_at', 'user_created_at',
                              →'collection_date'])

grupos_limpos = df.groupby("clean_tweet").groups
docs = list(grupos_limpos.keys())

topic_model = BERTopic.load("/content/drive/MyDrive/TCC/Data/TUEBR22_topics")

# Gerar embeddings (necessário para visualize_documents)
embedding_model = SentenceTransformer("sentence-transformers/
→paraphrase-multilingual-MiniLM-L12-v2")
embeddings = embedding_model.encode(docs, show_progress_bar=False)

# Criar uma pasta para salvar os gráficos

```

```

output_dir = "/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic"
os.makedirs(output_dir, exist_ok=True)

# Dicionário com os gráficos
graficos = {
    "barchart": lambda: topic_model.visualize_barchart(top_n_topics=20,
↳custom_labels=True),
    "term_rank": lambda: topic_model.visualize_term_rank(custom_labels=True),
    "term_rank_log": lambda: topic_model.visualize_term_rank(log_scale=True,
↳custom_labels=True),
    "topics": lambda: topic_model.visualize_topics(custom_labels=True),
    "heatmap": lambda: topic_model.visualize_heatmap(top_n_topics=20,
↳custom_labels=True),
    "hierarchy": lambda: topic_model.visualize_hierarchy(top_n_topics=20,
↳custom_labels=True),
    "documents": lambda: topic_model.visualize_documents(
        docs, embeddings=embeddings, custom_labels=True, hide_annotations=True
    )
}

# Salvar cada gráfico como HTML
for nome, func in graficos.items():
    print(f"Gerando e salvando: {nome}...")
    fig = func()
    caminho = os.path.join(output_dir, f"{nome}.html")
    fig.write_html(caminho)
    print(f" → Salvo em: {caminho}")

print(f"\n Todos os gráficos foram salvos em: {output_dir}")

```

```

Gerando e salvando: barchart...
 → Salvo em:
/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/barchart.html
Gerando e salvando: term_rank...
 → Salvo em:
/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/term_rank.html
Gerando e salvando: term_rank_log...
 → Salvo em:
/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/term_rank_log.html
Gerando e salvando: topics...
 → Salvo em: /content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/topics.html
Gerando e salvando: heatmap...
 → Salvo em:
/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/heatmap.html
Gerando e salvando: hierarchy...
 → Salvo em:
/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/hierarchy.html

```

Gerando e salvando: documents...

→ Salvo em:

/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/documents.html

Todos os gráficos foram salvos em:

/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic

```
[ ]: from IPython.display import HTML, display

with open("/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/barchart.
↳html", "r") as f:
    display(HTML(f.read()))
```

<IPython.core.display.HTML object>

```
[ ]: # Term Rank
from IPython.display import HTML, display

with open("/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/term_rank.
↳html", "r") as f:
    display(HTML(f.read()))
```

<IPython.core.display.HTML object>

```
[ ]: # Term Rank (Escala Log)
from IPython.display import HTML, display

with open("/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/term_rank_log.
↳html", "r") as f:
    display(HTML(f.read()))
```

<IPython.core.display.HTML object>

```
[ ]: # Topics (Intertopic Distance Map)
from IPython.display import HTML, display

with open("/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/topics.html",
↳"r") as f:
    display(HTML(f.read()))
```

<IPython.core.display.HTML object>

```
[ ]: # Heatmap
from IPython.display import HTML, display

with open("/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/heatmap.html",
↳"r") as f:
    display(HTML(f.read()))
```

<IPython.core.display.HTML object>

```
[ ]: # Hierarchy (Dendrograma)
from IPython.display import HTML, display

with open("/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/hierarchy.
↳html", "r") as f:
    display(HTML(f.read()))
```

<IPython.core.display.HTML object>

```
[ ]: # Documentos (Visualização dos Documentos)
from IPython.display import HTML, display

with open("/content/drive/MyDrive/TCC/Estaticos/graficos_bertopic/documents.
↳html", "r") as f:
    display(HTML(f.read()))
```

<IPython.core.display.HTML object>